
Numerik 1 2026 Kurzversion 0.06

Martin Reißel

24.06.2026

I	Einleitung	1
1	Was ist Numerik?	3
2	Numerische Verfahren für Wissenschaftliches Rechnen und Simulation	5
3	Numerische Verfahren für Data Science und Machine Learning	7
4	Zusammenfassung	9
II	Lineare Gleichungssysteme - Direkte Löser I	11
5	Überblick	13
6	Theorie	15
7	Gaußelimination	19
8	LU-Zerlegung	23
8.1	Ohne Zeilentausch	23
8.2	Beliebige reguläre Matrizen	26
9	Cholesky-Zerlegung	31
10	Zusammenfassung	35
III	Fehler in numerischen Prozessen	37
11	Überblick	39
12	Gut gestellte Probleme	41
13	Fehlergrößen	43
14	Fehlerquellen	45
14.1	Floating-Point-Darstellung von Zahlen	46
14.2	Eingabefehler, Kondition	49
14.3	Verfahrensfehler	51
14.4	Akkumulierter Rundungsfehler, Floating-Point-Arithmetik	52
15	Zusammenfassung	55

IV	Lineare Gleichungssysteme - Direkte Löser II	57
16	Überblick	59
17	Fehlerbetrachtung für lineare Gleichungssysteme	61
18	LU-Zerlegung mit Pivot-Strategie	65
19	Grundlagen	67
20	Orthogonale Transformationen	69
20.1	Verfahren von Givens	71
20.2	Verfahren von Householder	74
21	Zusammenfassung	79
V	Iterative Löser	81
22	Überblick	83
23	Grundlagen	85
24	Lineare stationäre einstufige Iterationen mit Fixpunkteigenschaft	87
24.1	Konstruktion	87
24.2	Konvergenz	89
24.3	Gesamtschritt-Verfahren (Jacobi)	91
24.4	Einzelschritt-Verfahren (Gauß-Seidel)	92
24.5	Nachiteration	93
24.6	Relaxationsverfahren	93
24.7	Symmetrische Varianten	95
25	Nichtstationäre Iterationen	97
25.1	Steepest-Descent-Verfahren (Methode des steilsten Abstiegs)	97
25.2	Konjugierte Gradienten (CG)	99
26	Dünn besetzte Matrizen	101
27	Vergleich verschiedener Verfahren	103
28	Zusammenfassung	105
VI	Eigenwerte und Eigenvektoren	107
29	Überblick	109
30	Motivation	111
31	Theorie	113
31.1	Komplexwertige Matrizen	113
31.2	Reellwertige Matrizen	116
31.3	Positiv definite Matrizen	118
31.4	Zusammenfassung	119
32	Vektoriteration	121
33	Jacobi-Verfahren	123
34	QR-Iteration	127
34.1	Hessenberg-Transformation	129

34.2	Shift-Technik, Deflation	130
35	Zusammenfassung	133
VII	Nullstellenprobleme	135
36	Überblick	137
37	Einschließungsverfahren	139
37.1	Bisektion	139
37.2	Regula-Falsi-Verfahren	141
38	Nicht-Einschließungsverfahren	143
38.1	Sekanten-Verfahren	143
38.2	Taylor-Reihen-basierte Verfahren, Newton-Verfahren	144
39	Stationäre Iterationen, Banachscher Fixpunktsatz	145
40	Newton-Verfahren	149
41	Zusammenfassung	153
VIII	Interpolation	155
42	Überblick	157
43	Polynom Interpolation	159
43.1	Grundlagen	159
43.2	Vandermonde System	159
43.3	Lagrange Polynome	160
43.4	Newton-Darstellung, dividierte Differenzen	161
44	Interpolation mit stückweise polynomialen Funktionen	167
44.1	Polynom-Splines	167
44.2	Kubische Polynom-Splines	168
44.2.1	Abschlussbedingungen	168
44.2.2	Direkte Berechnung	169
44.2.3	Berechnung über Momente	169
45	Interpolierende Kurven	173
46	Zusammenfassung	175
IX	Approximation	177
47	Überblick	179
48	Lineare Ausgleichsprobleme	181
48.1	Grundlagen	181
48.2	QR Zerlegung	183
48.3	CGLS	184
49	Pseudoinverse	187
50	Singulärwertzerlegung	189
51	Regularisierung schlecht konditionierter Probleme	193
51.1	Grundlagen	193

51.2	Abgeschnittene Singulärwertzerlegung (TSVD)	194
51.3	Tikhonov Regularisierung	195
51.4	Parameterwahl	196
52	Zusammenfassung	199
X	Integration	201
53	Überblick	203
54	Newton-Cotes Formeln	205
54.1	Grundlagen	205
54.2	Integralapproximation für $[a,b]=[0,1]$	206
54.3	Integralapproximation für beschränkte Intervalle $[a,b]$	209
54.4	Summierte Formeln	209
54.5	Fehlerbetrachtung	211
54.5.1	Einfache Newton-Cotes Formeln	211
54.5.2	Exaktheitsgrad	214
54.5.3	Summierte Newton-Cotes Formeln	214
55	Gauß-Integration	217
56	Extrapolation	221
57	Adaptive Integration	231
57.1	Problemstellung	231
57.2	Fehlerindikator	231
57.3	Adaptive Newton-Cotes Verfahren	233
58	Zusammenfassung	235
XI	Nicht restringierte Optimierung	237
59	Überblick	239
60	Grundlagen	241
61	Abstiegsverfahren	243
61.1	Konstruktion	243
61.2	Liniensuche	245
61.3	Methode des steilsten Abstiegs (Steepest-Descent)	247
61.4	Konjugierte Gradienten (CG)	247
62	Newton-Verfahren	251
62.1	Konstruktion	251
62.2	Trust-Region-Verfahren	254
62.3	Quasi-Newton-Verfahren	257
63	Nichtlineare Ausgleichsprobleme	261
64	Zusammenfassung	265
	Literaturverzeichnis	267

Teil I

Einleitung

Was ist Numerik?

- das heißt konkret:
 - „continuous mathematics“: reelle oder komplexe Zahlen sind beteiligt (in Abgrenzung zur diskreten Mathematik), dies umfasst Teile der Analysis und der linearen Algebra
 - Entwicklung von (effizienten) Berechnungsmethoden
 - bei Berechnung mit einem Computer: *Floating-Point*-Zahlen und -Arithmetik
 - Konsequenz: Studium von Rundungsfehlern
- Hintergrund: viele mathematische Objekte sind zwar wohldefiniert, aber nicht endlich berechenbar
- **bisher gesehen:** innermathematische Anwendung numerischer Verfahren
- mehr Beispiele hierzu:

- analog zu oben: Man berechne $\sin \alpha$
 - in der Analysis: Definition von $\sin \alpha$ üblicherweise durch unendliche Reihe (Taylorreihe):

$$\sin \alpha = \sum_{n=0}^{\infty} (-1)^n \frac{\alpha^{2n+1}}{(2n+1)!}$$

- aus der Analysis bekannt: diese Reihe konvergiert für jedes $\alpha \Rightarrow \sin$ wohldefiniert
 - aber: Grenzwert i. A. nicht endlich berechenbar $\Rightarrow$ numerische Verfahren zur näherungsweisen Berechnung
- analog: $\cos \alpha$
- $\int_a^b f(x) dx$ existiert immer für stetiges f , ist aber i. A. nicht unmittelbar berechenbar, wenn man keine Stammfunktion von f findet $\Rightarrow$ numerische Verfahren zur näherungsweisen Berechnung
- Berechnung von $\sqrt{x}$ (siehe Übung) i. A. nicht exakt durchführbar $\Rightarrow$ numerische Verfahren zur näherungsweisen Berechnung
- Verallgemeinerung: Nullstellen von Polynomen höheren Grades i. A. nicht direkt berechenbar $\Rightarrow$ numerische Verfahren zur näherungsweisen Berechnung
- ...

Numerische Verfahren für Wissenschaftliches Rechnen und Simulation

- Numerische Simulation: Verhaltensvorhersage eines physikalischen Systems durch mathematische Modelle und numerische Verfahren („digitaler Zwilling“)
- physikalisches System: z. B. beliebiges Bauteil, Auto, Flugzeug, Atom, Planet, ...
- wichtiges Einsatzgebiet der numerischen Simulation: Produktentwicklung in der Industrie
- **klassisches Vorgehen in der Produktentwicklung (vereinfacht):**
 - grobe Vorauslegung der Konstruktion
 - Prototyp bauen und testen
 - wenn Test erfolgreich: Prototyp kann in Serie gehen
 - wenn Test fehlgeschlagen: Ein veränderter Prototyp wird getestet
 - Iteration bis zum Erfolg
- **Alternative:**
 - virtueller Prototyp: vollständige Analyse durch *Numerische Simulation*
 - davon ausgehend: endgültige Auslegung der Konstruktion
 - am Ende: *Validierung* durch *einen* realen Prototypen
- Vorteil: spart i. d. R. Geld und Zeit (Extrembeispiel: Konstruktion eines Verkehrsflugzeugs)
- die numerische Mathematik stellt mathematische Grundlagen und effiziente Algorithmen zur Numerischen Simulation bereit.
- *wissenschaftliches Rechnen*: ähnlich, aber eher durch wissenschaftliches Interesse motiviert (z. B. Simulationen, um die Entstehung von Planeten zu verstehen, z.B. [Simulation des menschlichen Gehirns](#))
- Wissenschaftliches Rechnen und Numerische Simulation können extreme Anforderungen an die Leistungsfähigkeit von Computern stellen; siehe z.B. [Deep Sea Project](#)
- Entwicklung von effizienten Algorithmen für Supercomputer: Grenzgebiet zwischen Numerik und Informatik
- „The next big thing“: automatische Optimierung
 - bisher: Konstruktion gegeben, numerische Simulation erlaubt virtuelle Tests
 - besser: Man lasse den Computer die optimale Konstruktion finden
 - dazu:

- man hält gewisse Abmessungen im CAD-Modell variabel („Parametrisierung der Geometrie“)
- man finde iterativ den optimalen Parametersatz und damit die optimale Geometrie
- pro Iteration: eine numerische Simulation

Numerische Verfahren für Data Science und Machine Learning

- in den Bereichen Data Science (DS) und Machine Learning (ML) gibt es zahlreiche Anwendungen die auf numerischen Verfahren basieren

Zusammenfassung

- viele Objekte der Mathematik sind nicht endlich berechenbar
- Aufgabenstellungen müssen oft durch vereinfachte Ersatzprobleme approximiert werden
- mathematisch äquivalente Algorithmen können in Floating-Point-Arithmetik unterschiedliche Ergebnisse liefern
- ein großes Anwendungsgebiet der numerischen Mathematik: Numerische Simulation, wissenschaftliches Rechnen, Data Science und Machine Learning

Teil II

Lineare Gleichungssysteme - Direkte Löser I

Überblick

- das Lösen linearer Gleichungssysteme $Ax = b$ ist ein zentrales Grundproblem in Wissenschaft und Technik (technische Simulation mit Differentialgleichungen, Optimierung, Modellanpassung Data Science,...)
- wir betrachten eindeutig lösbare quadratische lineare Gleichungssysteme $Ax = b$
- eine elementare Methode zur Lösung solcher Systeme ist die Gaußelimination
- darauf aufbauend entwickeln wir die LU-Zerlegung, die das effiziente Lösen bei verschiedenen rechte Seiten ermöglicht
- Gaußelimination und LU-Zerlegung sind direkte Verfahren, in exakter Arithmetik liefern sie nach endlich vielen (Grund-) Rechenoperationen die exakte Lösung
- schränken wir die Klasse der Gleichungssysteme auf solche mit symmetrisch positiv definiten Matrizen ein, so können wir die LU-Zerlegung weiter vereinfachen und erhalten die Cholesky-Zerlegung, die etwa nur die Hälfte des Rechenaufwands der LU-Zerlegung benötigt.

- betrachte mehrere Gleichungen für eine bestimmte Anzahl von Unbekannten
- Unbekannte dürfen nur linear auftreten, wie in den folgenden Beispielen:

$$\begin{array}{cccccc} 5x = 7, & x + 2y = 5 & x + 2y = 5 & 5x + 3y = 8, & 5x = 7 \\ & 2x + y = 4, & 2x + y = 4, & & 10x = 3 \end{array}$$

- das System

$$\begin{array}{l} 3x^2 + y = 1 \\ x + 2y = 3 \end{array}$$

ist nichtlinear

- betrachten wir allgemein ein System von m Gleichungen für n Unbekannte, so erhalten wir:

$$\begin{array}{ccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \dots & + & a_{1n}x_n & = & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \dots & + & a_{2n}x_n & = & b_2 \\ \vdots & & & & & & \vdots & & \vdots \\ a_{m1}x_1 & + & a_{m2}x_2 & + & \dots & + & a_{mn}x_n & = & b_m \end{array}$$

mit

$$\begin{array}{ll} a_{ij} \in \mathbb{R} & \text{gegebene Koeffizienten,} \\ b_i \in \mathbb{R} & \text{gegebene rechte Seite,} \\ x_j \in \mathbb{R} & \text{gesuchte Unbekannte} \end{array}$$

- zur Abkürzung benutzt man die Matrix-Vektor-Schreibweise

$$Ax = b$$

mit Systemmatrix

$$A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n},$$

rechter Seite

$$b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix} \in \mathbb{R}^m,$$

Vektor der Unbekannten

$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n$$

und Matrix-Vektor Produkt

$$Ax = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} a_{11}x_1 + \dots + a_{1n}x_n \\ \vdots \\ a_{m1}x_1 + \dots + a_{mn}x_n \end{pmatrix}$$

- lineare Algebra:
 - ist $m \neq n$ (# Gleichungen $\neq$ # Unbekannte) dann gibt es in der Regel keine *eindeutige* Lösung sondern entweder keine Lösung oder unendlich viele
 - $m = n \Rightarrow Ax = b$ eindeutig lösbar falls A *regulär* ist

Definition 8

$A \in \mathbb{R}^{n \times n}$ heißt *regulär*, falls $\det(A) \neq 0$, *singulär*, falls $\det(A) = 0$.

- alternative Kriterien sind:
 - $\det(A) \neq 0 \Leftrightarrow$ Spalten $s_j = \begin{pmatrix} a_{1j} \\ \vdots \\ a_{nj} \end{pmatrix}, j = 1, \dots, n$, linear unabhängig
 - $\det(A) \neq 0 \Leftrightarrow$ Zeilen $z_i = (a_{i1}, \dots, a_{in}), i = 1, \dots, n$, linear unabhängig
- die Anzahl unabhängiger Spalten und unabhängiger Zeilen ist gleich und wird als Rang der Matrix $\text{rang}(A)$ bezeichnet
- $A \in \mathbb{R}^{n \times n}$ regulär $\Leftrightarrow \text{rang}(A) = n$, $A \in \mathbb{R}^{n \times n}$ singulär $\Leftrightarrow \text{rang}(A) < n$
- eine Matrix $A \in \mathbb{R}^{m \times n}$ definiert eine lineare Abbildung $\mathbb{R}^n \rightarrow \mathbb{R}^m$ durch $x \mapsto Ax$
- jede lineare Abbildung $\mathbb{R}^n \rightarrow \mathbb{R}^m$ ist als Matrix darstellbar
- $A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{p \times m}$ nacheinander ausgeführt ergibt eine neue lineare Abbildung $\mathbb{R}^n \rightarrow \mathbb{R}^p$:

$$\mathbb{R}^n \ni x \mapsto Ax \mapsto B(Ax) \in \mathbb{R}^p$$

- die Matrix C der neuen Abbildung ergibt sich durch Matrizenmultiplikation:

$$B = \begin{pmatrix} b_{11} & \dots & b_{1m} \\ \vdots & & \vdots \\ b_{p1} & \dots & b_{pm} \end{pmatrix} \in \mathbb{R}^{p \times m}, \quad A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n},$$

$$BA = \begin{pmatrix} (b_{11}a_{11} + \dots + b_{1m}a_{m1}) & \dots & (b_{11}a_{1n} + \dots + b_{1m}a_{mn}) \\ \vdots & & \vdots \\ (b_{p1}a_{11} + \dots + b_{pm}a_{m1}) & \dots & (b_{p1}a_{1n} + \dots + b_{pm}a_{mn}) \end{pmatrix} \in \mathbb{R}^{p \times n}$$

- Matrizenmultiplikation ist nur bei passender Dimension möglich
- in der Regel ist $AB \neq BA$ (auch bei $m = n$)
- die zu

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n}$$

transponierte Matrix ist

$$A^T = \begin{pmatrix} a_{11} & \cdots & a_{m1} \\ \vdots & & \vdots \\ a_{1n} & \cdots & a_{mn} \end{pmatrix} \in \mathbb{R}^{n \times m}$$

- A^T entsteht durch „Vertauschen von Zeilen mit Spalten“, d.h. „Spiegelung“ an der Diagonalen
- $(AB)^T = B^T A^T$
- ist $m = n$, so nennt man $A \in \mathbb{R}^{n \times n}$ quadratisch
- $A, B \in \mathbb{R}^{n \times n} \Rightarrow \det(AB) = \det(A) \det(B) \Rightarrow$ sind A, B regulär, dann ist auch AB regulär
- ist $A \in \mathbb{R}^{n \times n}$ regulär, so existiert die Inverse $A^{-1} \in \mathbb{R}^{n \times n}$ mit

$$A^{-1}A = AA^{-1} = I = \begin{pmatrix} 1 & & \\ & \ddots & \\ & & 1 \end{pmatrix}$$

- für A, B regulär ist $(AB)^{-1} = B^{-1}A^{-1}$
- auf $\mathbb{R}^n$ verwenden wir das euklidische Skalarprodukt

$$(x, y)_2 = \sum_{i=1}^n x_i y_i, \quad x, y \in \mathbb{R}^n$$

- betrachten wir $x, y \in \mathbb{R}^n$ als Matrizen

$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^{n \times 1}, \quad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^{n \times 1}$$

dann ist $x^T = (x_1, \dots, x_n) \in \mathbb{R}^{1 \times n}$ und

$$x^T y = (x_1, \dots, x_n) \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} = x_1 y_1 + \dots + x_n y_n = (x, y)_2 \in \mathbb{R}^{1 \times 1} = \mathbb{R}$$

- wir betrachten in folgendem Kapitel nur Systeme mit regulärem A , d.h. quadratische Systeme, die eindeutig lösbar sind
- um den „Betrag“ eines Vektors $x = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ zu beschreiben benutzt man gewöhnlich die *euklidische Norm*

$$\|x\|_2 = \sqrt{(x, x)_2} = \sqrt{x_1^2 + \dots + x_n^2}$$

- dies ist ein Spezialfall einer Vektornorm

Definition 9

$\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ heißt Norm auf $\mathbb{R}^n$ falls

1. $\|x\| = 0 \Leftrightarrow x = 0$
2. $\|\alpha x\| = |\alpha| \|x\| \quad \forall x \in \mathbb{R}^n \quad \forall \alpha \in \mathbb{R}$
3. $\|x + y\| \leq \|x\| + \|y\| \quad \forall x, y \in \mathbb{R}^n$

- häufig benutzte Normen sind:

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}, \quad 1 \leq p < \infty, \text{ zum Beispiel}$$

$$\begin{aligned} p = 1 & : \|x\|_1 = |x_1| + \dots + |x_n| \\ p = 2 & : \|x\|_2 = \sqrt{x_1^2 + \dots + x_n^2} \end{aligned}$$

- $\|x\|_\infty = \max_{i=1,\dots,n} |x_i|$
- die verschiedenen Normen auf $\mathbb{R}^n$ sind im folgenden Sinn äquivalent:
 - sind $\|\cdot\|, \|\cdot\|'$ zwei Normen auf $\mathbb{R}^n$ dann gibt es Konstanten α, β unabhängig von x mit

$$\alpha\|x\|' \leq \|x\| \leq \beta\|x\|' \quad \forall x \in \mathbb{R}^n$$

- wir verwenden Normen um Fehler zu messen:
 - betrachte $Ax = b, Ax' = b'$
 - vergleiche $\|b - b'\|$ mit $\|x - x'\|$
- benutze Norm, die einfach zu handhaben ist
- Norm beschreibt „Größe“ von Vektoren
- „Größe“ von Matrizen:
 - $A \in \mathbb{R}^{n \times n}$ definiert lineare Abbildung $\mathbb{R}^n \rightarrow \mathbb{R}^n$ durch $x \mapsto Ax$
 - „Größe“ von A :
 - wie stark wird ein Vektor maximal verlängert?
 - vergleiche $\|Ax\|$ mit $\|x\|$

! Satz 11

Ist $\|\cdot\|_V$ eine Norm auf $\mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$, dann definiert

$$\|A\|_I := \sup_{x \neq 0} \frac{\|Ax\|_V}{\|x\|_V} = \max_{\|x\|_V=1} \|Ax\|_V$$

eine *Matrixnorm* auf $\mathbb{R}^{n \times n}$, die von $\|\cdot\|_V$ induzierte Norm.

Gaußelimination

- erstes systematisches Verfahren für lineare Gleichungssysteme
- von Gauß in seinen astronomischen Arbeiten benutzt
- war vorher schon in China und im arabischen Kulturkreis bekannt
- wir betrachten das folgende Beispiel:

$$x_1 + x_2 + 3x_3 = 6 \quad (7.1)$$

$$3x_1 + 6x_2 + 10x_3 = 25 \quad (7.2)$$

$$x_1 + 4x_2 + 6x_3 = 15 \quad (7.3)$$

- versuche durch Skalieren/Subtrahieren die Gleichungen zu vereinfachen, d.h. Unbekannte zu eliminieren:

$$x_1 + x_2 + 3x_3 = 6 \quad (7.4)$$

$$3x_2 + x_3 = 7 \quad (7.5)$$

$$3x_2 + 3x_3 = 9 \quad (7.6)$$

- eliminieren wir noch x_2 , so erhalten wir

$$x_1 + x_2 + 3x_3 = 6 \quad (7.7)$$

$$3x_2 + x_3 = 7 \quad (7.8)$$

$$2x_3 = 2 \quad (7.9)$$

- die letzte Gleichung enthält nur eine Unbekannte und ist direkt lösbar:

$$x_3 = 1$$

- setzt man x_3 in die zweite Gleichung ein so erhalten wir

$$3x_2 + 1 = 7$$

und damit $x_2 = 2$

- analog folgt für die erste Gleichung $x_1 + 2 + 3 = 6$ und

$$x_1 = 1$$

- damit ist $x = (1, 2, 1)^T$ Lösung des Systems
- wie sehen die Umformungen in Matrixschreibweise aus?
- wir fassen dazu A, b im erweiterten System $(A|b)$ zusammen:

$$\left(\begin{array}{ccc|c} 1 & 1 & 3 & 6 \\ 3 & 6 & 10 & 25 \\ 1 & 4 & 6 & 15 \end{array} \right) \rightarrow \left(\begin{array}{ccc|c} 1 & 1 & 3 & 6 \\ 0 & 3 & 1 & 7 \\ 0 & 3 & 3 & 9 \end{array} \right) \rightarrow \left(\begin{array}{ccc|c} 1 & 1 & 3 & 6 \\ 0 & 3 & 1 & 7 \\ 0 & 0 & 2 & 2 \end{array} \right)$$

- *eliminiere* durch Skalierung/Subtraktion von Zeilen die *Matrixspalten* unterhalb der Diagonalen (schrittweises Entfernen von Unbekannten aus Gleichungen)
- erhalte *Dreiecksgestalt*, d.h. letzte Gleichung enthält eine Unbekannte, vorletzte 2, etc.
- aus Dreiecksgestalt kann wie oben durch *rückwärts einsetzen* die Lösung bestimmt werden
- zusammengefasst erhalten wir folgendes Schema:

$$\text{erweitertes System} \xrightarrow{\text{Spaltenelimination}} \text{Dreiecksgestalt} \xrightarrow{\text{Rückwärtseinsetzen}} \text{Lösung}$$

- funktioniert dieses Verfahren für alle regulären Systeme mit $A \in \mathbb{R}^{n \times n}$?
- betrachten wir als Beispiel das folgende erweiterte System

$$(A|b) = \left(\begin{array}{ccc|c} 1 & 2 & 3 & 8 \\ 3 & 6 & 10 & 25 \\ 1 & 4 & 6 & 15 \end{array} \right)$$

- für A gilt $\det(A) = -2$, d.h. $Ax = b$ ist immer eindeutig lösbar
- wenden wir den Eliminationsprozess an so erhalten wir nach dem ersten Schritt

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 8 \\ 0 & 0 & 1 & 1 \\ 0 & 2 & 3 & 7 \end{array} \right)$$

- der zweite Schritt ist so nicht direkt durchführbar, da das zweite Diagonalelement verschwindet
- deshalb tauschen wir die Reihenfolge der zweiten und dritten Gleichung (d.h. der entsprechenden Zeilen in $(A|b)$) und erhalten

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 8 \\ 0 & 2 & 3 & 7 \\ 0 & 0 & 1 & 1 \end{array} \right)$$

wobei die Lösung wieder mit Rückwärtseinsetzen berechenbar ist

- es gilt also:
 - bei 0 auf der Diagonalen kann zunächst nicht eliminiert werden
 - die aktuelle Zeile muss mit einer darunter liegenden Zeile vertauscht werden, so dass anschließend das Diagonalelement ungleich 0 ist
- wir nehmen zunächst an, dass alle Diagonalelemente $\neq 0$ sind
- allgemeiner Fall kommt später
- dort sehen wir dann auch, dass für A regulär immer ein geeigneter Zeilentauch möglich ist
- Verallgemeinerung auf $n \times n$ Systeme:
 - sei A, b gegeben, d.h.

$$\left(\begin{array}{ccc|c} a_{11} & \dots & a_{1n} & b_1 \\ a_{21} & \dots & a_{2n} & b_2 \\ \vdots & & \vdots & \vdots \\ a_{n1} & \dots & a_{nn} & b_n \end{array} \right)$$

- ist $a_{11} \neq 0$ so eliminiert man die erste Spalte durch

$$\begin{array}{lcl} \text{Zeile 2} & - & \frac{a_{21}}{a_{11}} \text{ Zeile 1} \\ \vdots & & \vdots \\ \text{Zeile } n & - & \frac{a_{n1}}{a_{11}} \text{ Zeile 1} \end{array}$$

so dass als neues System

$$\left(\begin{array}{cccc|c} a_{11}^{(1)} & a_{12}^{(1)} & \dots & a_{1n}^{(1)} & b_1^{(1)} \\ 0 & a_{22}^{(1)} & \dots & a_{2n}^{(1)} & b_2^{(1)} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & a_{n2}^{(1)} & \dots & a_{nn}^{(1)} & b_n^{(1)} \end{array} \right)$$

entsteht mit unveränderter erster Zeile

$$a_{1j}^{(1)} = a_{1j}, \quad j = 1, \dots, n, \quad b_1^{(1)} = b_1,$$

und

$$a_{ij}^{(1)} = a_{ij} - l_{i1}a_{1j}, \quad b_i^{(1)} = b_i - l_{i1}b_1, \quad i = 2, \dots, n, \quad j = 1, \dots, n,$$

d.h. die mit

$$l_{i1} = \frac{a_{i1}}{a_{11}}$$

skalierte erste Zeile wird von der i -ten Zeile subtrahiert

- damit haben wir $Ax = b$ in ein äquivalentes System $A^{(1)}x = b^{(1)}$ transformiert mit erster Spalte

$$s_1^{(1)} = \begin{pmatrix} a_{11}^{(1)} \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

- ist $a_{22}^{(1)} \neq 0$ kann analog die zweite Spalte eliminiert werden, etc.
- insgesamt erhalten wir nach $n - 1$ Schritten

$$(A|b) \rightarrow (A^{(1)}|b^{(1)}) \rightarrow \dots \rightarrow (A^{(n-1)}|b^{(n-1)})$$

mit

$$A^{(n-1)} = U = \begin{pmatrix} u_{11} & \dots & \dots & u_{1n} \\ 0 & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & u_{nn} \end{pmatrix}$$

- statt $Ax = b$ lösen wir dann $Ux = b^{(n-1)}$ durch Rückwärtseinsetzen:

$$\begin{array}{ccccccc} u_{11}x_1 + & \dots + & u_{1n-1}x_{n-1} + & u_{1n}x_n & = & b_1^{(n-1)} \\ & \ddots & & & & \vdots \\ & & u_{n-1n-1}x_{n-1} + & u_{n-1n}x_n & = & b_{n-1}^{(n-1)} \\ & & & u_{nn}x_n & = & b_n^{(n-1)} \end{array}$$

$$\begin{aligned}
 x_n &= \frac{b_n^{(n-1)}}{u_{nn}} \\
 \Rightarrow x_{n-1} &= \frac{b_{n-1}^{(n-1)} - u_{n-1n}x_n}{u_{n-1n-1}} \\
 x_{n-2} &= \frac{b_{n-2}^{(n-1)} - u_{n-2n}x_n - u_{n-2n-1}x_{n-1}}{u_{n-2n-2}} \\
 &\vdots
 \end{aligned}$$

8.1 Ohne Zeilentausch

- untersuchen den Eliminationsprozess von oben

$$\begin{array}{ccccccc} A & \rightarrow & A^{(1)} & \rightarrow & \dots & \rightarrow & A^{(n-1)} = U \\ b & \rightarrow & b^{(1)} & \rightarrow & \dots & \rightarrow & b^{(n-1)} \end{array}$$

und beschreiben ihn mathematisch präziser

- betrachten dann $Ax = b$, $Ax' = b'$ für identisches A :
 - Elimination liefert in beiden Fällen gleiches U
 - wie kann man sich doppelte Arbeit sparen?
- nehmen nach wie vor an, dass A regulär ist und wir keine Zeilen tauschen müssen
- sei zunächst

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad a_{11} \neq 0$$

- eliminiere a_{21} , indem von der zweiten Zeile $\frac{a_{21}}{a_{11}}$ mal Zeile 1 abgezogen wird
- das kann als Matrixmultiplikation geschrieben werden
- mit

$$F = \begin{pmatrix} 1 & 0 \\ -\frac{a_{21}}{a_{11}} & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ -l_{21} & 1 \end{pmatrix}$$

folgt

$$\begin{aligned} FA &= \begin{pmatrix} 1 & 0 \\ -\frac{a_{21}}{a_{11}} & 1 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \\ &= \begin{pmatrix} a_{11} & a_{12} \\ -\frac{a_{21}}{a_{11}}a_{11} + a_{21} & -\frac{a_{21}}{a_{11}}a_{12} + a_{22} \end{pmatrix} \\ &= \begin{pmatrix} a_{11} & a_{12} \\ 0 & a_{22} - \frac{a_{21}}{a_{11}}a_{12} \end{pmatrix} \end{aligned}$$

- analog folgt für die Elimination der ersten Spalte $A \rightarrow A^{(1)}$ im allgemeinen Fall

$$A^{(1)} = F_1 A, \quad F_1 = \begin{pmatrix} 1 & & & \\ -l_{21} & 1 & & \\ -l_{31} & & 1 & \\ \vdots & & & \ddots \\ -l_{n1} & & & & 1 \end{pmatrix} \in \mathbb{R}^{n \times n}$$

bzw. für die Elimination der k -ten Spalte $A^{(k-1)} \rightarrow A^{(k)}$

$$A^{(k)} = F_k A^{(k-1)}, \quad F_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & & 1 \end{pmatrix} \in \mathbb{R}^{n \times n},$$

mit $l_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}$

- die F_k heißen *Frobeniusmatrizen*
- mit den F_k erhalten wir

$$U = A^{(n-1)} = F_{n-1} \cdot \dots \cdot F_1 A$$

und

$$y = b^{(n-1)} = F_{n-1} \cdot \dots \cdot F_1 b$$

da auf b dieselben Zeilenoperationen angewandt werden

- alle F_k sind regulär ($\det F_k = 1$) so dass $Ax = b$ und $Ux = y$ wirklich äquivalent sind
- aus

$$U = F_{n-1} \cdot \dots \cdot F_1 A$$

folgt andererseits

$$A = F_1^{-1} \cdot \dots \cdot F_{n-1}^{-1} U$$

- für F_k rechnet man leicht nach, dass

$$F_k^{-1} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{k+1k} & 1 & \\ & & \vdots & & \ddots \\ & & l_{nk} & & & 1 \end{pmatrix}$$

- durch Ausmultiplizieren erhält man

$$L := F_1^{-1} \cdot \dots \cdot F_{n-1}^{-1} = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ \vdots & \ddots & & & \\ l_{n1} & \dots & l_{nn-1} & 1 \end{pmatrix}$$

und damit $A = LU$

! Satz 15

Sind alle $a_{kk}^{(k-1)} \neq 0$ so erzeugt die Gaußelimination eine LU -Zerlegung

$$A = LU$$

von A , wobei L eine untere Dreiecksmatrix mit Diagonale 1 und U eine obere Dreiecksmatrix ist.

$$L = \begin{pmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & \ddots & \ddots & \\ l_{n1} & \cdots & l_{nn-1} & 1 \end{pmatrix}, \quad U = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ & u_{22} & \cdots & u_{2n} \\ & & \ddots & \vdots \\ & & & u_{nn} \end{pmatrix}$$

- die 1 auf der Diagonalen von L braucht nicht explizit gespeichert zu werden
- der Rest von L, U „passt“ in A rein
- A selbst brauchen wir dann nicht mehr, da $Ax = b \Leftrightarrow LUx = b$
- L, U können direkt (ohne Zwischenspeicherung) im Laufe der Zerlegung in A eingetragen werden
- im nächsten Beispiel verwenden wir die LU-Zerlegung um den Fall mehrerer rechte Seiten zu behandeln

8.2 Beliebige reguläre Matrizen

- bisher haben wir vorausgesetzt, dass während der Elimination kein Diagonalelement verschwinden darf, d.h. $a_{kk}^{(k-1)} \neq 0$
- das Beispiel oben hat gezeigt, dass es reguläre Matrizen A gibt, die dies nicht erfüllen
- als Abhilfe haben wir dort Zeilen getauscht
- das funktioniert immer

! Lemma 19

Sei A regulär, $A^{(k-1)} = F_{k-1} \cdot \dots \cdot F_1 A$ mit Hilfe der Gaußelimination erzeugt und $a_{kk}^{(k-1)} = 0$. Dann gibt es in der k -ten Spalte unterhalb der Diagonalen immer mindestens ein Element $a_{ik}^{(k-1)} \neq 0, i > k$, d.h. ein erfolgreicher Zeilentauch ist immer möglich.

- wie baut man Zeilentauschen in LU –Matrixformalismus ein?
- bevor F_k die k -te Spalte eliminiert, muss eventuell getauscht werden
- Zeilentausch erfolgt durch Multiplikation mit einer *Permutationsmatrix*

$$P = \begin{pmatrix} & \begin{matrix} i \\ \vdots \\ 0 \\ \vdots \\ 1 \end{matrix} & \begin{matrix} j \\ \vdots \\ 1 \\ \vdots \\ 0 \end{matrix} & \\ \begin{pmatrix} 1 & & & \\ & \ddots & & \\ & & 1 & \\ & & & \ddots \end{pmatrix} & & & \\ \begin{pmatrix} \dots & \dots & \dots & \dots \\ & 1 & & \\ & & \ddots & \\ & & & 1 \end{pmatrix} & & & \\ \begin{pmatrix} \dots & \dots & \dots & \dots \\ & 1 & & \\ & & \ddots & \\ & & & 1 \end{pmatrix} & & & \end{pmatrix} \begin{matrix} i \\ j \end{matrix}$$

- PA tauscht Zeilen i und j , AP tauscht Spalten i und j
- für Permutationen gilt $P = P^T$ und $PP = I$, d.h. $P = P^{-1}$
- damit erhalten wir

$$U = F_{n-1}P_{n-1} \cdot \dots \cdot F_1P_1A$$

wobei U eine obere Dreiecksmatrix ist und P_i Permutationsmatrizen oder $P_i = I$ (was auch eine Permutation ist)

- ohne Permutation war

$$\begin{aligned} U &= F_{n-1} \cdot \dots \cdot F_1 A &= L^{-1} A \\ L &= (F_{n-1} \cdot \dots \cdot F_1)^{-1} &= F_1^{-1} \cdot \dots \cdot F_{n-1}^{-1} \end{aligned}$$

wobei L untere Dreiecksmatrix ist

- analoges Vorgehen mit Permutationen liefert

$$U = \tilde{L}^{-1}A, \quad \tilde{L} = (F_{n-1}P_{n-1} \cdot \dots \cdot F_1P_1)^{-1}$$

- Problem:
 - $\tilde{L}$ ist in der Regel keine untere Dreiecksmatrix
 - $\tilde{L}y = b$ ist nicht mit Vorwärtseinsetzen lösbar
- wie bekommt man Dreiecksgestalt zurück?
- benutze $P_iP_i = I \Rightarrow P_1P_2 \cdot \dots \cdot P_{n-1}P_{n-1} \cdot \dots \cdot P_2P_1 = I$
- damit erhalten wir:

$$\begin{aligned} U &= F_{n-1}P_{n-1} \cdot \dots \cdot F_2P_2F_1P_1A \\ &= (F_{n-1}P_{n-1} \cdot \dots \cdot F_2P_2F_1P_1) (P_1 \cdot \dots \cdot P_{n-1}P_{n-1} \cdot \dots \cdot P_1) A \\ &= (F_{n-1}P_{n-1} \cdot \dots \cdot F_2P_2F_1P_2 \cdot \dots \cdot P_{n-1}) (P_{n-1} \cdot \dots \cdot P_1) A, \end{aligned}$$

d.h. $U = L^{-1}PA$ bzw. $PA = LU$ mit Permutationsmatrix (nur Zeilentausch)

$$P = P_{n-1} \cdot \dots \cdot P_1$$

und

$$\begin{aligned} L &= (F_{n-1}P_{n-1} \cdot \dots \cdot F_2P_2F_1P_2 \cdot \dots \cdot P_{n-1})^{-1} \\ &\stackrel{P_i^{-1}=P_i}{=} P_{n-1} \cdot \dots \cdot P_2F_1^{-1}P_2F_2^{-1} \cdot \dots \cdot P_{n-1}F_{n-1}^{-1} \end{aligned}$$

- L hat jetzt Dreiecksgestalt:
 - setze

$$L_1 = F_1^{-1}, \quad L_k = P_k L_{k-1} P_k F_k^{-1}, \quad k = 2, \dots, n-1$$

d.h.

$$\begin{aligned} L_2 &= P_2 F_1^{-1} P_2 F_2^{-1} \\ L_3 &= P_3 L_2 P_3 F_3^{-1} = P_3 P_2 F_1^{-1} P_2 F_2^{-1} P_3 F_3^{-1} \\ &\vdots \\ L_{n-1} &= L \end{aligned}$$

- wir zeigen jetzt, dass alle L_k untere Dreiecksgestalt haben:

$$\bullet L_1 = F_1^{-1} = \begin{pmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & & \ddots & \\ l_{n1} & & & 1 \end{pmatrix}$$

- $L_2 = P_2 L_1 P_2 F_2^{-1}$:
 - in $P_2 L_1$ sind die Zeilen 2 und j vertauscht:

$$P_2 L_1 = \begin{pmatrix} 1 & & & & \\ l_{j1} & 0 & & & 1 \\ l_{31} & & 1 & & \\ \vdots & & & \ddots & \\ l_{j-11} & & & & 1 \\ l_{21} & 1 & & & 0 \\ l_{j+11} & & & & 1 & \\ \vdots & & & & & \ddots \\ l_{n1} & & & & & & 1 \end{pmatrix}$$

- $(P_2 L_1) P_2$ tauscht Spalte 2 und j :

$$P_2 L_1 P_2 = \begin{pmatrix} 1 & & & & & & & \\ l_{j1} & 1 & & & & & & \\ l_{31} & & 1 & & & & & \\ \vdots & & & \ddots & & & & \\ l_{j-11} & & & & 1 & & & \\ l_{21} & & & & & 1 & & \\ l_{j+11} & & & & & & 1 & \\ \vdots & & & & & & & \ddots \\ l_{n1} & & & & & & & & 1 \end{pmatrix}$$

- wegen

$$F_2^{-1} = \begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & l_{32} & 1 & & \\ & \vdots & & \ddots & \\ & l_{n2} & & & 1 \end{pmatrix}$$

erhalten wir schließlich

$$P_2 L_1 P_2 F_2^{-1} = \begin{pmatrix} 1 & & & & & & & \\ l_{j1} & 1 & & & & & & \\ l_{31} & l_{32} & 1 & & & & & \\ \vdots & \vdots & & \ddots & & & & \\ l_{j-11} & l_{j-12} & & & 1 & & & \\ l_{21} & l_{j2} & & & & 1 & & \\ l_{j+11} & l_{j+12} & & & & & 1 & \\ \vdots & \vdots & & & & & & \ddots \\ l_{n1} & l_{n2} & & & & & & & 1 \end{pmatrix}$$

- allgemein gilt (per Induktion) für $L_k = P_k L_{k-1} P_k F_k^{-1}$:
 - L_{k-1} ist gegeben durch

$$L_{k-1} = \begin{pmatrix} 1 & & & & & \\ * & \ddots & & & & \\ \vdots & \ddots & 1 & & & \\ * & \cdots & * & 1 & & \\ \vdots & & \vdots & & \ddots & \\ * & \cdots & * & & & 1 \end{pmatrix}$$

$k-1$

- $P_k L_{k-1} P_k$ tauscht in den ersten $k-1$ Spalten Zeile k mit Zeile $j > k$, d.h. im Block

$$\begin{pmatrix} * & & & & \\ \vdots & \ddots & & & \\ * & \cdots & * & & k \\ \vdots & & \vdots & & \\ * & \cdots & * & & j > k \\ \vdots & & \vdots & & \\ * & \cdots & * & & \end{pmatrix}$$

- Multiplikation mit F_k^{-1} vergrößert diesen Block um die k -te Spalte von F_k^{-1} , d.h.

$$L_k = P_k L_{k-1} P_k F_k^{-1} = \begin{pmatrix} 1 & & & & & \\ * & \ddots & & & & \\ \vdots & \ddots & 1 & & & \\ * & \dots & * & 1 & & \\ \vdots & & \vdots & l_{k+1k} & 1 & \\ \vdots & & \vdots & \vdots & & \ddots \\ * & \dots & * & l_{nk} & & 1 \end{pmatrix}$$

- damit haben alle L_k und somit auch $L = L_{n-1}$ untere Dreiecksgestalt

! Satz 20

Ist $A \in \mathbb{R}^{n \times n}$ regulär, so gibt es Permutationen $P_k, k = 1, \dots, n-1$, so dass mit $P = P_{n-1} \cdot \dots \cdot P_1$ gilt

$$PA = LU,$$

wobei L, U untere/obere Dreiecksmatrizen sind mit $\text{diag}(L) = 1$.

Cholesky-Zerlegung

- haben bis jetzt beliebige reguläre $A \in \mathbb{R}^{n \times n}$ betrachtet
- in einem wesentlichen Anwendungsgebiet (Partielle Differentialgleichungen) sind häufig spezielle Gleichungssysteme zu lösen

Definition 24

$A \in \mathbb{R}^{n \times n}$ heißt symmetrisch positiv definit (*spd*) falls $A = A^T$ und

$$(x, Ax)_2 > 0 \quad \forall x \in \mathbb{R}^n, x \neq 0.$$

Satz 25

Ist $A \in \mathbb{R}^{n \times n}$ spd, so ist A regulär und $a_{ii} > 0, i = 1, \dots, n$.

- wir betrachten jetzt die Gaußelimination für A spd
- A ist symmetrisch, d.h.

$$A = \left(\begin{array}{c|ccc} a_{11} & a_{21} & \dots & a_{n1} \\ a_{21} & & & \\ \vdots & & & \\ a_{n1} & & & \end{array} \middle| \begin{array}{c} B \end{array} \right)$$

- A ist spd, weshalb $a_{11} > 0$ so dass kein Zeilentausch notwendig ist
- wir eliminieren die erste Spalte durch Multiplikation mit F_1 von links:

$$F_1 A = \left(\begin{array}{c|ccc} a_{11} & a_{21} & \dots & a_{n1} \\ 0 & & & \\ \vdots & & & \\ 0 & & & \end{array} \middle| \begin{array}{c} \tilde{B} \end{array} \right)$$

- multiplizieren wir mit F_1^T zusätzlich von rechts, so folgt aus der Symmetrie

$$\begin{aligned}
 A^{(1)} = F_1 A F_1^T &= \left(\begin{array}{c|ccc} a_{11} & a_{21} & \dots & a_{n1} \\ 0 & & & \\ \vdots & & \tilde{B} & \\ 0 & & & \end{array} \right) \begin{pmatrix} 1 & -l_{21} & \dots & -l_{n1} \\ & 1 & & \\ & & \ddots & \\ & & & 1 \end{pmatrix} \\
 &= \left(\begin{array}{c|ccc} a_{11} & 0 & \dots & 0 \\ 0 & & & \\ \vdots & & \tilde{B} & \\ 0 & & & \end{array} \right),
 \end{aligned}$$

wobei $\tilde{B}$ nicht verändert wird

- $A^{(1)}$ ist symmetrisch, denn

$$(A^{(1)})^T = (F_1 A F_1^T)^T = F_1 A^T F_1^T \stackrel{A \text{ sym.}}{=} F_1 A F_1^T = A^{(1)}$$

- $A^{(1)}$ ist auch positiv definit:

- es gilt

$$(x, A^{(1)}x)_2 = x^T F_1 A F_1^T x = (F_1^T x)^T A (F_1^T x) = y^T A y = (y, A y)_2, \quad y = F_1^T x$$

- da F_1^T regulär ist, folgt aus $x \neq 0$ auch $y = F_1^T x \neq 0$
- da A spd ist, erhalten wir schließlich

$$(x, A^{(1)}x)_2 = (y, A y)_2 > 0 \quad \forall x \neq 0$$

- damit kann in $A^{(1)}$ ganz analog die zweite Spalte/Zeile beseitigt werden:

$$A^{(2)} = F_2 A^{(1)} F_2^T = F_2 F_1 A F_1^T F_2^T = \begin{pmatrix} * & & & \\ & * & & \\ & & * & \dots & * \\ & & \vdots & & \vdots \\ & & * & \dots & * \end{pmatrix}$$

wobei $A^{(2)}$ wieder spd ist

- nach $n - 1$ Schritten folgt

$$F_{n-1} \cdot \dots \cdot F_1 A F_1^T \cdot \dots \cdot F_{n-1}^T = \begin{pmatrix} a_{11} & & & \\ & a_{22}^{(1)} & & \\ & & \ddots & \\ & & & a_{nn}^{(n-1)} \end{pmatrix} =: D$$

mit D spd, d.h. alle Diagonalelemente sind größer 0

- damit ist

$$A = F_1^{-1} \cdot \dots \cdot F_{n-1}^{-1} D (F_{n-1}^T)^{-1} \cdot \dots \cdot (F_1^T)^{-1}$$

- für alle $C \in \mathbb{R}^{n \times n}$ regulär gilt $(C^T)^{-1} = (C^{-1})^T$ so dass

$$(F_{n-1}^T)^{-1} \cdot \dots \cdot (F_1^T)^{-1} = (F_{n-1}^{-1})^T \cdot \dots \cdot (F_1^{-1})^T = (F_1^{-1} \cdot \dots \cdot F_{n-1}^{-1})^T = \tilde{L}^T,$$

also ist $A = \tilde{L} D \tilde{L}^T$, $\tilde{L}$ untere Dreiecksmatrix mit 1 auf der Diagonalen

- da

$$D = \begin{pmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{pmatrix}$$

mit $d_i > 0$ gilt

$$D = EE^T, \quad E = \begin{pmatrix} \sqrt{d_1} & & \\ & \ddots & \\ & & \sqrt{d_n} \end{pmatrix}$$

und somit

$$A = \tilde{L}E E^T \tilde{L}^T = LL^T, \quad L = \tilde{L}E$$

! Satz 26

$A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit, kann wie folgt zerlegt werden:

- $A = \tilde{L}D\tilde{L}^T$, $\tilde{L}$ untere Dreiecksmatrix mit 1 auf der Diagonalen, D Diagonalmatrix mit positiven Diagonalelementen (*rationale Cholesky-Zerlegung*)
- $A = LL^T$, L untere Dreiecksmatrix (*Cholesky-Zerlegung*)

- Aufwand:
 - die Cholesky-Zerlegung basiert auf der Gaußelimination mit zusätzlicher Multiplikation mit F_k^T , was zunächst einen höheren Aufwand erwarten lässt
 - aber F_k^T eliminiert nur die entsprechende Zeile und hat sonst keinen Einfluss, d.h. es ist nichts weiter zu tun als 0 in die Matrix einzutragen
 - alle $A^{(k)}$ sind symmetrisch (oberes und unteres Dreieck sind identisch), weshalb nur eine Hälfte der Koeffizienten berechnet werden muss
 - Aufwand Cholesky $\sim \frac{1}{2}$ Aufwand Gauß $\sim \frac{1}{3}n^3$ Operationen
- für die praktische Durchführung wird in der Regel eine andere Form benutzt
- sei $A \in \mathbb{R}^{n \times n}$ spd mit $A = LL^T$,

$$L = \begin{pmatrix} l_{11} & & & \\ l_{21} & l_{22} & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \dots & l_{nn} \end{pmatrix} = \left(\begin{array}{c|ccc} l_{11} & & & \\ \hline l^{(2)} & & & L^{(2)} \end{array} \right)$$

mit

$$l^{(2)} = \begin{pmatrix} l_{21} \\ \vdots \\ l_{n1} \end{pmatrix} \in \mathbb{R}^{n-1}, \quad L^{(2)} = \begin{pmatrix} l_{22} & & \\ \vdots & \ddots & \\ l_{n2} & \dots & l_{nn} \end{pmatrix} \in \mathbb{R}^{(n-1) \times (n-1)}$$

dann ist

$$A = \left(\begin{array}{c|ccc} a_{11} & a_{21} & \dots & a_{n1} \\ \hline a_{21} & & & \\ \vdots & & B^{(2)} & \\ a_{n1} & & & \end{array} \right) = LL^T,$$

also

$$\begin{aligned} A &= \left(\begin{array}{c|ccc} l_{11} & 0 & \dots & 0 \\ \hline l_{21} & & & \\ \vdots & & L^{(2)} & \\ l_{n1} & & & \end{array} \right) \left(\begin{array}{c|ccc} l_{11} & l_{21} & \dots & l_{n1} \\ \hline 0 & & & \\ \vdots & & L^{(2)T} & \\ 0 & & & \end{array} \right) \\ &= \left(\begin{array}{c|ccc} l_{11}^2 & l_{11}l_{21} & \dots & l_{11}l_{n1} \\ \hline l_{11}l_{21} & & & \\ \vdots & & L^{(2)}L^{(2)T} + l^{(2)}l^{(2)T} & \\ l_{11}l_{n1} & & & \end{array} \right) \end{aligned}$$

- ein Koeffizientenvergleich liefert

$$\begin{aligned} l_{11}^2 &= a_{11} \Rightarrow l_{11} = \sqrt{a_{11}} \\ l_{11}l_{i1} &= a_{i1} \Rightarrow l_{i1} = \frac{a_{i1}}{\sqrt{a_{11}}}, \quad i = 2, \dots, n \end{aligned}$$

d.h. die erste Spalte von L (und damit $l^{(2)}$) ist bekannt

- die Submatrix $L^{(2)}$ ist noch unbekannt aber da

$$L^{(2)}L^{(2)T} = B^{(2)} - l^{(2)}l^{(2)T} =: A^{(2)}$$

mit

$$a_{ij}^{(2)} = a_{ij} - l_{i1}l_{j1}, \quad i, j \geq 2$$

kann aus $A^{(2)}$ analog die erste Spalte von $L^{(2)}$ (d.h. zweite Spalte von L) bestimmt werden

- diese Prozedur wird wiederholt, bis alle Spalten von L berechnet sind
- der k -te Schritt des Cholesky-Verfahrens besteht also aus

$$\left. \begin{aligned} l_{kk} &= \sqrt{a_{kk}^{(k)}} \\ l_{ik} &= \frac{a_{ik}^{(k)}}{l_{kk}} \end{aligned} \right\} \quad i = k+1, \dots, n \quad \left. \vphantom{\begin{aligned} l_{kk} &= \sqrt{a_{kk}^{(k)}} \\ l_{ik} &= \frac{a_{ik}^{(k)}}{l_{kk}} \end{aligned}} \right\} \text{ berechne } k\text{-te Spalte}$$

$$\left. \begin{aligned} a_{ij}^{(k+1)} &= a_{ij}^{(k)} - l_{ik}l_{jk} \end{aligned} \right\} \quad \begin{aligned} i &= k+1, \dots, n \\ j &= k+1, \dots, i \end{aligned} \quad \left. \vphantom{\begin{aligned} a_{ij}^{(k+1)} &= a_{ij}^{(k)} - l_{ik}l_{jk} \end{aligned}} \right\} \text{ berechne } A^{(k+1)}$$

- ist $A = LL^T$ bekannt, so verfährt man zur Lösung von $Ax = b$ wie üblich:
 - $Ax = b \Leftrightarrow LL^Tx = b \Leftrightarrow Ly = b, L^Tx = y$
 - $Ly = b$ wird durch Vorwärtseinsetzen, $L^Tx = y$ durch Rückwärtseinsetzen gelöst

Zusammenfassung

- die Gaußelimination transformiert durch systematische Umformung (Zeilesubtraktion) das System $Ax = b$ auf ein System in oberer Dreiecksform, das dann durch Rückwärtseinsetzen einfach zu lösen ist
- Zeilentausch ist in der Regel nötig (Division durch 0)
- Gaußelimination führt direkt zur LU-Zerlegung $PA = LU$ und ermöglicht effizientes Lösen für mehrere rechte Seiten b
- Rechenaufwand: $\mathcal{O}(n^3)$ für die Faktorisierung, $\mathcal{O}(n^2)$ pro rechter Seite für Vorwärts-/Rückwärtseinsetzen
- Cholesky-Zerlegung: Spezialfall für symmetrisch positiv definite Matrizen $A = LL^\top$, kein Zeilentausch nötig, halber Aufwand im Vergleich zu LU wegen Symmetrie der Matrizen

Teil III

Fehler in numerischen Prozessen

Überblick

- wann sind numerische Ergebnisse verlässlich sind und wie können Abweichungen systematisch beschrieben werden
- Gut gestellte Probleme und Bedeutung für numerische Lösbarkeit
- Fehlergrößen: Unterscheidung und Messung von absolutem/relativem Fehler
- Fehlerquellen (Eingabedaten, Rechnerarithmetik, verwendetes Verfahren)
- Floating-Point-Darstellung, inexakte Arithmetik, Rundung und Maschinengenauigkeit
- Sensitivität der Lösung gegenüber Störungen der Daten, Konditionszahlen
- Abweichungen durch Approximation/Abbruch (z. B. Iterationsstopp, Diskretisierung) und deren Abschätzung
- Fehlerfortpflanzung durch große Anzahl (aufeinander folgenden) Gleitkommaoperationen

Gut gestellte Probleme

- praktisch alle Aufgabenstellungen der numerischen Mathematik lassen sich als (eventuell sehr kompliziertes) Nullstellenproblem schreiben:
 - gegeben ist eine Funktion g und Eingabedaten x
 - suche y mit $g(x, y) = 0$
- folgende Eigenschaften sollte ein solches Problem sinnvollerweise erfüllen:
 - *Existenz*: es sollte Lösungen geben
 - *Eindeutigkeit*: zu jedem x sollte es *genau* ein y geben
 - *stetige Abhängigkeit* der Lösung y von den Eingabedaten x , d.h.

$$x' \rightarrow x \quad \Rightarrow \quad y' \rightarrow y$$

- stetige Abhängigkeit garantiert, dass (hinreichend) kleine Störungen der Eingabedaten nur kleine Veränderungen der Ergebnisse verursachen

Definition 30

Gilt für das Problem $g(x, y) = 0$ Existenz, Eindeutigkeit und stetige Abhängigkeit, so nennt man das Problem *gut gestellt* (well posed), ansonsten *schlecht gestellt* (ill posed)

- aus Existenz und Eindeutigkeit folgt, dass $g(x, y) = 0$ nach y aufgelöst werden kann
- dann existiert eine Lösungsfunktion f mit

$$g(x, y) = 0 \quad \Leftrightarrow \quad y = f(x)$$

d.h. f bildet die Eingabedaten auf die zugehörige Lösung ab

- *stetige Abhängigkeit* bedeutet damit

$$x' \rightarrow x \quad \Rightarrow \quad y' = f(x') \rightarrow f(x) = y$$

d.h. f ist stetig

- müssen schlecht gestellte Probleme behandelt werden, so approximiert man sie in der Regel durch gut gestellte Ersatzprobleme (Regularisierung), wobei die Approximationseigenschaften genau kontrolliert werden müssen

- wir betrachten ein gut gestelltes Problem

$$y = f(x), \quad x \text{ gegeben}, \quad f : X \rightarrow Y$$

wobei X, Y Vektorräume mit Normen $\|\cdot\|_X, \|\cdot\|_Y$ sind

- zur numerischen Behandlung wird f durch ein (diskretes, endliches) Problem $\hat{f}$ ersetzt, das anschließend in Floating-Point-Arithmetik als $\tilde{f}$ implementiert wird

$$\begin{array}{ccc} x & \xrightarrow{f} & y \quad \text{Aufgabenstellung} \\ \downarrow & & \\ x & \xrightarrow{\hat{f}} & \hat{y} \quad \text{Ersatzproblem} \\ \downarrow & & \\ \tilde{x} & \xrightarrow{\tilde{f}} & \tilde{y} \quad \text{FP-Implementierung des Ersatzproblems} \end{array}$$

- y ist die *exakte* Lösung zu den *exakten* Eingangsdaten x
- $\tilde{x}$ ist die Floating-Point-Zahl, die sich bei Umwandlung von x ins Floating-Point-Format ergibt
- $\hat{y}$ ist das Ergebnis des in *exakter Arithmetik* durchgeführten Ersatzproblems, wobei als Input die *exakten* Eingangsdaten x benutzt werden
- $\tilde{y}$ ist die vom Computer mit Hilfe der Floating-Point-Implementierung des Ersatzproblems berechnete Näherung, wobei als Input die Floating-Point-Version $\tilde{x}$ der exakten Daten x benutzt wird
- die Qualität eines Approximationsverfahrens wird dadurch bestimmt, wie weit $\tilde{y}$ von y abweicht
- um diese Abweichung quantitativ zu fassen, werden die folgenden Fehlergrößen definiert

Definition 33

Seien y und $\tilde{y}$ wie oben, dann heißt

- $e = \tilde{y} - y$ *Fehler*
- $e_a = \|\tilde{y} - y\|_Y$ *absoluter Fehler*
- $e_r = \frac{\|\tilde{y} - y\|_Y}{\|y\|_Y}$ für $\|y\|_Y \neq 0$ *relativer Fehler*

- in numerischen Verfahren versucht man den Fehler zu kontrollieren und dabei (wenn möglich) unter eine vom Benutzer vorgegebene Schranke ε zu drücken
- soll man besser den absoluten oder den relativen Fehler benutzen?
- vergleicht man die Ergebnisse des Beispiels, so fällt folgendes auf
 - in zweiten Fall ist e_a größer, aber e_r kleiner als im ersten
 - der dritte Fall hat einen sehr großen relativen Fehler, obwohl der absolute Fehler sehr klein ist
- der relative Fehler ist eigentlich aussagekräftiger, kann aber bei kleinen Absolutwerten von y wie in Fall 3 eine unnötig hohe Genauigkeit erzwingen
- deshalb benutzt man in der Praxis häufig ein Kriterium, das beide Fehlerarten kombiniert

Definition 35

Seien $\varepsilon_a, \varepsilon_r$ gegebene Schranken für den absoluten bzw. relativen Fehler. Beim *gemischten Fehlerkriterium* versucht man eine Approximation $\tilde{y}$ von y zu finden, mit

$$\|\tilde{y} - y\|_Y \leq \varepsilon_a + \|y\|_Y \varepsilon_r$$

- betrachten wir $\|\tilde{y} - y\|_Y \leq \varepsilon_a + \|y\|_Y \varepsilon_r$ genauer:
 - ist der exakte Absolutwert $\|y\|_Y$ sehr klein, dann dominiert ε_a und

$$e_a = \|\tilde{y} - y\|_Y \lesssim \varepsilon_a$$

- ist $\|y\|_Y$ sehr groß, dann dominiert auf der rechten Seite $\|y\|_Y \varepsilon_r$ und

$$\|\tilde{y} - y\|_Y \lesssim \|y\|_Y \varepsilon_r \quad \text{bzw.} \quad e_r = \frac{\|\tilde{y} - y\|_Y}{\|y\|_Y} \lesssim \varepsilon_r$$

Fehlerquellen

- wie wir oben gesehen haben, durchlaufen wir zur numerischen Behandlung eines gut gestellten Problems $y = f(x)$ folgende Schritte:

$$\begin{array}{rcl}
 x & \xrightarrow{f} & y \quad \text{Originalproblem} \\
 \downarrow & & \\
 x & \xrightarrow{\hat{f}} & \hat{y} \quad \text{Ersatzproblem} \\
 \downarrow & & \\
 \tilde{x} & \xrightarrow{\tilde{f}} & \tilde{y} \quad \text{FP-Implementierung des Ersatzproblems}
 \end{array}$$

- wir betrachten den Fehler

$$\tilde{y} - y = \tilde{f}(\tilde{x}) - f(x)$$

und versuchen die verschiedenen Fehlerquellen zu lokalisieren

- dazu formen wir den Fehler wie folgt um

$$\begin{aligned}
 \tilde{y} - y &= \tilde{f}(\tilde{x}) - \hat{f}(\tilde{x}) \\
 &\quad + \hat{f}(\tilde{x}) - f(\tilde{x}) \\
 &\quad + f(\tilde{x}) - f(x)
 \end{aligned}$$

und untersuchen die einzelnen Summanden

- *Eingabefehler* $f(\tilde{x}) - f(x)$
 - wegen der Umwandlung der Eingabedaten x in Floating-Point-Darstellung $\tilde{x}$ (siehe folgende Abschnitte) tritt dieser Fehler immer auf und wird deshalb auch als *unvermeidbarer* Fehler bezeichnet
 - die Größe dieses Fehleranteils hängt davon ab, wie empfindlich das Originalproblem f auf Störungen in den Eingabedaten reagiert
- *Verfahrensfehler* $\hat{f}(\tilde{x}) - f(\tilde{x})$
 - hier wird das in *exakter Arithmetik* gerechnete Ersatzproblem $\hat{f}$ mit dem Originalproblem f für den selben Input $\tilde{x}$ verglichen
 - die Größe dieses Fehleranteils hängt von der Qualität des Ersatzproblems ab
- *Akkumulierte Rundungsfehler* $\tilde{f}(\tilde{x}) - \hat{f}(\tilde{x})$

- das Ersatzproblem $\hat{f}$ besteht aus einer endlichen Anzahl von Elementaroperationen (Additionen, Multiplikationen etc.)
- bei der Floating-Point-Implementierung $\tilde{f}$ des Ersatzproblems werden diese Operationen durch ihre Floating-Point-Äquivalente ersetzt
- jede Floating-Point-Operation erzeugt in der Regel Rundungsfehler
- diese Fehler akkumulieren sich
- die Größe dieses Fehleranteils hängt davon ab, wie robust das Ersatzproblem auf eine Floating-Point-Implementierung reagiert
- in den folgenden Abschnitten werden die einzelnen Fehlerquellen genauer untersucht

14.1 Floating-Point-Darstellung von Zahlen

- bei der Anwendung numerischer Verfahren müssen grundsätzlich die Input-Daten x (reelle Zahlen) in eine für den Computer geeignete Form $\tilde{x}$, die entsprechende Floating-Point-Darstellung, umgewandelt werden
- welche Fehler dabei entstehen können, werden wir nun genauer untersuchen
- vom Taschenrechner ist man die Verarbeitung von Dezimalzahlen und -brüchen gewohnt
- dabei unterscheidet man im wesentlichen zwei Darstellungen
 - *Festkommadarstellung*
 - die Anzahl der Nachkommastellen ist fest
 - bei zwei Nachkommastellen erhalten wir z.B.

$$12.47 \quad - 1.00$$

- *Exponentialdarstellung*
 - die Zahl wird als Produkt einer Festpunktzahl und einer Zehnerpotenz dargestellt
- diese Darstellung wird auch als *Fließkommadarstellung* bezeichnet
- Dezimalbrüche sind Zahlen zur Basis 10:

$$\begin{aligned} 21.23E01 &= 1.23 \cdot 10^1 = 12.3 \\ 1.23E-01 &= 1.23 \cdot 10^{-1} = 0.123 \end{aligned}$$

$$12.34 = 1 \cdot 10^1 + 2 \cdot 10^0 + 3 \cdot 10^{-1} + 4 \cdot 10^{-2}$$

wobei die Ziffern ganze Zahlen zwischen 0 und 9 sind

- das geht analog auch für andere ganzzahlige Basen $b > 1$

Definition 37

Eine *Fließkommazahl* zur Basis b hat die Darstellung

$$v \cdot m \cdot b^e$$

Dabei ist

- b die *Basis*, eine natürliche Zahl > 1
- v das *Vorzeichen*, also ± 1
- m die *Mantisse*, ein b -adischer Bruch

$$m = m_{-k} \dots m_{-1} m_0 \cdot m_1 \dots m_l = m_{-k} b^k + \dots + m_l b^{-l} = \sum_{i=-k}^l m_i b^{-i}$$

mit Ziffern $m_i \in \{0, 1, \dots, b-1\}$ und Stellenzahl $k+l+1$

- e der *Exponent*, eine ganze Zahl

- in dieser Form sind Fließkommadarstellungen einer Zahl nicht eindeutig, denn durch Änderung des Exponenten erhält man z.B.

$$-43210000 = -43.21 \cdot 10^6 = -4.321 \cdot 10^7$$

- verlangt man von der Mantisse, dass sie genau eine Vorkommastelle hat die nicht verschwinden darf (d.h. $k=0$ und $m_0 \neq 0$), dann ist die Fließkommadarstellung für alle Zahlen $\neq 0$ eindeutig

! Lemma 39

Die *normalisierte* Fließkommadarstellung

$$\pm(m_0 \cdot m_1 \dots m_l) b^e = \pm \left(\sum_{i=0}^l m_i b^{-i} \right) b^e, \quad m_0 \neq 0$$

ist für Zahlen $\neq 0$ eindeutig

- es gibt reelle Zahlen, die nicht mit endlicher Mantissenlänge darstellbar sind (in keiner Basis, z.B. π , e , $\sqrt{2}$ etc.)
- im Computer steht nur eine endliche (feste) Anzahl von Stellen zur Verfügung
- Floating-Point-Formate fester Länge können also nur eine Teilmenge des $\mathbb{R}$ abbilden
- Eingabedaten müssen zunächst in das benutzte Floating-Point-Format umgewandelt werden, wobei in der Regel gerundet und abgeschnitten werden muss
- wie groß ist der maximale Fehler bei Umwandlung einer reellen Zahl in ein Floating-Point-Format?
- jede reelle Zahl $x \in \mathbb{R}$ kann als normalisierte Fließkommazahl zur Basis b mit „unendlich langer Mantisse“

$$\mathbb{R} \ni x = \pm(m_0 \cdot m_1 \dots m_l m_{l+1} \dots) b^e$$

$$b^0 \quad b^{-1} \dots b^{-l} b^{-(l+1)} \dots$$

dargestellt werden

- durch Runden erzeugt man aus x eine Fließkommazahl $\tilde{x}$ mit Stellenzahl $l+1$
- dazu wird in der Regel dasjenige $\tilde{x}$ ausgewählt, das x am nächsten liegt (*nearest* Rundung)
- dabei treten folgende Probleme auf
 - es kann zwei $\tilde{x}$ geben mit dem gleichen minimalen Abstand zu x , z.B. für $b=10$ und $l=2$ erhalten wir für $x=1.235$

$$\tilde{x}_1 = 1.23, \quad \tilde{x}_2 = 1.24, \quad |x - \tilde{x}_1| = |x - \tilde{x}_2| = 0.005$$

- $\tilde{x}$ muss eventuell renormalisiert werden, z.B. für $b=10$, $l=2$,

$$x = 9.996 \cdot 10^{-4} \quad \tilde{x} = 1.00 \cdot 10^{-3}$$

- ohne Berücksichtigung dieser Spezialfälle ist

$$\tilde{x} = \begin{cases} \pm(m_0.m_1 \dots m_l)b^e & \text{für } m_{l+1} < rd(\frac{b}{2}) \\ \pm(m_0.m_1 \dots (m_l + 1))b^e & \text{für } m_{l+1} \geq rd(\frac{b}{2}) \end{cases}$$

d.h. es wird auf- bzw. abgerundet

- für den Fehler erhalten wir

$$e_a = |\tilde{x} - x| \leq \frac{b}{2} b^{-(l+1)} b^e = \frac{1}{2} b^{-l} b^e$$

und für den relativen Fehler

$$e_r = \frac{e_a}{|x|} \leq \frac{\frac{1}{2} b^{-l} b^e}{(m_0 . m_1 \dots m_l \dots) b^e} \leq \frac{1}{2} b^{-l}$$

wobei es Zahlen gibt, so dass Gleichheit gilt

- man definiert die obere Schranke für e_r als Maschinengenauigkeit

Definition 41

Betrachten wir normierte Fließkommazahlen zur Basis b mit Mantissenlänge l , so bezeichnet

$$\varepsilon_M = \frac{1}{2} b^{-l}$$

die *Maschinengenauigkeit*

- Floating-Point-Darstellungen auf dem Computer arbeiten in der Regel mit der Basis $b = 2$
- früher wurden Floating-Point-Operationen als Unterprogramme in Programmbibliotheken implementiert, wodurch sehr viele proprietäre Formate entstanden
- heute werden Floating-Point-Operationen direkt von der Hardware ausgeführt
- die Standardisierung nach *IEEE 754* legt eine ganze Reihe von Floating-Point-Formaten mit $b = 2$ und das Verhalten von arithmetischen Grundoperationen fest
- die wichtigsten Formate sind single und double
- *single (np.float32)*:
 - eine single Zahl besteht aus einem Vorzeichen, einer 24-stelligen Mantisse und einem 8-stelligen Exponenten
 - wegen $b = 2$ ist $m_0 = 1$ für alle normalisierten Zahlen und wird deshalb nicht extra gespeichert
 - Vorzeichen sowie die (wesentlichen) Stellen der Mantisse und des Exponenten passen deshalb in ein 32bit Wort

$$vm_1 \dots m_{23} \tilde{e}_1 \dots \tilde{e}_8$$

- wegen der 24-stelligen Mantisse ist $l = 23$ und die Maschinengenauigkeit ist

$$\varepsilon_M = \frac{1}{2} 2^{-23} = 2^{-24} \approx 5.960464e - 08$$

was etwa 7 gültigen Dezimalstellen entspricht

- der Exponent wird in einer vorzeichenlosen 8bit int-Zahl $\tilde{e}$ kodiert, d.h. $\tilde{e} \in \{0, \dots, 255\}$
- den tatsächlichen Wert e erhält man durch

$$e = \tilde{e} - 127 = \tilde{e} - (2^7 - 1), \quad e \in \{-127, \dots, 128\}$$

- für *normale* Zahlen ist $e \in \{-126, \dots, 127\}$, die Exponenten -127 und 128 sind für Spezialfälle reserviert:
 - $e = 128$ wird bei verschwindender Mantisse als $\pm\infty$, sonst als NaN (**N**ot **a** **N**umber) interpretiert
 - $e = -127$ wird bei verschwindender Mantisse als 0, sonst als „Subnormal“ interpretiert
- *double* (*np.float64*):
 - ist analog zu single als 64bit Format aufgebaut

$$vm_1 \dots m_{52} \tilde{e}_1 \dots \tilde{e}_{11}$$

- mit $l = 52$ erhalten wir $\varepsilon_M = 2^{-53} \approx 1.110223e - 16$, also ungefähr 16 gültigen Dezimalstellen
 - der Exponent e berechnet sich nach $e = \tilde{e} - (2^{10} - 1) = \tilde{e} - 1023$
 - *half* (*np.float16*):
 - ist analog zu single als 16bit Format aufgebaut
- $$vm_1 \dots m_{10} \tilde{e}_1 \dots \tilde{e}_5$$
- mit $l = 10$ erhalten wir $\varepsilon_M = 2^{-11} \approx 4.882812e - 04$, also ungefähr 3 gültigen Dezimalstellen
 - der Exponent e berechnet sich nach $e = \tilde{e} - (2^4 - 1) = \tilde{e} - 15$
 - oft findet man auch extended double (80bit) Formate
 - als Standard benutzt IEEE754 *nearest even Rundung*, d.h. nearest Rundung, wobei bei Mehrdeutigkeiten dasjenige $\tilde{x}$ benutzt wird, dessen letzte Mantissenstelle gleich 0 ist
 - bei der Umwandlung von Dezimalbrüchen in das Binärformat entstehen selbst bei einfachen Beispielen Rundungsfehler

14.2 Eingabefehler, Kondition

- wir untersuchen nun das Verhalten des Eingabefehlers $f(\tilde{x}) - f(x)$
- da unser Problem gut gestellt ist, ist f stetig, d.h. für alle $\varepsilon > 0$ gibt es ein (hinreichend kleines) $\delta > 0$ so dass

$$\|f(\tilde{x}) - f(x)\|_Y < \varepsilon \quad \text{falls} \quad \|\tilde{x} - x\|_X < \delta$$

- $\tilde{x} - x$ ist die Abweichung zwischen x und seiner Floating-Point-Darstellung $\tilde{x}$, die in der Regel *nicht* beliebig klein wird
- wie das folgende Beispiel zeigt, kann das zu Problemen führen
- um die Zusammenhänge zwischen Fehlern auf der Input-Seite x und Fehlern auf der Output-Seite $f(x)$ zu beschreiben, führt man Konditionszahlen ein

Definition 45

Sei $f : X \rightarrow Y$. Die *kleinsten* Konstanten κ_a, κ_r mit

$$\|f(x') - f(x)\|_Y \leq \kappa_a \|x' - x\|_X$$

$$\frac{\|f(x') - f(x)\|_Y}{\|f(x)\|_Y} \leq \kappa_r \frac{\|x' - x\|_X}{\|x\|_X}, \quad \|x' - x\|_X \leq \delta$$

heißen *absolute Kondition* bzw. *relative Kondition* von f in x und hängen von den benutzten Normen, x und δ ab.

- für stetig differenzierbares f kann man relativ einfach obere Schranken für die Konditionszahlen herleiten
- wir betrachten zunächst den einfachen Fall $f : \mathbb{R} \rightarrow \mathbb{R}$
- nach dem Mittelwertsatz gilt

$$f(x') = f(x) + f'(\xi)(x' - x)$$

mit

$$\xi \in \text{co}(x, x') = \{z \mid z = x + t(x' - x), t \in [0, 1]\} = [\min(x, x'), \max(x, x')]$$

und damit

$$|f(x') - f(x)| = |f'(\xi)| |x' - x| \leq \max_{\xi \in \text{co}(x, x')} |f'(\xi)| |x' - x|,$$

- für alle x' mit $|x' - x| \leq \delta$ gilt

$$\text{co}(x, x') \subset [x - \delta, x + \delta],$$

und somit

$$\xi \in \text{co}(x, x') \quad \Rightarrow \quad |\xi - x| \leq \delta$$

- damit erhalten wir schließlich die folgenden Abschätzungen

! Satz 48

Für stetig differenzierbares f gelten die Abschätzungen

$$\kappa_a \leq \max_{|\xi - x| \leq \delta} |f'(\xi)|, \quad \kappa_r \leq \frac{|x|}{|f(x)|} \max_{|\xi - x| \leq \delta} |f'(\xi)|$$

- da die Abschätzung des Maximums oft recht aufwändig ist, ersetzt man sie häufig durch eine einfachere Näherung
- für zweimal stetig differenzierbares f folgt aus der Taylor-Entwicklung von f um x

$$f(x') = f(x) + f'(x)(x' - x) + R(x', x), \quad R(x', x) = O(|x' - x|^2)$$

- ist $|x' - x|$ klein, dann vernachlässigt man das Restglied R und erhält die Näherung (Linearisierung)

$$f(x') \approx f(x) + f'(x)(x' - x)$$

- somit ist

$$|f(x') - f(x)| \approx |f'(x)| |x' - x|$$

und

$$\kappa_a \approx |f'(x)|$$

- analog erhält man für die Verstärkung des relativen Fehlers

$$\kappa_r \approx \frac{|x|}{|f(x)|} |f'(x)|$$

Definition 50

Die Näherungen

$$\kappa_a \approx |f'(x)|, \quad \kappa_r \approx \frac{|x|}{|f(x)|} |f'(x)|$$

bezeichnet man als *Fehlerfortpflanzungsformeln* der *differentiellen Fehleranalyse*

- für allgemeines $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ geht man analog vor
- der Mittelwertsatz nimmt in diesem Fall die Form

$$f(x') = f(x) + \left(\int_0^1 f'((x + t(x' - x))) dt \right) (x' - x)$$

an, wobei

$$f'(x) = \begin{pmatrix} \partial_{x_1} f_1(x) & \dots & \partial_{x_n} f_1(x) \\ \vdots & & \vdots \\ \partial_{x_1} f_m(x) & \dots & \partial_{x_n} f_m(x) \end{pmatrix}$$

die *Jacobi-Matrix* von f in x ist

- benutzt man in $\mathbb{R}^n$ die Norm $\|\cdot\|_X$ und in $\mathbb{R}^m$ die Norm $\|\cdot\|_Y$ so folgt

$$\begin{aligned} \|f(x') - f(x)\|_Y &\leq \left\| \int_0^1 f'((x + t(x' - x))) dt \right\|_{X,Y} \|x' - x\|_X \\ &\leq \max_{\xi \in \text{co}(x, x')} \|f'(\xi)\|_{X,Y} \|x' - x\|_X, \end{aligned}$$

wobei $\|\cdot\|_{X,Y}$ die zugehörige induzierte Matrixnorm ist

- für alle x' mit $\|x' - x\|_X \leq \delta$ gilt dann wieder

$$\begin{aligned} \kappa_a &\leq \max_{\|x' - x\|_X \leq \delta} \|f'(\xi)\|_{X,Y} \\ \kappa_r &\leq \frac{\|x\|_X}{\|f(x)\|_Y} \max_{\|x' - x\|_X \leq \delta} \|f'(\xi)\|_{X,Y} \end{aligned}$$

- analog erhalten wir für die *Fehlerfortpflanzungsformeln* der *differentiellen Fehleranalyse*

$$\kappa_a \approx \|f'(x)\|_{X,Y}, \quad \kappa_r \approx \frac{\|x\|_X}{\|f(x)\|_Y} \|f'(x)\|_{X,Y}$$

14.3 Verfahrensfehler

- das Originalproblem f muss, je nach Aufgabenstellung, durch ein *Ersatzproblem* $\hat{f}$ endlich bzw. diskret gemacht werden
- für das selbe f können in der Regel viele verschiedene Ersatzprobleme $\hat{f}$ konstruiert werden
- die Differenz zwischen dem in *exakter Arithmetik* gerechneten Ersatzproblem $\hat{f}$ und dem Originalproblem f für den selben Input $\tilde{x}$ definiert den Verfahrensfehler

$$\hat{f}(\tilde{x}) - f(\tilde{x})$$

- Ziel bei der Konstruktion eines Ersatzproblems ist es, den Verfahrensfehler mit möglichst wenig Aufwand möglichst klein zu machen
- in der Regel hängen Qualität und Aufwand des Ersatzproblems von einem *Diskretisierungsparameter* $h > 0$ ab
- ein vernünftig gewähltes Ersatzproblem geht für $h \rightarrow 0$ in das Originalproblem über, d.h. der Verfahrensfehler geht für $h \rightarrow 0$ gegen 0 (und der Aufwand in der Regel gegen unendlich)

14.4 Akkumulierter Rundungsfehler, Floating-Point-Arithmetik

- der akkumulierte Rundungsfehler $\tilde{f}(\tilde{x}) - \hat{f}(\tilde{x})$ vergleicht das in *exakter* Arithmetik gerechnete Ersatzproblem $\hat{f}$ mit einer seiner Floating-Point-Implementierungen $\tilde{f}$
- zu einem Ersatzproblem kann es mehrere Floating-Point-Implementierung geben, die mathematisch äquivalent (d.h. in exakter Arithmetik identisch) sind, aber unterschiedliche Floating-Point-Ergebnisse liefern
- das Ersatzproblem $\hat{f}$ setzt sich in der Regel aus einer großen Anzahl von elementaren Rechenoperationen $\hat{f}_i$ zusammen, die aus $\tilde{x}$ über Zwischenergebnisse $\hat{x}_i$ schließlich $\hat{f}(\tilde{x})$ erzeugen

$$\tilde{x} \xrightarrow{\hat{f}_1} \hat{x}_1 \xrightarrow{\hat{f}_2} \hat{x}_2 \xrightarrow{\hat{f}_3} \dots \xrightarrow{\hat{f}_p} \hat{f}(\tilde{x})$$

- in der Implementierung $\tilde{f}$ werden die $\hat{f}_i$ durch ihr Floating-Point-Äquivalent $\tilde{f}_i$ ersetzt

$$\tilde{x} \xrightarrow{\tilde{f}_1} \tilde{x}_1 \xrightarrow{\tilde{f}_2} \tilde{x}_2 \xrightarrow{\tilde{f}_3} \dots \xrightarrow{\tilde{f}_p} \tilde{f}(\tilde{x})$$

- aufgrund der endlichen Mantissenlänge können selbst diese elementaren Operationen keine exakten Ergebnisse liefern, was wir anhand einiger Grundrechenarten jetzt genauer untersuchen werden
- die Floating-Point-Implementierungen der Grundrechenarten bezeichnen wir mit $\oplus, \ominus, \odot, \oslash$
- die Arbeitsweise von $\oplus, \odot$ soll anhand von Beispielen mit $b = 10$ erklärt werden, die realen Implementierungen im Binärsystem arbeiten analog
- wir betrachten zunächst die Addition bzw. Subtraktion
 - die Subtraktion ist eine Addition mit Vorzeichenwechsel, muss also nicht gesondert betrachtet werden
 - für die Addition von Zahlen mit gleichem Vorzeichen wie z.B.

$$1.23 \cdot 10^0 \oplus 7.89 \cdot 10^{-3}$$

werden folgende Schritte durchlaufen:

- Angleichen der Exponenten

$$\begin{array}{rcl} 1.23 & \cdot & 10^0 \\ 0.00789 & \cdot & 10^0 \end{array}$$

- Addieren der Mantissen

$$1.23789 \cdot 10^0$$

- Runden der Mantisse und gegebenenfalls Renormalisieren des Ergebnisses

$$1.24 \cdot 10^0$$

- bei verschiedenen Vorzeichen wie z.B.

$$1.23 \cdot 10^0 \ominus 1.22 \cdot 10^0$$

erhalten wir analog:

- Angleichen der Exponenten

$$\begin{array}{r} 1.23 \cdot 10^0 \\ -1.22 \cdot 10^0 \end{array}$$

- Addieren der Mantissen

$$0.01 \cdot 10^0$$

- durch Runden der Mantisse und Renormalisierung

$$1. \cdot 10^0$$

erhalten wir zwei Stellen in der Mantisse, über die wir nichts wissen

- diesen Effekt nennt man *Auslöschung*
- die Mantisse könnte also 1.00 aber auch 1.99 sein, was einem relativen Fehler von bis zu 100% entspricht
- deshalb sollten Auslöschungen möglichst vermieden werden
- IEEE754 füllt die unbekannten Ziffern bei Auslöschungen mit 0 auf
- die Multiplikation ist wesentlich unkritischer, da keine Auslöschungen auftreten können
- zur Berechnung von

$$1.23 \cdot 10^4 \odot (-4.56 \cdot 10^7)$$

werden folgende Schritte nötig:

- Bestimmung des Vorzeichens, hier —
- Addition der Exponenten, $4 + 7 = 11$
- Multiplikation der Mantissen

$$1.23 \cdot 4.56 = 5.6088$$

- Rundung auf Mantissenlänge und eventuell Renormalisierung

$$-5.61 \cdot 10^{11}$$

- wegen der jeweils durchgeführten Rundung der Mantisse sind $\oplus, \odot$ *nicht assoziativ*:
 - wir betrachten normierte Floating-Point-Zahlen mit dreistelliger Mantisse

$$x = 1.00 \cdot 10^2 = 100$$

$$y = 4.00 \cdot 10^{-1} = 0.4$$

- $x \oplus y$ ergibt

$$\begin{array}{r} 1.00 \cdot 10^2 \\ \oplus 0.004 \cdot 10^2 \\ \hline 1.00 \cdot 10^2 \end{array}$$

d.h. $x \oplus y = 100$, so dass

$$((x \oplus y) \oplus y) \oplus y = 100$$

- für $y \oplus y$ erhalten wir

$$\oplus \begin{array}{rcl} 4.00 & \cdot & 10^{-1} \\ 4.00 & \cdot & 10^{-1} \\ 8.00 & \cdot & 10^{-1} \end{array}$$

d.h. $y \oplus y = 8.00 \cdot 10^{-1}$ und $(y \oplus y) \oplus y = 1.20 \cdot 10^0$, so dass

$$((y \oplus y) \oplus y) \oplus x = 1.01 \cdot 10^2$$

- kommen wir nun zurück zu der Floating-Point-Implementierung $\tilde{f}$

$$\tilde{x} \xrightarrow{\tilde{f}_1} \tilde{x}_1 \xrightarrow{\tilde{f}_2} \tilde{x}_2 \xrightarrow{\tilde{f}_3} \dots \xrightarrow{\tilde{f}_p} \tilde{f}(\tilde{x})$$

unseres Ersatzproblems

$$\tilde{x} \xrightarrow{\hat{f}_1} \hat{x}_1 \xrightarrow{\hat{f}_2} \hat{x}_2 \xrightarrow{\hat{f}_3} \dots \xrightarrow{\hat{f}_p} \hat{f}(\tilde{x})$$

- um zu verstehen, welche Fehler durch die einzelnen $\tilde{f}_i$ entstehen, vergleichen wir schrittweise die Zwischenergebnisse $\hat{x}_i$ und $\tilde{x}_i$:

- für den ersten Schritt erhalten wir

$$\tilde{x}_1 - \hat{x}_1 = \tilde{f}_1(\tilde{x}) - \hat{f}_1(\tilde{x}),$$

- da beides Mal derselbe Input $\tilde{x}$ benutzt wird, handelt es sich hierbei um den *lokalen Fehler*, der durch das Runden des Ergebnisses bei der Floating-Point-Operation $\tilde{f}_1$ entsteht
- im zweiten Schritt erhalten wir

$$\begin{aligned} \tilde{x}_2 - \hat{x}_2 &= \tilde{f}_2(\tilde{x}_1) - \hat{f}_2(\hat{x}_1) \\ &= \tilde{f}_2(\tilde{x}_1) - \hat{f}_2(\tilde{x}_1) + \hat{f}_2(\tilde{x}_1) - \hat{f}_2(\hat{x}_1) \end{aligned}$$

- $\tilde{f}_2(\tilde{x}_1) - \hat{f}_2(\tilde{x}_1)$ ist wieder der lokale Fehler durch $\tilde{f}_2$
- der Term

$$\hat{f}_2(\tilde{x}_1) - \hat{f}_2(\hat{x}_1)$$

beschreibt die *Fehlerverstärkung* des Fehlers $\tilde{x}_1 - \hat{x}_1$ aus dem ersten Schritt

- alle weiteren Schritte verlaufen analog zum zweiten
- in jedem Schritt wird also der vorherige Fehler verstärkt und ein neuer lokaler Fehler hinzu addiert
- deswegen bezeichnet man diesen Fehlertyp als *akkumulierten Rundungsfehler*
- zur quantitativen Analyse der Fehlerverstärkung können die Konditionsabschätzungen von oben benutzt werden
- falls Rundungsfehler das Ergebnis stark verfälschen (d.h. der akkumulierte Fehler sehr groß ist) nennt man eine Implementierung *instabil*, ansonsten ist sie *stabil*

Zusammenfassung

- gut gestellte Probleme besitzen eine eindeutige Lösung, die stetig von den Eingabedaten abhängt
- schlecht gestellte Probleme werden durch gut gestellte Probleme angenähert
- bei der numerischen Umsetzung

$$\begin{array}{ccc}
 x & \xrightarrow{f} & y \quad \text{Originalproblem} \\
 \downarrow & & \\
 x & \xrightarrow{\hat{f}} & \hat{y} \quad \text{Ersatzproblem} \\
 \downarrow & & \\
 \tilde{x} & \xrightarrow{\tilde{f}} & \tilde{y} \quad \text{FP-Implementierung des Ersatzproblems}
 \end{array}$$

betrachten wir die folgende Zerlegung des Gesamtfehlers

$$\tilde{f}(\tilde{x}) - f(x) = \underbrace{\tilde{f}(\tilde{x}) - \hat{f}(\tilde{x})}_{\text{akk. Rundungsfehler}} + \underbrace{\hat{f}(\tilde{x}) - f(\tilde{x})}_{\text{Verfahrensfehler}} + \underbrace{f(\tilde{x}) - f(x)}_{\text{Eingabefehler}}$$

- man versucht, den absoluten oder relativen Gesamtfehler (oder ein gemischtes Kriterium) zu kontrollieren
- in der Praxis sollten alle drei Fehlerarten in der gleichen Größenordnung gehalten werden, zu große Genauigkeit bei einer Fehlerart allein reduziert den Gesamtfehler nicht nennenswert und erhöht eventuell den Aufwand beträchtlich
- die einzelnen Fehlerarten lassen sich wie folgt charakterisieren
- *Eingabefehler*
 - beschreibt die Empfindlichkeit des Originalproblems f gegenüber Störungen in den Eingabedaten x
 - ist bestimmt durch die Kondition von f und damit im Wesentlichen nicht veränderbar
 - gut gestellte aber schlecht konditionierte Probleme können sich genauso verhalten wie schlecht gestellte Probleme
- *Verfahrensfehler*
 - beurteilt die Qualität des Ersatzproblems
 - wird in der Regel für jedes Ersatzproblem separat untersucht (z.B. Konvergenztheorie)
 - kann durch ein „besseres“ Ersatzproblem kleiner gemacht werden

- *Akkumulierter Rundungsfehler*
 - kann durch eine „günstigere“ Floating-Point-Implementierung des Ersatzproblems verändert werden (Summationsreihenfolgen, Vermeidung von Auslöschungen etc.)
 - stabile Implementierungen sind robust gegenüber Rundungsfehlereffekten

Teil IV

Lineare Gleichungssysteme - Direkte Löser II

KAPITEL 16

Überblick

- wir betrachten die LU-Zerlegung und untersuchen ihr Fehlerverhalten
- eine wichtige Rolle spielt dabei die Kondition des Gleichungssystems $Ax = b$
- für eine numerisch berechnete Näherung $\tilde{x}$ untersuchen wir den Zusammenhang zwischen Residuum $r = b - A\tilde{x}$ und Fehler $x - \tilde{x}$
- wir nutzen vorhandene Freiheitsgrade im LU-Algorithmus geschickt aus, um seine numerische Stabilität zu erhöhen
- wir entwickeln völlig andere Eliminationsverfahren die in einem gewissen Sinn (Konditionszahlen) optimal sind und vergleichen sie mit der LU-Zerlegung

Fehlerbetrachtung für lineare Gleichungssysteme

- betrachten $Ax = b$, A quadratisch, regulär
- Problem ist gut gestellt mit

$$f(A, b) = A^{-1}b = x$$

- betrachten zunächst die Kondition des Problems, d.h. wie empfindlich reagiert x auf Störungen in A, b
- das gestörte System sei $\tilde{A}\tilde{x} = \tilde{b}$
- setze $\Delta A = \tilde{A} - A$, $\Delta b = \tilde{b} - b$ und $\Delta x = \tilde{x} - x$

! Satz 58

Sei $\|\cdot\|$ eine Norm auf $\mathbb{R}^n$, A regulär und $\|\Delta A\| < \frac{1}{\|A^{-1}\|}$, wobei die durch $\|\cdot\|$ induzierte Matrixnorm benutzt wird. Dann ist $\tilde{A}$ regulär, und es gilt

$$\frac{\|\Delta x\|}{\|x\|} \leq \frac{\kappa(A)}{1 - \kappa(A) \frac{\|\Delta A\|}{\|A\|}} \left(\frac{\|\Delta A\|}{\|A\|} + \frac{\|\Delta b\|}{\|b\|} \right)$$

mit $\kappa(A) = \|A\| \|A^{-1}\|$.

- $\kappa(A)$ sollte also so klein wie möglich sein
- Problem: $\kappa(A) = \|A\| \|A^{-1}\|$ ist wegen $\|A^{-1}\|$ in der Regel nicht berechenbar
- es gibt numerische Verfahren, die $\kappa(A)$ mit vertretbarem Aufwand abschätzen (z.B. LAPACK Expert Driver Routinen)
- generell sollte man vor der Berechnung versuchen, die Kondition des Problems zu verbessern
- betrachte $Ax = b$ und U, V regulär

$$Ax = b \iff \underbrace{UAV}_{\tilde{A}} \underbrace{V^{-1}x}_{\tilde{x}} = \underbrace{Ub}_{\tilde{b}}$$

- $\tilde{A}\tilde{x} = \tilde{b}$ heißt *vorkonditioniertes System*
- löse $\tilde{A}\tilde{x} = \tilde{b}$ mit $\kappa(\tilde{A})$ statt $Ax = b$ mit $\kappa(A)$
- optimal (aber nicht praktikabel) ist z. B.

$$U = A^{-1}, V = I \Rightarrow \tilde{A} = I \Rightarrow \kappa(\tilde{A}) = 1$$

- Praxis: suche für U bzw. V Näherungen für A^{-1} , die billig zu berechnen sind
- einfachste Ansätze:
 - skaliere Zeilen

$$U = \begin{pmatrix} u_1 & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & u_n \end{pmatrix}, \quad V = I$$

- skaliere Spalten

$$U = I, \quad V = \begin{pmatrix} v_1 & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & v_n \end{pmatrix}$$

- skaliere beides

$$U = \begin{pmatrix} u_1 & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & u_n \end{pmatrix}, \quad V = \begin{pmatrix} v_1 & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & v_n \end{pmatrix}$$

- „vernünftige“ Skalierung:
 - alle Gleichungen sollten Koeffizienten gleicher Größenordnung haben $\Rightarrow$ Zeilenvektoren sollten gleiche Norm haben
 - alle Unbekannten sollten mit gleichem Gewicht auftauchen $\Rightarrow$ Spalten gleicher Norm
- Spalten bzw. Zeilen auf gleiche Norm bringen nennt man *Äquilibrieren*
- äquilibriert man Spalten und Zeilen gleichzeitig, so führt dies auf ein nichtlineares Gleichungssystem mit $2n-2$ Unbekannten, das schwieriger zu lösen ist als das ursprüngliche lineare Gleichungssystem
- deshalb werden in der Regel entweder nur Zeilen oder nur Spalten äquilibriert
- der Satz von oben liefert eine reine Konditionsaussage für das exakte Problem, keine Fehlerabschätzung für ein numerisches Verfahren
- ein stabiler Algorithmus verhält sich analog, so dass die Abschätzung als *a-priori Fehlerschätzer* dienen kann
- bei *a-posteriori Methoden* wird zunächst eine Näherungslösung bestimmt und dann getestet, ob sie akzeptabel ist
- dies liefert in der Regel genauere Aussagen als a-priori Methoden
- ein Beispiel dafür ist im nächsten Satz angegeben
- wir betrachten dabei das folgende Szenario:
 - A und b sind nur bis auf Störungen B bzw. c bekannt

- $Ax = b$ ist also mit einer „gewissen Unschärfe“ (z.B. Rundungsfehler oder Messungenauigkeiten) gegeben
- in dieser Lage ist es nicht sinnvoll, nach der exakten Lösung x zu fragen
- stattdessen: Plausibilitätsprüfung einer vorgelegten Näherungslösung $\tilde{x}$
- ist $\tilde{x}$ Lösung eines Systems „innerhalb der Unschärfe“, dann kann man $\tilde{x}$ als Lösung akzeptieren

! Satz 61 (Prager, Oettli)

Für $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$ sei

$$|x| = (|x_1|, \dots, |x_n|)^T, \quad |A| = (|a_{ij}|)_{i,j}.$$

Ist $\det(A) \neq 0$ und $\tilde{x}$ eine Näherungslösung von $Ax = b$ mit *Residuum* $\tilde{r} = b - A\tilde{x}$ sowie

$$B \in \mathbb{R}^{n \times n}, \quad c \in \mathbb{R}^n, \quad b_{ij} \geq 0, \quad c_i \geq 0 \quad \forall i, j$$

mit

$$|\tilde{r}| \leq B|\tilde{x}| + c.$$

Dann gibt es $|\Delta A| \leq B$ und $|\Delta b| \leq c$ so dass $\tilde{x}$ *exakte* Lösung von

$$(A + \Delta A)\tilde{x} = b + \Delta b$$

ist. Die Ungleichheitszeichen sind jeweils komponentenweise anzuwenden.

- in der Praxis wird das Resultat in der Regel wie folgt benutzt
 - wir nehmen an, dass die Störungen ähnlich wie A, b aussehen und setzen

$$B = \varepsilon|A|, \quad c = \varepsilon|b|$$

mit ε unbekannt

- berechne $\tilde{x}$ mit einem numerischen Verfahren und damit $\tilde{r}$
- bestimme jetzt das kleinste ε mit

$$|\tilde{r}| \leq B|\tilde{x}| + c = \varepsilon \underbrace{(|A| |\tilde{x}| + |b|)}_s$$

d.h.

$$\varepsilon = \max_i \frac{|\tilde{r}_i|}{\tilde{s}_i}$$

unter der Voraussetzung, dass $\tilde{s}_i \neq 0$

- ist dies möglich, dann gilt

$$|\tilde{r}| \leq \varepsilon|A| |\tilde{x}| + \varepsilon|b|$$

und nach dem Satz von Prager-Oettli ist $\tilde{x}$ die exakte Lösung von

$$(A + \Delta A)\tilde{x} = b + \Delta b$$

mit

$$|\Delta A| \leq \varepsilon|A|, \quad |\Delta b| \leq \varepsilon|b|,$$

d.h. ε liefert eine obere Abschätzung für die Störung der Eingabedaten

- damit stellt die Berechnung von ε eine Art „numerische Rückwärtsanalyse“ dar

LU-Zerlegung mit Pivot-Strategie

- die LU -Zerlegung transformiert $Ax = b$ in $Ux = y$ mit $U = F_{n-1}P_{n-1} \dots F_1P_1A$, d.h. ein lineares Gleichungssystem in ein anderes
- $Ux = y$ einfach lösbar (Rückwärtseinsetzen)
- wie verändert sich das Problem dadurch, d.h., wie verändert sich die Kondition und damit die Fehlerverstärkung?
- vergleichen jetzt $\kappa(A)$ mit $\kappa(U)$
- kann man das verhindern oder mindestens abmildern?
- wir betrachten nun den allgemeinen Fall

$$U = L^{-1}PA, \quad L^{-1} = F_{n-1}P_{n-1} \cdot \dots \cdot F_2P_2F_1P_1, \quad P = P_{n-1} \cdot \dots \cdot P_1$$

- mit $\kappa_\infty(BC) \leq \kappa_\infty(B)\kappa_\infty(C)$ folgt

$$\kappa_\infty(U) \leq \kappa_\infty(L^{-1})\kappa_\infty(P)\kappa_\infty(A),$$

$$\kappa_\infty(L^{-1}) \leq \kappa_\infty(F_{n-1})\kappa_\infty(P_{n-1}) \dots \kappa_\infty(F_2)\kappa_\infty(P_2)\kappa_\infty(F_1)\kappa_\infty(P_1) \dots \kappa_\infty(P_{n-1}),$$

$$\kappa_\infty(P) \leq \kappa_\infty(P_{n-1}) \dots \kappa_\infty(P_1)$$

- $\kappa_\infty(U)$ kann nicht stark anwachsen, wenn $\kappa_\infty(F_k), \kappa_\infty(P_k)$ klein sind
- wegen $\kappa_\infty(C) = \|C\|_\infty \|C^{-1}\|_\infty, \|\cdot\|_\infty$ Zeilensummennorm, folgt
 - für die Permutationsmatrizen P_k folgt wegen $P_k = P_k^{-1}$,

$$\|P_k\|_\infty = \|P_k^{-1}\|_\infty = 1$$

und somit

$$\kappa_\infty(P_k) = 1$$

- für die Frobeniusmatrizen F_k und ihre Inversen

$$\|F_k\|_\infty = \|F_k^{-1}\|_\infty = 1 + \max_{i>k} |l_{ik}|$$

und somit

$$\kappa_{\infty}(F_k) = \left(1 + \max_{i>k} |l_{ik}|\right)^2,$$

$$l_{ik} = \frac{\tilde{a}_{ik}^{(k-1)}}{\tilde{a}_{kk}^{(k-1)}}, \quad \tilde{A}^{(k-1)} = P_k F_{k-1} P_{k-1} \cdot \dots \cdot F_1 P_1 A,$$

wobei $\tilde{A}^{(k-1)}$ die Systemmatrix *nach* dem k -ten Zeilentauch und *vor* der Elimination der k -ten Spalte ist

- $\kappa_{\infty}(P_k)$ ist optimal
- $\kappa_{\infty}(F_k)$ wird dann klein, wenn $\max_{i>k} |l_{ik}|$ klein ist
- die Koeffizienten

$$l_{ik} = \frac{\tilde{a}_{ik}^{(k-1)}}{\tilde{a}_{kk}^{(k-1)}}$$

werden durch den zuletzt durchgeführten Zeilentauch (d.h. die Permutation P_k) beeinflusst

- bisher haben wir Zeilen immer so getauscht, dass anschließend *irgendein* Element ungleich 0 auf der Diagonale steht
- jetzt tauschen wir die Zeilen so, dass das *betragsgrößte* Element der Spalte auf die Diagonale kommt, d.h. nach dem Zeilentauch gilt

$$|\tilde{a}_{kk}^{(k-1)}| \geq |\tilde{a}_{ik}^{(k-1)}| \quad \forall i > k$$

- damit ist

$$|\tilde{l}_{ik}| \leq 1$$

und

$$\kappa_{\infty}(F_k) \leq 4$$

- zusammen erhalten wir

$$\kappa_{\infty}(U) \leq \kappa_{\infty}(L^{-1}) \kappa_{\infty}(P) \kappa_{\infty}(A) \leq \kappa_{\infty}(F_{n-1}) \dots \kappa_{\infty}(F_1) \kappa_{\infty}(A),$$

also

$$\kappa_{\infty}(U) \leq 4^{n-1} \kappa_{\infty}(A)$$

- dieses Verfahren nennt sich *Spalten-Pivot-Suche*
- *Praktische Durchführung:*
 - $PA = LU$, mit anderem P, L, U
 - Suchschleife zur Bestimmung der P_k
 - sonst keine Änderung nötig

KAPITEL 19

Grundlagen

Orthogonale Transformationen

- betrachten k -ten Schritt des LU -Verfahrens $A^{(k)} = F_k P_k A^{(k-1)}$ mit

$$\begin{pmatrix} a_{kk}^{(k-1)} \\ a_{k+1k}^{(k-1)} \\ \vdots \\ a_{nk}^{(k-1)} \end{pmatrix} \xrightarrow{F_k P_k} \begin{pmatrix} a_{kk}^{(k)} \\ 0 \\ \vdots \\ 0 \end{pmatrix},$$

d.h., die Spalte ab der Diagonale abwärts wird auf ein Vielfaches des ersten Einheitsvektors transformiert

- damit ist

$$\kappa(A^{(k)}) \leq \kappa(T_k) \kappa(A^{(k-1)}), \quad T_k = F_k P_k$$

bzw.

$$\kappa(U) \leq \kappa(T_{n-1}) \dots \kappa(T_1) \kappa(A)$$

- eine optimale Abschätzung für $\kappa(U)$ erhalten wir, wenn $\kappa(T_k) = 1$ für alle k
- $T_k = F_k P_k, P_k$ Permutationsmatrix,

$$F_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1k} & \ddots & \\ & & \vdots & \ddots & \\ & & -l_{nk} & & 1 \end{pmatrix}, \quad l_{ik} = \frac{\tilde{a}_{ik}^{(k-1)}}{\tilde{a}_{kk}^{(k-1)}}$$

- für κ_∞ erhält man analog

$$\kappa_\infty(T_k) = (1 + \max_{i>k} |l_{ik}|)^2$$

- damit ist

$$\kappa_{\infty}(T_k) = 1 \iff l_{ik} = 0 \quad \forall i > k,$$

d.h. $\kappa_{\infty}(T_k) = 1$ nur wenn nichts zu eliminieren ist, sonst ist $\kappa_{\infty}(T_k) > 1$

- wie kann man das verbessern:
 - andere Norm: es gibt keine induzierte Norm, so dass $\kappa(T_k) = 1$, falls mindestens ein $l_{ik} \neq 0$
 - anderes T_k (mit geeigneter Norm)
- wir suchen also $Q \in \mathbb{R}^{n \times n}$ mit folgenden Eigenschaften:
 - Q regulär
 - $\|Qx\| = \|x\| \quad \forall x \in \mathbb{R}^n$ weshalb auch

$$\|x\| = \|QQ^{-1}x\| = \|Q^{-1}x\| \quad \forall x \in \mathbb{R}^n$$

gelten muss, so dass $\|Q\| = 1$, $\|Q^{-1}\| = 1$ und damit $\kappa(Q) = 1$

- mit Q muss man Spalten eliminieren können, d.h.

$$Q \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} a \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad a \in \mathbb{R}$$

- aus der Geometrie kennt man solche linearen Abbildungen, nämlich Drehungen und Spiegelungen:
 - sie sind regulär (Inverse ist gegeben durch Drehung um negativen Winkel bzw. nochmaliges Spiegeln)
 - die euklidische Länge (d.h. die Vektornorm $\|\cdot\|_2$) der gedrehten/gespiegelten Vektoren ändert sich nicht
 - Eliminationseigenschaft: weisen wir später nach
- Dreh- und Spiegelmatrizen Q sind Spezialfall von orthonormalen Matrizen

Definition 70

$Q \in \mathbb{R}^{n \times n}$ heißt *orthonormale Matrix*, falls die Spaltenvektoren q_j paarweise senkrecht stehen und Länge 1 haben, d.h.

$$(q_i, q_j)_2 = \delta_{ij} = \begin{cases} 1 & i = j \\ 0 & \text{sonst} \end{cases}.$$

Satz 71

Sind $Q, \tilde{Q} \in \mathbb{R}^{n \times n}$ orthonormal, dann gilt:

- $QQ^T = Q^T Q = I$, d.h. $Q^{-1} = Q^T$
- $\kappa_2(Q) = 1$
- $Q\tilde{Q}$ ist orthonormal

20.1 Verfahren von Givens

- benutzen Drehungen zum Eliminieren der Spalten
- nach dem letzten Kapitel gilt in $\mathbb{R}^2$:

- ist $u = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}$ gegeben, $\|u\|_2 \neq 0$, so gilt

$$Qu = \pm \|u\|_2 e_1$$

mit

$$Q = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}, \quad c = \pm \frac{u_1}{\|u\|_2}, \quad s = \pm \frac{u_2}{\|u\|_2}$$

- für $\|u\|_2 = 0$ ist $u = (0, 0)^T$ so dass mit $Q = I$ folgt

$$Qu = \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \pm \|u\|_2 e_1$$

- die Verallgemeinerung für $u = (u_1, \dots, u_{i-1}, u_i, u_{i+1}, \dots, u_n)^T \in \mathbb{R}^n$ sieht wie folgt aus:
 - sei $w = \sqrt{u_1^2 + u_i^2}$, $i > 1$
 - setze für $w = 0$, $Q_{i1} = I$ bzw. für $w \neq 0$

$$Q_{i1} = \begin{pmatrix} c & & & & s & & \\ & 1 & & & & & \\ & & \ddots & & & & \\ & & & 1 & & & \\ -s & & & & c & & \\ & & & & & 1 & \\ & & & & & & \ddots \\ & & & & & & & 1 \end{pmatrix}, \quad c = \pm \frac{u_1}{w}, \quad s = \pm \frac{u_i}{w}$$

- dann ist Q_{i1} orthonormal und

$$Q_{i1}u = \begin{pmatrix} \pm w \\ u_2 \\ \vdots \\ u_{i-1} \\ 0 \\ u_{i+1} \\ \vdots \\ u_n \end{pmatrix}$$

d.h. u_i wird eliminiert und außer u_1, u_i wird nichts verändert

- benutzt man das positive Vorzeichen, so liefert die Anwendung auf die erste Spalte $s_1 = (a_{11}, \dots, a_{n1})^T$ von A

$$s_1^{(2)} = Q_{21}s_1 = \begin{pmatrix} \sqrt{a_{11}^2 + a_{21}^2} \\ 0 \\ a_{31} \\ \vdots \\ a_{n1} \end{pmatrix}$$

- analog kann man die restlichen Elemente der Spalte eliminieren:

$$s_1^{(3)} = Q_{31}s_1^{(2)} = \begin{pmatrix} \sqrt{a_{11}^2 + a_{21}^2 + a_{31}^2} \\ 0 \\ 0 \\ a_{41} \\ \vdots \\ a_{n1} \end{pmatrix} \quad \dots \quad s_1^{(n)} = \underbrace{Q_{n1} \cdot \dots \cdot Q_{21}}_{=: Q_1} s_1 = \begin{pmatrix} \|s_1\|_2 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

- insgesamt:
 - $Q_1 = Q_{n1} \cdot \dots \cdot Q_{21}$ ist orthonormal
 - $Q_1 s_1 = \|s_1\| e_1$ und damit

$$Q_1 A = \begin{pmatrix} * & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \vdots & & \vdots \\ 0 & * & \dots & * \end{pmatrix}$$

- Wiederholung desselben Algorithmus für die Spalten 2 bis $n - 1$ liefert

$$Q_{n-1} \cdot \dots \cdot Q_1 A = U$$

mit Q_i orthonormal und wegen $Q_i^{-1} = Q_i^T$

$$A = \underbrace{Q_1^T \cdot \dots \cdot Q_{n-1}^T}_Q U$$

- damit ist folgender Satz bewiesen

! Satz 74

$A \in \mathbb{R}^{n \times n}$, dann existiert Q orthonormal mit $A = QU$, U obere Dreiecksmatrix:

- praktische Durchführung:
 - gehe analog zur LU -Zerlegung vor:

$$Ax = b, \quad A = QR \quad \Leftrightarrow \quad Qy = b, \quad Rx = y$$

- letzter Teil funktioniert wie bei LU -Zerlegung, da R obere Dreiecksmatrix ist
- die Handhabung von Q ist komplizierter, da Q keine untere Dreiecksmatrix ist
- betrachte Elimination des Koeffizienten an Position i, j :
 - alle vorhergehenden Koeffizienten sind schon eliminiert, d.h.

$$A \rightarrow \tilde{A} = \begin{pmatrix} \tilde{a}_{11} & \dots & \dots & \dots & \dots & \dots & \tilde{a}_{1n} \\ 0 & \ddots & & & & & \vdots \\ \vdots & \ddots & \tilde{a}_{j-1,j-1} & & & & \vdots \\ \vdots & & 0 & \tilde{a}_{jj} & & & \vdots \\ \vdots & & \vdots & 0 & \tilde{a}_{j+1,j+1} & & \vdots \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ \vdots & & \vdots & 0 & \vdots & & \vdots \\ \vdots & & \vdots & \tilde{a}_{ij} & \vdots & & \vdots \\ \vdots & & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & \tilde{a}_{nj} & \tilde{a}_{nj+1} & \dots & \tilde{a}_{nn} \end{pmatrix}$$

- benutze

$$Q_{ij} = \begin{pmatrix} 1 & & & & & & & \\ & \ddots & & & & & & \\ & & 1 & & & & & \\ & & & c & & & s & \\ & & & & 1 & & & \\ & & & & & \ddots & & \\ & & & & & & 1 & \\ & & -s & & & & & c \\ & & & & & & & & 1 \\ & & & & & & & & & \ddots \\ & & & & & & & & & & 1 \end{pmatrix}$$

mit geeignetem c, s

- $Q_{ij}\tilde{A}$ verändert nur die i -te und j -te Zeile von $\tilde{A}$:

$$\tilde{a}_{jk} \rightarrow c\tilde{a}_{jk} + s\tilde{a}_{ik}, \quad \tilde{a}_{ik} \rightarrow -s\tilde{a}_{jk} + c\tilde{a}_{ik}$$

- anders als bei LU Zerlegung werden *beide* Zeilen verändert (auch die Diagonalzeile)
- Q_{ij} eliminiert $\tilde{a}_{ij}$, d.h. $\tilde{a}_{ij} \rightarrow 0$
- Q_{ij} ist durch *eine* reelle Zahl (Drehwinkel) definiert $\Rightarrow$ frei gewordener Platz von $\tilde{a}_{ij}$ reicht, um alle Informationen (Drehwinkel) für Q_{ij} abzuspeichern
- in der Praxis wird nicht der Drehwinkel benutzt, da zur Berechnung trigonometrische Funktionen nötig wären, was sehr aufwändig ist
- speichere z.B. $s = \sin(\alpha)$ und berechne $c = \pm\sqrt{1-s^2} \Rightarrow$ Vorzeichen von c ist nicht klar
- Lösung:
 - habe bei Wahl von Q_{ij} noch Vorzeichen frei
 - wähle Vorzeichen so, dass $c \geq 0$, d.h. der Drehwinkel liegt in $[-\frac{\pi}{2}, \frac{\pi}{2}]$
 - speichere $\rho_{ij} = s$ an Stelle von $\tilde{a}_{ij}$
- nach Transformation sämtlicher Spalten finden wir im Speicherbereich der Ausgangsmatrix folgende Werte:

$$\begin{array}{cccc} r_{11} & \cdots & \cdots & r_{1n} \\ \rho_{21} & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ \rho_{n1} & \cdots & \rho_{nn-1} & r_{nn} \end{array}$$

wobei

$r_{ij}, j \geq i$ die Koeffizienten der oberen Dreiecksmatrix R sind und aus den ρ_{ij} die Matrizen Q_{ij} rekonstruiert werden können

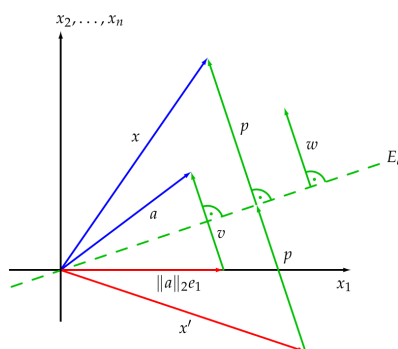
- zur Lösung von $Qy = b$ benutzen wir die ρ_{ij} :
 - $y = Q^T b = Q_{n-1} \cdot \dots \cdot Q_1 b = Q_{nn-1} \cdot \dots \cdot Q_{n1} \cdot \dots \cdot Q_{21} b$
 - betrachte ρ_{ij} in Eliminationsreihenfolge und rekonstruiere Q_{ij} über

$$s = \rho_{ij}, \quad c = \sqrt{1 - s^2}$$

- führe Matrix-Vektor-Produkte aus
- dabei verändert $Q_{ij}z$, $z \in \mathbb{R}^n$ nur z_i und z_j
- Auflösen von $Rx = y$ erfolgt wieder durch Rückwärtseinsetzen

20.2 Verfahren von Householder

- außer Drehungen erhalten auch Spiegelungen $\|\cdot\|_2$
- wir versuchen die erste Spalte $a = (a_{11}, \dots, a_{n1})^T$ von A auf $\pm\|a\|_2 e_1$ zu spiegeln
- wir betrachten zunächst nur $\|a\|_2 e_1$, negatives Vorzeichen geht analog



- a legt Spiegelebene E_a fest
- E_a steht senkrecht auf v , $v = a - \|a\|_2 e_1$
- um ein beliebiges x an E_a zu spiegeln wird
 - der Anteil von x senkrecht E_a bestimmt

$$p = w(w, x)_2, \quad w = \frac{v}{\|v\|_2}$$

- p zweimal von x subtrahiert:

$$x' = x - 2p = x - 2w(w, x)_2 = x - 2ww^T x = (I - 2ww^T)x,$$

wobei $ww^T \in \mathbb{R}^{n \times n}$

- insgesamt ist die Spiegelung an E_a also eine lineare Abbildung mit Matrix

$$Q_+ = I - 2ww^T, \quad w = \frac{v}{\|v\|_2}, \quad v = a - \|a\|_2 e_1$$

Satz 79

- $Q_+ = I - 2ww^T$, $\|w\|_2 = 1$ definiert eine Spiegelung an der Hyperebene $(w, x)_2 = 0$
- $Q_+ = Q_+^T = Q_+^{-1}$ und damit orthonormal
- $\det(Q_+) = -1$ d.h. die Orientierung kehrt sich um
- soll a auf $-\|a\|_2 e_1$ gespiegelt werden, ist

$$Q_- = I - 2ww^T, \quad w = \frac{v}{\|v\|_2}, \quad v = a + \|a\|_2 e_1$$

- benutzen wir $Q_1 = Q_+$ bzw. $Q_1 = Q_-$ zu $a = a_1$, a_1 erste Spalte von A , und multiplizieren beide Seiten von $Ax = b$, so erhalten wir

$$Q_1 Ax = Q_1 b, \quad Q_1 A = \begin{pmatrix} * & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \vdots & & \vdots \\ 0 & * & \dots & * \end{pmatrix}$$

mit $\kappa_2(Q_1) = 1$, da Q_1 orthonormal ist

- Wiederholung desselben Arguments auf der $(n-1) \times (n-1)$ Untermatrix ergibt schließlich

$$Q_{n-1} \dots Q_1 Ax = Q_{n-1} \dots Q_1 b$$

mit

$$Q_{n-1} \dots Q_1 A = R$$

obere Dreiecksmatrix

- somit ist

$$A = QR, \quad Q = (Q_{n-1} \dots Q_1)^{-1} = Q_1 \dots Q_{n-1}$$

- Q ist orthonormal, d.h. auch mit Spiegelungen erhalten wir eine (andere) QR-Zerlegung von A
- dieser Zugang heißt *Householder-Verfahren*
- praktische Durchführung:
 - betrachte $A = (a_1, \dots, a_n)$
 - bestimme im ersten Schritt

$$Q_1 = I - \frac{2}{\|v\|_2^2} vv^T, \quad v = a_1 \mp \|a_1\|_2 e_1$$

dann gilt $Q_1 a_1 = \pm \|a_1\|_2 e_1$ und damit

$$Q_1 A = (Q_1 a_1, Q_1 a_2, \dots, Q_1 a_n) = (\pm \|a_1\|_2 e_1, Q_1 a_2, \dots, Q_1 a_n)$$

d.h. die erste Spalte ist eliminiert, die Matrix Q_1 muss auf die restlichen Spalten von links her multipliziert werden

- wiederhole analoge Berechnungen für die Schritte $2, \dots, n-1$
- Frage:
 - wie bestimmt und speichert man die Q_i ?
 - wie berechnet man $Q_i z, z \in \mathbb{R}^n$ effizient?
- Berechnen und Speichern der Q_i am Beispiel von Q_1 :
 - berechne $\|a_1\|_2$ und speichere Ergebnis zwischen
 - bestimme

$$v = a_1 \mp \|a_1\|_2 e_1 = \begin{pmatrix} a_{11} \mp \|a_1\|_2 \\ a_{21} \\ \vdots \\ a_{n1} \end{pmatrix}$$

- v unterscheidet sich nur in der erste Komponente von der ersten Spalte der Ausgangsmatrix $\Rightarrow$ speichere v über erste Spalte
- wähle freies Vorzeichen, so dass Auslöschungen vermieden werden:

$$\begin{aligned} a_{11} < 0 &\Rightarrow \text{"-"} \Rightarrow Q_1 a_1 = \|a_1\|_2 e_1 \\ a_{11} \geq 0 &\Rightarrow \text{"+"} \Rightarrow Q_1 a_1 = -\|a_1\|_2 e_1 \end{aligned}$$

- für Q_1 brauchen wir auch $\|v\|_2$:

$$\begin{aligned} \|v\|_2^2 &= (\mp |a_{11}| \mp \|a_1\|_2)^2 + a_{21}^2 + \dots + a_{n1}^2 \\ &= \|a_1\|_2^2 + 2|a_{11}| \|a_1\|_2 + \underbrace{a_{11}^2 + a_{21}^2 + \dots + a_{n1}^2}_{=\|a_1\|_2^2} \\ &= 2(\|a_1\|_2^2 + |a_{11}| \|a_1\|_2) \end{aligned}$$

- die Produkte $Q_1 z$ werden wie folgt effizient berechnet:
 - betrachte

$$Q_1 z = \left(I - \frac{2}{\|v\|_2^2} v v^T\right) z = z - \frac{2}{\|v\|_2^2} v \underbrace{v^T z}_{(v,z)_2} = z - \frac{2(v,z)_2}{\|v\|_2^2} v$$

- berechne zuerst $(v, z)_2$ und dann $\alpha = \frac{2(v,z)_2}{\|v\|_2^2}$, wobei $\|v\|_2^2$ wie oben beschafft wird
- schließlich ist

$$Q_1 z = z - \alpha v$$

- Speicherproblem:
 - v belegt ganze Spalte ab Diagonalelement abwärts
 - auf Diagonale sollte aber $\pm \|a_1\|_2$ stehen
 - speichere Diagonale in separatem Vektor ab
 - insgesamt:
 - wiederhole $n - 1$ Spiegelungen
 - erhalte $Rx = Q^T b$, $Q^T = Q_{n-1} \cdot \dots \cdot Q_1$
 - jedes Q_i ist aus v_i rekonstruierbar
 - $Rx = Qb$ ist durch Rückwärtseinsetzen lösbar
- behandle mehrere rechte Seiten wie üblich

- Aufwand:

$$\sim \frac{4}{3}n^3 \quad \cong \quad 2 \cdot \text{LU} \quad \cong \quad \frac{1}{2} \cdot \text{Givens}$$

Zusammenfassung

- wir haben die Stabilität und Genauigkeit beim Lösen linearer Gleichungssysteme genauer untersucht und robuste Faktorisierungsverfahren kennenlernen
- Fehlerbetrachtung:
 - Zusammenhang zwischen Residuum $r = b - A\hat{x}$, (Vorwärts-) Fehler $x - \hat{x}$ und Kondition $\kappa(A)$
 - Prager-Oettli: Rückwärts-Fehler, Interpretation einer berechneten Lösung $\hat{x}$ als exakte Lösung eines gestörten Problems
- LU-Zerlegung mit Pivot-Strategie
- Praktische Konsequenz: effizientes Lösen über Vorwärts-/Rückwärtseinsetzen und Fehlerkontrolle
- Orthogonale Transformationen: optimal bezüglich Konditionsverhalten bei Zerlegung, gute numerische Stabilität
- Givens-Rotationen: Drehungen
- Householder-Reflexionen: Spiegelungen, halber Aufwand von Givens, Standardwerkzeug für QR-Zerlegung
- Spalteneliminatoren mit optimaler Kondition sind im wesentlichen Drehungen und Spiegelungen

Teil V

Iterative Löser

- wir betrachten $Ax = b$, $A \in \mathbb{R}^{n \times n}$ regulär
- direkte Löser liefern bei exakter Arithmetik nach endlich vielen Schritten die exakte Lösung x
- gibt es auch völlig anders strukturierte Algorithmen dafür?
- wir versuchen Lösungen eines linearen Gleichungssystems $Ax = b$ iterativ näherungsweise zu bestimmen:
 - starte mit Anfangsnäherung $x^{(0)}$ von x
 - erzeuge Folge von Näherungen

$$x^{(0)} \rightarrow x^{(1)} \rightarrow \dots \rightarrow x^{(k)} \rightarrow \dots$$

so lange bis $x^{(k)}$ nahe genug an x liegt

- ein Iterationsverfahren ist durch die *Verfahrensvorschriften* ϕ_k definiert:

$$x^{(k+1)} = \phi_k(x^{(k)}, x^{(k-1)}, \dots, x^{(0)})$$

- um Konvergenz zu untersuchen, benötigen wir die folgenden Begriffe

Definition 83

Ein Iterationsverfahren

$$x^{(k+1)} = \phi_k(x^{(k)}, x^{(k-1)}, \dots, x^{(0)})$$

- ist *konvergent*, falls $x^{(k)} \xrightarrow{k \rightarrow \infty} x$ für alle $x^{(0)} \in \mathbb{R}^n$
- hat die *Fixpunkteigenschaft*, falls $x = \phi_k(x, \dots, x)$ für x mit $Ax = b$

Der *Fehler* $e^{(k)}$ und das *Residuum* $r^{(k)}$ im k -ten Schritt sind gegeben durch

$$e^{(k)} = x - x^{(k)}, \quad r^{(k)} = b - Ax^{(k)}.$$

Benutzt man zur Berechnung von $x^{(k+1)}$ nur den direkten Vorgänger $x^{(k)}$ und keine $x^{(j)}$, $j < k$, d.h.

$$x^{(k+1)} = \phi_k(x^{(k)}),$$

mit *Verfahrensvorschriften* $\phi_k : \mathbb{R}^n \rightarrow \mathbb{R}^n$, so heißt das Verfahren *einstufig*, sonst *mehrstufig*.

Ist $\phi_k = \phi$, d.h. wird in jedem Schritt die selbe Vorschrift benutzt, so heißt das Verfahren *stationär*, sonst *instationär*.

- wir betrachten zunächst nur einstufige, stationäre Verfahren, d.h.

$$x^{(k+1)} = \phi(x^{(k)}),$$

mit *Verfahrensvorschriften* $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^n$

Lineare stationäre einstufige Iterationen mit Fixpunkteigenschaft

24.1 Konstruktion

- es sei x die (eindeutige) Lösung von $Ax = b$, $A \in \mathbb{R}^{n \times n}$ regulär
- eine einstufige, stationäre Iteration

$$x^{(k+1)} = \phi(x^{(k)})$$

zum Startwert $x^{(0)}$ sollte idealerweise folgende Eigenschaften besitzen:

- $x^{(k)} \xrightarrow{k \rightarrow \infty} x$ für alle $x^{(0)}$ (**Konvergenz**)
- $x = \phi(x) \Leftrightarrow Ax = b$ (**Fixpunkteigenschaft**)
- entsprechende Iterationen mit linear affiner Verfahrensvorschrift ϕ kann man auf verschiedenen Wegen herleiten
- **Matrix-Splitting:**
 - wir zerlegen die Matrix A in $A = M - N$
 - aus $Ax = b$ wird dann $Mx = Nx + b$ und für invertierbares M schließlich $x = M^{-1}(Nx + b)$
 - die Iteration

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b$$

besitzt damit automatisch die Fixpunkteigenschaft

- **Residuen-Iteration:**
 - benutzen wir beim Matrix-Splitting $N = M - A$ so erhalten wir

$$\begin{aligned} x^{(k+1)} &= M^{-1}(M - A)x^{(k)} + M^{-1}b \\ &= x^{(k)} + M^{-1}(b - Ax^{(k)}) \\ &= x^{(k)} + M^{-1}r^{(k)} \end{aligned}$$

mit **Residuum** $r^{(k)} = b - Ax^{(k)}$

- die Matrix M nennt man **Vorkonditionierer**
- **einstufige lineare stationäre Iteration:**
 - für $\phi(x) = Bx + d$ (linear affin) erhalten wir die Iteration

$$x^{(k+1)} = Bx^{(k)} + d$$

- fordern wir die Fixpunkteigenschaft $x = \phi(x)$ für x mit $Ax = b$, so erhalten wir

$$x = Bx + d$$

bzw.

$$(I - B)x = d$$

- da A regulär sein soll folgt mit $x = A^{-1}b$

$$d = (I - B)x = (I - B)A^{-1}b$$

- $I - B$ ist invertierbar denn für $b = 0$ ist $x = A^{-1}0 = 0$ die einzige Lösung von $(I - B)x = 0$
- damit erhalten wir schließlich

$$\begin{aligned} x^{(k+1)} &= Bx^{(k)} + d \\ &= Bx^{(k)} + (I - B)A^{-1}(b - Ax^{(k)}) + (I - B)A^{-1}Ax^{(k)} \\ &= x^{(k)} + (I - B)A^{-1}r^{(k)} \\ &= x^{(k)} + M^{-1}r^{(k)} \\ &= M^{-1}(Nx + b) \end{aligned}$$

mit $M = A(I - B)^{-1}$, $N = A - M$

- damit sind also
 - Matrixsplitting mit $A = M - N$
 - Residueniteration mit Vorkonditionierer M
 - lineare stationäre Iteration mit Fixpunkteigenschaft und $M = A(I - B)^{-1}$ bzw. $B = I - M^{-1}A$, $d = M^{-1}b = (I - B)A^{-1}b$

äquivalent

- für die folgenden Konvergenzbetrachtungen benutzen wir die letzte Darstellung

Definition 84

Ein *einstufiges, stationäres lineares Iterationsverfahren* ist gegeben durch

$$x^{(k+1)} = \phi(x^{(k)}) = Bx^{(k)} + d$$

wobei B die *Iterationsmatrix* und d der *Iterationsvektor* ist.

Satz 85

Besitzt das einstufige, stationäre lineare Iterationsverfahren $x^{(k+1)} = \phi(x^{(k)}) = Bx^{(k)} + d$ die Fixpunkteigenschaft $x = \phi(x)$, dann gilt für $e^{(k)} = x - x^{(k)}$, $r^{(k)} = b - Ax^{(k)}$

$$Ae^{(k)} = r^{(k)}$$

24.2 Konvergenz

- wir betrachten für das reguläre Gleichungssystem $Ax = b$ die Iteration

$$x^{(k+1)} = \phi(x^{(k)}) = Bx^{(k)} + d$$

mit Fixpunkteigenschaft

$$x = \phi(x) = Bx + d \Leftrightarrow Ax = b$$

- für den Fehler $e^{(k)} = x^{(k)} - x$ erhalten wir

$$e^{(k+1)} = x^{(k+1)} - x = \phi(x^{(k)}) - \phi(x) = Bx^{(k)} + d - (Bx + d) = Be^{(k)}$$

und somit

$$e^{(k)} = B^k e^{(0)}$$

- damit gilt

$$e^{(k)} \xrightarrow{k \rightarrow \infty} 0$$

für beliebiges $e^{(0)}$ genau dann, wenn

$$B^k \xrightarrow{k \rightarrow \infty} 0$$

- genauere Aussagen dazu liefert uns der Spektralradius von B

Definition 86

Der *Spektralradius* von B ist

$$\rho(B) = \max \{ |\lambda| \mid \lambda \text{ Eigenwert von } B \}.$$

- der Spektralradius hat folgende wichtige Eigenschaften

Satz 87

Für den Spektralradius gilt:

- $\rho(B) \leq \|B\|_I$ für alle *induzierten* Matrixnormen
- $\forall \varepsilon > 0$ existiert eine von einer Vektornorm $\|\cdot\|_V$ *induzierte* Matrixnorm $\|\cdot\|_I$, d.h.

$$\|B\|_I = \sup_{w \neq 0} \frac{\|Bw\|_V}{\|w\|_V}$$

mit $\|B\|_I \leq \rho(B) + \varepsilon$.

Satz 88

Es gilt $B^k \xrightarrow{k \rightarrow \infty} 0$ genau dann, wenn $\rho(B) < 1$.

- damit erhalten wir das folgende zentrale Ergebnis

! Satz 89

Die Iteration $x^{(k+1)} = \phi(x^{(k)}) = Bx^{(k)} + d$ konvergiert für alle $x^{(0)}$ genau dann, wenn $\rho(B) < 1$.

- weiterhin gilt der folgende Zusammenhang

! Satz 90 (Gelfand)

Für jede beliebige Matrixnorm $\|\cdot\|_M$ gilt

$$\rho(B) = \lim_{k \rightarrow \infty} \|B^k\|_M^{\frac{1}{k}}.$$

Für induzierte Matrixnormen $\|\cdot\|_I$ konvergiert $\|B^k\|_I^{\frac{1}{k}}$ von oben gegen $\rho(B)$.

- schränkt man die Klasse der betrachteten Matrizen A, M ein, so kann man für die Residueniteration explizit eine Norm angeben, in der sie *monoton* konvergiert

! Lemma 92

Sei $A \in \mathbb{R}^{n \times n}$ spd, dann ist durch

$$(x, y)_A = x^T A y, \quad \|x\|_A = \sqrt{(x, x)_A}$$

ein Skalarprodukt und eine Norm auf $\mathbb{R}^n$ definiert.

! Satz 93

Sei A, M spd. Ist $2M - A$ spd, so konvergiert

$$\phi(x) = Bx + d, \quad B = I - M^{-1}A, \quad d = M^{-1}b$$

und $\|B\|_A = \|B\|_M = \rho(B) < 1$.

! Satz 94

Sei A spd, ϕ wie im letzten Satz. Ist $M + M^T - A$ spd, dann ist M regulär und ϕ konvergiert mit $\rho(B) \leq \|B\|_A < 1$.

- wann bricht man die Iteration ab?
- beobachte $\|e^{(k)}\| = \|x - x^{(k)}\| \Rightarrow$ geht nicht, da x unbekannt
- teste $\|r^{(k)}\| = \|Ae^{(k)}\|$:
 - meistens wird $r^{(k)}$ sowieso berechnet (Residueniteration: $x_{k+1} = x^{(k)} + M^{-1}r^{(k)}$)
 - ist $\|A\|$ sehr klein, so kann wegen $\|r^{(k)}\| \leq \|A\| \|e^{(k)}\|$ das Residuum $r^{(k)}$ klein sein, obwohl $\|e^{(k)}\|$ groß ist
 - Skalierungen des Systems $Ax = b$ beeinflussen die Residuen:
 - betrachte

$$\begin{aligned} Ax = b, \quad M &\Rightarrow x_{k+1} = x^{(k)} + M^{-1}Ae^{(k)} \\ \gamma Ax = \gamma b, \quad \gamma M &\Rightarrow \tilde{x}^{(k+1)} = \tilde{x}^{(k)} + \frac{1}{\gamma}M^{-1}\gamma A\tilde{e}^{(k)} = \tilde{x}'_k + M^{-1}A\tilde{e}'^{(k)} \end{aligned}$$

- mit $x^{(0)} = \tilde{x}^{(0)}$ ist auch $e^{(0)} = \tilde{e}^{(0)}$ und somit $x^{(k)} = \tilde{x}^{(k)}$, $e^{(k)} = e'_k \forall k$
- andererseits ist

$$r^{(k)} = b - Ax^{(k)}, \quad \tilde{r}^{(k)} = \gamma(b - A\tilde{x}^{(k)}) = \gamma r^{(k)},$$

d.h. das Residuum skaliert mit γ

- deswegen prüft man nicht $\|r^{(k)}\|$ sondern $\frac{\|r^{(k)}\|}{\|r_0\|}$

24.3 Gesamtschritt-Verfahren (Jacobi)

- wir splitten A auf in $A = D - E - F$,

$$D = \begin{pmatrix} a_{11} & & \\ & \ddots & \\ & & a_{nn} \end{pmatrix}, \quad E = - \begin{pmatrix} 0 & \dots & \dots & 0 \\ a_{21} & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ a_{n1} & \dots & a_{nn-1} & 0 \end{pmatrix}, \quad F = - \begin{pmatrix} 0 & a_{12} & \dots & a_{1n} \\ & \ddots & \ddots & \vdots \\ & & \ddots & a_{n-1n} \\ & & & 0 \end{pmatrix}$$

- gilt $a_{ii} \neq 0$ so ist D invertierbar
- wir benutzen nun $M = D$, $N = D - A = E + F$ und erhalten die Iteration

$$x^{(k+1)} = D^{-1}(b + (E + F)x^{(k)}) = x^{(k)} + M^{-1}r^{(k)} = D^{-1}(E + F)x^{(k)} + D^{-1}b$$

- wegen

$$D^{-1} = \begin{pmatrix} \frac{1}{a_{11}} & & \\ & \ddots & \\ & & \frac{1}{a_{nn}} \end{pmatrix}$$

ergibt dies komponentenweise

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)}), \quad i = 1, \dots, n$$

- dieses Verfahren heißt *Jacobi-Verfahren* oder auch *Gesamtschritt-Verfahren*

! Satz 97

Ist A *streng diagonaldominant*, d.h.

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \forall i = 1, \dots, n$$

dann konvergiert das Jacobi-Verfahren (und A ist regulär).

24.4 Einzelschritt-Verfahren (Gauß-Seidel)

- betrachten das Jacobi-Verfahren komponentenweise

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n,$$

d.h. für die neue Komponente $x_i^{(k+1)}$ benutzt man auf der rechten Seite die alten Komponenten $x_j^{(k)}$

- wenn das Verfahren konvergiert, sollte $x^{(k+1)}$ „genauer“ als $x^{(k)}$ sein
- wenn $x_i^{(k+1)}$ berechnet wird, kennt man schon $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$
- wir benutzen auf der rechten Seite von $x_i^{(k+1)}$ diese neuen Komponenten
- damit erhalten wir das *Gauß-Seidel-Verfahren*, das auch *Einzelschritt-Verfahren* genannt wird

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n$$

- spalten wir A wieder in $A = D - E - F$ auf, so ist

$$\begin{aligned} x^{(k+1)} &= D^{-1}(b + Ex^{(k+1)} + Fx^{(k)}) \\ \Leftrightarrow Dx^{(k+1)} &= b + Ex^{(k+1)} + Fx^{(k)} \\ \Leftrightarrow (D - E)x^{(k+1)} &= Fx^{(k)} + b \\ &= (A + F)x^{(k)} + b - Ax^{(k)} \\ &= (D - E)x^{(k)} + r^{(k)} \\ \Leftrightarrow x^{(k+1)} &= x^{(k)} + (D - E)^{-1}r^{(k)} \end{aligned}$$

- das Gauß-Seidel-Verfahren ist also ein Splitting-Verfahren mit

$$M = D - E, \quad N = F$$

bzw. eine lineare stationäre Iteration mit

$$B = (D - E)^{-1}F, \quad d = (D - E)^{-1}b$$

! Satz 100

Ist A strikt diagonaldominant, dann gilt

$$\|B_{GS}\|_{\infty} \leq \|B_J\|_{\infty} < 1$$

d.h. das Gauß-Seidel-Verfahren konvergiert und $\|e^{(k)}\|_{\infty}$ fällt mindestens so schnell ab wie beim Jacobi-Verfahren.

! Satz 101 (Stein-Rosenberg)

Sei $A = D - E - F$, D invertierbar, $E + F \geq 0$ (komponentenweise) und $\rho(B_J) < 1$. Dann gilt

$$\rho(B_{GS}) < \rho(B_J).$$

! Satz 102

Ist A spd, so konvergiert das Gauß-Seidel-Verfahren ohne weitere Voraussetzungen. Die Konvergenz ist monoton in der Norm $\|\cdot\|_A$.

24.5 Nachiteration

- je besser M die Matrix A approximiert desto schneller konvergiert die Residueniteration
- betrachte direkten Löser wie z.B. LU-Zerlegung
- wegen Rundungsfehler generiert man $\tilde{L}\tilde{U} \approx A$ (P lassen wir der Einfachheit halber mal weg) und $\tilde{x}$ mit $A\tilde{x} \approx b$
- $\tilde{L}, \tilde{U}$ sind Dreiecksmatrizen, also leicht invertierbar
- wir erzeugen eine lineare stationäre Iteration nach mit $M = \tilde{L}\tilde{U}$, die sogenannte *Nachiteration*, die die Qualität von $\tilde{x}$ verbessert
- praktische Durchführung:
 - bestimme mit LU-Zerlegung $\tilde{x}, \tilde{L}, \tilde{U}$
 - setze $x_0 = \tilde{x}$ und wiederhole:
 - $r^{(k)} = b - Ax^{(k)}$
 - $p^{(k)} = M^{-1}r^{(k)}$ d.h. $\tilde{L}\tilde{U}p^{(k)} = r^{(k)}$
 - $x^{(k+1)} = x^{(k)} + p^{(k)}$

$$\text{bis } \frac{\|p^{(k)}\|}{\|x^{(k+1)}\|} < \varepsilon$$

- ohne Rundungsfehler wäre $M = LU = A$ und $M^{-1} = A^{-1}$ d.h. nach einem Schritt hätten wir die exakte Lösung
- mit Rundungsfehlern können wir das nicht erwarten, aber trotzdem ist in der Regel

$$B = I - M^{-1}A = I - (\tilde{L}\tilde{U})^{-1}A$$

nahe an der Nullmatrix und damit $\rho(B) \ll 1$, d.h. der Iterationsprozess sollte sehr schnell konvergieren

- Problem: $r^{(k)}$ liegt nahe bei 0, d.h. Fehler durch Auslöschung spielen eventuell eine große Rolle
- deswegen wird $r^{(k)}$ oft in höherer Genauigkeit berechnet
- $r^{(k)}$ mit hoher Genauigkeit $\Rightarrow$ eine Iteration reicht
- $r^{(k)}$ mit gleicher Genauigkeit wie LU-Zerlegung lohnt sich ebenfalls, da Stabilitätseigenschaften verbessert werden
- schlechte Konvergenz ist ein Indikator für schlechte Kondition des Ausgangsproblems $Ax = b$

24.6 Relaxationsverfahren

- wir betrachten nochmal das Jacobi-Verfahren

$$x^{(k+1)} = x^{(k)} + D^{-1}r^{(k)}$$

$$\text{d.h. } M_J = D, \quad B_J = I - M_J^{-1}A = D^{-1}(E + F)$$

- fügen wir vor dem Korrekturterm einen Parameter $\omega \in \mathbb{R}$ ein und versuchen darüber die Konvergenz günstig zu beeinflussen so erhalten wir das *relaxierte Jacobi-Verfahren* J_ω

$$x^{(k+1)} = x^{(k)} + \omega D^{-1}r^{(k)}$$

$$\text{d.h. } M_{J_\omega} = \frac{1}{\omega}D \text{ und}$$

$$\begin{aligned} B_{J_\omega} &= I - M_{J_\omega}^{-1}A = I - \omega D^{-1}(D - E - F) = (1 - \omega)I + \omega D^{-1}(E + F) \\ &= (1 - \omega)I + \omega B_J \\ d_{J_\omega} &= M_{J_\omega}^{-1}b = \omega D^{-1}b \\ &= \omega d_J \end{aligned}$$

- komponentenweise gilt dann

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)}), \quad i = 1, \dots, n$$

- die Konvergenz wird durch $\rho(B_{J_\omega})$ bestimmt, d.h. von den Eigenwerten von B_{J_ω}
- wegen $B_{J_\omega} = (1 - \omega)I + \omega B_J$ gilt

$$\lambda_i(B_{J_\omega}) = 1 - \omega + \omega \lambda_i(B_J)$$

! Satz 103

Ist $\omega \in (0, 1]$, und konvergiert das Jacobi-Verfahren dann konvergiert auch das relaxierte Jacobi-Verfahren

- wähle ω so, dass $\rho(B_{J_\omega})$ möglichst klein ist, d.h. ω_{opt} ist Lösung von

$$\max_{i=1, \dots, n} |1 - \omega + \omega \lambda_i(B_J)| \rightarrow \min$$

- für einfaches B_J kann man ω_{opt} angeben

! Satz 104

Hat B_J reelle Eigenwerte $-1 < \lambda_1 \leq \dots \leq \lambda_n < 1$ (damit ist auch $\rho(B_J) < 1$, d.h. das Jacobi-Verfahren konvergiert), so ist $\rho(B_{J_\omega})$ minimal für

$$\omega_{opt} = \frac{2}{2 - \lambda_1 - \lambda_n}$$

- das Problem beim Ergebnis des letzten Satzes ist, dass die Eigenwerte λ_i in der Regel nicht bekannt sind
- sie werden deswegen entweder abgeschätzt oder ω wird empirisch bestimmt
- für J_ω gilt

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)}), \quad i = 1, \dots, n$$

- wendet man den Gauß-Seidel-Trick an, so erhält man das *SOR-Verfahren* (Successive-Over-Relaxation)

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \frac{1}{a_{ii}}(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)}), \quad i = 1, \dots, n$$

- für SOR gilt also:

$$M_{SOR} = \frac{1}{\omega}D - E$$

und

$$B_{SOR} = (I - \omega D^{-1}E)^{-1}((1 - \omega)I + \omega D^{-1}F),$$

$$d_{SOR} = M_{SOR}^{-1}b$$

! Satz 107 (Kahan)

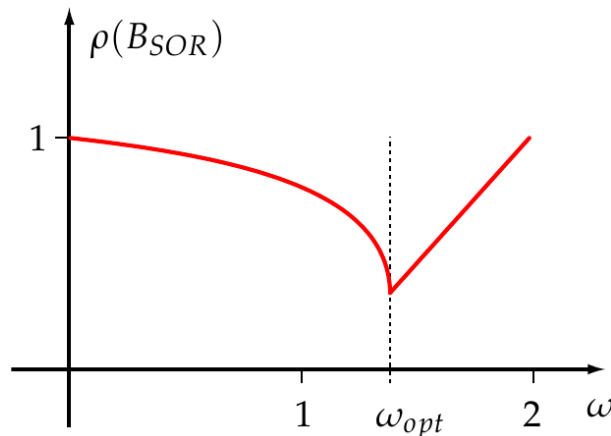
$\rho(B_{SOR}) \geq |\omega - 1|$, d.h. SOR kann für $\omega \notin (0, 2)$ nicht konvergent sein

! Satz 108 (Ostrowski, Reich)

Ist A spd dann konvergiert das SOR-Verfahren genau dann wenn $\omega \in (0, 2)$. Die Konvergenz ist monoton in $\|\cdot\|_A$

- für eine spezielle Klasse von Matrizen kann man ω_{opt} angeben (A konsistent geordnet mit A -Property, B_J hat nur reelle Eigenwerte und $\rho(B_J) < 1$):

$$\omega_{opt} = \frac{2}{1 + \sqrt{1 - \rho(B_J)^2}} > 1, \quad \rho(B_{SOR}) = \omega_{opt} - 1 < \rho(B_J)$$



- deswegen ist es günstiger ω zu groß als zu klein abzuschätzen

24.7 Symmetrische Varianten

- für das Gauß-Seidel-Verfahren ist $M_{GS} = D - E$, so dass für A spd M_{GS} in der Regel nicht mehr spd ist
- es gibt Anwendungen (PCG, Multigrid) wo M spd erwünscht ist
- betrachten nochmal die komponentenweise Darstellung des Jacobi- bzw. Gauß-Seidel-Verfahrens
- für das Jacobi-Verfahren hatten wir

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)}), \quad i = 1, \dots, n$$

- durchlaufen wir den Index i in umgekehrter Reihenfolge, so ändert sich nichts
- das Gauß-Seidel-Verfahren entsteht durch Benutzung der bereits berechneten Komponenten von $x^{(k+1)}$
- durchlaufen wir i von 1 bis n , so benutzen wir für $x_i^{(k+1)}$ alle $x_j^{(k+1)}$ mit $j < i$ und erhalten wie oben

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n$$

- Vorkonditionierer, Iterationsmatrix und Iterationsvektor sind gegeben durch

$$M_{GS} = D - E, \quad B_{GS} = (D - E)^{-1} F, \quad d_{GS} = M_{GS}^{-1} b$$

- durchlaufen wir i von n bis 1, so benutzen wir für $x_i^{(k+1)}$ alle $x_j^{(k+1)}$ mit $j > i$ und erhalten als neues Verfahren das Rückwärts-Gauß-Seidel-Verfahren

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k)} - \sum_{j>i} a_{ij} x_j^{(k+1)} \right), \quad i = n, \dots, 1$$

- Vorkonditionierer, Iterationsmatrix und Iterationsvektor sind dann

$$M_{RGS} = D - F, \quad B_{RGS} = (D - F)^{-1} E, \quad d_{RGS} = M_{RGS}^{-1} b$$

- für das *symmetrische Gauß-Seidel-Verfahren* führt man zunächst einen Gauß-Seidel Schritt und dann einen Rückwärts-Gauß-Seidel Schritt durch

$$x^{(k)} \xrightarrow{GS} B_{GS} x^{(k)} + d_{GS} \xrightarrow{RGS} x^{(k+1)} = B_{RGS} (B_{GS} x^{(k)} + d_{GS}) + d_{RGS}$$

d.h.

$$x^{(k+1)} = \underbrace{B_{RGS} B_{GS}}_{=: B_{SGS}} x^{(k)} + \underbrace{B_{RGS} d_{GS} + d_{RGS}}_{=: d_{SGS}}$$

und somit:

$$\begin{aligned} B_{SGS} &= (D - F)^{-1} E (D - E)^{-1} F, \\ M_{SGS} &= (D - E) D^{-1} (D - F), \\ d_{SGS} &= M_{SGS}^{-1} b \end{aligned}$$

! Satz 109

Ist A symmetrisch und $a_{ii} > 0, i = 1, \dots, n$, so ist M_{SGS} spd.

Nichtstationäre Iterationen

25.1 Steepest-Descent-Verfahren (Methode des steilsten Abstiegs)

- in diesem Abschnitt benutzen wir statt $x^{(k)}$ die Notation x_k , d.h. x_k bezeichnet hier den k -ten Iterationsvektor, nicht die k -te Komponente des Vektors x
- starten mit einfacher stationärer Iteration

$$x_{k+1} = x_k + M^{-1}r_k$$

- fügen Parameter α_k ein

$$x_{k+1} = x_k + \alpha_k M^{-1}r_k$$

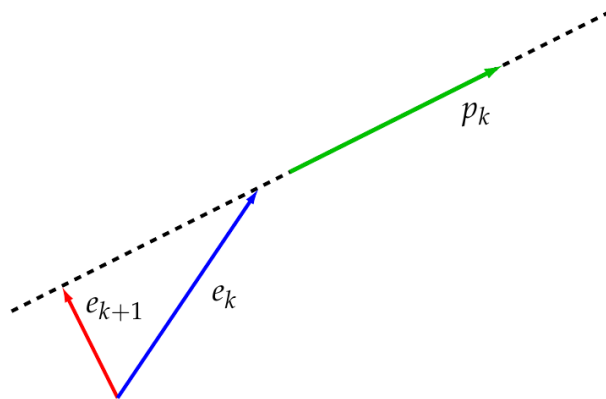
und bestimmen in *jedem Schritt* α_k so, dass x_{k+1} möglichst „günstige“ Eigenschaften hat

- in der Regel erhalten wir für unterschiedliche k auch unterschiedliche α_k , weshalb die Iteration im Gegensatz zu den oben besprochenen Relaxationsverfahren nicht mehr stationär ist ($x_{k+1} = \Phi_k(x_k)$)
- x_{k+1} „günstig“ bedeutet, dass

$$e_{k+1} = x - x_{k+1} = x - x_k - \alpha_k M^{-1}r_k = e_k - \alpha_k p_k, \quad p_k = M^{-1}r_k,$$

in einer geeigneten Norm möglichst klein werden soll

- Geometrie:
 - e_{k+1} liegt auf einer Geraden durch e_k in Richtung p_k
 - $\|e_{k+1}\|_2$ wird minimal genau dann wenn p_k senkrecht auf e_{k+1}



- $\|e_{k+1}\|_2 \leq \|e_k\|_2$, da schlimmstenfalls e_k senkrecht auf p_k stehen kann und damit $\alpha_k = 0$ ist
- Verallgemeinerung:
 - ist $[x, y]$ ein beliebiges Skalarprodukt auf $\mathbb{R}^n$ dann definiert

$$\|x\|_{[\cdot, \cdot]} = \sqrt{[x, x]}$$

eine Norm auf $\mathbb{R}^n$

- $\|e_{k+1}\|_{[\cdot, \cdot]} = \|e_k - \alpha_k p_k\|_{[\cdot, \cdot]}$ wird minimal, falls p_k senkrecht auf e_{k+1} d.h.

$$0 = [p_k, e_{k+1}] = [p_k, e_k - \alpha_k p_k] = [p_k, e_k] - \alpha_k [p_k, p_k] \quad \Leftrightarrow \quad \alpha_k = \frac{[p_k, e_k]}{[p_k, p_k]}$$

- zusammen haben wir folgendes Verfahren konstruiert:
 - starte mit $x_0 \in \mathbb{R}^n$ und wiederhole

$$\begin{aligned} r_k &= b - Ax_k \\ p_k &= M^{-1}r_k \\ \alpha_k &= \frac{[p_k, e_k]}{[p_k, p_k]} \\ x_{k+1} &= x_k + \alpha_k p_k \end{aligned}$$

- x_{k+1} ist mit wenigen Operationen zu berechnen
- $\|e_k\|_{[\cdot, \cdot]}$ ist monoton nicht wachsend
- Problem: bei der Berechnung von α_k muss $[p_k, e_k]$ bestimmt werden mit $e_k = x - x_k$, wobei x unbekannt ist
- dieses Problem kann durch eine spezielle Wahl des Skalarprodukts (und damit der Norm) gelöst werden
- mit A spd ist

$$[x, y] = (x, y)_A = (x, Ay)_2 = x^T Ay$$

ein Skalarprodukt mit zugehöriger Norm

$$\|x\|_{[\cdot, \cdot]} = \|x\|_A = \sqrt{x^T A x}$$

- damit erhalten wir

$$\alpha_k = \frac{(p_k, e_k)_A}{(p_k, p_k)_A} = \frac{(p_k, Ae_k)_2}{(p_k, Ap_k)_2} = \frac{(p_k, r_k)_2}{(p_k, Ap_k)_2},$$

d.h. α_k ist über p_k, r_k berechenbar

- benutzt man noch

$$r_{k+1} = b - Ax_{k+1} = b - A(x_k + \alpha_k p_k) = b - Ax_k - \alpha_k A p_k = r_k - \alpha_k A p_k$$

so erhalten wir für A spd das folgende Verfahren:

- starte mit x_0 , $r_0 = b - Ax_0$ und wiederhole

$$p_k = M^{-1} r_k$$

$$s_k = A p_k$$

$$\alpha_k = \frac{(p_k, r_k)_2}{(p_k, s_k)_2}$$

$$x_{k+1} = x_k + \alpha_k p_k$$

$$r_{k+1} = r_k - \alpha_k s_k$$

- Konvergenz:
 - wir wissen, dass $\|e_k\|_A$ nicht anwachsen kann
 - $\|e_k\|_A$ könnte aber stagnieren

! Satz 112

A, M spd, dann konvergiert das obige Verfahren und

$$\|e_{k+1}\|_A \leq \frac{\frac{\lambda_{max}}{\lambda_{min}} - 1}{\frac{\lambda_{max}}{\lambda_{min}} + 1} \|e_k\|_A$$

wobei $\lambda_{min}, \lambda_{max}$ kleinster bzw. größter Eigenwert von $M^{-1}A$ ist.

25.2 Konjugierte Gradienten (CG)

- beim Steepest-Descent-Verfahren wird e_{k+1} so bestimmt, dass p_k senkrecht auf e_{k+1} d.h.

$$0 = (p_k, e_{k+1})_A = (p_k, r_{k+1})_2$$

- für vorherige Suchrichtungen $p_j, j < k$ gilt das nicht, d.h. in der Regel ist

$$0 \neq (p_j, e_{k+1})_A, \quad j < k$$

- Idee: bestimme e_{k+1} so dass

$$(p_k, e_{k+1})_A = (p_{k-1}, e_{k+1})_A = 0$$

- $\|e_{k+1}\|_A$ wird über einen zweidimensionalen Unterraum minimiert, wodurch wir ein schnelleres Abklingen des Fehlers erwarten
- geschicktes Ausrechnen liefert für $M = I$ und A spd das klassische *CG-Verfahren* (Conjugate Gradients):
 - x_0 gegeben, $p_0 = r_0 = b - Ax_0$
 - wiederhole für $k \geq 0$:

$$\begin{aligned}x_{k+1} &= x_k + \alpha_k p_k \\r_{k+1} &= r_k - \alpha_k A p_k \quad \alpha_k = \frac{(r_k, r_k)_2}{(p_k, A p_k)_2} \\p_{k+1} &= r_{k+1} + \beta_k p_k \quad \beta_k = \frac{(r_{k+1}, r_{k+1})_2}{(r_k, r_k)_2}\end{aligned}$$

! Satz 116

Ist A spd so gilt für das CG-Verfahren:

- $(p_j, e_{k+1})_A = 0 \quad \forall j = 0, \dots, k$
- nach höchstens n Schritten ist $e_k = 0$
- $\|e_k\|_A \leq 2\left(\frac{\sqrt{\beta}-1}{\sqrt{\beta}+1}\right)^k \|e_0\|_A, \beta = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$

- die Rekursionsvorschrift für $M \neq I$ ist etwas aufwändiger und liefert das *vorkonditionierte CG-Verfahren* (PCG-Verfahren, **P**reconditioned **C**onjugate **G**radient) für M, A spd:

- x_0 gegeben, $r_0 = b - Ax_0, p_0 = q_0 = M^{-1}r_0$
- wiederhole für $k \geq 0$:

$$\begin{aligned}x_{k+1} &= x_k + \alpha_k p_k \\r_{k+1} &= r_k - \alpha_k A p_k \quad \alpha_k = \frac{(q_k, r_k)_2}{(p_k, A p_k)_2} \\q_{k+1} &= M^{-1}r_{k+1} \\p_{k+1} &= q_{k+1} + \beta_k p_k \quad \beta_k = \frac{(q_{k+1}, r_{k+1})_2}{(q_k, r_k)_2}\end{aligned}$$

- als Vorkonditionierer M benutzt man unter anderem:

- $M = \text{diag}(A) = \begin{pmatrix} a_{11} & & \\ & \ddots & \\ & & a_{nn} \end{pmatrix} = M_J$
- M aus einem symmetrischen, stationären Verfahren, wie z.B. SGS

$$M = M_{SGS} = (D - E)D^{-1}(D - E)^T$$

- unvollständige Cholesky Zerlegung

Dünn besetzte Matrizen

- bei (fast) allen Iterationsverfahren besteht der wesentliche Aufwand in einem Matrix-Vektor-Produkt
- der Aufwand dafür ist proportional zu der Anzahl der Matrixelemente
- bei vollbesetzten Matrizen in $\mathbb{R}^{n \times n}$ erhalten wir also $\mathcal{O}(n^2)$
- geht man davon aus, das man höchstens n Iterationen durchführt, so erhält man wieder, wie bei den direkten Lösern, einen Gesamtaufwand von $\mathcal{O}(n^3)$
- in Anwendungen treten aber oft Gleichungssysteme auf, deren Matrix dünn besetzt ist (sparse), d.h. die Anzahl n_{nz} der Elemente ungleich 0 ist sehr gering (deutlich unter 1%)
- berücksichtigt man bei Matrix-Vektor-Produkten die dünne Besetzungsstruktur der Matrix, so reduziert sich der Aufwand auf $\mathcal{O}(n_{nz})$
- in vielen realen Anwendungen ist $n_{nz} \sim n$, so dass der Aufwand dann nur $\mathcal{O}(n)$ ist
- dünn besetzte Matrizen werden nicht im Standardformat als zweidimensionales Array abgelegt, sondern in einem angepassten Format, dass einerseits das einfache Implementieren von Matrix-Vektor-Produkten zulässt und andererseits möglichst wenig Speicherplatz benötigt
- ein gängiges Datenformat dafür ist Compressed Sparse Row (CSR):
 - bei CSR werden nur die Nichtnullelemente der Matrix abgelegt sowie entsprechende Indexinformationen, in welcher Zeile bzw. Spalte der betreffende Eintrag in der Matrix zu finden ist
 - betrachten wir als Beispiel die Matrix

$$\begin{pmatrix} 1 & 0 & 7 & 0 & 8 \\ 6 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 4 & 5 & 0 & 0 & 0 \end{pmatrix}$$

- in CSR werden dafür folgende Daten gespeichert

```

      val    1  7  8  6  3  2  4  5
col-ind  1  3  5  1  2  4  1  2
row-ind  1  4  6  7  9

```

- im Vektor `val` stehen alle Nichtnullelemente der Matrix in zeilenweiser Anordnung (deshalb Compressed Sparse Row)
- der Index-Vektor `col-ind` enthält für jedes Element in `val` den zugehörigen Spaltenindex j
- die Einträge in `row-ind` geben an, ab dem wievielten Element in `val` jeweils eine neue Zeile beginnt (i -Index), d.h. in unserem Beispiel gehören die Elemente 1-3 zu Zeile 1, 4-5 zu Zeile 2, 6 zu Zeile 3 und 7-8 zu Zeile 4
- zum Speichern der Matrix in CSR benötigen wir damit 8 Floats und 13 Integer (statt 20 Floats im vollbesetzten Fall)
- bei großen dünn besetzten Matrizen ist das Verhältnis noch wesentlich günstiger

Vergleich verschiedener Verfahren

- zum Abschluss vergleichen wir das Verhalten der verschiedenen iterativen Verfahren anhand eines praxisrelevanten Beispiels

Zusammenfassung

- (große) lineare Gleichungssysteme $Ax = b$ iterativ näherungsweise lösen, wenn direkte Verfahren zu teuer sind
- stationäre einstufige Iterationen (Fixpunktform):

$$x^{(k+1)} = Bx^{(k)} + c,$$

- Konvergenz genau dann, wenn $\rho(B) < 1$
- Jacobi-Verfahren (Gesamtschritt): alle Komponenten aus $x^{(k)}$ gleichzeitig aktualisieren (gut parallelisierbar)
- Gauß–Seidel (Einzelschritt): nutzt neue Komponenten sofort innerhalb eines Schritts (konvergiert oft schneller als Jacobi)
- Nachiteration (iterative Verbesserung): aus Residuen Korrekturen berechnen, um eine vorhandene Approximation zu verbessern
- Relaxationsverfahren: Dämpfung/Überrelaxation zur Beschleunigung (Parameter ω)
- Symmetrische Varianten: z. B. symmetrisches Gauß–Seidel, wichtig als Baustein/Preconditioner
- Nichtstationäre Iterationen: Iterationsfunktionen ändern sich mit k
- Steepest Descent: schrittweise Minimierung, aber teils langsame Konvergenz
- Konjugierte Gradienten (CG): für symmetrisch positiv definite A , effizient, oft mit Preconditioning
- dünn besetzte Matrizen: Sparsity ausnutzen, Matrix-Vektor-Produkte als Kernoperation
- Vergleich der Verfahren: Konvergenzgeschwindigkeit, Residuen- und Fehlerverhalten

Teil VI

Eigenwerte und Eigenvektoren

- wir suchen λ und $x \neq 0$, so dass

$$Ax = \lambda x$$

- aus der Algebra wissen wir, dass es dafür keine direkten Verfahren geben kann
- wir untersuchen Iterationsverfahren zur Bestimmung
 - eines einzelnen Eigenwert/Eigenvektor-Paars λ, x
 - mehrerer bzw. aller Eigenwerte/Eigenvektoren λ_k, x_k
- schließlich betrachten wir noch das verallgemeinerte Eigenwertproblem

$$Ax = \lambda Bx, \quad x \neq 0$$

- in vielen Anwendungen spielen Resonanzphänomene eine wichtige Rolle
- wir sehen also, dass das System auf eine Anregung mit ganz bestimmten Frequenzen sehr kritisch reagiert
- dies kann im Extremfall sogar zu ernsthaften Schäden führen
- ein bekanntes Beispiel aus der Praxis ist der Einsturz der Tacoma-Narrows Brücke (USA, 1940), bei dem periodisch auftretende Seitenwinde derart extreme Schwingungsamplituden verursachten, dass große Teile der Brücke zerstört wurden
- deshalb ist es wichtig, dass man die Frequenzen, auf die ein System so empfindlich reagiert (*Eigenfrequenzen*), genau kennt
- für den Federschwinger von oben ist (unter der Annahme einer linearen Feder, d.h. Hookesches Gesetz) diese Frequenz gegeben durch

$$f = \frac{1}{2\pi} \sqrt{\frac{k}{m}},$$

wobei k die Federsteifigkeit und m die Masse der an der Feder befestigten Kugel ist

- kennen wir diese Frequenz, so können wir bei unserem Beispiel von oben den Anfahrprozess so abändern, dass die Schwingungsamplituden (und damit auch die Belastungen) während des Hochfahrens reduziert werden
- bei komplexen Bauteile ist das Schwingungsverhalten sehr viel komplizierter
- sie besitzen nicht nur eine sondern sehr viele Eigenfrequenzen
- die zugehörigen Schwingungsformen (*Modes*) können ebenfalls sehr komplex sein (die Schwingungsformen beim Federschwinger war einfach nur auf-und-ab)
- sowohl Eigenfrequenzen als auch Schwingungsformen lassen sich nur für wenige Spezialfälle analytisch berechnen
- analog zu dem Beispiel aus der Strukturmechanik (Kurbel) aus dem ersten Kapitel betrachtet man deswegen wieder ein Ersatzproblem (Finite Elemente)
- das Ersatzmodell beschreibt letztlich das Bauteilverhalten über große, dünn besetzte Matrizen
- die Eigenfrequenzen und Eigenschwingungsformen ergeben sich aus den (verallgemeinerten) Eigenwerten und Eigenvektoren dieser Matrizen

31.1 Komplexwertige Matrizen

- $\lambda \in \mathbb{C}$ ist *Eigenwert* der Matrix $A \in \mathbb{C}^{n \times n}$ falls es einen *Eigenvektor* $v \in \mathbb{C}^n$, $v \neq 0$, gibt mit

$$Av = \lambda v$$

- eine äquivalente Bedingung dafür ist

$$0 = \det(A - \lambda I) = p_A(\lambda),$$

d.h. λ ist Nullstelle des *charakteristischen Polynoms* p_A

- p_A hat genau Grad n , d.h.

$$p_A(\lambda) = (\lambda_1 - \lambda) \cdot \dots \cdot (\lambda_n - \lambda),$$

$\lambda_1, \dots, \lambda_n \in \mathbb{C}$ sind die (nicht notwendigerweise verschiedenen) Eigenwerte von A

- ist $\lambda_i = \lambda_j$, $i \neq j$, so heißt λ_i *mehrfacher* Eigenwert
- $v_i \in \mathbb{C}^n$ ist *Eigenvektor* von A zu λ_i falls $v_i \neq 0$, $Av_i = \lambda_i v_i$
- ist v_i Eigenvektor zu λ_i , dann ist für $c \in \mathbb{C}$, $c \neq 0$, auch $c \cdot v_i$ Eigenvektor
- bei mehrfachen Eigenwerten unterscheidet man die *algebraische* Vielfachheit, d.h. die Vielfachheit der Nullstelle im charakteristischen Polynom p_A , und die *geometrische* Vielfachheit, d.h. die Anzahl der linear unabhängigen Eigenvektoren zu dem mehrfachen Eigenwert
- für beide Werte gilt der Zusammenhang:

$$1 \leq \text{geometrische Vielfachheit} \leq \text{algebraische Vielfachheit}$$

! Satz 125

Ist $A \in \mathbb{C}^{n \times n}$ und sind $v, w \in \mathbb{C}^n$ Eigenvektoren zu Eigenwerten $\lambda \neq \mu$, so sind v, w linear unabhängig.

 Definition 126

Ist $A \in \mathbb{C}^{n \times n}$ und $V \in \mathbb{C}^{n \times n}$, V regulär, so heißt

$$VAV^{-1}$$

Ähnlichkeitstransformation von A .

 Satz 127

Ist $A \in \mathbb{C}^{n \times n}$ und $V \in \mathbb{C}^{n \times n}$, V regulär, dann haben A und VAV^{-1} die selben Eigenwerte.

 Satz 128 (Jordan-Form)

Für jede Matrix $A \in \mathbb{C}^{n \times n}$ gibt es eine reguläre Matrix $V \in \mathbb{C}^{n \times n}$ mit

$$A = VJV^{-1}, \quad J = \begin{pmatrix} J_1 & & \\ & \ddots & \\ & & J_m \end{pmatrix}.$$

Dabei ist J eine Blockdiagonalmatrix mit

$$J_i = \begin{pmatrix} \lambda_i & 1 & & \\ & \ddots & \ddots & \\ & & \lambda_i & 1 \\ & & & \lambda_i \end{pmatrix} \in \mathbb{C}^{n_i \times n_i}.$$

- jede Matrix $A \in \mathbb{C}^{n \times n}$ ist also durch eine Ähnlichkeitstransformation $A = VJV^{-1}$ Block-diagonalisierbar, wobei auf der Diagonale von J die Eigenwerte zu finden sind
- zu jedem Block J_i gibt es genau einen Eigenvektor zum Eigenwert λ_i
- steht das erste Diagonalelement $J_{i,11}$ von J_i in der Matrix J an Position J_{kk} , dann ist die k -te Spalte v_k von V Eigenvektor zu λ_i
- besitzt $A \in \mathbb{C}^{n \times n}$ genau n linear unabhängige Eigenvektoren v_i mit Eigenwerten λ_i , dann gilt

$$A = V \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix} V^{-1}, \quad V = (v_1, \dots, v_n)$$

d.h. A ist diagonalisierbar

- was ändert sich, wenn man für V nur spezielle reguläre Matrizen zulässt?

 Definition 129

Eine Matrix $U \in \mathbb{C}^{n \times n}$ heißt *unitär* wenn $UU^H = U^H U = I$. Dabei ist $U^H = \bar{U}^T$.

 Satz 131 (Schur-Form)

Für jede Matrix $A \in \mathbb{C}^{n \times n}$ gibt es eine unitäre Matrix $U \in \mathbb{C}^{n \times n}$ mit

$$A = UTU^H, \quad T = \begin{pmatrix} \lambda_1 & t_{12} & \dots & t_{1n} \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & t_{n-1n} \\ 0 & \dots & 0 & \lambda_n \end{pmatrix}.$$

- jede Matrix $A \in \mathbb{C}^{n \times n}$ kann also durch eine *unitäre* Ähnlichkeitstransformation auf obere Dreiecksgestalt T transformiert werden, wobei auf der Diagonale von T die Eigenwerte von A stehen

Definition 132

Eine Matrix $A \in \mathbb{C}^{n \times n}$ heißt *unitär diagonalisierbar*, wenn es eine unitäre Matrix $U \in \mathbb{C}^{n \times n}$ gibt, so dass

$$A = U\Lambda U^H, \quad \Lambda = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & \lambda_n \end{pmatrix}.$$

- um unitäre Diagonalisierbarkeit zu untersuchen betrachten wir spezielle Matrizen

Definition 133

Eine Matrix $A \in \mathbb{C}^{n \times n}$ heißt *normal* wenn $AA^H = A^H A$.

- für die unitäre Diagonalisierbarkeit erhalten wir das folgende Resultat

Satz 134

Eine Matrix $A \in \mathbb{C}^{n \times n}$ ist genau dann unitär diagonalisierbar, wenn A *normal* ist.

Definition 138

Eine Matrix $A \in \mathbb{C}^{n \times n}$ heißt *hermitesch* wenn $A = A^H$.

Satz 140

$A \in \mathbb{C}^{n \times n}$ sei normal, d.h. $AA^H = A^H A$. Die Eigenwerte von A sind alle reell, genau dann wenn A hermitesch ist, d.h. $A = A^H$.

- der folgende Satz liefert eine erste Aussage, wo man die λ_i zu suchen hat

Satz 143 (Gerschgorin)

Sei $A \in \mathbb{C}^{n \times n}$ und

$$K = \bigcup_{i=1}^n K_i, \quad K_i = \{z \mid z \in \mathbb{C}, |z - a_{ii}| \leq \sum_{j \neq i} |a_{ij}|\}.$$

Dann gilt für die Eigenwerte λ_i von A dass $\lambda_i \in K \forall i$.

- das folgende Resultat kann in der Praxis sehr nützlich sein

! Satz 145

Die Eigenwerte von

$$A = \begin{pmatrix} B & C \\ 0 & D \end{pmatrix} \in \mathbb{C}^{n \times n}, \quad B \in \mathbb{C}^{p \times p}, \quad C \in \mathbb{C}^{p \times n-p}, \quad D \in \mathbb{C}^{n-p \times n-p},$$

sind genau die Eigenwerte von B und D .

Ist λ ein Eigenwert sowohl von B als auch D , so addieren sich die algebraischen Vielfachheiten.

- hat A diese Struktur, dann kann das Eigenwertproblem für A in zwei kleinere und damit potentiell einfachere Eigenwertprobleme für B und D zerlegt werden
- für die Eigenwerte von A spielt C also gar keine Rolle
- für die Eigenvektoren von A stimmt das nicht
- genauere Aussagen dazu liefert der folgende Satz

! Satz 147 (Roth's removal rule)

Die Matrizen

$$A = \begin{pmatrix} B & C \\ 0 & D \end{pmatrix}, \quad \tilde{A} = \begin{pmatrix} B & 0 \\ 0 & D \end{pmatrix}$$

sind ähnlich, d.h. es gibt eine reguläre Matrix V mit

$$A = V \tilde{A} V^{-1}$$

genau dann, wenn die *Sylvester-Gleichung*

$$BX - XD = C$$

eine Lösung X besitzt.

In diesem Fall ist

$$V = \begin{pmatrix} I & -X \\ 0 & I \end{pmatrix}, \quad V^{-1} = \begin{pmatrix} I & X \\ 0 & I \end{pmatrix}.$$

31.2 Reellwertige Matrizen

- wir betrachten jetzt reellwertige Matrizen $A \in \mathbb{R}^{n \times n}$
- wie das folgende Beispiel zeigt, haben reellwertige Matrizen A nicht immer nur reelle Eigenwerte

! Satz 151

Ist $\lambda = a + jb$, $a, b \in \mathbb{R}$, $b \neq 0$, Eigenwert von $A \in \mathbb{R}^{n \times n}$ zum Eigenvektor $v = u + jw$, $u, w \in \mathbb{R}^n$, dann ist $\bar{\lambda} = a - jb$ Eigenwert zum Eigenvektor $\bar{v} = u - jw$.

v und $\bar{v}$ sind linear unabhängig, genauso wie $u = \Re(v)$ und $w = \Im(v)$.

- unter Berücksichtigung dieser Ergebnisse lassen sich die meisten Aussagen für komplexwertige Matrizen auf reellwertige Matrizen übertragen, wobei die Beweise größtenteils analog verlaufen

! Satz 152 (Reelle Jordan-Form)

Für jede Matrix $A \in \mathbb{R}^{n \times n}$ gibt es eine reguläre Matrix $V \in \mathbb{R}^{n \times n}$ so dass

$$A = VJV^{-1}, \quad J = \begin{pmatrix} J_1 & & \\ & \ddots & \\ & & J_m \end{pmatrix}, \quad J_i = \begin{pmatrix} \Lambda_i & I & & \\ & \ddots & \ddots & \\ & & \Lambda_i & I \\ & & & \Lambda_i \end{pmatrix} \in \mathbb{C}^{n_i \times n_i}.$$

Für $\lambda_i \in \mathbb{R}$ ist $\Lambda_i = \lambda_i$, sonst ist

$$\Lambda_i = \begin{pmatrix} a_i & b_i \\ -b_i & a_i \end{pmatrix},$$

wobei $\lambda_i = a_i \pm jb_i \in \mathbb{C}$ die Eigenwerte von Λ_i sind.

! Satz 153 (Reelle Schur-Form)

Für jede Matrix $A \in \mathbb{R}^{n \times n}$ gibt es eine orthonormale Matrix $Q \in \mathbb{R}^{n \times n}$ mit

$$A = QTQ^T, \quad T = \begin{pmatrix} T_{11} & T_{12} & \cdots & T_{1m} \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & T_{m-1m} \\ 0 & \cdots & 0 & T_{mm} \end{pmatrix}.$$

Dabei ist entweder $T_{ii} \in \mathbb{R}$ oder $T_{ii} \in \mathbb{R}^{2 \times 2}$.

Im ersten Fall ist dann $T_{ii} = \lambda_i$ ein Eigenwert von A .

Im zweiten Fall ist T_{ii} eine 2×2 Matrix mit den komplex konjugierten Eigenwerten $\lambda_i, \bar{\lambda}_i$ von A

- bei der reellen Jordan-Form konnten wir mit Ähnlichkeitstransformationen $A = VJV^{-1}$ die Diagonalblöcke zu den komplex konjugierten Eigenwerten immer auf die Standardform

$$T = \begin{pmatrix} a & b \\ -b & a \end{pmatrix}$$

bringen, wobei V eine reguläre Matrix ist

- bei der reellen Schur-Form dürfen wir für die Ähnlichkeitstransformationen aber nur orthonormale Matrizen Q verwenden
- in diesem Fall muss A noch zusätzliche Voraussetzungen erfüllen
- wir betrachten zunächst $A \in \mathbb{R}^{2 \times 2}$ und übertragen das Resultat anschließend auf den allgemeinen Fall

! Satz 154

Eine Matrix $A \in \mathbb{R}^{2 \times 2}$ mit komplexen Eigenwerten $\lambda = a + jb, \bar{\lambda} = a - jb, b \neq 0$, ist genau dann orthonormal auf die Standardform

$$T = \begin{pmatrix} a & b \\ -b & a \end{pmatrix}$$

transformierbar, wenn A normal ist.

! Satz 156

$A \in \mathbb{R}^{n \times n}$ ist genau dann orthonormal diagonalisierbar, d.h.

$$A = Q\Lambda Q^T, \quad \Lambda = \begin{pmatrix} \Lambda_1 & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \dots & 0 & \Lambda_m \end{pmatrix}, \quad \text{mit } \Lambda_i = \lambda_i \in \mathbb{R} \quad \text{oder} \quad \Lambda_i = \begin{pmatrix} a_i & b_i \\ -b_i & a_i \end{pmatrix}$$

wenn A normal ist, d.h. $A^T A = A A^T$.

! Satz 159

$A \in \mathbb{R}^{n \times n}$ sei normal, d.h. $AA^T = A^T A$. Die Eigenwerte von A sind alle reell, genau dann wenn A symmetrisch ist, d.h. $A = A^T$.

31.3 Positiv definite Matrizen

🔊 Definition 161

$A \in \mathbb{R}^{n \times n}$ heißt positiv definit, falls

$$(x, Ax)_2 = x^T A x > 0 \quad \forall x \in \mathbb{R}^n, \quad x \neq 0$$

! Satz 162

A ist genau dann positiv definit, wenn

$$A_s = \frac{1}{2}(A + A^T)$$

positiv definit ist.

! Satz 163

Ist A symmetrisch, so ist A genau dann positiv definit, wenn alle Eigenwerte positiv sind.

- ohne Symmetrie gilt das i.A. nicht

31.4 Zusammenfassung

- das Eigenwertproblem ist also ein Nullstellenproblem für das charakteristische Polynom p_A
- aus der Algebra wissen wir, dass es ab Polynomgrad 5 *keinen* endlichen Algorithmus zur Lösung dieses Problems gibt
- numerische Algorithmen können deswegen nur iterative Verfahren sein
- in vielen Anwendungsfällen (z.B. Bestimmung von Resonanzfrequenzen) ist man nur an einer kleinen Anzahl von Eigenwerten und Eigenvektoren interessiert
- benötigt man alle Eigenwerte und Eigenvektoren, dann ist es naheliegend, die Matrix A durch Ähnlichkeitstransformationen auf (reelle) Jordan- bzw. Schur-Form zu transformieren
- die Jordan-Form beinhaltet detailliertere Informationen als die Schur-Form, ihre Berechnung ist allerdings schlecht gestellt
- deshalb benutzt man Algorithmen die A näherungsweise auf auf Schur-Form $A = UTU^H$ bzw. reelle Schur-Form $A = QTQ^T$ transformieren
- dazu benötigt man nur unitäre/orthonormale Transformationen (Householder, Givens), die numerisch sehr stabil sind
- ist A unitär diagonalisierbar, d.h. $A = U\Lambda U^H$, also $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n) \in \mathbb{C}^{n \times n}$, so sind die Spaltenvektoren von U die Eigenvektoren von A
- ist A reell und orthonormal diagonalisierbar, also $A = Q\Lambda Q^T$, $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n) \in \mathbb{R}^{n \times n}$, so sind die Spaltenvektoren von Q die Eigenvektoren von A

- wir betrachten normale Matrizen $A \in \mathbb{C}^{n \times n}$
- damit ist A unitär diagonalisierbar, d.h. es existiert eine Orthonormalbasis von Eigenvektoren $u_1, \dots, u_n$
- jedes $x \in \mathbb{C}^n$ kann als $x = \sum_i x_i u_i$, $x_i = (u_i, x)_2$ geschrieben werden und

$$Ax = \sum_i \lambda_i x_i u_i \quad \text{bzw.} \quad A^k x = \sum_i \lambda_i^k x_i u_i$$

- gilt $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ so dominiert der Anteil $\lambda_1^k x_1 u_1$ (falls $x_1 \neq 0$) in $A^k x$
- wird x oft genug mit A multipliziert, so bleibt (im wesentlichen) ein Vielfaches von v_1 übrig
- ist $|\lambda_1| > 1$ so wird $\|A^k x\|_2$ immer größer
- um die damit verbundenen numerischen Probleme zu vermeiden normalisiert man nach jeder Multiplikation mit A den neu berechneten Vektor und erhält somit die *Vektoriteration*
 - wähle $x^{(0)}$ mit $\|x^{(0)}\|_2 = 1$, $x_1^{(0)} = (x^{(0)}, v_1) \neq 0$
 - wiederhole

$$\begin{aligned} y^{(k+1)} &= Ax^{(k)} \\ x^{(k+1)} &= \frac{y^{(k+1)}}{\|y^{(k+1)}\|_2} \\ \mu^{(k+1)} &= (x^{(k)}, y^{(k+1)})_2 = \frac{(x^{(k)}, Ax^{(k)})_2}{(x^{(k)}, x^{(k)})_2} \end{aligned}$$

- $x^{(k)}$ ist eine Näherung von $c \cdot v_1$, $\mu^{(k)}$ eine Näherung von λ_1

! Satz 167

$A \in \mathbb{C}^{n \times n}$ sei normal mit Eigenwerten $|\lambda_1| = |\lambda_2| = \dots |\lambda_p| > |\lambda_{p+1}| \geq \dots \geq |\lambda_n|$ und Eigenvektoren u_i . Dann gilt für die Vektoriteration

$$\|y^{(k+1)}\|_2 \xrightarrow{k \rightarrow \infty} |\lambda_1| = \rho(A), \quad |\lambda_1| - \|y^{(k+1)}\|_2 = \mathcal{O}\left(\left|\frac{\lambda_{p+1}}{\lambda_1}\right|^{2k}\right),$$

falls $\sum_{i=1}^p |(u_i, x^{(0)})_2|^2 \neq 0$.

Ist zusätzlich $p = 1$ so folgt

$$\mu^{(k)} \xrightarrow{k \rightarrow \infty} \lambda_1, \quad |\lambda_1 - \mu^{(k+1)}| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^{2k}\right).$$

Für $p = 1$ und $\lambda_1 > 0$ konvergiert $x^{(k)}$ gegen ein Vielfaches von u_1 .

- wie das folgende Beispiel zeigt, muss die Vektoriteration nicht immer konvergieren

Jacobi-Verfahren

- wir versuchen ein Verfahren zu konstruieren, das Näherungen für *alle* Eigenwerte und -vektoren liefert
- dazu beschränken wir uns auf symmetrische Matrizen A , d.h. es gibt orthonormale Matrizen Q mit

$$Q^T A Q = \Lambda = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix}, \quad Q = (v_1, \dots, v_n)$$

wobei λ_i, v_i die Eigenwerte und Eigenvektoren von A sind

- wir wissen, dass es kein direktes Verfahren geben kann um die λ_i, v_i zu bestimmen und somit kann auch Q nicht in endlich vielen Schritten erzeugt werden
- deshalb versuchen wir, A iterativ mit einfachen orthonormalen Matrizen Q_k auf näherungsweise Diagonalgestalt zu transformieren

$$A^{(0)} = A, \quad A^{(k)} = Q_k^T A^{(k-1)} Q_k$$

- die „einfachsten“ Q_k , die wir kennen, sind Givens-Rotationen

$$Q_k = \begin{pmatrix} 1 & & & & & & & & \\ & \ddots & & & & & & & \\ & & 1 & & & & & & \\ & & & c & & & & s & \\ & & & & 1 & & & & \\ & & & & & \ddots & & & \\ & & & & & & 1 & & \\ & & & & & & & c & \\ & & & & & & & & 1 & \\ & & & & & & & & & s & \\ & & & & & & & & & & 1 \end{pmatrix}$$

- $A^{(k)}$ soll „näher“ an Λ sein als $A^{(k-1)}$, d.h. die Nebendiagonalelemente sollten möglichst kleinen Betrag haben
- das folgende Lemma liefert eine erste Idee dazu

! Lemma 173

Ist $A \in \mathbb{R}^{n \times n}$ symmetrisch und $Q \in \mathbb{R}^{n \times n}$ eine Drehmatrix

$$Q = \begin{pmatrix} 1 & & & & & & & & & \\ & \ddots & & & & & & & & \\ & & 1 & & & & & & & \\ & & & c & & & & & & \\ & & & & 1 & & & & & \\ & & & & & \ddots & & & & \\ & & & & & & 1 & & & \\ & & & & & & & c & & \\ & & & & & & & & 1 & \\ & & & & & & & & & \ddots \\ & & & & & & & & & & 1 \end{pmatrix} \begin{matrix} i_0 \\ j_0 \end{matrix}$$

mit

$$c = \cos(\alpha), \quad s = \sin(\alpha), \quad \alpha = \begin{cases} \frac{\pi}{4} & a_{i_0 i_0} = a_{j_0 j_0} \\ \frac{1}{2} \arctan\left(\frac{2a_{i_0 j_0}}{a_{j_0 j_0} - a_{i_0 i_0}}\right) & a_{i_0 i_0} \neq a_{j_0 j_0} \end{cases}.$$

Dann gilt $b_{i_0 j_0} = 0$ für $B = Q^T A Q$.

- $A^{(k)}$ soll „näher“ an Λ sein als $A^{(k-1)}$, d.h. die Nebendiagonalelemente von $A^{(k)}$ sollten möglichst klein sein
- das legt folgende Strategie nahe:
 - suche das betragsgrößte Nebendiagonalelement $a_{i_0 j_0}^{(k-1)}$ von $A^{(k-1)}$
 - dabei kann $j_0 > i_0$ angenommen werden, da alle Matrizen $A^{(k-1)}$ symmetrisch sind
 - bestimme Q_k mit Hilfe des Lemmas so, dass $a_{i_0 j_0}^{(k)} = 0$
- die Multiplikation $Q_k^T A^{(k-1)} Q_k$ verändert Einträge in den Spalten und Zeilen mit Index i_0 und j_0
- wurde dort im vorherigen Schritt eine 0 erzeugt, so kann diese jetzt wieder verschwinden, d.h. wir eliminieren *nicht* nach und nach die Nebendiagonalelemente
- trotzdem haben wir damit etwas gewonnen

! Satz 175

Für die Quadratsumme der Nebendiagonalelemente der Matrizen $A^{(k)}$

$$N(A^{(k)}) = \sum_{i \neq j} (a_{ij}^{(k)})^2$$

gilt

$$N(A^{(k+1)}) \leq \alpha N(A^{(k)}), \quad \alpha \in [0, 1),$$

d.h. sie konvergiert *streng monoton* fallend gegen 0.

! Satz 176

Sind $\lambda_i, a_{ii}^{(k)}$ aufsteigend sortiert, so gilt

$$|a_{ii}^{(k)} - \lambda_i| \leq \sqrt{N(A^{(k)})},$$

d.h. die sortierten Diagonalelemente von $A^{(k)}$ konvergieren gegen die Eigenwerte von A

- wir berechnen also $A^{(k)} = Q_k^T A^{(k-1)} Q_k$ und benutzen

$$\text{diag}(A^{(k)}) = \begin{pmatrix} a_{11}^{(k)} & & \\ & \ddots & \\ & & a_{nn}^{(k)} \end{pmatrix}$$

als Näherung für Λ , also als Näherung für *alle* Eigenwerte λ_i

- wie man eine Approximation der Eigenvektoren erhält, kann man sich durch folgende Überlegungen plausibel machen:

- nach oben ist

$$A^{(k)} = Q_k^T \cdot \dots \cdot Q_1^T A \underbrace{Q_1 \cdot \dots \cdot Q_k}_{=: Q^{(k)}}$$

und deshalb

$$A = Q^{(k)} A^{(k)} Q^{(k)T}$$

- andererseits ist $A = Q \Lambda Q^T$, $Q = (v_1, \dots, v_n)$
- ist $A^{(k)}$ eine gute Näherung von Λ (d.h. sind die Nebendiagonalelemente klein), so liefern die Spalten von $Q^{(k)}$ eine Approximation der v_i
- zur Implementierung erweitert man die Iteration um

$$Q^{(0)} = I, \quad Q^{(k)} = Q^{(k-1)} Q_k$$

- wir betrachten die Unterraum-Iteration für $A \in \mathbb{R}^{n \times n}$ mit n Startvektoren, d.h. der Unterraum hat volle Dimension:
 - wähle $X^{(0)} = (x_1^{(0)}, \dots, x_n^{(0)})$ mit $x_j^{(0)}$ orthonormal
 - wiederhole für $k = 0, 1, \dots$

$$\begin{aligned} Y^{(k+1)} &= AX^{(k)} \\ \tilde{Q}^{(k+1)} R^{(k+1)} &= Y^{(k+1)} \\ X^{(k+1)} &= \tilde{Q}^{(k+1)} \\ T^{(k+1)} &= X^{(k)T} Y^{(k+1)} = X^{(k)T} AX^{(k)} \end{aligned}$$

- damit ist

$$\begin{aligned} T^{(k)} &= X^{(k-1)T} AX^{(k-1)} \\ &= X^{(k-1)T} Y^{(k)} \\ &= X^{(k-1)T} \tilde{Q}^{(k)} R^{(k)} \\ &= (X^{(k-1)T} X^{(k)}) R^{(k)} \\ &= Q^{(k)} R^{(k)} \end{aligned}$$

- $Q^{(k)} R^{(k)}$ ist also eine QR-Zerlegung von $T^{(k)}$ mit

$$Q^{(k)} = X^{(k-1)T} X^{(k)}$$

- da alle $X^{(k)}$ orthonormal sind gilt auch

$$\begin{aligned}
 T^{(k+1)} &= X^{(k)T} A X^{(k)} \\
 &= X^{(k)T} A X^{(k-1)} X^{(k-1)T} X^{(k)} \\
 &= X^{(k)T} Y^{(k)} X^{(k-1)T} X^{(k)} \\
 &= X^{(k)T} \tilde{Q}^{(k)} R^{(k)} X^{(k-1)T} X^{(k)} \\
 &= X^{(k)T} X^{(k)} R^{(k)} X^{(k-1)T} X^{(k)} \\
 &= R^{(k)} (X^{(k-1)T} X^{(k)}) \\
 &= R^{(k)} Q^{(k)}
 \end{aligned}$$

- für $X^{(0)} = I$ ist $T^{(1)} = A$
- mit $A^{(k)} = T^{(k+1)}$ erhalten wir schließlich die *QR-Iteration*:
 - starte mit $A^{(0)} = A$
 - wiederhole:
 - zerlege $A^{(k)} = Q^{(k)} R^{(k)}$
 - berechne $A^{(k+1)} = R^{(k)} Q^{(k)}$
- wegen

$$A^{(k+1)} = R^{(k)} Q^{(k)} = Q^{(k)T} Q^{(k)} R^{(k)} Q^{(k)} = Q^{(k)T} A^{(k)} Q^{(k)}$$

ist

$$A^{(k)} = Q^{(k-1)T} \cdot \dots \cdot Q^{(0)T} A Q^{(0)} \cdot \dots \cdot Q^{(k-1)}$$

also

$$A = \hat{Q}^{(k)} A^{(k)} \hat{Q}^{(k)T}, \quad \hat{Q}^{(k)} = Q^{(0)} \cdot \dots \cdot Q^{(k-1)}$$

- hat A die reelle Schur-Form $A = QTQ^T$ so gilt unter geeigneten Voraussetzungen an A

$$A^{(k)} \xrightarrow{k \rightarrow \infty} T, \quad \hat{Q}^{(k)} \xrightarrow{k \rightarrow \infty} Q$$

- für symmetrische Matrizen erhalten wir folgende Aussage

! Satz 180

Ist $A \in \mathbb{R}^{n \times n}$ symmetrisch mit Eigenwerten $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$, dann gilt für die mit dem QR-Verfahren erzeugten Matrizen $A^{(k)}$

$$A^{(k)} \xrightarrow{k \rightarrow \infty} \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix}$$

- im nächsten Abschnitt werden wir detailliertere Aussagen zur Konvergenz kennen lernen

34.1 Hessenberg-Transformation

- in der Standard-Form des letzten Kapitels ist der Aufwand pro Schritt der QR-Iteration im wesentlichen eine Householder-Zerlegung, also $\mathcal{O}(n^3)$, und ist somit extrem hoch
- ist es möglich, A vorab in eine Form zu bringen, die den Aufwand der QR-Zerlegungen senkt?
- dazu benutzen wir Householder-Matrizen auf verkürzten Spalten von A :

- wir betrachten die erste Spalte s_1 und die um das Diagonalelement verkürzte erste Spalte $\tilde{s}_1$

$$s_1 = \begin{pmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{pmatrix}, \quad \tilde{s}_1 = \begin{pmatrix} a_{21} \\ \vdots \\ a_{n1} \end{pmatrix}$$

und konstruieren die Householder-Matrix $\tilde{Q}_1$ die $\tilde{s}_1$ eliminiert, d.h.

$$\tilde{Q}_1 \tilde{s}_1 = \alpha_1 \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

- erweitern wir $\tilde{Q}_1 \in \mathbb{R}^{n-1 \times n-1}$ zu

$$Q_1 := \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & & & \\ \vdots & & \tilde{Q}_1 & \\ 0 & & & \end{pmatrix}$$

dann ist $Q_1 \in \mathbb{R}^{n \times n}$ orthonormal mit

$$Q_1 A = \begin{pmatrix} a_{11} & * & \dots & \dots & * \\ \alpha_1 & * & \dots & \dots & * \\ 0 & \vdots & & & \vdots \\ \vdots & \vdots & & & \vdots \\ 0 & * & \dots & \dots & * \end{pmatrix}, \quad Q_1 A Q_1^T = Q_1 A Q_1 = \begin{pmatrix} a_{11} & * & \dots & \dots & * \\ \alpha_1 & * & \dots & \dots & * \\ 0 & \vdots & & & \vdots \\ \vdots & \vdots & & & \vdots \\ 0 & * & \dots & \dots & * \end{pmatrix}$$

- analoges Vorgehen für Spalte 2 bis $n - 2$ liefert

$$H = \underbrace{Q_{n-2} \cdot \dots \cdot Q_1}_Q A \underbrace{Q_1^T \cdot \dots \cdot Q_{n-2}^T}_{Q^T = Q^{-1}} = \begin{pmatrix} * & & \dots & & * \\ * & \ddots & & & \vdots \\ 0 & \ddots & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & * & * \end{pmatrix}$$

- $H = Q A Q^{-1}$ hat dieselben Eigenwerte wie A und besitzt *Hessenberg-Form*, d.h. das um die erste Nebendiagonale reduzierte untere Dreieck ist identisch 0
- für das QR-Verfahren bringt die Hessenberg-Form folgenden Vorteil:
 - bei der QR-Zerlegung einer Hessenberg-Matrix muss pro Spalte nur das Subdiagonalelement eliminiert werden, was jeweils mit *einer einzigen* Givens-Rotation erledigt werden kann, so dass der Gesamtaufwand nur noch $\mathcal{O}(n^2)$ statt $\mathcal{O}(n^3)$ ist
 - hat $A^{(k)}$ Hessenberg-Form und wurde mit Givens-Matrizen in $A^{(k)} = Q^{(k)} R^{(k)}$ zerlegt, so hat $A^{(k+1)} = R^{(k)} Q^{(k)}$ wieder Hessenberg-Form
 - startet man also das QR-Verfahren mit $A^{(0)}$ in Hessenberg-Form, so haben alle $A^{(k)}$ Hessenberg-Form, d.h. alle QR-Zerlegungen können mit Givens-Rotationen mit $\mathcal{O}(n^2)$ Operationen durchgeführt werden
- ist A symmetrisch, so vereinfacht sich das ganze nochmals:
 - die Hessenberg-Transformierte H ist wegen $H = Q A Q^{-1} = Q A Q^T$ ebenfalls symmetrisch und muss deshalb tridiagonal sein:

$$H = \begin{pmatrix} * & * & 0 & \dots & 0 \\ * & * & * & \ddots & \vdots \\ 0 & * & * & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & * \\ 0 & \dots & 0 & * & * \end{pmatrix}$$

- wird das QR-Verfahren mit tridiagonalem $A^{(0)}$ gestartet, so sind alle $A^{(k)}$ tridiagonal und der Aufwand für die Zerlegung $A^{(k)} = Q^{(k)} R^{(k)}$ und die Matrixmultiplikation $R^{(k)} Q^{(k)}$ ist jeweils $\mathcal{O}(n)$
- starten wir die für unsere Beispielmatrix A die QR-Iteration mit der Hessenberg-Matrix H statt mit A , so benötigen wir für eine ähnliche Genauigkeit deutlich weniger Iterationen
- für Hessenberg-Matrizen gibt es folgendes notwendige und hinreichende Konvergenzkriterium

! Satz 182 (Parlett)

$H \in \mathbb{R}^{n \times n}$ sei eine irreduzible Hessenbergmatrix, d.h. $h_{i+1,i} \neq 0$, $i = 1, \dots, n-1$. Der QR-Algorithmus angewandt auf H erzeugt eine Folge von Hessenberg-Matrizen $H^{(k)}$, die genau dann gegen eine obere Block-Dreiecks-Matrix T der reellen Schur-Zerlegung von H konvergiert, wenn in jeder Menge von Eigenwerten von H mit gleichem Betrag höchstens zwei mit gerader und zwei mit ungerader algebraischer Vielfachheit vorhanden sind.

- in der Praxis geht man wie folgt vor:
 - man transformiert A zunächst auf Hessenbergform H
 - ist H reduzibel, dann ist es eine Block-Matrix

$$H = \begin{pmatrix} H_{11} & H_{12} & \dots & H_{1m} \\ 0 & H_{22} & \dots & H_{2m} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & H_{mm} \end{pmatrix},$$

wobei die H_{kk} irreduzible Hessenberg-Matrizen sind

- damit sind die Eigenwerte von H gerade die Eigenwerte der H_{kk}
- somit kann die Berechnung der Eigenwerte der reduziblen Hessenberg-Matrix H auf das Berechnen der Eigenwerte der m irreduziblen Hessenberg-Matrizen H_{kk} zurück geführt werden

34.2 Shift-Technik, Deflation

- mit der Hessenberg-Transformation haben wir den Aufwand reduziert, aber die Konvergenzgeschwindigkeit kann noch verbessert werden
- dazu benutzen wir den folgenden Zusammenhang:
 - zerlege statt A die Matrix

$$A - \mu I = QR$$

- dann hat

$$\tilde{A} = RQ + \mu I$$

wegen $Q^T = Q^{-1}$ dieselben Eigenwerte wie A , denn

$$\begin{aligned}
 \tilde{A} &= RQ + \mu I \\
 &= Q^T Q R Q + \mu I \\
 &= Q^T (A - \mu I) Q + \mu I \\
 &= Q^T A Q - \mu Q^T Q + \mu I \\
 &= Q^T A Q
 \end{aligned}$$

- damit bauen wir in das QR-Verfahren einen *Shift* ein, d.h. wir ändern die Iteration in

$$\begin{aligned}
 A^{(k)} - \mu^{(k)} I &= Q_k R_k \\
 A^{(k+1)} &= R_k Q_k + \mu^{(k)} I
 \end{aligned}$$

wobei $\mu^{(k)}$ geeignet zu bestimmen ist

- der zusätzliche Aufwand für den Shift ist gering ($\sim 2n$)
- wie wählt man $\mu^{(k)}$ geschickt?
- wir untersuchen ausschließlich den Fall, dass A nur reelle Eigenwerte besitzt (zur Behandlung komplexer Eigenwerte sind zusätzliche Überlegungen notwendig):
 - betrachte den 2×2 Block

$$B^{(k)} := \begin{pmatrix} a_{n-1,n-1}^{(k)} & a_{n-1,n}^{(k)} \\ a_{nn-1}^{(k)} & a_{nn}^{(k)} \end{pmatrix}$$

und bestimme dessen Eigenwerte γ_1, γ_2

- als $\mu^{(k)}$ benutzen wir den Eigenwert γ_i , für den $|\gamma_i - a_{nn}^{(k)}|$ minimal wird
- mit dieser Wahl konvergiert $a_{nn-1}^{(k)}$ schnell gegen 0 und $a_{nn}^{(k)}$ gegen einen Eigenwert λ_n von A , d.h. ist $l > k$ groß genug, so ist

$$A^{(l)} = \begin{pmatrix} * & \dots & \dots & \dots & * \\ * & \ddots & & & \vdots \\ & \ddots & \ddots & & \vdots \\ & & * & * & * \\ & & & \varepsilon & \tilde{\lambda}_n \end{pmatrix}$$

wobei ε klein und $\tilde{\lambda}_n$ eine gute Näherung für λ_n ist

- damit haben wir einen Eigenwert approximiert und die restlichen Eigenwerte sind ausschließlich durch die ersten $n - 1$ Spalten und Zeilen der Matrix bestimmt
- deshalb streichen wir die letzte Zeile und Spalte und wiederholen das Verfahren auf der verbliebenen Restmatrix
- diese Reduktionstechnik nennt sich *Deflation*
- insgesamt erhalten wir folgendes Verfahren:
 - setze $k = 0$
 - bestimme aus A die Hessenbergmatrix $A^{(0)}$ und setze $Q^{(0)} = I$
 - wiederhole bis $n = 1$:
 - wiederhole bis $|a_{nn-1}^{(k)}| \leq \varepsilon$:
 - bestimme $\mu^{(k)}$ aus $B^{(k)}$
 - zerlege $A^{(k)} - \mu^{(k)} I$ in $Q_k R_k$ mit Givens-Rotationen
 - $A^{(k+1)} = R_k Q_k + \mu^{(k)} I$
 - $Q^{(k+1)} = Q^{(k)} Q_k$
 - $k := k + 1$

- $n := n - 1$
- die Diagonalelemente von $A^{(k)}$ sind Approximationen der Eigenwerte
- die Spalten von $Q^{(k)}$ sind Approximationen der zugehörigen Eigenvektoren
- die Implementierung in dieser Form zählt zu den Standardverfahren und findet sich so auch in den einschlägigen Bibliotheken (z.B. LAPACK)

Zusammenfassung

- Eigenwerte und Eigenvektoren beschreiben grundlegende Eigenschaften linearer Abbildungen (Stabilität, Schwingungen, Hauptachsen) und treten in vielen Anwendungen auf
- Spektraleigenschaften und Normalformen (Jordan, Schur) für komplexwertige und reellwertige Matrizen, Unterschiede in Diagonalisierbarkeit und Struktur
- Positiv definite Matrizen
- Iterative Verfahren für dominante Eigenpaare, Vektoriteration
- Zielgenaues Finden einzelner Eigenwerte: inverse Iteration und Rayleigh-Quotienten-Iteration (schnelle lokale Konvergenz)
- Mehrere Eigenwerte gleichzeitig: Unterraum-Iteration
- Jacobi-Verfahren: rotationsbasiertes Verfahren speziell für symmetrische Matrizen (Diagonalisieren durch Drehungen)
- QR-Iteration: Standardverfahren zur Eigenwertberechnung, Hessenberg-Transformation und Beschleunigung via Shift und Deflation.
- Verallgemeinerte Eigenwertprobleme: statt $Ax = \lambda x$ betrachtet man das Paar (A, B)

$$Ax = \lambda Bx$$

und numerische Lösung über (verallgemeinerte) Hessenberg-Transformation und QZ-Iteration.

Teil VII

Nullstellenprobleme

- bisher haben wir immer lineare Gleichungssysteme $Ax = b$ gelöst, d.h. Nullstellen

$$f(x) = 0, \quad f : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad f(x) = Ax - b$$

gesucht

- jetzt betrachten wir $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ nichtlinear
- Problem:
 - Eindeutigkeit ($f(x) = x^2 - 1$)
 - keine direkten Verfahren möglich (Nullstellen von Polynomen mit Grad ≥ 5)
- betrachten deshalb ausschließlich iterative Verfahren
- Konvergenz:
 - f linear: $\forall x_0$ muss $x_k \xrightarrow{k \rightarrow \infty} x$ gelten (global)
 - für f nichtlinear können wir dies nicht mehr erwarten und müssen uns mit Konvergenz für $x_0 \in X \subset \mathbb{R}^n$ begnügen (*lokale Konvergenz*)
- Vorgehensweise in den nächsten Abschnitten:
 - betrachten vorwiegend den eindimensionalen Fall $f : \mathbb{R} \rightarrow \mathbb{R}$
 - starten mit intuitiver geometrischer Herleitung einiger Verfahren
 - definieren allgemeinen Iterationsprozess für $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ (Erweiterung des linearen Falls), beweisen ein hinreichendes Konvergenzkriterium inklusive Fehlerabschätzungen und untersuchen die Konvergenzgeschwindigkeit
 - wenden die allgemeinen Aussagen auf das wichtigste Verfahren (Newton-Verfahren) an, um den eindimensionalen Fall detailliert zu untersuchen
 - zum Abschluss gehen wir noch auf eine Verallgemeinerung für $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ ein

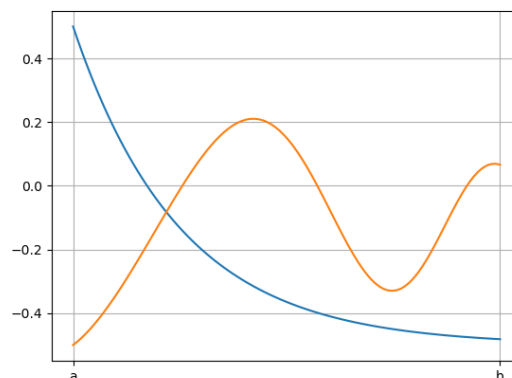
Einschließungsverfahren

37.1 Bisektion

- wir setzen $f : \mathbb{R} \rightarrow \mathbb{R}$ stetig voraus
- ist $a_0, b_0 \in \mathbb{R}$ mit $f(a_0) < 0, f(b_0) > 0$ oder $f(a_0) > 0, f(b_0) < 0$, also

$$f(a_0) \cdot f(b_0) < 0,$$

dann muss in $[a_0, b_0]$ mindestens eine Nullstelle von f liegen



- betrachte $x_0 = \frac{a_0 + b_0}{2}$, d.h. die Intervallmitte
- ist $f(x_0) \cdot f(a_0) < 0$ dann liegt sicher eine Nullstelle in $[a_0, x_0]$ sonst in $[x_0, b_0]$
- setze also

$$\begin{aligned} a_1 &= a_0, & b_1 &= x_0 & \text{falls } f(x_0) \cdot f(a_0) < 0, \\ a_1 &= x_0, & b_1 &= b_0 & \text{sonst} \end{aligned}$$

und wiederhole dieselben Überlegungen für $[a_1, b_1]$

- halbiere Intervalle fortlaufend
- erhalten Folge x_k die gegen eine Nullstelle konvergiert
- man iteriert so lange, bis die Intervalllänge $|b_k - a_k|$ oder $|f(x_k)|$ hinreichend klein ist
- Konvergenzgeschwindigkeit:
 - $x \in [a_0, b_0]$, $x_0 = \frac{a_0 + b_0}{2}$ und somit

$$|e_0| = |x - x_0| \leq \frac{b_0 - a_0}{2}$$

- analog gilt

$$|e_k| = |x - x_k| \leq \frac{b_k - a_k}{2}$$

- wegen der Intervallhalbierung haben wir

$$b_k - a_k = \frac{b_{k-1} - a_{k-1}}{2} = \frac{b_{k-2} - a_{k-2}}{4} = \dots = \frac{b_0 - a_0}{2^k}$$

und damit

$$|e_k| \leq \frac{b_0 - a_0}{2^{k+1}}$$

bzw. für den relativen Fehler

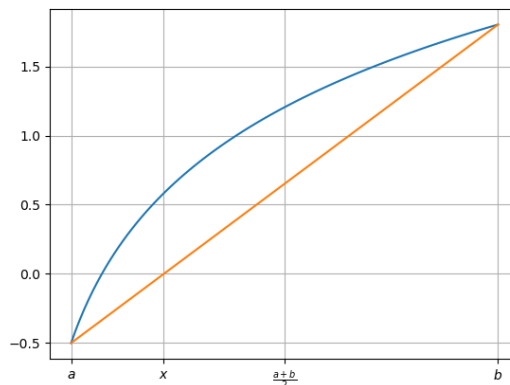
$$|e_{k,r}| \leq \frac{b_0 - a_0}{|x|} \cdot \frac{1}{2^{k+1}}$$

- Vorteil:
 - f muss nur stetig sein
 - Fehler einfach abschätzbar
 - einfach zu implementieren
- Nachteil:
 - ein Anfangsintervall $[a_0, b_0]$ mit $f(a_0) \cdot f(b_0) < 0$ muss bekannt sein
 - die Konvergenz ist relativ langsam, 10 Schritte liefern etwa 3 Dezimalstellen, denn

$$\frac{|e_{k+10,r}|}{|e_{k,r}|} \approx \frac{\frac{b_0-a_0}{|x|} \cdot \frac{1}{2^{k+11}}}{\frac{b_0-a_0}{|x|} \cdot \frac{1}{2^{k+1}}} = \frac{1}{2^{10}} = \frac{1}{1024}$$

37.2 Regula-Falsi-Verfahren

- wir modifizieren das Bisektions-Verfahren, um die Konvergenzgeschwindigkeit zu erhöhen:
 - benutze für x_k nicht einfach Intervallmitte $x_k = \frac{a_k+b_k}{2}$, sondern zusätzliche Information:
 - verbinde $(a_k, f(a_k)), (b_k, f(b_k))$ durch eine Gerade s



- x_k sei jetzt die Nullstelle der Geraden s :

$$s(x) = f(a_k) + \frac{f(b_k) - f(a_k)}{b_k - a_k} \cdot (x - a_k)$$

$$s(x_k) = 0 \quad \Leftrightarrow \quad x_k = a_k - \frac{b_k - a_k}{f(b_k) - f(a_k)} f(a_k) = \frac{a_k f(b_k) - b_k f(a_k)}{f(b_k) - f(a_k)}$$

- zusammen erhalten wir das *Regula-Falsi-Verfahren*:

- wähle a_0, b_0 so, dass $f(a_0) \cdot f(b_0) < 0$
- berechne je Schritt:

$$x_k = \frac{a_k f(b_k) - b_k f(a_k)}{f(b_k) - f(a_k)} = a_k - \frac{f(a_k)}{\frac{f(b_k) - f(a_k)}{b_k - a_k}}$$

und setze

$$\begin{aligned} a_{k+1} &= a_k, & b_{k+1} &= x_k & \text{falls } f(x_k) \cdot f(a_k) < 0, \\ a_{k+1} &= x_k, & b_{k+1} &= b_k & \text{sonst} \end{aligned}$$

- Konvergenzgeschwindigkeit:
 - unter einigen zusätzlichen Voraussetzungen an f kann man zeigen, dass

$$|e_{k+1}| \leq \alpha |e_k|, \quad \alpha < 1$$

gilt

- für $\alpha < \frac{1}{2}$ wäre das Regula-Falsi-Verfahren schneller als das Bisektions-Verfahren
- dies trifft nicht immer zu, wie das folgende Beispiel zeigt

Nicht-Einschließungsverfahren

38.1 Sekanten-Verfahren

- wir ändern das Regula-Falsi-Verfahren geringfügig ab:
 - seien x_{-1}, x_0 gegeben
 - bestimme für $k = 0, 1, \dots$ die neue Näherung x_{k+1} als Nullstelle der Geraden durch die Punkte

$$(x_{k-1}, f(x_{k-1})), \quad (x_k, f(x_k)),$$

also

$$x_{k+1} = x_k - \frac{f(x_k)}{\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}}$$

- damit erhalten wir das *Sekanten-Verfahren*
- das Sekanten-Verfahren ist *kein* Einschließungsverfahren
- konvergiert das Sekanten-Verfahren, so ist es in der Regel schneller als das Bisektions- und Regula-Falsi-Verfahren
- das Sekanten-Verfahren ist nur lokal konvergent, d.h. es konvergiert nur, wenn die Startwerte genügend nahe an der gesuchten Nullstelle liegen
- für die Konvergenzgeschwindigkeit des Sekanten-Verfahrens erhalten wir folgende Aussage

! Satz 196

Ist f zweimal stetig differenzierbar in einer hinreichend kleinen Umgebung X der gesuchten Nullstelle x . Dann konvergiert das Sekanten-Verfahren für alle $x_{-1}, x_0 \in X$ und es gibt eine Konstante $c > 0$ so dass

$$|e_k| \leq c |e_{k-1}|^{\frac{1+\sqrt{5}}{2}}$$

gilt für $k \rightarrow \infty$

38.2 Taylor-Reihen-basierte Verfahren, Newton-Verfahren

- ist $f(x) = 0$ und f hinreichend oft differenzierbar, x_k gegeben, so liefert eine Taylor-Entwicklung von f um x_k

$$f(y) = f(x_k) + (y - x_k)f'(x_k) + \frac{(y - x_k)^2}{2}f''(x_k) + \dots$$

bzw. für $y = x$

$$0 = f(x) = f(x_k) + (x - x_k)f'(x_k) + \frac{(x - x_k)^2}{2}f''(x_k) + \dots$$

- Idee:
 - breche Taylor-Reihe nach einigen Termen ab
 - die abgebrochene Reihe ist ein Polynom, welches die Funktion f in einer Umgebung von x_k approximiert
 - bestimme eine Nullstelle $\tilde{x}$ dieses Polynoms und setze

$$x_{k+1} = \tilde{x}$$

- das Newton-Verfahren ist nur lokal konvergent, d.h. es konvergiert nur, wenn die Startwerte genügend nahe an der gesuchten Nullstelle liegen
- es ist das gebräuchlichste Verfahren für nichtlineare Nullstellenprobleme
- genauere Aussagen über das Konvergenzverhalten leiten wir aus den allgemeinen Betrachtungen des nächsten Kapitels her

Stationäre Iterationen, Banachscher Fixpunktsatz

- wir betrachten allgemeine stationäre Iterationen $x_{k+1} = \Phi(x_k)$ und leiten Aussagen über deren Konvergenz und Konvergenzgeschwindigkeit her
- wir hatten bei linearen Gleichungssystemen schon einmal eine spezielle Variante dieser Verfahren betrachtet
 - um $f(x) = b - Ax = 0$ zu lösen haben wir es in ein Fixpunktproblem

$$x = \Phi(x) = Bx + d$$

umgewandelt, so dass

$$x = \Phi(x) \Leftrightarrow b - Ax = 0$$

- $x = \Phi(x)$ haben wir dann versucht, iterativ mit $x_{k+1} = \Phi(x_k)$ zu lösen
- für den Fehler $e_k = x - x_k$ war

$$e_{k+1} = x - x_{k+1} = \Phi(x) - \Phi(x_k) = B(x - x_k) = Be_k$$

- $x_k \xrightarrow{k \rightarrow \infty} x$ für beliebiges x_0 genau dann, wenn $\rho(B) < 1$
- ist $\rho(B) < 1$ dann gibt es eine von einer Vektornorm $\|\cdot\|_V$ Matrixnorm $\|\cdot\|_M$, so dass

$$\|B\|_M = \|\Phi'(x)\|_M < 1$$

- wir wollen jetzt allgemeiner $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ nichtlinear betrachten und analog vorgehen
- wir wandeln Nullstellenproblem so in Fixpunktproblem um, dass gilt

$$x = \Phi(x) \Rightarrow f(x) = 0$$

- wegen der Mehrdeutigkeit von $f(x) = 0$ muss die Umkehrung ($\Leftarrow$) nicht notwendig gelten, d.h. es kann auch nur eine Nullstelle Fixpunkt sein
- da f nichtlinear ist, hat Φ in der Regel nicht mehr die einfache Gestalt $Bx + d$ wie im linearen Fall
- wir betrachten wieder zu gegebenem x_0 die *Picard-Iteration*

$$x_{k+1} = \Phi(x_k)$$

- aus der Fixpunkteigenschaft folgt für den Fehler

$$e_{k+1} = x - x_{k+1} = \Phi(x) - \Phi(x_k)$$

- wie verhält sich e_k für $k \rightarrow \infty$?
- im linearen Fall $\Phi(y) = By + d$ hatten wir gesehen, dass $\|\Phi'(x)\|_M = \|B\|_M < 1$ die wesentliche Voraussetzung für Konvergenz war
- ist f nichtlinear, so ist Φ in der Regel auch nichtlinear
 - wir betrachten den eindimensionalen Fall $\Phi : \mathbb{R} \rightarrow \mathbb{R}$
 - ist Φ differenzierbar, dann folgt aus dem Mittelwertsatz

$$e_{k+1} = \Phi(x) - \Phi(x_k) = \Phi'(\xi_k)(x - x_k) = \Phi'(\xi_k)e_k$$

mit $\xi_k \in \text{co}(x, x_k)$ und somit

$$|e_{k+1}| \leq |\Phi'(\xi_k)| \cdot |e_k|$$

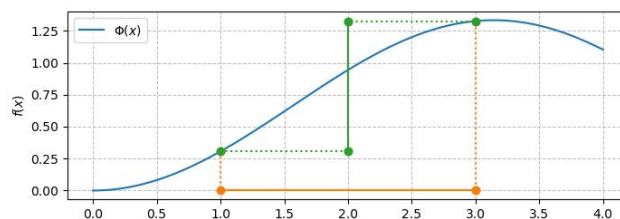
- für $|\Phi'(\xi_k)| \leq \alpha < 1 \forall k$, α unabhängig von k , folgt

$$|e_k| \xrightarrow{k \rightarrow \infty} 0$$

- geometrisch folgt aus $|\Phi'| < 1$

$$|\Phi(x) - \Phi(y)| < |x - y|,$$

d.h. Φ kontrahiert



- im Gegensatz zum linearen Fall ist hier $\Phi'(\xi)$ nicht konstant für alle $\xi \in \mathbb{R}$, d.h. Φ kann in einer Teilmenge des $\mathbb{R}$ kontrahieren, in einer anderen Teilmenge dagegen nicht
- jetzt verallgemeinern wir diese Überlegungen auf $\mathbb{R}^n$

Definition 205

- $x_k, x \in \mathbb{R}^n$, x_k konvergiert gegen x in $\|\cdot\|$ falls $\lim_{k \rightarrow \infty} \|x_k - x\| = 0$
- Φ heißt *Kontraktion* bezüglich $\|\cdot\|$ auf $X \subset \mathbb{R}^n$ falls

$$\Phi : X \rightarrow X$$

und ein $\alpha < 1$ unabhängig von x, y existiert mit

$$\|\Phi(x) - \Phi(y)\| \leq \alpha \|x - y\| \quad \forall x, y \in X$$

- die Iteration $x_{k+1} = \Phi(x_k)$ heißt *lokal konvergent* gegen $x \in X \subset \mathbb{R}^n$ falls

$$\lim_{k \rightarrow \infty} x_k = x \quad \forall x_0 \in X$$

- der folgende Satz liefert ein hinreichendes Konvergenzkriterium

! Fixpunktsatz von Banach 207

Sei $X \subset \mathbb{R}^n$ abgeschlossen, $\Phi : X \rightarrow X$ eine Kontraktion auf X mit

$$\|\Phi(x) - \Phi(y)\| \leq \alpha \|x - y\| \quad \forall x, y \in X,$$

$\alpha < 1$ unabhängig von x, y . Dann hat Φ genau einen Fixpunkt $x = \Phi(x)$ in X . Die Picard-Iteration

$$x_{k+1} = \Phi(x_k)$$

konvergiert $\forall x_0 \in X$ gegen x . Es gelten die Abschätzungen

$$\|x - x_k\| \leq \frac{\alpha^k}{1 - \alpha} \|x_1 - x_0\| \quad \text{a-priori Abschätzung}$$

$$\|x - x_k\| \leq \frac{\alpha}{1 - \alpha} \|x_k - x_{k-1}\| \quad \text{a-posteriori Abschätzung}$$

🔊 Definition 213

Die Iteration $x_{k+1} = \Phi(x_k)$ heißt

- *linear konvergent* falls es eine Konstante $0 \leq c < 1$ unabhängig von k gibt mit

$$\|e_{k+1}\| \leq c \|e_k\| \quad \forall k$$

- *konvergent mit Ordnung m* falls es eine Konstante $0 \leq c$ unabhängig von k gibt mit

$$\lim_{k \rightarrow \infty} \|e_k\| = 0 \quad \text{und} \quad \|e_{k+1}\| \leq c \|e_k\|^m \quad \forall k$$

- ist Φ stetig differenzierbar, so kann man allgemeine Aussagen über die Konvergenzordnung treffen
- wir beschränken uns hier auf den eindimensionalen Fall $\Phi : \mathbb{R} \rightarrow \mathbb{R}$
- die Ergebnisse lassen sich entsprechend auf den höherdimensionalen Fall verallgemeinern

! Satz 215

Ist $m \geq 2$, x ein Fixpunkt von $\Phi : \mathbb{R} \rightarrow \mathbb{R}$ und Φ m -mal stetig differenzierbar in x mit

$$\Phi'(x) = \Phi''(x) = \dots = \Phi^{(m-1)}(x) = 0,$$

dann gibt es ein abgeschlossenes Intervall $X = [x - \delta, x + \delta]$, $\delta > 0$, so dass die Iteration

$$x_{k+1} = \Phi(x_k)$$

für alle $x_0 \in X$ mindestens mit Ordnung m konvergiert.

- analog zeigt man das folgende Ergebnis

! Satz 216

Ist $m \geq 2$, x ein Fixpunkt von $\Phi : \mathbb{R} \rightarrow \mathbb{R}$ und Φ $(m + 1)$ -mal stetig differenzierbar in x mit

$$\Phi'(x) = \Phi''(x) = \dots = \Phi^{(m-1)}(x) = 0, \quad \Phi^{(m)}(x) \neq 0,$$

dann gibt es ein abgeschlossenes Intervall $X = [x - \delta, x + \delta]$, $\delta > 0$, so dass die Iteration

$$x_{k+1} = \Phi(x_k)$$

für alle $x_0 \in X$ *genau* mit Ordnung m konvergiert

- schließlich brauchen wir noch ein Abbruchkriterium für die Iteration

- $|x_{k+1} - x_k| < \varepsilon$:

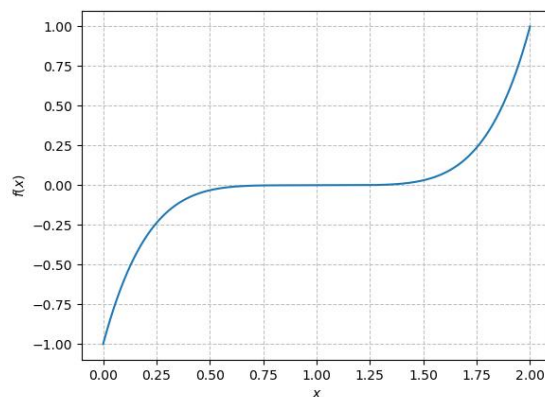
- die a-posteriori Abschätzung des Satzes von Banach liefert

$$|e_{k+1}| \leq \frac{\alpha}{1 - \alpha} |x_{k+1} - x_k|$$

- ist α nicht zu nahe bei 1, so liefert $|x_{k+1} - x_k|$ eine brauchbare Abschätzung für $|e_{k+1}|$

- $|f(x_k)| < \varepsilon$:

- problematisch bei schlecht konditionierten Problemen



- selbst wenn $|x_k - x|$ „groß“ ist, kann $|f(x_k)|$ schon sehr klein sein

- Schätzung $\hat{\alpha}$ von α :

- nach dem Satz von Banach gilt $|e_{k+1}| \leq \frac{\alpha}{1 - \alpha} |x_{k+1} - x_k|$

- α ist in der Regel unbekannt bzw. mühsam zu bestimmen

- versuche α durch $\hat{\alpha}$ zu schätzen

- ist Φ differenzierbar, so gilt $|\Phi'(\xi)| \leq \alpha$

- setze

$$\alpha \approx |\Phi'(x_k)| \approx \left| \frac{\Phi(x_k) - \Phi(x_{k-1})}{x_k - x_{k-1}} \right| = \left| \frac{x_{k+1} - x_k}{x_k - x_{k-1}} \right| =: \hat{\alpha}$$

- benutze dann

$$|\hat{e}_{k+1}| := \frac{\hat{\alpha}}{1 - \hat{\alpha}} |x_{k+1} - x_k|$$

als Schätzer für $|e_{k+1}|$

- breche ab, wenn $|\hat{e}_k| \leq \varepsilon$

- $\hat{\alpha}$ wird in jedem Schritt neu berechnet, weshalb diese Art der Fehlerschätzung oft genauere Ergebnisse liefert als die a-posteriori Formel mit festem α

Newton-Verfahren

- wir benutzen die Ergebnisse von oben um das Newton-Verfahren

$$x_{k+1} = \Phi(x_k), \quad \Phi(y) = y - \frac{f(y)}{f'(y)}$$

genauer zu untersuchen

- für den Fixpunkt $x = \Phi(x)$ gilt $f(x) = 0$
- um eine möglichst hohe Konvergenzgeschwindigkeit zu erhalten, sollen nach dem Satz über Konvergenzordnungen möglichst viele Ableitungen von Φ im Fixpunkt $x = \Phi(x)$ verschwinden:

$$\begin{aligned} \Phi'(x) &= \left(x - \frac{f(x)}{f'(x)}\right)' \\ &= 1 - \frac{(f'(x))^2 - f(x)f''(x)}{(f'(x))^2} \\ &= \frac{f(x)f''(x)}{(f'(x))^2} \\ \Phi''(x) &= \frac{(f'(x))^3 f''(x) + f(x)(f'(x))^2 f'''(x) - 2f(x)f'(x)(f''(x))^2}{(f'(x))^4} \end{aligned}$$

- ist $f'(x) \neq 0$ (damit Division keine Probleme macht), so ist $\Phi'(x) = 0$ und $\Phi''(x) = \frac{f''(x)}{f'(x)}$ im allgemeinen ungleich 0
- damit können wir den Satz über Konvergenzordnungen mit $m = 2$ anwenden und erhalten

! Satz 218

$f : \mathbb{R} \rightarrow \mathbb{R}$ sei zweimal stetig differenzierbar mit $f(x) = 0$, $f'(x) \neq 0$, d.h. x ist eine *einfache* Nullstelle von f . Liegt x_0 nahe genug an x , dann konvergiert das Newton-Verfahren *quadratisch* Konvergenz x .

- was passiert bei *mehrfachen Nullstellen*, d.h. wenn $f(x) = 0$, $f'(x) = 0$?
- $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$, weshalb für x_k nahe bei x die Quotientenbildung Probleme machen könnte
- betrachten jetzt den allgemeinen Fall, dass x eine q -fache Nullstelle, $q > 1$, von f ist:

$$f(x) = 0, \quad f'(x) = 0, \quad \dots, \quad f^{(q-1)}(x) = 0, \quad f^{(q)}(x) \neq 0$$

- damit kann f geschrieben werden (Taylorentwicklung um x) als

$$f(y) = (y - x)^q g(y), \quad g(x) \neq 0$$

- für die Ableitung von f erhalten wir

$$f'(y) = q(y - x)^{q-1} g(y) + (y - x)^q g'(y)$$

und damit

$$\frac{f(y)}{f'(y)} = \frac{(y - x)^q g(y)}{q(y - x)^{q-1} g(y) + (y - x)^q g'(y)} = \frac{(y - x) g(y)}{q g(y) + (y - x) g'(y)}$$

- da $g(x) \neq 0$ ist folgt

$$\lim_{y \rightarrow x} \frac{f(y)}{f'(y)} = 0,$$

so dass der Quotient im Newton-Verfahren auch im Grenzwert existiert

- berechnet man mit der Darstellung von f auch

$$\Phi'(y) = \left(y - \frac{f(y)}{f'(y)} \right)' = \frac{f(y) f''(y)}{(f'(y))^2}$$

so erhält man

$$\Phi'(x) = \lim_{y \rightarrow x} \Phi'(y) = 1 - \frac{1}{q}, \quad q > 1$$

- damit ist $\Phi'(x) > 0$ und wir können Satz über Konvergenzordnungen nicht anwenden

! Satz 220

Für mehrfache Nullstellen ist das Newton-Verfahren wohldefiniert und konvergiert nur *linear* in einer Umgebung von x .

- wie kann man bei mehrfachen Nullstellen die Konvergenz beschleunigen
- sei x eine q -fache Nullstelle von f , d.h.

$$f(x) = f'(x) = \dots = f^{(q-1)}(x) = 0, \quad f^{(q)}(x) \neq 0$$

- Variante 1:

- setze $\tilde{\Phi}(y) = y - q \frac{f(y)}{f'(y)}$ und iteriere $x_{k+1} = \tilde{\Phi}(x_k)$
- durch Ausrechnen erhält man $\tilde{\Phi}'(x) = 0$ und damit also (mindestens) quadratische Konvergenz
- der Nachteil dieser Variante ist, dass die Vielfachheit q bekannt sein muss

- Variante 2

- da x eine q -fache Nullstelle ist, erhalten wir aus einer Taylorentwicklung von f

$$f(y) = \frac{1}{q!} (y - x)^q f^{(q)}(\xi), \quad f'(y) = \frac{1}{(q-1)!} (y - x)^{q-1} f^{(q)}(\eta)$$

mit $\xi, \eta \in \text{co}(x, y)$

- da $f^{(q)}(x) \neq 0$ ist folgt für y nahe x , dass $f^{(q)}(\xi) \neq 0$ und $f^{(q)}(\eta) \neq 0$ und damit

$$\tilde{f}(y) = \frac{f(y)}{f'(y)} = \frac{1}{q}(y-x) \frac{f^{(q)}(\xi)}{f^{(q)}(\eta)},$$

d.h. x ist einfache Nullstelle von $\tilde{f} = \frac{f}{f'}$

- wird das Newton-Verfahren auf $\tilde{f}$ angewandt, so konvergiert es quadratisch
- die Vielfachheit q wird hier also nicht explizit benutzt
- Nachteil ist die kompliziertere Iterationsvorschrift

$$\Phi(y) = y - \frac{\tilde{f}(y)}{\tilde{f}'(y)} = y - \frac{\frac{f(y)}{f'(y)}}{\frac{(f'(y))^2 - f(y)f''(y)}{(f'(y))^2}} = y - \frac{f(y)f'(y)}{(f'(y))^2 - f(y)f''(y)}$$

in der pro Schritt neben f und f' auch noch f'' auszuwerten ist

- bisher haben wir das Newton-Verfahren nur auf skalare Nullstellenprobleme $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = 0$ angewandt
- was ändert sich für Systeme, d.h. für $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$?
- auch hier leiten wir das Verfahren über die Taylorentwicklung

$$f(y) = f(x_k) + (Df)(x_k)(y - x_k) + R$$

her, wobei

$$(Df)(x_k) = \begin{pmatrix} \partial_1 f_1(x_k) & \dots & \partial_n f_1(x_k) \\ \vdots & & \vdots \\ \partial_1 f_n(x_k) & \dots & \partial_n f_n(x_k) \end{pmatrix}$$

die *Jacobi-Matrix* von f in x_k ist

- durch Weglassen des Restgliedes R erhält man

$$0 = f(x_k) + (Df)(x_k)(x_{k+1} - x_k)$$

und damit für reguläres $Df(x_k)$

$$x_{k+1} = x_k - ((Df)(x_k))^{-1} f(x_k)$$

- analog zum skalaren Fall zeigt man

! Satz 222

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}^n, f(x) = 0$ und $(Df)(x)$ regulär (was einer einfachen Nullstelle im eindimensionalen entspricht), so konvergiert das Newton-Verfahren lokal quadratisch.

- das Analogon zur mehrfachen Nullstelle ist der Fall $Df(x)$ singulär
- die Behandlung ist etwas komplizierter als im skalaren Fall

Zusammenfassung

- Problemstellung: Nullstellen einer Funktion finden, also x^* mit

$$f(x^*) = 0$$

- Einschließungsverfahren: Start mit Intervall $[a, b]$ und Garantie, dass die Nullstelle darin bleibt (robust, aber oft langsam)
- Bisektion: halbiert $[a, b]$ wiederholt, sichere Konvergenz
- Regula Falsi: ersetzt Halbierung durch Schnittpunkt der Sekante mit der x -Achse, behält aber ein einschließendes Intervall
- ITP-Verfahren: kombiniert Bisektion und Sekantenidee, um Sicherheit und Geschwindigkeit zu verbinden und Nachteile von Regula Falsi zu vermeiden
- Nicht-Einschließungsverfahren: arbeiten mit Näherungen $x^{(k)}$ ohne garantiertes Intervall (oft schneller, kann aber scheitern)
- Sekanten-Verfahren: Newton-ähnlich ohne Ableitung, nutzt die beiden letzten Näherungen zur Steigungsapproximation
- Newton-Verfahren: nutzt Ableitung

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

mit guter Konvergenz nahe der Nullstelle

- Konvergenztheorie mit Fixpunkt-Satz von Banach
- Polynom-Nullstellen (Umformung als Eigenwertproblem, Durand–Kerner, Aberth–Ehrlich)
- Intervallarithmetik, Intervall-Newton zur garantierten Einschließung

Teil VIII

Interpolation

- gegeben sind Punkte $(x_0, y_0), \dots, (x_n, y_n)$, z.B. Messwerte
- gesucht ist ein funktionaler Zusammenhang zwischen x_i und y_i , d.h. eine Funktion so dass

$$f(x_i) = y_i \quad \forall i = 0, \dots, n,$$

d.h. die Funktion f *interpoliert* die Werte (x_i, y_i)

- ist f gefunden, so wird es in der Regel an *Zwischenstellen* $x \neq x_i$ ausgewertet
- f sollte möglichst einfach handhabbar sein, z.B.
 - ein Polynom

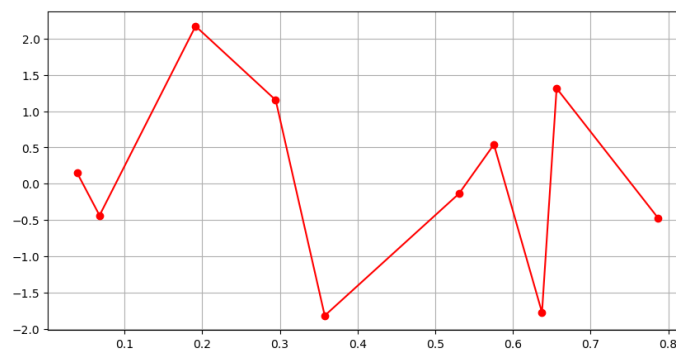
$$f(x) = c_0 + c_1 x + \dots + c_m x^m$$

- eine Überlagerung trigonometrischer Funktionen

$$f(x) = c_0 + c_1 \cos(x) + c_2 \cos(2x) + \dots + c_{n-1} \cos((n-1)x)$$

- ein stückweises Polynom wie

$$f|_{[x_i, x_{i+1}]}(x) = c_i + \tilde{c}_i x$$



- in allen Fällen sind Koeffizienten so zu bestimmen, dass $f(x_i) = y_i$
- grundsätzliche Fragen sind:

- geht das immer eindeutig
- wie muss f aussehen (z.B. Polynomgrad, Anzahl der cos-Terme)
- wie wertet man f effizient an Zwischenstellen aus

Polynom Interpolation

43.1 Grundlagen

- gegeben seien die $n + 1$ Punkte (x_i, y_i) , $i = 0, \dots, n$, d.h. $n + 1$ Interpolationsbedingungen sind zu erfüllen
- damit brauchen wir ein Polynom mit mindestens $n + 1$ Koeffizienten, also mit Grad n :

$$f(x) = p_n(x) = c_0 + c_1x + \dots + c_nx^n$$

- kann man $c_0, \dots, c_n$ so bestimmen, dass $p_n(x_i) = y_i \forall i$?

43.2 Vandermonde System

- aus der Interpolationsbedingung $p_n(x_i) = y_i$ erhalten wir

$$\begin{array}{rclcl} y_0 & = & p_n(x_0) & = & c_0 + c_1x_0 + \dots + c_nx_0^n \\ \vdots & & \vdots & & \vdots \\ y_n & = & p_n(x_n) & = & c_0 + c_1x_n + \dots + c_nx_n^n \end{array}$$

also ein lineares Gleichungssystem für die Koeffizienten $c_0, \dots, c_n$ in der Form

$$\underbrace{\begin{pmatrix} 1 & x_0 & \dots & x_0^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \dots & x_n^n \end{pmatrix}}_A \underbrace{\begin{pmatrix} c_0 \\ \vdots \\ c_n \end{pmatrix}}_c = \underbrace{\begin{pmatrix} y_0 \\ \vdots \\ y_n \end{pmatrix}}_y$$

- ist $A \in \mathbb{R}^{(n+1) \times (n+1)}$ regulär, so können wir für beliebige y_i -Werte immer ein interpolierendes Polynom vom Grad $\leq n$ finden

! Satz 225

Ist $x_i \neq x_j \forall i \neq j$ so gibt es genau ein Polynom $p_n(x)$ mit Grad $\leq n$, das $(x_0, y_0), \dots, (x_n, y_n)$ interpoliert.

- praktische Durchführung:

- stelle das lineare Gleichungssystem $Ac = y$ auf und löse es
- damit sind die Polynomkoeffizienten $c_0, \dots, c_n$ bekannt und $p_n(x) = c_0 + c_1x + \dots + c_nx^n$ erfüllt $p_n(x_i) = y_i, i = 0, \dots, n$
- die Auswertung an Zwischenstellen erfolgt
 - direkt über $p_n(x) = c_0 + c_1x + \dots + c_nx^n$, d.h. berechne x^i, c_ix^i und summiere, was teuer und rundungsfehleranfällig ist
 - mit dem *Horner-Schema*:
 - $p_n(x)$ wird wie folgt umgeformt

$$\begin{aligned}
 p_n(x) &= c_0 + c_1x + \dots + c_nx^n \\
 &= c_0 + x(c_1 + c_2x + \dots + c_nx^{n-1}) \\
 &= c_0 + x(c_1 + x(c_2 + \dots + c_nx^{n-2})) \\
 &\vdots \\
 &= c_0 + x\left(c_1 + x\left(c_2 + x\left(\dots + x\left(c_{n-1} + xc_n\right)\dots\right)\right)\right)
 \end{aligned}$$

- berechne schrittweise von innen nach außen

$$\begin{aligned}
 q_0 &= c_n \\
 q_1 &= c_{n-1} + x \cdot q_0 = c_{n-1} + xc_n \\
 q_2 &= c_{n-2} + x \cdot q_1 = c_{n-2} + x(c_{n-1} + xc_n) = c_{n-2} + xc_{n-1} + x^2c_n
 \end{aligned}$$

also

$$q_0 = c_n, \quad q_k = c_{n-k} + xq_{k-1}, \quad k = 1, \dots, n$$

und damit $p_n(x) = q_n$

43.3 Lagrange Polynome

- Polynome mit reellen Koeffizienten und Grad $\leq n$

$$p(x) = c_0 + c_1x + \dots + c_nx^n$$

bilden einen $(n+1)$ -dimensionalen reellen Vektorraum Π_n , wobei die Koeffizienten c_j die Koordinaten von p bezüglich der Basen (Basispolynome)

$$M_j(x) = x^j$$

sind (*Monome*)

- aus der linearen Algebra wissen wir, dass man in einem Vektorraum unterschiedliche Basen verwenden kann und somit unterschiedliche Darstellungen des selben Elements des Vektorraums erhält
- Idee:
 - wir benutzen statt Monomen eine andere Basis für das Interpolationspolynom
 - wir konstruieren die Basis, so dass das Interpolationsproblem einfacher zu lösen ist, z.B. ohne Lösung eines linearen Gleichungssystems $Ac = b$
- wir betrachten zu $x_0, \dots, x_n$ die *Lagrange-Polynome*

$$L_j(x) = \frac{x - x_0}{x_j - x_0} \cdot \dots \cdot \frac{x - x_{j-1}}{x_j - x_{j-1}} \cdot \frac{x - x_{j+1}}{x_j - x_{j+1}} \cdot \dots \cdot \frac{x - x_n}{x_j - x_n}$$

- L_j ist wohldefiniert falls $x_i \neq x_j \forall i \neq j$ und hat Grad n
- außerdem gilt

$$L_j(x_i) = \begin{cases} 1 & \text{für } i = j \\ 0 & \text{sonst} \end{cases}$$

- die Lagrange Polynome $L_j(x)$, $j = 0, \dots, n$, bilden eine Basis des Vektorraums Π_n :
 - wir müssen zeigen, dass jedes beliebige Polynom der Form

$$p(x) = \sum_{j=0}^n d_j x^j$$

sich als Linearkombination der L_j schreiben lässt

- mit $y_i = p(x_i)$ definieren wir das Polynom

$$q(x) = \sum_{j=0}^n y_j L_j(x)$$

vom Grad n mit

$$q(x_i) = \sum_{j=0}^n y_j L_j(x_i) = \sum_{j=0}^n y_j \delta_{i,j} = y_i = p(x_i), \quad i = 0, \dots, n$$

- damit interpolieren die Polynome p, q die selben Werte (x_i, y_i)
- nach oben ist das Interpolationspolynom zu (x_i, y_i) eindeutig bestimmt, also gilt $p(x) = q(x)$
- mit den Lagrange-Polynomen L_j können wir das Interpolationspolynom nun ganz einfach ermitteln
- setzt man

$$p_n(x) = \sum_{j=0}^n y_j L_j(x)$$

dann ist der Grad von $p_n(x)$ höchstens n und es gilt

$$p_n(x_i) = \sum_{j=0}^n y_j L_j(x_i) = \sum_{j=0}^n y_j \delta_{i,j} = y_i \quad \forall i = 0, \dots, n,$$

d.h. $p_n(x)$ ist das gesuchte Interpolationspolynom

- damit erhalten wir folgendes Verfahren:
 - berechne aus den x_i die Lagrange Polynome $L_0, \dots, L_n$
 - berechne mit den y_i dann

$$p_n(x) = \sum_{j=0}^n y_j L_j(x)$$

43.4 Newton-Darstellung, dividierte Differenzen

- was ist zu berücksichtigen, wenn ein weiterer Datenpunkt (x_{n+1}, y_{n+1}) hinzu kommt?
 - Vandermonde:
 - die Basis $M_j(x) = x^j$, $j = 0, \dots, n$, wird um $M_{n+1}(x) = x^{n+1}$ erweitert
 - die Dimension von $Ac = y$ wächst auf $n + 2$
 - Lagrange:
 - statt Basen

$$L_j(x) = \prod_{k=0, k \neq j}^n \frac{x - x_k}{x_j - x_k}, \quad j = 0, \dots, n,$$

haben wir jetzt

$$\tilde{L}_j(x) = \prod_{k=0, k \neq j}^{n+1} \frac{x - x_k}{x_j - x_k} = L_j(x) \frac{x - x_{n+1}}{x_j - x_{n+1}}, \quad j = 0, \dots, n,$$

sowie

$$\tilde{L}_{n+1}(x) = \prod_{k=0}^n \frac{x - x_k}{x_{n+1} - x_k},$$

- es müssen also alle „alten“ Basen L_j verändert und ein neues Basiselement $\tilde{L}_{n+1}$ generiert werden
- geht das mit einer anderen Basis von Π_n eventuell besser?
- wir betrachten die Polynome

$$N_j(x) = \prod_{k=0}^{j-1} (x - x_k), \quad j = 0, \dots, n,$$

also

$$\begin{aligned} N_0(x) &= 1, \\ N_1(x) &= x - x_0 \\ N_2(x) &= (x - x_0)(x - x_1) \\ &\vdots \end{aligned}$$

- wegen

$$N_j(x) = x^j + \sum_{k=0}^{j-1} n_{jk} x^k$$

ist der Grad von N_j genau j , weshalb die Polynome N_j wieder eine Basis von Π_n bilden

- für N_j gilt

$$N_j(x_i) = \prod_{k=0}^{j-1} (x_i - x_k) = \begin{cases} 0 & \text{für } i < j \\ \prod_{k=0}^{j-1} (x_i - x_k) & \text{für } j \leq i \end{cases}$$

- versuchen wir nun das Interpolationspolynom in der Form

$$p_n(x) = \sum_{j=0}^n c_j N_j(x)$$

zu bestimmen, so erhalten wir

$$p_n(x_i) = \sum_{j=0}^n c_j N_j(x_i) = \sum_{j=0}^i c_j N_j(x_i) = y_i, \quad i = 1, \dots, n,$$

also das lineare Gleichungssystem $Lc = y$ mit unterer Dreiecksmatrix

$$L = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 1 & x_1 - x_0 & 0 & & 0 \\ 1 & x_2 - x_0 & (x_2 - x_0)(x_2 - x_1) & \ddots & \vdots \\ \vdots & \vdots & \vdots & \ddots & 0 \\ 1 & x_n - x_0 & (x_n - x_0)(x_n - x_1) & \dots & \prod_{k=0}^{n-1} (x_n - x_k) \end{pmatrix}$$

- kommt nun ein weiterer Datenpunkt (x_{n+1}, y_{n+1}) hinzu, dann
 - bleiben die bisherigen Basiselemente $N_0, \dots, N_n$ unverändert und werden durch N_{n+1} zu einer Basis von Π_{n+1} ergänzt
 - wird das Gleichungssystem $Lc = y$ um eine zusätzliche Zeile unten und eine zusätzliche Spalte rechts erweitert, d.h. alle bisherigen Einträge bleiben unverändert
- wir müssen also die „alten“ Basiselemente nicht mehr anfassen, allerdings müssen wir (ähnlich wie bei Vandermonde) ein neues Gleichungssystem lösen
- wie wir gleich sehen werden, ist das aufgrund der speziellen Struktur von L mit wenig Aufwand zu bewerkstelligen (vorwärts einsetzen)
- wir gehen dazu schrittweise vor, d.h. wir starten zunächst mit dem Datenpunkt (x_0, y_0) und nehmen dann immer einen weiteren Punkt dazu
- wir betrachten den ersten Datenpunkt (x_0, y_0) :

- das Interpolationspolynom p_0 dazu ist

$$p_0(x) = c_0 N_0(x) = c_0$$

und aus $Lc = y$ folgt in diesem Fall

$$c_0 = y_0 =: d_{00},$$

d.h.

$$p_0(x) = d_{00}$$

- wir nehmen jetzt (x_1, y_1) dazu:

- wir erhalten

$$p_1(x) = c_0 N_0(x) + c_1 N_1(x) = c_0 + c_1(x - x_0)$$

und damit für $Lc = y$

$$c_0 = y_0 =: d_{00}$$

$$\begin{pmatrix} 1 & 0 \\ 1 & x_1 - x_0 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \end{pmatrix},$$

- $c_0 = y_0 = d_{00}$ ist aus dem ersten Schritt schon bekannt
- für c_1 erhalten wir mit $d_{10} := y_1$

$$c_1 = \frac{y_1 - c_0}{x_1 - x_0} = \frac{d_{10} - d_{00}}{x_1 - x_0} =: d_{11}$$

also

$$p_1(x) = d_{00} N_0(x) + d_{11} N_1(x) = d_{00} + d_{11}(x - x_0)$$

bzw.

$$p_1(x) = p_0(x) + d_{11} N_1(x)$$

- analog folgt bei der Hinzunahme von (x_2, y_2) :
 - das Interpolationspolynom ist

$$\begin{aligned} p_2(x) &= c_0 N_0(x) + c_1 N_1(x) + c_2 N_2(x) \\ &= c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) \end{aligned}$$

und für $Lc = y$ erhalten wir

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & x_1 - x_0 & 0 \\ 1 & x_2 - x_0 & (x_2 - x_0)(x_2 - x_1) \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix},$$

- $c_0 = d_{00}$, $c_1 = d_{11}$ sind aus den vorherigen Schritten schon bekannt
- für c_2 erhalten wir die Gleichung

$$c_0 + c_1(x_2 - x_0) + c_2(x_2 - x_0)(x_2 - x_1) = y_2$$

bzw.

$$\underbrace{c_0 + c_1(x_1 - x_0)}_{y_1} + c_1(x_2 - x_1) + c_2(x_2 - x_0)(x_2 - x_1) = y_2$$

- mit $d_{20} := y_2$, $d_{10} = y_1$ erhalten wir

$$c_1 + c_2(x_2 - x_0) = \frac{y_2 - y_1}{x_2 - x_1} = \frac{d_{20} - d_{10}}{x_2 - x_1} =: d_{21}$$

- wegen $c_1 = d_{11}$ ist

$$c_2 = \frac{d_{21} - c_1}{x_2 - x_0} = \frac{d_{21} - d_{11}}{x_2 - x_0} =: d_{22}$$

und damit

$$\begin{aligned} p_2(x) &= p_1(x) + d_{22} N_2(x) \\ &= d_{00} N_0(x) + d_{11} N_1(x) + d_{22} N_2(x) \\ &= d_{00} + d_{11}(x - x_0) + d_{22}(x - x_0)(x - x_1) \end{aligned}$$

- per Induktion kann man dies für weitere Punkte fortsetzen
- die Lösung des Gleichungssystems $Lc = y$ erhalten wir also durch folgende Schritte:
 - für $i = 0, \dots, n$:
 - setze $d_{i0} = y_i$
 - berechne für $j = 0, \dots, i$

$$d_{ij} = \frac{d_{ij-1} - d_{i-1j-1}}{x_i - x_{i-j}}$$

- $c_i = d_{ii}$
- bei gegebenen Punkten (x_i, y_i) bauen wir damit punktweise (Zeile für Zeile) folgendes Schema auf

$$\begin{array}{c|ccc} x_0 & y_0 & = & d_{00} \\ & & & \searrow \\ x_1 & y_1 & = & d_{10} \rightarrow d_{11} \\ & & & \searrow \\ x_2 & y_2 & = & d_{20} \rightarrow d_{21} \rightarrow d_{22} \\ & \vdots & & \vdots \rightarrow \vdots \rightarrow \ddots \end{array}$$

- wird ein Punkt hinzugefügt, wird das Schema lediglich um eine neue Zeile erweitert
- für das Interpolationspolynom erhalten wir

$$\begin{aligned}
 p_n(x) &= \sum_{j=0}^n d_{jj} N_j(x) \\
 &= d_{00} + d_{11}(x - x_0) + d_{22}(x - x_0)(x - x_1) + \dots + d_{nn}(x - x_0)(x - x_1) \cdot \dots \cdot (x - x_{n-1})
 \end{aligned}$$

- damit haben wir folgendes Resultat erzielt

! Satz 229

Das Interpolationspolynom in *Newton-Darstellung* lautet

$$p_n(x) = d_{00} + d_{11}(x - x_0) + \dots + d_{nn}(x - x_0) \cdot \dots \cdot (x - x_{n-1}),$$

mit *dividierten Differenzen* d_{ij}

$$\begin{aligned}
 d_{i0} &= y_i & i &= 0, \dots, n \\
 d_{ij} &= \frac{d_{ij-1} - d_{i-1j-1}}{x_i - x_{i-j}} & i &= 0, \dots, n, \quad j = 0, \dots, i.
 \end{aligned}$$

- zur Auswertung schreiben wir $p_n(x)$ als

$$p_n(x) = d_{00} + (x - x_0)(d_{11} + (x - x_1)(d_{22} + \dots (x - x_{n-1})d_{nn}) \dots)$$

und erhalten analog zum Horner-Schema

$$q_0 = d_{nn}, \quad q_k = d_{n-k, n-k} + (x - x_{n-k})q_{k-1} \quad k = 1, \dots, n$$

und $p_n(x) = q_n$

Interpolation mit stückweise polynomialen Funktionen

44.1 Polynom-Splines

- Probleme bei Interpolationspolynom:
 - bei steigender Zahl von Punkten wächst auch der Polynomgrad
 - bei höherem Polynomgrad kann es zu sehr eigenartigem Verhalten des Interpolationspolynoms an Zwischenstellen kommen (Überschwinger, Instabilitäten)
- in konkreten Anwendungen können tausende von Punkte auftreten, so dass die Polynominterpolation unbrauchbar ist
- Lösung:
 - begrenze Polynomgrad (z.B. ≤ 3)
 - stücke mehrere Polynome aneinander
 - fordere glatte Übergänge zwischen einzelnen Polynomstücken

Definition 234

- für $x_0 < x_1 < \dots < x_n$ heißt $\Omega_n = \{x_0, \dots, x_n\}$ Unterteilung des Intervalls $[x_0, x_n]$.
- eine Funktion s_k mit
 - $s_k|_{[x_i, x_{i+1}]}$ ist Polynom vom Grad $\leq k$
 - s_k ist C^{k-1} auf $[x_0, x_n]$
 heißt *Polynomspline* der Ordnung k zu Ω_n .
- $S_k(\Omega_n)$ ist die Menge aller Polynom-Splines der Ordnung k zur Unterteilung $\Omega_n = \{x_0, \dots, x_n\}$.

- $S_k(\Omega_n)$ ist ein Vektorraum
- mit der folgenden Überlegung kann man sich die Dimension des Raums ableiten:
 - auf den n Teilintervallen $[x_i, x_{i+1}]$, $i = 0, \dots, n-1$, haben wir jeweils ein Polynom vom Grad $\leq k$, d.h. ein Polynom mit $k+1$ Koeffizienten

- an den inneren Punkten $x_i, i = 1, \dots, n-1$, müssen k Stetigkeitsbedingungen erfüllt sein
- insgesamt haben wir also $n(k+1)$ freie Parameter und $(n-1)k$ (unabhängige) Nebenbedingungen, so dass wir eine Dimension $n(k+1) - (n-1)k = n+k$ erwarten
- genauere Untersuchungen liefern das folgende Resultat

! Satz 236

In $S_k(\Omega_n)$ gibt es genau $k+n$ linear unabhängige Splines, d.h. $\dim(S_k(\Omega_n)) = k+n$.

44.2 Kubische Polynom-Splines

- in der Praxis werden häufig kubische Polynomsplines ($k=3$) benutzt, d.h.
 - $s_3|_{[x_i, x_{i+1}]} = a_i + b_i x + c_i x^2 + d_i x^3$
 - s_3 ist zweimal stetig differenzierbar auf $[x_0, x_n]$, d.h. insbesondere an den Nahtstellen $x_i, i = 1, \dots, n-1$

44.2.1 Abschlussbedingungen

- wir versuchen jetzt die a_i, b_i, c_i, d_i so zu bestimmen, dass s_3 die Punkte (x_i, y_i) interpoliert, d.h. $s_3(x_i) = y_i, i = 0, \dots, n$
- insgesamt haben wir n Intervalle $[x_i, x_{i+1}], i = 0, \dots, n-1$, so dass $4n$ Koeffizienten a_i, b_i, c_i, d_i zu bestimmen sind
- als Bedingungsgleichungen erhalten wir:
 - Interpolation:

$$s_3(x_i) = y_i, \quad i = 0, \dots, n \quad \Rightarrow \quad n+1 \text{ Stück}$$

- die Glattheitsbedingungen an den inneren Punkten $x_i, i = 1, \dots, n-1$ liefern

$$\begin{aligned} C^0 : \quad s_3(x_i)_- &= s_3(x_i)_+ & \Rightarrow & \quad n-1 \text{ Stück} \\ C^1 : \quad s'_3(x_i)_- &= s'_3(x_i)_+ & \Rightarrow & \quad n-1 \text{ Stück} \\ C^2 : \quad s''_3(x_i)_- &= s''_3(x_i)_+ & \Rightarrow & \quad n-1 \text{ Stück} \end{aligned}$$

- damit haben wir $4n-2$ Gleichungen für $4n$ Unbekannte und brauchen zwei zusätzliche *Abschlussbedingungen*
- die gängigsten Abschlussbedingungen sind:
 - *natürlicher Spline*:

$$s''_3(x_0) = s''_3(x_n) = 0,$$

d.h. die Krümmung am Rand verschwindet

- *periodischer Spline*:

$$s'_3(x_0) = s'_3(x_n), \quad s''_3(x_0) = s''_3(x_n)$$

- *Hermite Spline*:

$$s_3'(x_0) = u, \quad s_3'(x_n) = v,$$

u, v gegeben

- wir untersuchen den natürlichen Spline genauer, die anderen Fälle lassen sich analog bearbeiten

44.2.2 Direkte Berechnung

- wir schreiben zunächst die Bedingungen in Gleichungsform

- auf $[x_i, x_{i+1}]$, $i = 0, \dots, n-1$ gilt

$$s_3(x) = a_i + b_i x + c_i x^2 + d_i x^3$$

$$s_3'(x) = b_i + 2c_i x + 3d_i x^2$$

$$s_3''(x) = 2c_i + 6d_i x$$

- Interpolationseigenschaft und Stetigkeit sind erfüllt, falls

$$\begin{aligned} a_i + b_i x_i + c_i x_i^2 + d_i x_i^3 &= y_i \\ a_i + b_i x_{i+1} + c_i x_{i+1}^2 + d_i x_{i+1}^3 &= y_{i+1} \end{aligned}, \quad i = 0, \dots, n-1$$

- Stetigkeit der ersten Ableitung liefert

$$b_{i-1} + 2c_{i-1}x_i + 3d_{i-1}x_i^2 = b_i + 2c_i x_i + 3d_i x_i^2, \quad i = 1, \dots, n-1$$

und Stetigkeit der zweiten Ableitung bedeutet

$$2c_{i-1} + 6d_{i-1}x_i = 2c_i + 6d_i x_i, \quad i = 1, \dots, n-1$$

- schließlich erhalten wir aus der natürlichen Abschlussbedingung

$$2c_0 + 6d_0 x_0 = 0$$

$$2c_{n-1} + 6d_{n-1} x_n = 0$$

- alle Gleichungen sind linear in a_i, b_i, c_i, d_i , so dass wir insgesamt ein lineares Gleichungssystem der Dimension $4n$ erhalten
- für $x_i \neq x_j \forall i \neq j$ ist dies stets eindeutig lösbar (auch bei periodischen bzw. Hermite Abschlussbedingungen)

Satz 238

Seien (x_i, y_i) , $i = 0, \dots, n$ gegeben, $x_0 < x_1 < \dots < x_n$. Dann gibt es genau einen kubischen Spline s_3 mit natürlicher (periodischer/Hermite) Abschlussbedingung.

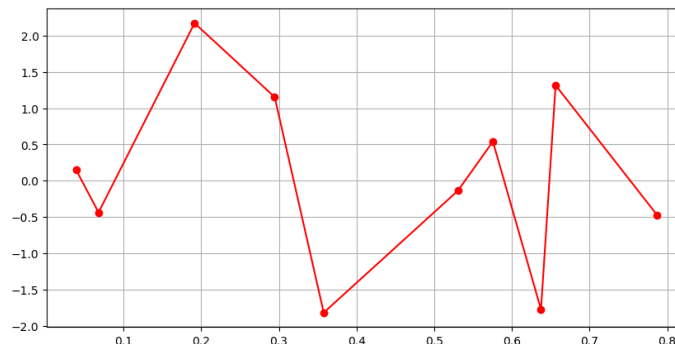
- s_3 zeigt einen sehr glatten Verlauf zwischen den Stützstellen und ist von f kaum zu unterscheiden

44.2.3 Berechnung über Momente

- die direkte Berechnung aus dem letzten Abschnitt ist nicht sehr effizient:
 - die Dimension des linearen Gleichungssystems ist $4n$
 - die Systemmatrix hat wenig Struktur
- Idee:
 - s_3 ist C^2 und ist stückweise polynomial mit Grad ≤ 3
 - s_3'' ist damit stetig und stückweise linear mit $s_3''(x_0) = s_3''(x_n) = 0$ bei natürlichen Randbedingungen, also ein Polygonzug
 - ein Polygonzug ist bestimmt durch die Funktionswerte an den Stützstellen, d.h. durch

$$\beta_i = s_3''(x_i), \quad i = 0, \dots, n$$

- wir versuchen also zunächst die *Momente* β_i (und damit den Polygonzug s_3'') zu bestimmen und integrieren dann zweimal um schließlich s_3 zu erhalten



- wir betrachten s_3 auf $[x_i, x_{i+1}]$, $i \in \{0, \dots, n-1\}$ fest
 - s_3 ist ein Polynom mit Grad ≤ 3

$$s_3(x_i) = y_i, \quad s_3(x_{i+1}) = y_{i+1}$$

- s_3' ist ein Polynom mit Grad ≤ 2

$$s_3'(x_i) =: \alpha_i, \quad s_3'(x_{i+1}) =: \alpha_{i+1}$$

- s_3'' ist ein Polynom vom Grad ≤ 1 (also eine Gerade), mit

$$s_3''(x_i) = \beta_i, \quad s_3''(x_{i+1}) = \beta_{i+1}$$

so dass

$$s_3''(x) = \beta_i + \frac{x - x_i}{x_{i+1} - x_i}(\beta_{i+1} - \beta_i) = \beta_i + \frac{\beta_{i+1} - \beta_i}{h_i}(x - x_i)$$

mit $h_i = x_{i+1} - x_i$

- Integration von $s_3''(z)$ über $[x_i, x]$ liefert

$$\begin{aligned} \int_{x_i}^x s_3''(z) dz &= \int_{x_i}^x \left(\beta_i + \frac{\beta_{i+1} - \beta_i}{h_i}(z - x_i) \right) dz \\ \Leftrightarrow s_3'(x) - s_3'(x_i) &= \beta_i(x - x_i) + \frac{\beta_{i+1} - \beta_i}{2h_i}(x - x_i)^2 \end{aligned}$$

und somit

$$s_3'(x) = \alpha_i + \beta_i(x - x_i) + \frac{\beta_{i+1} - \beta_i}{2h_i}(x - x_i)^2$$

- integrieren wir $s_3'(z)$ über $[x_i, x]$ so folgt

$$s_3(x) = y_i + \alpha_i(x - x_i) + \frac{\beta_i}{2}(x - x_i)^2 + \frac{\beta_{i+1} - \beta_i}{6h_i}(x - x_i)^3$$

- mit $x = x_{i+1}$ erhalten wir

$$\begin{aligned}
 \alpha_{i+1} &= s'_3(x_{i+1}) \\
 &= \alpha_i + \beta_i(x_{i+1} - x_i) + \frac{\beta_{i+1} - \beta_i}{2h_i}(x_{i+1} - x_i)^2 \\
 &= \alpha_i + \beta_i h_i + \frac{\beta_{i+1} - \beta_i}{2h_i} h_i^2
 \end{aligned}$$

und damit

$$\alpha_{i+1} - \alpha_i = \frac{\beta_i + \beta_{i+1}}{2} h_i$$

- mit $x = x_{i+1}$ folgt

$$\begin{aligned}
 y_{i+1} &= s_3(x_{i+1}) \\
 &= y_i + \alpha_i(x_{i+1} - x_i) + \frac{\beta_i}{2}(x_{i+1} - x_i)^2 + \frac{\beta_{i+1} - \beta_i}{6h_i}(x_{i+1} - x_i)^3 \\
 &= y_i + \alpha_i h_i + \frac{\beta_i}{2} h_i^2 + \frac{\beta_{i+1} - \beta_i}{6} h_i^2
 \end{aligned}$$

so dass

$$\frac{y_{i+1} - y_i}{h_i} = \alpha_i + \frac{\beta_i}{3} h_i + \frac{\beta_{i+1}}{6} h_i$$

- betrachten wir jetzt einen inneren Punkt $x_i, i \in \{1, \dots, n-1\}$ und das Intervall $[x_i, x_{i+1}]$ bzw. $[x_{i-1}, x_i]$, so folgt

$$\begin{aligned}
 \frac{y_{i+1} - y_i}{h_i} &= \alpha_i + \frac{\beta_i}{3} h_i + \frac{\beta_{i+1}}{6} h_i \\
 \frac{y_i - y_{i-1}}{h_{i-1}} &= \alpha_{i-1} + \frac{\beta_{i-1}}{3} h_{i-1} + \frac{\beta_i}{6} h_{i-1}
 \end{aligned}$$

und damit

$$\begin{aligned}
 \frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} &= \alpha_i - \alpha_{i-1} + \frac{\beta_i}{3} h_i + \frac{\beta_{i+1}}{6} h_i - \frac{\beta_{i-1}}{3} h_{i-1} - \frac{\beta_i}{6} h_{i-1} \\
 &= \frac{\beta_{i-1} + \beta_i}{2} h_{i-1} + \frac{\beta_i}{3} h_i + \frac{\beta_{i+1}}{6} h_i - \frac{\beta_{i-1}}{3} h_{i-1} - \frac{\beta_i}{6} h_{i-1} \\
 &= \frac{h_{i-1}}{6} \beta_{i-1} + \frac{h_{i-1} + h_i}{3} \beta_i + \frac{h_i}{6} \beta_{i+1}
 \end{aligned}$$

bzw.

$$h_{i-1} \beta_{i-1} + 2(h_{i-1} + h_i) \beta_i + h_i \beta_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right)$$

für $i = 1, \dots, n-1$

- somit habe wir also $n-1$ lineare Gleichungen für die $n+1$ Unbekannten $\beta_0, \beta_1, \dots, \beta_{n-1}, \beta_n$
- aus der natürlichen Abschlussbedingung wissen wir aber, dass $\beta_0 = \beta_n = 0$
- zusammen bleibt also ein lineares Gleichungssystem der Dimension $n-1$ übrig

$$\underbrace{\begin{pmatrix} 2(h_0 + h_1) & h_1 & & & \\ h_1 & 2(h_1 + h_2) & h_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & h_{n-2} & \\ & & & h_{n-2} & 2(h_{n-2} + h_{n-1}) \end{pmatrix}}_{=:A} \underbrace{\begin{pmatrix} \beta_1 \\ \vdots \\ \vdots \\ \beta_{n-1} \end{pmatrix}}_{\beta} = \underbrace{\begin{pmatrix} \gamma_1 \\ \vdots \\ \vdots \\ \gamma_{n-1} \end{pmatrix}}_{\gamma}$$

mit

$$\gamma_i = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right), \quad h_i = x_{i+1} - x_i$$

- A ist streng diagonaldominant und damit regulär, so dass die β_i eindeutig bestimmt sind
- damit kennen wir die Momente β_i , d.h. s_3''
- wie erhält man s_3 ?
- nach oben gilt:

$$s_3|_{[x_i, x_{i+1}]}(x) = y_i + \alpha_i(x - x_i) + \frac{\beta_i}{2}(x - x_i)^2 + \frac{\beta_{i+1} - \beta_i}{6h_i}(x - x_i)^3$$

- bis auf die α_i sind alle Koeffizienten bekannt
- die α_i können nun berechnet werden:

$$\alpha_i = \frac{y_{i+1} - y_i}{h_i} - \frac{1}{3}\beta_i h_i - \frac{1}{6}\beta_{i+1} h_i$$

- zusammengefasst erhalten wir folgendes Verfahren zur Bestimmung eines natürlichen, kubischen Splines s_3 :
 - berechne aus x_i die $h_i = x_{i+1} - x_i$ und stelle

$$A = \begin{pmatrix} 2(h_0 + h_1) & h_1 & & & \\ h_1 & 2(h_1 + h_2) & h_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & h_{n-2} \\ & & & h_{n-2} & 2(h_{n-2} + h_{n-1}) \end{pmatrix} \in \mathbb{R}^{(n-1) \times (n-1)}$$

auf

- A ist symmetrisch, tridiagonal und streng diagonaldominant, also sehr einfach mit direkten oder iterativen Lösern zu bearbeiten
- berechne aus y_i, h_i die rechte Seite $\gamma = (\gamma_1, \dots, \gamma_{n-1})^T$,

$$\gamma_i = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right)$$

- löse $A\beta = \gamma$ und setze $\beta_0 = \beta_n = 0$
- berechne α_i durch

$$\alpha_i = \frac{y_{i+1} - y_i}{h_i} - \frac{1}{3}\beta_i h_i - \frac{1}{6}\beta_{i+1} h_i, \quad i = 0, \dots, n-1$$

- auf $[x_i, x_{i+1}]$ ist $s_3(x)$ dann gegeben durch

$$y_i + \alpha_i(x - x_i) + \frac{\beta_i}{2}(x - x_i)^2 + \frac{\beta_{i+1} - \beta_i}{6h_i}(x - x_i)^3$$

- Auswertung erfolgt dann z.B. mit dem Horner-Schema (analog Newton-Polynom)

Interpolierende Kurven

- bisher haben wir zu Datenpunkten

$$(x_i, y_i), \quad x_i, y_i \in \mathbb{R}$$

Funktionen $f : \mathbb{R} \rightarrow \mathbb{R}$ gesucht (z.B. Polynome oder Splines) mit

$$f(x_i) = y_i \quad \forall i$$

- damit das funktioniert, haben wir immer angenommen, dass die x_i paarweise verschieden sind
- was können wir tun, wenn das nicht mehr der Fall ist?
- statt einfacher Funktionen benutzen wir jetzt Kurven zur Interpolation

Definition 243

Das Bild einer stetigen Abbildung

$$\gamma : \mathbb{R} \supset [a, b] \rightarrow \mathbb{R}^m, \quad t \mapsto \gamma(t)$$

heißt *Kurve* in $\mathbb{R}^m$ mit *Kurvenparameter* $t \in [a, b]$

- unsere neue Aufgabenstellung lautet jetzt:
 - gegeben $(t_i, z_i), t_i \in [a, b], z_i \in \mathbb{R}^m, i = 0, \dots, n$
 - finde $f : [a, b] \rightarrow \mathbb{R}^m$ mit $f(t_i) = z_i \quad \forall i$
- für f werden wir die „einfachen“ Polynome bzw. Splines von oben verwenden
- interpolierende polynomiale Kurven:
 - wir benutzen

$$f(t) = \begin{pmatrix} p_1(t) \\ \vdots \\ p_m(t) \end{pmatrix}$$

wobei die $p_j(t)$ Polynome vom Grad n sind

- wegen

$$\mathbb{R}^m \ni z_i = \begin{pmatrix} z_{i1} \\ \vdots \\ z_{im} \end{pmatrix}$$

folgt aus $f(t_i) = z_i$

$$p_j(t_i) = z_{ij}, \quad i = 0, \dots, n, \quad j = 1, \dots, m,$$

d.h. wir haben m „einfache“ Interpolationspolynome p_j zu den Daten $(t_i, z_{ij}), i = 0, \dots, n$, zu bestimmen

- interpolierende Spline-Kurven:

- wir benutzen

$$f(t) = \begin{pmatrix} s_1(t) \\ \vdots \\ s_m(t) \end{pmatrix}$$

wobei die $s_j(t)$ Splines sind

- aus $f(t_i) = z_i$ folgt

$$s_j(t_i) = z_{ij}, \quad i = 0, \dots, n, \quad j = 1, \dots, m,$$

d.h. wir haben wieder m „einfache“ interpolierende Splines s_j zu den Daten $(t_i, z_{ij}), i = 0, \dots, n$, zu berechnen

Zusammenfassung

- Problemstellung: aus Stützstellen (x_i, y_i) eine Funktion f konstruieren, die die Daten genau trifft, also $f(x_i) = y_i$, und das Ergebnis sinnvoll auswerten
- Polynominterpolation:
 - Polynom p vom Grad $\leq n$ durch $n + 1$ Punkte, typische Fragestellungen sind Existenz/Eindeutigkeit und Fehlerverhalten
 - Vandermonde-System: Bestimmung der Polynomkoeffizienten über ein lineares Gleichungssystem (konzeptionell einfach, numerisch oft ungünstig)
 - Lagrange-Form: Darstellung von p als Summe geeigneter Basis-Polynome
 - Newton-Form und dividierte Differenzen: inkrementelle Darstellung, gut für das Hinzufügen neuer Stützstellen
 - Baryzentrische Form: numerisch stabile und schnelle Auswertung
 - Approximationseigenschaften, Interpolationsfehler, Einfluss der Stützstellenwahl
 - Hermite-Interpolation: nicht nur Funktionswerte, sondern auch Ableitungen vorgeben; Verbindung zu Taylor-Polynomen
 - Bernstein-Polynome: alternative Basis mit guten Formeigenschaften (z. B. Positivität, geometrische Interpretation)
- Stückweise polynomiale Interpolation:
 - statt eines globalen Polynoms lokale Polynome pro Intervall, mit Glattheitsbedingungen
 - Polynom-Splines, kubische Splines, Abschlussbedingungen und verschiedene Rechenwege (direkt, über Momente, über Hermite-/Bernstein-Form)
 - B-Splines: lokale Basis mit guter numerischer Stabilität, Konstruktion und Zusammenhang zu de-Boor-Darstellung
 - Monotone Interpolation: Formtreue (z. B. keine Überschwinger), Parameterwahl und praktische Umsetzung
- Interpolierende Kurven: Erweiterung auf parametrische Kurven (z. B. für Geometrie/CAD)

Teil IX

Approximation

Überblick

- aus Daten und Modellen eine „beste“ Näherung bestimmen, wenn exaktes Lösen nicht möglich oder nicht sinnvoll ist
- Lineare Ausgleichsprobleme: gesuchte Lösung minimiert den Fehler im Sinne kleinster Quadrate, z. B.

$$\min_x \|Ax - b\|_2$$

- Numerische Lösung: stabile Verfahren über QR-Zerlegung sowie iterative Varianten für große Probleme (CGLS)
- Pseudoinverse: beschreibt die Least-Squares-Lösung und den Umgang mit nicht eindeutig lösbaren Systemen
- Singulärwertzerlegung (SVD): zentrales Werkzeug, um Rang, Richtungen und Verstärkung von Fehlern in A zu analysieren
- Schlecht konditionierte Probleme: kleine Datenfehler können große Lösungsfehler erzeugen (Overfitting), erst Regularisierung macht die Aufgabe wieder stabil
- Regularisierungsmethoden: abgeschnittene SVD (TSVD) und Tikhonov-Regularisierung als Standardansätze
- Parameterwahl: Grad der Regularisierung, Gleichgewicht zwischen Datenanpassung und Glättung/Stabilität

Lineare Ausgleichsprobleme

48.1 Grundlagen

- gegeben sind Punkte $(x_i, y_i) \in \mathbb{R}^2, i = 1, \dots, m$
- die Punkte (x_i, y_i) sollen nicht interpoliert, sondern nur durch eine Funktion approximiert werden
- was heißt „nahe“:
- betrachte die Abweichung der Funktionswerte an jeder Stützstelle x_i :

$$e_i = f(x_i) - y_i$$

- summiere die e_i^2 auf:

$$\tilde{e} = \sqrt{\sum_{i=1}^m e_i^2} = \|e\|_2, \quad e = \begin{pmatrix} e_1 \\ \vdots \\ e_m \end{pmatrix}$$

- bestimme f so, dass $\tilde{e}$ minimal wird
- lassen wir statt einer Geraden f ein Polynom vom Grad $n - 1$ zu, so erhalten wir folgende Aufgabenstellung:
 - zu $(x_i, y_i), i = 1, \dots, m$, soll ein Polynom

$$f(x) = p_1 + p_2x + \dots + p_nx^{n-1}$$

gesucht werden, so dass

$$\|e\|_2 = \left\| \begin{pmatrix} f(x_1) - y_1 \\ \vdots \\ f(x_m) - y_m \end{pmatrix} \right\|_2$$

minimal wird

- ist $m = n$, so erhalten wir die klassische Polynominterpolation und damit $\|e\|_2 = 0$
- wir betrachten hier $n \leq m$, d.h. die Anzahl der Punkte ist groß gegenüber der Anzahl der Polynomkoeffizienten (was sinnvoll ist, da bei hohem Polynomgrad Stabilitätsprobleme auftreten)
- wir bringen das Problem jetzt in eine neue Form

$$\begin{aligned}
 \|e\|_2 &= \left\| \begin{pmatrix} f(x_1) - y_1 \\ \vdots \\ f(x_m) - y_m \end{pmatrix} \right\|_2 \\
 &= \left\| \begin{pmatrix} p_1 + p_2 x_1 + \dots + p_n x_1^{n-1} - y_1 \\ \vdots \\ p_1 + p_2 x_m + \dots + p_n x_m^{n-1} - y_m \end{pmatrix} \right\|_2 \\
 &= \left\| \underbrace{\begin{pmatrix} 1 & x_1 & \dots & x_1^{n-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_m & \dots & x_m^{n-1} \end{pmatrix}}_{A \in \mathbb{R}^{n \times m}} \underbrace{\begin{pmatrix} p_1 \\ \vdots \\ p_n \end{pmatrix}}_{p \in \mathbb{R}^n} - \underbrace{\begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix}}_{y \in \mathbb{R}^m} \right\|_2 \\
 &= \|Ap - y\|_2
 \end{aligned}$$

- wir können die Approximationsaufgabe lösen, falls

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2$$

lösbar ist

! Satz 249

Sei $A \in \mathbb{R}^{m \times n}$, $m \geq n$, $\text{rang}(A) = n$ (also maximal), dann hat für jedes $b \in \mathbb{R}^m$ das Minimalproblem

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2$$

genau eine Lösung $\hat{x} \in \mathbb{R}^n$. Die Lösung $\hat{x}$ löst auch die *Normalgleichungen*

$$A^T A \hat{x} = A^T b$$

wobei $A^T A \in \mathbb{R}^{n \times n}$ regulär ist.

- der letzte Satz liefert damit einen ersten Algorithmus zur numerischen Behandlung der Polynomapproximation
 - stelle Normalgleichungssystem $A^T A x = A^T b$ mit

$$A = \begin{pmatrix} 1 & x_1 & \dots & x_1^{n-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_m & \dots & x_m^{n-1} \end{pmatrix}, \quad b = \begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix}, \quad x = \begin{pmatrix} p_1 \\ \vdots \\ p_n \end{pmatrix}$$

auf und löse es ($A^T A$ ist spd, so dass das Cholesky-Verfahren oder iterative Methoden benutzt werden können)

- die Lösung x liefert die Polynomkoeffizienten
- das Approximationsproblem kann wie folgt verallgemeinert werden:
 - statt Polynomen kann man auch Linearkombinationen beliebiger Basisfunktionen $\phi_1, \dots, \phi_n : \mathbb{R} \rightarrow \mathbb{R}$ benutzen
 - suche eine approximierende Funktion f als Linearkombination dieser Basisfunktionen

$$f(x) = p_1 \phi_1(x) + \dots + p_n \phi_n(x)$$

und bestimme die Koeffizienten $p_1, \dots, p_n$ so, dass

$$\left\| \begin{pmatrix} f(x_1) - y_1 \\ \vdots \\ f(x_m) - y_m \end{pmatrix} \right\|_2$$

minimal wird

- mit $\phi_1(x) = 1, \phi_2(x) = x, \dots, \phi_n(x) = x^{n-1}$ erhalten wir daraus das oben untersuchte polynomiale Approximationsproblem
- analog zu oben zeigt man

$$\left\| \begin{pmatrix} f(x_1) - y_1 \\ \vdots \\ f(x_m) - y_m \end{pmatrix} \right\|_2 = \|Ap - y\|_2, \quad A = \begin{pmatrix} \phi_1(x_1) & \dots & \phi_n(x_1) \\ \vdots & & \vdots \\ \phi_1(x_m) & \dots & \phi_n(x_m) \end{pmatrix},$$

d.h. lediglich A hat sich verändert

48.2 QR Zerlegung

- die Normalgleichungen sind für die Numerik wenig geeignet:

$$A^T A = \begin{pmatrix} 1 & x_1 & \dots & x_1^{n-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_m & \dots & x_m^{n-1} \end{pmatrix}^T \begin{pmatrix} 1 & x_1 & \dots & x_1^{n-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_m & \dots & x_m^{n-1} \end{pmatrix} = \left(\sum_k x_k^{i-1} x_k^{j-1} \right)_{i,j=1,\dots,n}$$

- das Aufsummieren der Potenzen ist sehr rundungsfehleranfällig
- in der Praxis zeigt sich, dass eine numerische Lösung von $A^T A x = A^T b$ für größere m, n fast immer unbrauchbar ist
- deshalb suchen wir ein anderes Verfahren um $\|Ax - b\|_2$ zu minimieren
- dazu betrachten wir wieder orthonormale Matrizen Q (vergleiche Givens-Verfahren, Householder-Verfahren)
- mit $\|Qx\|_2 = \|x\|_2$ folgt

$$\|Ax - b\|_2 = \|Q(Ax - b)\|_2$$

und $\|Ax - b\|_2$ wird minimal falls $\|QAx - Qb\|_2$ minimal wird

- wir versuchen $Q = Q_n \cdot \dots \cdot Q_1$, Q_i orthonormal, so zu wählen, dass $\|QAx - Qb\|_2$ einfach zu minimieren ist
- wir benutzen als $Q_1, \dots, Q_n$ Householder-Matrizen um nacheinander die Spalten $1, \dots, n$ von A zu eliminieren, d.h.

$$QA = \underbrace{Q_m \cdot \dots \cdot Q_1 A}_m = \begin{pmatrix} * & \dots & \dots & * \\ 0 & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & * \\ 0 & \dots & 0 & 0 \\ \vdots & & \vdots & \vdots \\ 0 & \dots & 0 & 0 \end{pmatrix} =: \begin{pmatrix} R \\ 0 \end{pmatrix}$$

wobei $R \in \mathbb{R}^{n \times n}$ eine obere Dreiecksmatrix ist, die regulär ist, falls A vollen Rang n hat

- benutzen wir noch

$$Qb = \begin{pmatrix} c \\ d \end{pmatrix}, \quad c \in \mathbb{R}^n, \quad d \in \mathbb{R}^{m-n},$$

so folgt

$$\begin{aligned}\min_{x \in \mathbb{R}^n} \|Ax - b\|_2 &= \min_{x \in \mathbb{R}^n} \|QAx - Qb\|_2 \\ &= \min_{x \in \mathbb{R}^n} \left\| \begin{pmatrix} R \\ 0 \end{pmatrix} x - \begin{pmatrix} c \\ d \end{pmatrix} \right\|_2 \\ &= \min_{x \in \mathbb{R}^n} \sqrt{\|Rx - c\|_2^2 + \|d\|_2^2}\end{aligned}$$

- für die beiden Terme in der Wurzel gilt:
 - $\|d\|_2^2$ ist nicht beeinflussbar, sondern durch b fix gegeben
 - $\|Rx - c\|_2^2$ muss durch x so klein wie möglich gemacht werden
 - da $R \in \mathbb{R}^{n \times n}$ regulär hat $Rx - c = 0$ eine eindeutige Lösung,
 - da R eine obere Dreiecksmatrix ist, kann die Lösung von $Rx = c$ einfach durch rückwärts einsetzen bestimmt werden

48.3 CGLS

- in vielen Anwendungen ist $\|Ax - b\|_2$ für sehr große, dünn besetzte Matrizen A zu minimieren
- eine Lösung mittels Householder-Transformationen ist sehr rechenintensiv ($\approx mn^2$ Operationen für $A \in \mathbb{R}^{m \times n}$)
- betrachten wir also nochmals die Normalgleichungen

$$A^T A x = A^T b$$

- $A^T A$ ist in der Regel dicht besetzt und sehr groß, so dass direkte Verfahren zum Lösen des Gleichungssystems einen ähnlich hohen Aufwand wie die Householder-Transformationen mit sich bringen
- $A^T A$ ist aber positiv definit, so dass einige der iterativen Löser anwendbar sind
- die explizite Berechnung von $A^T A$ sollte vermieden werden (Kondition)
- die Anwendung von $A^T A$ auf einen Vektor v ist aber wegen

$$A^T A v = A^T (A v)$$

leicht mit zwei Matrix-Vektor-Produkten zu bewerkstelligen

- nach diesen Vorüberlegungen formulieren wir das klassische CG-Verfahren

$$\begin{aligned}x_{k+1} &= x_k + \alpha_k p_k \\ r_{k+1} &= r_k - \alpha_k \tilde{A} p_k \quad \alpha_k = \frac{(r_k, r_k)_2}{(p_k, \tilde{A} p_k)_2} \\ p_{k+1} &= r_{k+1} + \beta_k p_k \quad \beta_k = \frac{(r_{k+1}, r_{k+1})_2}{(r_k, r_k)_2}\end{aligned}$$

für die Normalgleichungen

$$\tilde{A} x = \tilde{b}, \quad \tilde{A} = A^T A, \quad \tilde{b} = A^T b$$

so um, dass $A^T A$ nicht explizit berechnet werden muss:

- für das Residuum gilt

$$r_k = \tilde{b} - \tilde{A} x_k = A^T b - A^T A x_k = A^T (b - A x_k) = A^T s_k, \quad s_k = b - A x_k$$

mit

$$s_{k+1} = b - Ax_{k+1} = b - A(x_k + \alpha_k p_k) = s_k - \alpha_k A p_k$$

- für den Nenner von α_k folgt

$$(p_k, \tilde{A} p_k)_2 = (p_k, A^T A p_k)_2 = (A p_k, A p_k)_2$$

- zusammen erhalten wir folgendes Verfahren

Definition 252

Das *CGLS-Verfahren* (Conjugate Gradient Least Squares) ist definiert durch:

- x_0 gegeben, $s_0 = b - Ax_0$, $p_0 = r_0 = A^T s_0$
- wiederhole für $k \geq 0$:

$$x_{k+1} = x_k + \alpha_k p_k$$

$$s_{k+1} = s_k - \alpha_k A p_k \quad \alpha_k = \frac{(r_k, r_k)_2}{(A p_k, A p_k)_2}$$

$$r_{k+1} = A^T s_{k+1}$$

$$p_{k+1} = r_{k+1} + \beta_k p_k \quad \beta_k = \frac{(r_{k+1}, r_{k+1})_2}{(r_k, r_k)_2}$$

Satz 253

Die Iterierten x_k aus dem CGLS-Verfahren minimieren $\|Ax - b\|_2$ über dem affinen Unterraum

$$x_0 + \text{span}(A^T r_0, (A^T A) A^T r_0, \dots, (A^T A)^{k-1} A^T r_0)$$

- in den letzten Abschnitten haben wir für $A \in \mathbb{R}^{m \times n}$, $m \geq n$ mit maximalem Rang $\text{rang}(A) = n$ das Minimalproblem

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2$$

betrachtet

- es hat für jedes $b \in \mathbb{R}^m$ genau eine Lösung $\hat{x} \in \mathbb{R}^n$ (Normalgleichungen)
- wie sieht das für allgemeine A aus, also m, n beliebig und $\text{rang}(A)$ nicht unbedingt maximal?
- der Minimierer ist also nicht eindeutig
- um im allgemeinen Fall Eindeutigkeit zu erhalten muss eine weitere Bedingung in das Minimalproblem eingefügt werden:

$$X = \{z \mid \|Az - b\|_2 \text{ minimal}\}, \quad x = \underset{z \in X}{\operatorname{argmin}} \|z\|_2$$

- unter allen Minimierern von $\|Az - b\|_2$ wird also derjenige ausgesucht, der selbst noch die kleinste Norm besitzt

! Satz 256

Sei $A \in \mathbb{R}^{m \times n}$ beliebig, $b \in \mathbb{R}^m$. Dann gibt es genau eine Lösung $x \in \mathbb{R}^n$ Minimalproblems. Diese hängt linear von b , d.h. es existiert eine eindeutige, von b unabhängige Matrix $A^+ \in \mathbb{R}^{n \times m}$ mit

$$x = A^+b, \quad \forall b \in \mathbb{R}^m.$$

A^+ heißt *Pseudoinverse* zu A .

! Satz 258

Die Pseudoinverse A^+ zu A ist durch die folgenden vier *Penrose-Axiome* festgelegt

$$(A^+A)^T = A^+A, \quad (AA^+)^T = AA^+, \quad A^+AA^+ = A^+, \quad AA^+A = A$$

- A^+b ist also die eindeutige Lösung des Minimalproblems von oben

- wie berechnet man A^+b ?
- mit Hilfe von Householder-Transformationen kann man die Berechnung von Ausgleichsproblemen so erweitern, dass man damit auch die Pseudoinverse bestimmen kann
- es handelt sich dabei um ein direktes (nicht iteratives) Verfahren
- dieses Verfahren wird in der Praxis selten eingesetzt, da es effizientere (iterative) Verfahren gibt, die auf den Ergebnissen des nächsten Abschnitts beruhen

Singulärwertzerlegung

- die Singulärwertzerlegung einer Matrix A steht in engem Zusammenhang mit den hier betrachteten Approximationsaufgaben bzw. der Pseudoinversen A^+

! Satz 260

Sei $A \in \mathbb{R}^{m \times n}$ beliebig. Dann gibt es orthonormale Matrizen $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$, so dass

$$U^T A V = \Sigma$$

mit

$$\Sigma \in \mathbb{R}^{m \times n}, \quad \Sigma = \text{diag}(\sigma_1, \dots, \sigma_p), \quad p = \min(m, n)$$

und

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0.$$

$U^T A V = \Sigma$ bzw. $A = U \Sigma V^T$ heißt *Singulärwertzerlegung* von A mit *Singulärwerten* σ_i .

- im folgenden Satz fassen wir die wichtigsten Eigenschaften der Singulärwertzerlegung zusammen

! Satz 262

Es sei $A = U \Sigma V^T$ eine Singulärwertzerlegung der Matrix $A \in \mathbb{R}^{m \times n}$. Dann gilt:

- $V \Sigma^T U^T$ ist eine Singulärwertzerlegung von A^T , d.h. die Singulärwerte von A und A^T sind identisch
- gilt $\sigma_1 \geq \dots \geq \sigma_r > \sigma_{r+1} = \dots = \sigma_p = 0$ dann ist $\text{rang}(A) = r$
- $\sigma_1^2, \dots, \sigma_p^2$ sind Eigenwerte von $A^T A$ bzw. $A A^T$, alle weiteren Eigenwerte der beiden Matrizen sind gleich 0
- die Spalten von U, V sind die Eigenvektoren von $A A^T$ bzw. $A^T A$
- ist $r = \text{rang}(A)$, $A = U \Sigma V^T$ mit

$$\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0) \in \mathbb{R}^{m \times n}$$

dann ist

$$A^+ = V\Sigma^+U^T, \quad \Sigma^+ = \text{diag}\left(\frac{1}{\sigma_1}, \dots, \frac{1}{\sigma_r}, 0, \dots, 0\right) \in \mathbb{R}^{n \times m}$$

- wie ermittelt man eine Singulärwertzerlegung?
 - nach oben wissen wir, dass $\sigma_1^2, \dots, \sigma_p^2$ die Eigenwerte und die Spalten von U, V die Eigenvektoren von AA^T bzw. A^TA sind
 - im Prinzip kann man die Singulärwertzerlegung also durch Lösen von Eigenwert- bzw. Eigenvektorproblemen für die symmetrischen Matrizen AA^T bzw. A^TA bestimmen
 - ein mögliches Verfahren dafür wäre die QR-Iteration
 - analog zu CGLS möchte man die explizite Berechnung von AA^T, A^TA vermeiden
 - durch geschickte Ausnutzung der Eigenschaften der Singulärwertzerlegung kann man die beiden wesentlichen Teile des QR-Verfahrens für symmetrische Matrizen (Hessenberg-Transformation auf Tridiagonalgestalt, QR-Iteration mit Tridiagonalmatrizen) so übertragen, dass ein Algorithmus entsteht, der direkt auf der Matrix A arbeitet
- dazu benutzen wir die folgende Eigenschaft

! Lemma 264

Ist $A \in \mathbb{R}^{m \times n}$ und sind $S \in \mathbb{R}^{m \times m}, T \in \mathbb{R}^{n \times n}$ orthonormale Matrizen, dann haben A und SAT dieselben Singulärwerte.

- wir benutzen also orthonormale Matrizen S, T um A so umzuformen, dass die Singulärwerte leichter zu bestimmen sind
- dazu können wir ohne Einschränkung $m \geq n$ annehmen (ansonsten betrachten wir A^T)
- wir eliminieren zuerst mit einer Householder-Matrix S_1 die erste Spalte von A

$$A \rightarrow S_1 A = \begin{pmatrix} * & * & * & \dots & * \\ 0 & * & * & \dots & * \\ 0 & * & * & \dots & * \\ \vdots & \vdots & & & \vdots \\ 0 & * & * & \dots & * \end{pmatrix}$$

- dann eliminieren wir mit einer Householder-Matrix T_1 die um eins verkürzte erste Zeile von $S_1 A$

$$S_1 A \rightarrow S_1 A T_1 = \begin{pmatrix} * & * & 0 & \dots & 0 \\ 0 & * & * & \dots & * \\ 0 & * & * & \dots & * \\ \vdots & \vdots & & & \vdots \\ 0 & * & * & \dots & * \end{pmatrix}$$

- bei diesem Schritt bleibt die erste Spalte von $S_1 A$ unverändert
- analog kann man alle weiteren Spalten und Zeilen behandeln und erhält schließlich nach n Schritten

$$SAT = \begin{pmatrix} * & * & 0 & \dots & 0 \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & * & * & 0 \\ \vdots & & \ddots & * & * \\ 0 & \dots & \dots & 0 & * \\ 0 & \dots & \dots & \dots & 0 \\ \vdots & & & & \vdots \\ 0 & \dots & \dots & \dots & 0 \end{pmatrix} = \begin{pmatrix} R \\ 0 \end{pmatrix}, \quad R \in \mathbb{R}^{n \times n}$$

wobei R eine obere Bidiagonal-Matrix ist und

$$S = S_n \cdot \dots \cdot S_1, \quad T = T_1 \cdot \dots \cdot T_{n-1}$$

- nach dem Lemma von oben haben A und $(R, 0)^T$ und damit A und R dieselben Singulärwerte
- diese Transformation auf Bidiagonalgestalt ist also das Analogon zur Hessenbergtransformation bei der Eigenwertberechnung
- im nächsten Schritt starten wir die folgende Iteration:
 - $R_0 = R$
 - wiederhole für $k > 0$

$$R_k^T = Q_{k+1} R_{k+1}, \quad R_{k+1}^T = Q_{k+2} R_{k+2}$$

- es werden immer zwei Schritte $R_k \rightarrow R_{k+2}$ durchgeführt
- R_k^T ist eine untere Bidiagonal-Matrix und $R_k^T = Q_{k+1} R_{k+1}$ eine QR -Zerlegung (z.B. mit dem Givens- oder Householder-Verfahren)
- analog zur Hessenberg-Transformation für Tridiagonal-Matrizen zeigt man, dass R_{k+1}, R_{k+2} auch wieder obere Bidiagonal-Matrizen sind
- außerdem gilt

$$R_{k+2} = Q_{k+2}^T R_{k+1}^T = Q_{k+2}^T (Q_{k+1}^T R_k^T)^T = Q_{k+2}^T R_k Q_{k+1}$$

- da Q_{k+1}, Q_{k+2} orthonormal sind, haben R_{k+2} und R_k identische Singulärwerte
- die Iteration von oben ist das Analogon einer QR -Iteration ohne Shift angewandt auf die Matrix $R^T R$
- R_{2k} konvergiert gegen eine Diagonalmatrix, die die Singulärwerte enthält
- aus den orthonormalen Transformationsmatrizen und den Matrizen T, S aus dem ersten Teil können die Matrizen U und V approximiert werden

Regularisierung schlecht konditionierter Probleme

51.1 Grundlagen

- wie wir oben bereits gesehen haben, kann die numerische Behandlung schlecht konditionierter Probleme sehr schwierig sein
- wir betrachten hier Methoden, um schlecht konditionierte Gleichungssysteme bzw. schlecht konditionierte lineare Ausgleichsprobleme (Pseudoinverse) zu behandeln
- im letzten Beispiel versagt selbst unser ‚stabilstes‘ Verfahren (QR) vollständig
- wie können wir solche Probleme trotzdem vernünftig behandeln?
- dazu müssen wir das genaue Verhalten bei der Fehlerfortpflanzung von b nach x untersuchen
- dabei werden die Singulärwerte von A eine wichtige Rolle spielen
- für lineare Ausgleichsprobleme gelten analoge Aussagen
- wir betrachten zunächst einen Spezialfall
- die Aussagen der letzten Beispiele lassen sich verallgemeinern
- dabei genügt es, die Pseudoinverse zu betrachten, da die Inverse regulärer Matrizen ja nur ein Spezialfall davon ist

! Satz 270

Es sei $A \in \mathbb{R}^{m \times n}$ mit Singulärwertzerlegung $A = U\Sigma V^T$,

$$\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0) \in \mathbb{R}^{m \times n}.$$

Für $x = A^+b$, $\Delta x = A^+\Delta b$ gilt die Abschätzung

$$\frac{\|\Delta x\|_2}{\|x\|_2} \leq \frac{\sigma_1}{\sigma_r} \frac{\|\Delta b\|_2}{\|P_r U^T b\|_2},$$

mit $P_r = \text{diag}(\underbrace{1, \dots, 1}_r, 0, \dots, 0) \in \mathbb{R}^{m \times m}$. Es gibt immer ein b und Δb , so dass

$$\frac{\|\Delta x\|_2}{\|x\|_2} = \frac{\sigma_1}{\sigma_r} \frac{\|\Delta b\|_2}{\|b\|_2}.$$

- dieses Ergebnis motiviert die folgende Definition der Konditionszahl für beliebige Matrizen

🔊 Definition 271

Es sei $A \in \mathbb{R}^{m \times n}$ mit Singulärwertzerlegung $A = U\Sigma V^T$,

$$\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0) \in \mathbb{R}^{m \times n}, \quad \sigma_1 \geq \dots \geq \sigma_r > 0.$$

Dann ist die Konditionszahl κ_2 von A gegeben durch

$$\kappa_2(A) = \frac{\sigma_1}{\sigma_r}$$

- in den nächsten Abschnitten werden wir nun Verfahren untersuchen, mit denen man brauchbare Näherungslösungen für schlecht konditionierte Gleichungssysteme bzw. lineare Ausgleichsprobleme erzeugen kann

51.2 Abgeschnittene Singulärwertzerlegung (TSVD)

- wie wir oben gesehen haben, werden die Konditionsprobleme durch die Singulärwerte σ_i von A verursacht, für die

$$\frac{\sigma_1}{\sigma_i}$$

groß ist

- lassen wir diese σ_i einfach weg, dann erhalten wir die abgeschnittene Singulärwertzerlegung (Truncated Singular Value Decomposition, **TSVD**)

🔊 Definition 273

Es sei $A \in \mathbb{R}^{m \times n}$ mit Singulärwertzerlegung $A = U\Sigma V^T$,

$$\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0) \in \mathbb{R}^{m \times n}, \quad \sigma_1 \geq \dots \geq \sigma_r > 0.$$

Für $\alpha > 0$ ist $A_\alpha = U\Sigma_\alpha V^T$,

$$\Sigma_\alpha = \text{diag}(\sigma_1, \dots, \sigma_k, 0, \dots, 0) \in \mathbb{R}^{m \times n}, \quad \frac{\sigma_k}{\sigma_1} \geq \sqrt{\alpha} > \frac{\sigma_{k+1}}{\sigma_1}$$

eine *abgeschnittene Singulärwertzerlegung* von A

- aus der Definition folgt direkt, dass

$$\kappa_2(A_\alpha) \leq \frac{1}{\sqrt{\alpha}}$$

- für großes α gilt:
 - A_α hat gute Kondition und

$$x_\alpha = A_\alpha^+ b = V \Sigma_\alpha^+ U^T b$$

ist problemlos zu berechnen

- A_α, A_α^+ sind unter Umständen keine guten Approximationen von A, A^+ , so dass x_α mit der gesuchten Lösung wenig zu tun hat
- für kleines α gilt:
 - A_α, A_α^+ sind gute Approximationen von A, A^+
 - A_α hat schlechte Kondition
- es kommt also wesentlich auf die Wahl des Parameters α an, wie wir auch im folgenden Beispiel sehen
- das Verhalten des Residuums im letzten Beispiel ist kein Zufall

! Satz 275

Sei $x_\alpha = A_\alpha^+ b$. Für $\alpha \downarrow 0$ konvergiert x_α gegen $x = A^+ b$ und das Residuum $\|b - Ax_\alpha\|_2$ ist monoton nicht wachsend.

51.3 Tikhonov Regularisierung

- die Berechnung der Pseudoinversen $x = A^+ b$ ist äquivalent zum Minimierungsproblem

$$\|Ax - b\|_2 \rightarrow \min, \quad \|x\|_2 \rightarrow \min$$

- wir betrachten jetzt für $\alpha > 0$ die Funktion

$$J_\alpha(x) = \|Ax - b\|_2^2 + \alpha \|x\|_2^2$$

und minimieren sie über x

- dazu formen wir J_α zunächst um

$$\begin{aligned} J_\alpha(x) &= \|Ax - b\|_2^2 + \alpha \|x\|_2^2 \\ &= \left\| \begin{pmatrix} Ax \\ \sqrt{\alpha} x \end{pmatrix} - \begin{pmatrix} b \\ 0 \end{pmatrix} \right\|_2^2 \\ &= \left\| \underbrace{\begin{pmatrix} A \\ \sqrt{\alpha} I \end{pmatrix}}_{A_\alpha} x - \underbrace{\begin{pmatrix} b \\ 0 \end{pmatrix}}_{b_\alpha} \right\|_2^2 \\ &= \|A_\alpha x - b_\alpha\|_2^2 \end{aligned}$$

- wegen $\alpha > 0$ hat A_α vollen Rang, so dass das Minimierungsproblem eindeutig lösbar ist und der Minimierer x_α über die Normalgleichungen

$$A_\alpha^T A_\alpha x_\alpha = A_\alpha^T b_\alpha$$

bestimmt werden kann

- setzen wir A_α, b_α ein, so erhalten wir

$$(A^T, \alpha I) \begin{pmatrix} A \\ \alpha I \end{pmatrix} x_\alpha = (A^T, \alpha I) b_\alpha$$

bzw.

$$(A^T A + \alpha I) x_\alpha = A^T b$$

- damit haben wir also die Normalgleichungen für $\|Ax - b\|_2 \rightarrow \min$ erhalten, ergänzt um einen Zusatzterm αI , der dafür sorgt, dass die Systemmatrix $A^T A + \alpha I$ symmetrisch positiv definit ist

Definition 277

Für $A \in \mathbb{R}^{m \times n}$, $\alpha > 0$ ist

$$x_\alpha = (A^T A + \alpha I)^{-1} A^T b$$

die *Tikhonov-Regularisierung* von $x = A^+ b$ zum Parameter α .

Satz 278

Sei x_α die Tikhonov-Regularisierung von $x = A^+ b$. Für $\alpha \downarrow 0$ konvergiert x_α gegen $x = A^+ b$ und das Residuum $\|b - Ax_\alpha\|_2$ ist monoton nicht wachsend.

- damit haben wir ein ähnliches Verhalten wie bei TSVD
 - zu großes α liefert schlechte Approximationsqualität
 - zu kleines α stabilisiert zu wenig
- eine vernünftige Parameterwahl ist entscheidend

51.4 Parameterwahl

- es gibt zahlreiche Strategien, um den Regularisierungsparameter α zu wählen
- die bekannteste davon arbeitet nach folgender Idee:
 - sei b die ungestörte rechte Seite
 - b ist nicht bekannt, sondern nur die gestörte Version $\tilde{b}$
 - $\tilde{b}$ sei mit einer absoluten Genauigkeit δ gegeben, d.h.

$$\|\tilde{b} - b\|_2 = \|\Delta b\|_2 \leq \delta,$$

$\tilde{x}_\alpha$ sei eine Regularisierung von $A^+ \tilde{b}$

- wähle das größte α , so dass

$$\|\tilde{b} - A\tilde{x}_\alpha\|_2$$

in der Größenordnung von δ liegt

- man soll also das Residuum nicht unnötig klein machen, da sonst die Gefahr besteht, dass man wieder in den Bereich schlechter Kondition gerät
- das gewählte α hängt von δ und $\tilde{b}$ ab

 Definition 280 (Diskrepanz-Prinzip nach Morozov)

Seien $A, \tilde{b}, \delta$ gegeben, $\|\tilde{b} - b\|_2 \leq \delta$ und $\tilde{x}_\alpha$ eine Regularisierung von $A^+\tilde{b}$. Dann wählt man den Regularisierungsparameter gemäß

$$\alpha(\delta, \tilde{b}) = \sup_{\alpha > 0} (\|\tilde{b} - A\tilde{x}_\alpha\|_2 \leq \tau\delta),$$

mit $\tau \geq 1$ fest.

- wir wenden jetzt das Diskrepanz-Prinzip auf die Tikhonov-Regularisierung an
- schlecht gestellte Gleichungssysteme oder lineare Ausgleichsprobleme spielen in der Praxis eine große Rolle

Zusammenfassung

- wir lösen Approximationsaufgaben, indem wir statt $Ax = b$ typischerweise ein Ausgleichsproblem formulieren:

$$\min_x \|Ax - b\|_2$$

- für zuverlässige Lösungen sind stabile Zerlegungen entscheidend (QR, CGLS,...)
- die Pseudoinverse A^+ liefert eine einheitliche Beschreibung der Least-Squares-Lösung, auch bei Rangdefizit, und liefert die Lösung mit minimaler Norm
- die SVD ist das zentrale Analysewerkzeug, sie erklärt Rang, Kondition und wie Messfehler in b zu Fehlern in x verstärkt werden
- bei schlecht konditionierten Problemen braucht man Regularisierung, um stabile, sinnvolle Näherungen zu erhalten
- wichtige Regularisierungsverfahren sind TSVD (kleine Singulärwerte abschneiden) und Tikhonov

$$\min_x (\|Ax - b\|_2^2 + \alpha \|x\|_2^2)$$

- der Regularisierungsparameter α steuert den Kompromiss zwischen guter Datenanpassung und Stabilität
- eine geeignete Parameterwahl ist sehr wichtig

Teil X

Integration

- wir betrachten folgende Aufgabenstellung: gegeben f stetig, berechne

$$\int_a^b f(x) dx$$

für endliche a, b

- aus der Analysis ist folgendes bekannt:
 - es ist $\int_a^b f(x) dx = F(b) - F(a)$, wobei F die Stammfunktion von f ist
 - ist f stetig, so existiert F immer
 - F ist oft nicht in geschlossener Form darstellbar, z.B. für $f(x) = e^{-x^2}$
- deswegen braucht man Methoden um $\int_a^b f(x) dx$ numerisch zu nähern
- Hauptanwendungsgebiete sind Methoden zur Approximation von Differentialgleichungen (Algorithmen für gewöhnliche Differentialgleichungen, Finite Elemente)
- die Grundidee für numerische Integrationsverfahren ist:
 - nähere f auf $[a, b]$ durch eine Funktion g , die einfach integrierbar ist (d.h. die Stammfunktion von g ist einfach zu bestimmen)
 - benutze dann $\int_a^b f(x) dx \approx \int_a^b g(x) dx$

54.1 Grundlagen

- für $-\infty < a < b < \infty$, $f \in C[a, b]$, sollen Approximationen von

$$\int_a^b f(x) dx$$

berechnet werden.

- dazu wird an $n + 1$ **äquidistanten Stützstellen** $x_k = a + (b - a)\frac{k}{n} \in [a, b]$, $k = 0, \dots, n$, der Integrand f ausgewertet, ein **Interpolationspolynom** g vom Grad n durch die Punkte $(x_k, f(x_k))$ erzeugt und dann

$$\int_a^b g(x) dx$$

als Approximation des ursprünglichen Integrals benutzt

- dabei erhält man immer die Darstellung

$$\int_a^b g(x) dx = (b - a) \sum_{k=0}^n \beta_k f(x_k),$$

wobei die **Gewichte** $\beta_k \in \mathbb{R}$ nur von n , aber nicht von a, b, f abhängig sind

- für $h = b - a$ erhält man die Fehlerabschätzung

$$\int_a^{a+h} f(x) dx = h \sum_{k=0}^n \beta_k f(x_k) + \mathcal{O}(h^{n+2}),$$

d.h. prinzipiell kann man durch mehr Stützstellen die Ordnung beliebig erhöhen (und damit für kleines h auch die Genauigkeit)

- in der Praxis geht das nicht, da der Polynomgrad von g entsprechend anwächst und dann die üblichen Probleme (Oszillationen) auftreten
- deshalb greift man zur Erhöhung der Genauigkeit auf **summierte Newton-Cotes Formeln** zurück
- dazu zerlegt man $[a, b]$ in m Teilintervalle $[a_i, b_i]$

$$a = a_0 < b_0 = a_1 < b_1 = a_2 < \dots < b_{m-2} = a_{m-1} < b_{m-1} = b,$$

betrachtet

$$\int_a^b f(x) dx = \sum_{i=0}^{m-1} \int_{a_i}^{b_i} f(x) dx$$

und wendet dann auf jedes Teilintegral die Approximation von oben mit niedrigem Polynomgrad n an

54.2 Integralapproximation für $[a,b]=[0,1]$

- auf dem Intervall $[0, 1]$ betrachten wir die **äquidistanten Stützstellen** $x_k = \frac{k}{n}$, $k = 0, \dots, n$
- das Interpolationspolynom g durch die Punkte $(x_k, f(x_k))$ ist gegeben als

$$g(x) = \sum_{k=0}^n f(x_k) L_k(x),$$

wobei L_k die zu den x_k gehörenden **Lagrange-Polynome**

$$L_k(x) = \frac{x - x_0}{x_k - x_0} \cdot \dots \cdot \frac{x - x_{k-1}}{x_k - x_{k-1}} \cdot \frac{x - x_{k+1}}{x_k - x_{k+1}} \cdot \dots \cdot \frac{x - x_n}{x_k - x_n}$$

sind

- somit erhalten wir die folgende Approximation

$$\int_0^1 f(x) dx \approx \int_0^1 g(x) dx = \sum_{k=0}^n f(x_k) \int_0^1 L_k(x) dx = \sum_{k=0}^n \beta_k f(x_k)$$

- wegen $x_k = \frac{k}{n}$ hängen die Gewichte

$$\beta_k = \int_0^1 L_k(x) dx$$

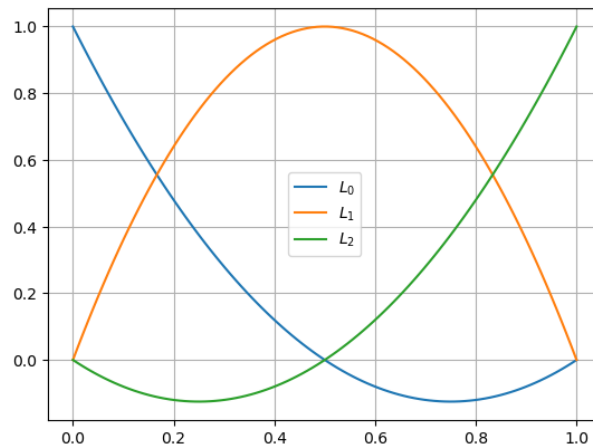
nur von n ab und sind einfach berechenbar

- wir berechnen zunächst die Lagrange-Polynome. Für $n = 2$ erhalten wir

$$L_0(x) = (x - 1)(2x - 1)$$

$$L_1(x) = 4x(1 - x)$$

$$L_2(x) = x(2x - 1)$$



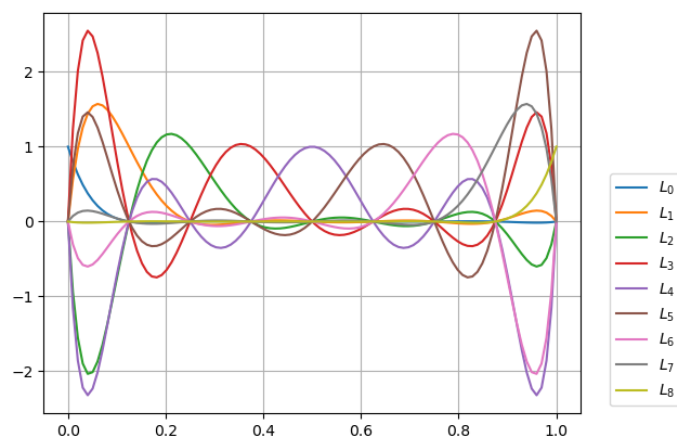
- zur Berechnung der Gewichte müssen wir nur noch die Lagrange-Polynome über das Intervall $[0, 1]$ integrieren:

$$\beta_0 = 1/6, \quad \beta_1 = 2/3, \quad \beta_2 = 1/6$$

- für $n = 1, \dots, 8$ ergibt sich für β_i analog

$n \backslash i$	0	1	2	3	4	5	6	7	8
1	$\frac{1}{2}$	$\frac{1}{2}$	—	—	—	—	—	—	—
2	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{6}$	—	—	—	—	—	—
3	$\frac{1}{8}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{1}{8}$	—	—	—	—	—
4	$\frac{7}{90}$	$\frac{16}{45}$	$\frac{2}{15}$	$\frac{16}{45}$	$\frac{7}{90}$	—	—	—	—
5	$\frac{19}{288}$	$\frac{25}{96}$	$\frac{25}{144}$	$\frac{25}{144}$	$\frac{25}{96}$	$\frac{19}{288}$	—	—	—
6	$\frac{41}{840}$	$\frac{9}{35}$	$\frac{9}{280}$	$\frac{34}{105}$	$\frac{9}{280}$	$\frac{9}{35}$	$\frac{41}{840}$	—	—
7	$\frac{751}{17280}$	$\frac{3577}{17280}$	$\frac{49}{640}$	$\frac{2989}{17280}$	$\frac{2989}{17280}$	$\frac{49}{640}$	$\frac{3577}{17280}$	$\frac{751}{17280}$	—
8	$\frac{989}{28350}$	$\frac{2944}{14175}$	$-\frac{464}{14175}$	$\frac{5248}{14175}$	$-\frac{454}{2835}$	$\frac{5248}{14175}$	$-\frac{464}{14175}$	$\frac{2944}{14175}$	$\frac{989}{28350}$

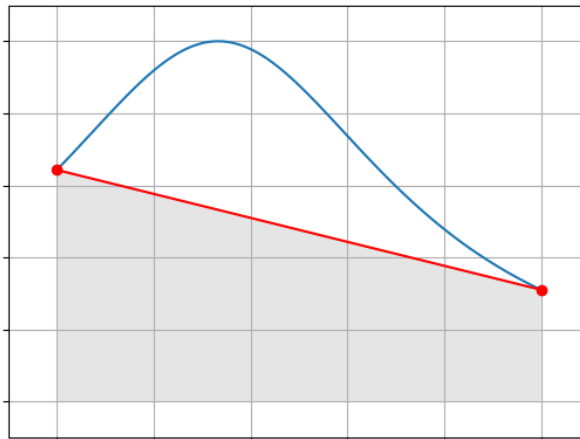
- bei $n = 8$ treten negative Gewichte auf. Dies hängt mit dem stark oszillierenden Verhalten der Lagrange-Polynome achten Grades zusammen:



- in Floating-Point-Arithmetik können diese negativen Gewichte selbst bei relativ einfachen Integranden zu unerwünschten Auslöschungen und damit zu unbrauchbaren Ergebnissen führen, weshalb man in der Regel nur relativ kleine n benutzt
- fassen wir jetzt alles zusammen, dann erhalten wir für unsere Integralapproximation

$$\int_0^1 f(x) dx \approx \sum_{k=0}^n \beta_k f(x_k), \quad \beta_k = \int_0^1 L_k(x) dx$$

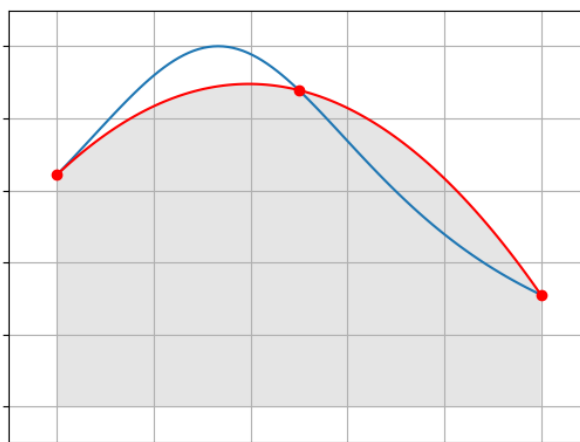
- für $n = 1$ wird also ein lineares Polynom zur Approximation des Integranden f verwendet, dass diesen bei $x_0 = 0$ und $x_1 = 1$ interpoliert:



- wir approximieren also das exakte Integral (Fläche unter der blauen Kurve) durch die Trapez-Fläche unterhalb des Interpolationspolynoms (rote Kurve)
- deshalb nennt man dies auch **Trapez-Regel**

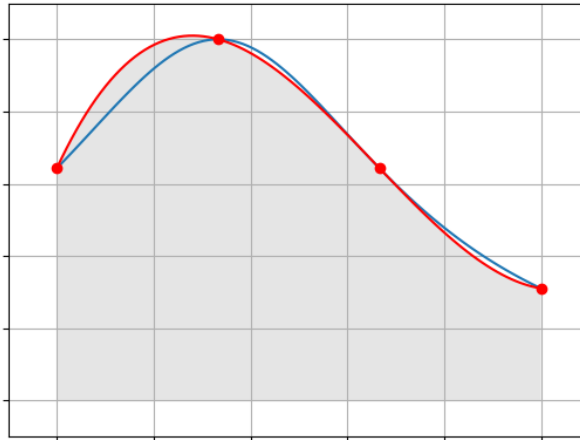
$$NC_1(f) = \frac{f(0)}{2} + \frac{f(1)}{2}$$

- für $n = 2$ erhalten wir die **Simpson-Regel**, bei der ein quadratisches Polynom das f in $0, \frac{1}{2}, 1$ interpoliert zum Einsatz kommt



$$NC_2(f) = \frac{f(0)}{6} + \frac{2f(\frac{1}{2})}{3} + \frac{f(1)}{6}$$

- für $n = 3$ erhalten wir die $\frac{3}{8}$ -Regel mit kubischem Interpolationspolynom



$$NC_3(f) = \frac{f(0)}{8} + \frac{3f(\frac{1}{3})}{8} + \frac{3f(\frac{2}{3})}{8} + \frac{f(1)}{8}$$

54.3 Integralapproximation für beschränkte Intervalle [a,b]

- Integrale über beliebige beschränkte Intervalle führen wir durch Substitution auf Integrale über $[0, 1]$ zurück, die wir dann wie oben beschrieben approximieren

$$\int_a^b f(x) dx = (b-a) \int_0^1 f(a + (b-a)y) dy \approx (b-a) \sum_{k=0}^n \beta_k f(a + (b-a)\frac{k}{n}),$$

wobei die Gewichte β_k die selben wie oben sind und nur von n , aber nicht von a, b, f abhängen

- setzen wir nun $x_k = a + (b-a)\frac{k}{n}$, so erhalten wir die **Standarddarstellung der Newton-Cotes Approximation**

$$\int_a^b f(x) dx \approx (b-a) \sum_{k=0}^n \beta_k f(x_k), \quad x_k = a + (b-a)\frac{k}{n}.$$

54.4 Summierte Formeln

- offensichtlich kann man mit wachsendem n in gewissem Umfang die Fehler reduzieren. Wie das erste Beispiel im letzten Kapitel zeigt, kann es bei großem n dabei zu Problemen kommen (oszillierende Interpolationspolynome)
- deswegen geht man zur Steigerung der Approximationsqualität anders vor
- man zerlegt $[a, b]$ in m Teilintervalle $[a_i, b_i]$

$$a = a_0 < b_0 = a_1 < b_1 = a_2 < \dots < b_{m-2} = a_{m-1} < b_{m-1} = b,$$

betrachtet

$$\int_a^b f(x) dx = \sum_{i=0}^{m-1} \int_{a_i}^{b_i} f(x) dx$$

und wendet dann auf jedes Teilintegral die Approximation von oben an

$$\int_a^b f(x) dx = \sum_{i=0}^{m-1} \int_{a_i}^{b_i} f(x) dx \approx \sum_{i=0}^{m-1} (b_i - a_i) \sum_{k=0}^n \beta_k f(x_{k,i}), \quad x_{k,i} = a_i + (b_i - a_i) \frac{k}{n}.$$

- im einfachsten Fall sind alle Teilintervalle gleich groß, d.h.

$$h = \frac{b-a}{m} = b_i - a_i, \quad a_i = a + ih, \quad b_i = a + (i+1)h$$

und man erhält

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{m-1} \sum_{k=0}^n \beta_k f(x_{k,i}), \quad x_{k,i} = a + ih + h \frac{k}{n},$$

was sich einfach implementieren lässt

- für die äquidistante **summierte Trapez-Regel** ($n = 1, \beta_0 = \beta_1 = \frac{1}{2}$) erhalten wir bei $m = 3$ Intervallen

$$NCS_{1,3}(f) = \frac{h(f(a) + 2f(a+h) + 2f(a+2h) + f(a+3h))}{2}$$

- bei m Intervallen ist

$$\begin{aligned} NCS_{1,m}(f) &= h \sum_{i=0}^{m-1} \frac{1}{2} (f(a_i) + f(b_i)) \\ &= \frac{h}{2} \sum_{i=0}^{m-1} (f(a_i) + f(a_{i+1})) \\ &= \frac{h}{2} (f(a_0) + f(a_1) + f(a_1) + f(a_2) + \dots + f(a_{m-1}) + f(a_m)), \end{aligned}$$

also

$$NCS_{1,m}(f) = \frac{h}{2} (f(\hat{x}_0) + 2f(\hat{x}_1) + \dots + 2f(\hat{x}_{m-1}) + f(\hat{x}_m)), \quad h = \frac{b-a}{m}, \quad \hat{x}_j = a + jh.$$

- analog erhalten wir für die äquidistante **summierte Simpson-Regel** ($n = 2, \beta_0 = \beta_2 = \frac{1}{6}, \beta_1 = \frac{4}{6}$) bei $m = 3$ Intervallen

$$NCS_{2,3}(f) = \frac{h(f(a) + 4f(a + \frac{h}{2}) + 2f(a+h) + 4f(a + \frac{3h}{2}) + 2f(a+2h) + 4f(a + \frac{5h}{2}) + f(a+3h))}{6}$$

- für beliebiges m ist

$$NCS_{2,m}(f) = h \sum_{i=0}^{m-1} \frac{1}{6} \left(f(a_i) + 4f\left(\frac{a_i + a_{i+1}}{2}\right) + f(a_{i+1}) \right), \quad a_i = a + ih, \quad h = \frac{b-a}{m}.$$

Mit $\hat{x}_j = a + (b-a) \frac{j}{2m} = a + j \frac{h}{2}$ folgt

$$\hat{x}_{2i} = a_i, \quad \hat{x}_{2i+1} = \frac{a_i + a_{i+1}}{2}$$

$$\begin{aligned}
 NCS_{2,m}(f) = \frac{h}{6} & (f(\hat{x}_0) + 4f(\hat{x}_1) + f(\hat{x}_2) \\
 & + f(\hat{x}_2) + 4f(\hat{x}_3) + f(\hat{x}_4) \\
 & \vdots \\
 & + f(\hat{x}_{2m-2}) + 4f(\hat{x}_{2m-1}) + f(\hat{x}_{2m}))
 \end{aligned}$$

also

$$NCS_{2,m}(f) = \frac{h}{6} (f(\hat{x}_0) + 4f(\hat{x}_1) + 2f(\hat{x}_2) + 4f(\hat{x}_3) + \dots + 2f(\hat{x}_{2m-2}) + 4f(\hat{x}_{2m-1}) + f(\hat{x}_{2m}))$$

54.5 Fehlerbetrachtung

- wie oben beschrieben werden in der Praxis fast ausschließlich die summierten Formeln eingesetzt
- wir wollen hier für den äquidistanten Fall Fehlerabschätzungen herleiten
- durch Erhöhen der Anzahl der Teilintervalle m , also Reduktion von $h = \frac{b-a}{m}$ kann offensichtlich die Approximationsqualität verbessert werden
- es stellt sich nun die Frage, wie stark bei den einzelnen Verfahren die Fehler auf Änderungen in h (bzw. m) reagieren. Wünschenswert ist ein möglichst schneller Abfall des Fehlers wenn h klein wird
- deshalb betrachten man die **Fehlerasymptotik** für $h \rightarrow 0$ bzw. $m \rightarrow \infty$ und versucht den Fehler durch Potenzen von h abzuschätzen
- wir werden die Fehlerabschätzung anhand der äquidistanten summierten Trapez-Regel und Simpson-Regel ausführlich erläutern
- die Vorgehensweise lässt sich dann analog auf beliebige äquidistante summierte Newton-Cotes Verfahren übertragen

54.5.1 Einfache Newton-Cotes Formeln

- im ersten Schritt werden wir zunächst ein *einzelnes Teilintervall* einer summierten Newton-Cotes Regel betrachten und untersuchen, wie sich der Fehler dafür verhält, wenn die Intervallbreite h gegen 0 geht
- ohne Einschränkung können wir dazu das Intervall $[0, h]$ verwenden
- NC_n integriert Polynome vom Grad $\leq n$ exakt (in exakter Arithmetik). Zusammen mit einer Taylor-Entwicklung des Integranden f erhalten wir damit relativ einfach eine erste Abschätzung
- für $n + 1$ -mal stetig differenzierbares f gilt nach Taylor

$$\begin{aligned}
 f(x) &= f(0) + f'(0)x + \dots + \frac{1}{n!} f^{(n)}(0)x^n + \frac{1}{(n+1)!} f^{(n+1)}(\xi(x))x^{n+1} \\
 &= T_n(x) + R_{n+1}(x)
 \end{aligned}$$

mit Taylor-Polynom T_n mit Ordnung $\leq n$ und Restglied R_{n+1}

- damit ist

$$\begin{aligned}
 \int_0^h f(x) dx &= \int_0^h T_n(x) dx + \int_0^h R_{n+1}(x) dx \\
 &= h \sum_{i=0}^n \beta_i T_n(x_i) + \int_0^h R_{n+1}(x) dx \\
 &= h \sum_{i=0}^n \beta_i f(x_i) + \int_0^h R_{n+1}(x) dx - h \sum_{i=0}^n \beta_i R_{n+1}(x_i)
 \end{aligned}$$

und für den Fehler erhalten wir

$$\begin{aligned} E_n(h) &= \int_0^h f(x) dx - h \sum_{i=0}^n \beta_i f(x_i) \\ &= \int_0^h R_{n+1}(x) dx - h \sum_{i=0}^n \beta_i R_{n+1}(x_i) \end{aligned}$$

- ist $\bar{f}^{(n+1)}$ eine obere Schranke von $|f^{(n+1)}|$ auf $[0, h]$, gilt also

$$|f^{(n+1)}(x)| \leq \bar{f}^{(n+1)} \quad \forall x \in [0, h]$$

und sind alle $\beta_i \geq 0$ (was für $n \leq 7$ der Fall ist), so folgt

$$\begin{aligned} |E_n(h)| &= \left| \int_0^h R_{n+1}(x) dx - h \sum_{i=0}^n \beta_i R_{n+1}(x_i) \right| \\ &\leq \left| \int_0^h R_{n+1}(x) dx \right| + \left| h \sum_{i=0}^n \beta_i R_{n+1}(x_i) \right| \\ &\leq \int_0^h |R_{n+1}(x)| dx + h \sum_{i=0}^n \beta_i |R_{n+1}(x_i)| \\ &\leq \bar{f}^{(n+1)} \int_0^h \frac{x^{n+1}}{(n+1)!} dx + h \bar{f}^{(n+1)} \sum_{i=0}^n \beta_i \frac{x_i^{n+1}}{(n+1)!} \\ &\leq \bar{f}^{(n+1)} \frac{h^{n+2}}{(n+2)!} + h \bar{f}^{(n+1)} \frac{h^{n+1}}{(n+1)!} \sum_{i=0}^n \beta_i \end{aligned}$$

und wegen $\sum_{i=0}^n \beta_i = 1$ schließlich

$$|E_n(h)| \leq \bar{f}^{(n+1)} \frac{h^{n+2}}{(n+2)!} + \bar{f}^{(n+1)} \frac{h^{n+2}}{(n+1)!} = h^{n+2} \frac{n+3}{(n+2)!} \bar{f}^{(n+1)}$$

also z.B.

$$|E_1(h)| \leq \frac{2h^3}{3} \bar{f}^{(2)}$$

$$|E_2(h)| \leq \frac{5h^4}{24} \bar{f}^{(3)}$$

$$|E_3(h)| \leq \frac{h^5}{20} \bar{f}^{(4)}$$

- diese Abschätzung ist sehr pessimistisch
- um genauere Aussagen zu erhalten betrachten wir jetzt die Taylor-Entwicklung des Fehlers

$$E_n(h) = \int_0^h f(x) dx - NC_n(h),$$

nach h um $h = 0$ und erhalten

$$E_1(h) = -\frac{h^3 \frac{d^2}{dh^2} f(h) \Big|_{h=0}}{12} + O(h^4)$$

$$E_2(h) = -\frac{h^5 \frac{d^4}{dh^4} f(h) \Big|_{h=0}}{2880} + O(h^6)$$

$$E_3(h) = -\frac{h^5 \left. \frac{d^4}{dh^4} f(h) \right|_{h=0}}{6480} + O(h^6)$$

$$E_4(h) = -\frac{h^7 \left. \frac{d^6}{dh^6} f(h) \right|_{h=0}}{1935360} + O(h^8)$$

- die Vorfaktoren der führenden Terme sind betragsmäßig deutlich kleiner als bei unserer Abschätzung oben
- darüber hinaus haben wir bei $E_2(h)$ als führende Ordnung 5 und nicht 4
- unter Benutzung der Approximationseigenschaft für Interpolationspolynome

$$f(z) - p_n(z) = \frac{f^{(n+1)}(\xi(z))}{(n+1)!} \prod_{i=0}^n (z - x_i)$$

kann man für allgemeines n folgende Fehlerdarstellung herleiten

! Satz 287

Sei $f \in C^{2+n}[0, h]$, $I_n(f)$ wie oben. Dann gibt es ein $\xi \in [0, h]$ mit

$$E_n(h) = \int_0^h f(x) dx - NC_n(h) = \begin{cases} \left(\frac{h}{n}\right)^{n+3} \int_0^n \frac{t \prod_{k=0}^n (t-k)}{(n+2)!} dt f^{(n+2)}(\xi) & \text{für } n \text{ gerade} \\ \left(\frac{h}{n}\right)^{n+2} \int_0^n \frac{t \prod_{k=0}^n (t-k)}{(n+1)!} dt f^{(n+1)}(\xi) & \text{für } n \text{ ungerade} \end{cases}$$

- damit erhalten wir die selben führenden Terme wie oben

$$E_1(h) = -\frac{h^3}{12} f^{(2)}(\xi)$$

$$E_2(h) = -\frac{h^5}{2880} f^{(4)}(\xi)$$

$$E_3(h) = -\frac{h^5}{6480} f^{(4)}(\xi)$$

$$E_4(h) = -\frac{h^7}{1935360} f^{(6)}(\xi)$$

- in der Literatur wir statt h häufig $\tilde{h} = h/n$ benutzt
- damit verändern sich die Fehlerdarstellungen wie folgt

! Korollar 288

Sei $f \in C^{n+2}[0, h]$, $I_n(f)$ wie oben und $\tilde{h} = h/n$. Dann gibt es ein $\xi \in [0, h]$ mit

$$E_n(\tilde{h}) = \int_0^h f(x) dx - NC_n(h) = \begin{cases} \tilde{h}^{n+3} \int_0^n \frac{t \prod_{k=0}^n (t-k)}{(n+2)!} dt f^{(n+2)}(\xi) & \text{für } n \text{ gerade} \\ \tilde{h}^{n+2} \int_0^n \frac{t \prod_{k=0}^n (t-k)}{(n+1)!} dt f^{(n+1)}(\xi) & \text{für } n \text{ ungerade} \end{cases}$$

54.5.2 Exaktheitsgrad

Definition 290

Ist $I(f)$ eine Integralapproximation die für alle Polynome f mit Grad $\leq l$ die exakten Werte liefert, dann nennt man l den *Exaktheitsgrad* von I .

- die (pessimistischeren) Fehlerabschätzungen von oben sind ein Spezialfall der folgenden Aussage

Satz 291

Es sei $h > 0$ und

$$I(f) = h \sum_{i=0}^n \beta_i f(x_i) \approx \int_0^h f(x) dx, \quad x_i = \alpha_i h,$$

eine Integralapproximation mit α_i, β_i unabhängig von h und Exaktheitsgrad l . Dann gilt für $f \in C^{l+1}$, $|f^{(l+1)}| \leq M$

$$|E(h)| = \left| \int_0^h f(x) dx - I(f) \right| = \mathcal{O}(h^{l+2}).$$

54.5.3 Summierte Newton-Cotes Formeln

- im letzten Abschnitt hatten wir für den Fehler der einfachen Trapezregel

$$E_1(h) = I(h) - NC_1(h) = \int_0^h f(x) dx - \frac{h}{2} (f(0) + f(h)),$$

die Darstellung

$$E_1(h) = -\frac{h^3}{12} f^{(2)}(\xi)$$

- wir haben also Konvergenz dritter Ordnung für ein *einzelnes* Teilintegral.
- benutzen wir dieses Ergebnis in der summierten Trapezregel so folgt

$$\begin{aligned} E_m &= \int_a^b f(x) dx - NCS_{1,m}(f) \\ &= \sum_{i=0}^{m-1} \int_{a_i}^{a_i+h} f(x) dx - \frac{h}{2} (f(a_i) + f(a_i+h)) \\ &= -\sum_{i=0}^{m-1} \frac{h^3}{12} f''(\xi_i) \end{aligned}$$

also

$$|E_m| \leq m \frac{h^3}{12} \max_{\xi \in [a,b]} |f''(\xi)|$$

und wegen $h = \frac{b-a}{m}$ schließlich

$$|E_m| \leq \frac{1}{m^2} \frac{(b-a)^3}{12} \max_{\xi \in [a,b]} |f''(\xi)|$$

! Satz 292

Die äquidistante summierte Trapez-Regel konvergiert quadratisch in m , falls f hinreichend glatt ist.

- bei der Simpson-Regel (und allen anderen Newton-Cotes Verfahren) geht man völlig analog vor
- der Fehler auf einem Teilintervall ist nach dem letzten Abschnitt

$$E_2(h) = -\frac{h^5}{2880} f^{(4)}(\xi)$$

- wir haben Konvergenz fünfter Ordnung für ein *einzelnes* Teilintegral
- benutzen wir dieses Ergebnis wieder für alle Teilintervalle, so erhalten wir für die äquidistante summierte Simpson-Regel die Fehlerabschätzung

$$|E(h)| = \left| \int_a^b f(x) dx - NCS_{2,m}(f) \right| \leq \frac{1}{m^4} \frac{(b-a)^5}{2880} \max_{\xi \in [a,b]} |f^{(4)}(\xi)|.$$

! Satz 293

Die äquidistante summierte Simpson-Regel konvergiert also mit Ordnung 4 in m , falls f hinreichend glatt ist.

- wir betrachten jetzt wieder den nicht-summierten Fall, d.h. ein einziges Intervall
- die Herleitung der Newton-Cotes-Verfahren besteht aus folgenden Schritten:
 - gebe äquidistante Stützstellen $x_i = a + i\tilde{h}$, $\tilde{h} = \frac{b-a}{n}$ vor
 - bestimme über die Lagrange-Polynome die Gewichte β_i
 - approximiere $\int_a^b f(x) dx \approx (b-a) \sum_{i=0}^n \beta_i f(x_i) =: I_n(f)$
- die Fehlerbetrachtung lieferte

$$\int_a^b f(x) dx - I_n(f) \sim \begin{cases} h^{n+3} f^{(n+2)}(\xi) & n \text{ gerade} \\ h^{n+2} f^{(n+1)}(\xi) & n \text{ ungerade} \end{cases}$$

- der Fehler verhält sich also wie $\tilde{h}^{n+3}$ (bzw. $\tilde{h}^{n+2}$)
- für Polynome mit Grad $\leq n+1$ (n ist der Fehler wegen des Faktors $f^{(n+2)}(\xi)$ ($f^{(n+1)}(\xi)$) gleich 0
- das ist wenig überraschend, da wir bei der Konstruktion der Verfahren im wesentlichen die $n+1$ Parameter β_i als „freie Parameter“ haben
- kann man diese Aussagen bei gleichem Aufwand (gleicher Stützstellenzahl) noch verbessern?
- Idee:
 - bei Newton-Cotes werden die x_i willkürlich festgelegt
 - wir versuchen jetzt die x_i möglichst günstig in $[a, b]$ zu verteilen
- wir haben nun in

$$\int_{-1}^1 f(x) dx \approx \sum_{i=0}^n \beta_i f(x_i)$$

$2(n+1)$ Parameter β_i und x_i frei, so dass wir bestenfalls erwarten können, dass damit beliebige Polynome vom Grad $\leq 2n+1$ (also mit $2(n+1)$ Koeffizienten) exakt integriert werden können

- wie wir sehen werden, können wir dieses Optimum erreichen
- die folgenden Überlegungen gehen zurück auf Gauß
- wir benutzen sogenannte *Orthogonalpolynome*
 - wir definieren ein Skalarprodukt

$$(f, g) = \int_a^b f(x)g(x) dx$$

für Funktionen auf $[a, b]$ und betrachten Polynome $P_k(x)$ vom Grad k mit führendem Koeffizienten 1

$$P_k(x) = p_{k0} + p_{k1}x + \dots + p_{kk-1}x^{k-1} + x^k$$

- Polynome $P_k, k = 0, \dots, m$, dieses Typs heißen orthogonal, falls

$$(P_k, P_l) = \int_a^b P_k(x)P_l(x) dx = \gamma_k \delta_{kl}, \quad \gamma_k \neq 0$$

ist

! Satz 297

Die Orthogonalpolynome P_k sind durch die oben definierten Eigenschaften eindeutig bestimmt. Sie können durch die Rekursion

$$\begin{aligned} P_0(x) &= 1 \\ P_1(x) &= x - \frac{(x, 1)}{(1, 1)} \\ P_k(x) &= (x - a_k)P_{k-1}(x) - b_k P_{k-2}(x) \end{aligned} \quad \begin{aligned} a_k &= \frac{(xP_{k-1}, P_{k-1})}{(P_{k-1}, P_{k-1})} \\ b_k &= \frac{(P_{k-1}, P_{k-1})}{(P_{k-2}, P_{k-2})} \end{aligned}$$

berechnet werden.

- wie man sieht, haben $P_k, k = 1, \dots, 4$, genau k Nullstellen in $(-1, 1)$
- das gilt für alle k

! Satz 299

Das Orthogonalpolynom P_n vom Grad n hat genau n verschiedene reelle Nullstellen $x_{ni}, i = 0, \dots, n-1$. Sie liegen alle im Intervall (a, b)

- wir benutzen jetzt die Nullstellen $x_0, \dots, x_n$ von P_{n+1} als Stützstellen, interpolieren ein Polynom durch $(x_i, f(x_i))$ und integrieren

📌 Definition 300

$x_0, \dots, x_n$ seien die Nullstellen des $n+1$ -ten Orthogonalpolynoms P_{n+1} und L_i die zugehörigen Lagrange-Polynome. Dann ist

$$G_n(f) = \sum_{i=0}^n \beta_i f(x_i), \quad \beta_i = \int_a^b L_i(x) dx$$

die *Integralnäherung nach Gauß*.

- ab jetzt betrachten wir das Standard-Intervall $(a, b) = (-1, 1)$
- denn allgemeine Fall können wir über eine Integraltransformation wieder einfach darauf zurück führen

! Lemma 302

G_n integriert Polynome vom Grad $\leq 2n + 1$ exakt.

! Lemma 303

Bei G_n sind alle Gewichte β_i positiv.

- G_n integriert also Polynome bis zum Grad $2n + 1$ exakt mit Gewichten $\beta_i > 0 \forall i$
- damit können wir direkt die Taylorreihen basierte Fehlerabschätzung von Newton-Cotes (die dort so nur bis $n = 7$ anwendbar war) auf G_n anwenden und erhalten

$$|E_n(h)| = \left| \int_0^h f(x) dx - G_n(f) \right| \leq h^{2n+3} \frac{2n+4}{(2n+3)!} \bar{f}^{(2n+2)}$$

für $|f^{(2n+2)}| \leq \bar{f}^{(2n+2)}$ auf $[0, h]$ also z.B.

$$|E_0(h)| \leq \frac{2h^3}{3} \bar{f}^{(1)}$$

$$|E_1(h)| \leq \frac{h^5}{20} \bar{f}^{(2)}$$

$$|E_2(h)| \leq \frac{h^7}{630} \bar{f}^{(3)}$$

- diese Abschätzung ist sehr pessimistisch.
- um genauere Aussagen zu erhalten betrachten wir jetzt die Taylor-Entwicklung des Fehlers

$$E_n(h) = \int_0^h f(x) dx - G_n(h),$$

nach h um $h = 0$ und erhalten

$$E_0(h) = \frac{h^3}{24} \frac{d^2}{dh^2} f(h) \Big|_{h=0} + O(h^4)$$

$$E_1(h) = \frac{h^5}{4320} \frac{d^4}{dh^4} f(h) \Big|_{h=0} + O(h^6)$$

$$E_2(h) = \frac{h^7}{2016000} \frac{d^6}{dh^6} f(h) \Big|_{h=0} + O(h^8)$$

$$E_3(h) = \frac{h^9}{1778112000} \frac{d^8}{dh^8} f(h) \Big|_{h=0} + O(h^{10})$$

$$E_4(h) = \frac{h^{11}}{2534876467200} \frac{d^{10}}{dh^{10}} f(h) \Big|_{h=0} + O(h^{12})$$

- wie bei Newton-Cotes kann man hier wieder folgendes Resultat nachweisen

! Satz 304

Sei $f \in C^{2n+2}[a, b]$, $G_n(f)$ wie oben und $\tilde{h} = \frac{b-a}{n}$. Dann gibt es ein $\xi \in (a, b)$ mit

$$\int_a^b f(x) dx - G_n(f) = c_n \tilde{h}^{2n+3} f^{(2n+2)}(\xi)$$

• Praxis:

- die x_i liegen in (a, b) , d.h. nie auf dem Rand
- für verschiedene n sind die x_i komplett verschieden, d.h. beim Übergang von n auf $n+1$ kommt nicht nur eine neue Stützstelle hinzu, sondern alle Stützstellen verändern sich
- die x_i hängen von a, b ab:
 - x_i werden für $[-1, 1]$ tabelliert
 - für beliebiges $[a, b]$ erhält man:

$$\tilde{x}_i = \frac{b-a}{2} x_i + \frac{b+a}{2}$$

d.h. die relative Verteilung innerhalb des Intervalls bleibt erhalten

- analog hängen auch die Gewichte β_i von (a, b) ab und werden für $[-1, 1]$ tabelliert:
 - für $[-1, 1]$ gilt $\sum_{i=0}^n \beta_i = 2$
 - für beliebiges $[a, b]$ erhält man

$$\tilde{\beta}_i = \frac{b-a}{2} \beta_i$$

- die Gauß-Formeln können auch als summierte Varianten eingesetzt werden
- Nachteil ist, dass anders als bei Newton-Cotes keine Stützstellen auf den Intervallrändern liegen, d.h. der Aufwand für GS_n ist vergleichbar mit dem für NCS_{n+1}
- andererseits ist die Fehlerordnung für $GS_n \sim \frac{1}{m^{2n+2}}$, für NCS_n aber nur $\sim \frac{1}{m^{n+2}}$ für n gerade bzw. $\sim \frac{1}{m^{n+1}}$ für n ungerade, so dass bei größerem n GS_{n-1} effizienter als NCS_n ist
- der Integrand f ist ausreichend oft differenzierbar, so dass unsere Fehlerabschätzungen von oben anwendbar sind
- die Fehlerverläufe bestätigen unsere theoretischen Aussagen
- ist f nicht ausreichend oft differenzierbar, so fallen die Konvergenzraten ab

- summierte Newton Cotes Formeln sind einfach anzuwenden:
 - der Polynomgrad kann niedrig gehalten werden, was Instabilitäten vermeidet
 - die Implementierung ist einfach
- ein typischer Vertreter ist die summierte Trapez-Regel, die im äquidistanten Fall gegeben ist durch

$$\begin{aligned} T_h(f) &= \frac{h}{2}(f(a_0) + 2f(a_1) + \dots + 2f(a_{m-1}) + f(a_m)) \\ &= \sum_{k=0}^{m-1} h\left(\frac{1}{2}f(a_k) + \frac{1}{2}f(a_{k+1})\right) \end{aligned}$$

mit m Trapezen der Breite $h = \frac{b-a}{m}$ und $a_k = a + k h$

- für den Fehler $E_h(f)$ hatten wir oben die Abschätzung

$$|E_h(f)| \leq \frac{mh^3}{12} \max_{\xi \in [a,b]} |f''(\xi)| = h^2 \frac{b-a}{12} \max_{\xi \in [a,b]} |f''(\xi)|, \quad h = \frac{b-a}{m},$$

erhalten, d.h. $|E_h(f)| \sim h^2$ für $h \rightarrow 0$

- um kleine Fehler zu erhalten kann man
 - h klein machen, d.h. die Anzahl m der Teilintervalle erhöhen, was den Aufwand hoch treibt
 - die Ordnung erhöhen (d.h. einen höheren Polynomgrad benutzen) was ebenfalls mehr Aufwand bedeutet und durch die Instabilitäten bei hohem Polynomgrad nach oben begrenzt ist
- kann man mit einfachen Mitteln hochgenaue, stabile Verfahren herleiten?
- wir betrachten das Fehlerverhalten der äquidistanten summierten Trapez-Regel nochmal genauer
- der Schlüssel dazu ist die Fehlerentwicklung nach Euler-Maclaurin
- zu ihrer Herleitung benutzt man spezielle Polynome

Definition 309

Die Bernoulli-Polynome b_k sind definiert durch

$$b_0(x) = 1$$

und

$$b'_k(x) = kb_{k-1}(x), \quad \int_0^1 b_k(x) dx = 0, \quad k \geq 1.$$

! Lemma 311

Für die Bernoulli Polynome b_k , $k \geq 0$, gilt:

- Symmetrie: $b_k(1-x) = (-1)^k b_k(x)$
- Randwerte:
 - $b_k(1) = b_k(0)$ für $k \neq 1$
 - $b_{2l+1}(1) = b_{2l+1}(0) = 0$, $l \geq 1$

Definition 312

Die Werte

$$\begin{aligned} B_0 &= b_0(0) = b_0(1), \\ B_1 &= b_1(0) = -b_1(1), \\ B_k &= b_k(0) = b_k(1), \quad k \geq 2 \end{aligned}$$

heißen Bernoulli-Zahlen.

! Satz 313 (Euler–Maclaurin, einfache Trapezregel auf $[0, 1]$)

Ist $f \in C^{2n}([0, 1])$ dann gilt

$$\int_0^1 f(x) dx = \frac{f(0) + f(1)}{2} - \sum_{l=1}^{n-1} \frac{B_{2l}}{(2l)!} \left(f^{(2l-1)}(1) - f^{(2l-1)}(0) \right) + R_{2n}^{[0,1]},$$

mit Restterm

$$R_{2n}^{[0,1]} = -\frac{1}{(2n)!} \int_0^1 b_{2n}(x) f^{(2n)}(x) dx.$$

! Satz 314 (Euler–Maclaurin, summierte Trapezregel auf $[a, b]$)

Ist $f \in C^{2n}([a, b])$, $h = \frac{b-a}{m}$, $a_k = a + kh$. Dann gilt

$$\int_a^b f(x) dx = T_h(f) - \sum_{l=1}^{n-1} \frac{B_{2l}}{(2l)!} h^{2l} \left(f^{(2l-1)}(b) - f^{(2l-1)}(a) \right) + R_{2n}^{[a,b]},$$

mit

$$T_h(f) = h \left(\frac{1}{2} f(a_0) + \sum_{k=1}^{m-1} f(a_k) + \frac{1}{2} f(a_m) \right),$$

und Restterm

$$R_{2n}^{[a,b]} = -\frac{h^{2n}}{(2n)!} \sum_{k=0}^{m-1} \int_0^1 b_{2n}(t) f^{(2n)}(x_k + ht) dt.$$

- für $f \in C^{2n}([a, b])$ folgt damit aus dem letzten Satz

$$\begin{aligned} T_h(f) &= \int_a^b f(x) dx + \sum_{l=1}^{n-1} h^{2l} \underbrace{\frac{B_{2l}}{(2l)!} (f^{(2l-1)}(b) - f^{(2l-1)}(a))}_{\tau_l} \\ &\quad + h^{2n} \underbrace{\frac{\sum_{k=0}^{m-1} \int_0^1 b_{2n}(t) f^{(2n)}(a_k + ht) dt}{(2n)!}}_{R_{2n}(h)} \end{aligned}$$

- die τ_l sind unabhängig von h
- für $R_{2n}(h)$ erhalten wir die Abschätzung

$$\begin{aligned} |R_{2n}(h)| &= \left| \frac{1}{(2n)!} \sum_{k=0}^{m-1} \int_0^1 b_{2n}(t) f^{(2n)}(a_k + ht) dt \right| \\ &\leq \frac{1}{(2n)!} \sum_{k=0}^{m-1} \int_0^1 |b_{2n}(t)| |f^{(2n)}(a_k + ht)| dt \\ &\leq \frac{1}{(2n)!} \max_{t \in [0,1]} |b_{2n}(t)| \sum_{k=0}^{m-1} \int_0^1 |f^{(2n)}(a_k + ht)| dt \\ &= \frac{1}{(2n)!} \max_{t \in [0,1]} |b_{2n}(t)| \int_a^b |f^{(2n)}(x)| dx \\ &\leq (b-a) \frac{1}{(2n)!} \max_{t \in [0,1]} |b_{2n}(t)| \max_{x \in [a,b]} |f^{(2n)}(x)| \end{aligned}$$

- $R_{2n}(h)$ ist also unabhängig von h beschränkt
- damit erhalten wir das folgende Resultat

! Satz 316 (Euler–Maclaurin)

Für $f \in C^{2n}[a, b]$, $n \geq 1$, $h = \frac{b-a}{m}$ ist

$$T_h(f) = \int_a^b f(x) dx + \tau_1 h^2 + \tau_2 h^4 + \dots + \tau_{n-1} (h^2)^{n-1} + R_{2n}(h) (h^2)^n$$

mit

$$|R_{2n}(h)| \leq r_n |b-a|,$$

wobei alle τ_k und auch r_n nicht von h (und damit auch nicht von m) abhängen sondern nur von a, b, f

- wir benutzen nun zur Approximation von $\int_a^b f(x) dx$ die summierte Trapez-Regel mit zwei unterschiedlichen Intervalllängen $h_0 = \frac{b-a}{m_0}$ bzw. $h_1 = \frac{b-a}{m_1}$, d.h.

$$T_{h_0}(f) = \int_a^b f(x)dx + \tau_1 h_0^2 + R_4(h_0)h_0^4$$

$$T_{h_1}(f) = \int_a^b f(x)dx + \tau_1 h_1^2 + R_4(h_1)h_1^4$$

- setze

$$T = \alpha_0 T_{h_0} + \alpha_1 T_{h_1}$$

und bestimme α_0, α_1 so, dass

$$T = \int_a^b f(x)dx + \beta_0 h_0^4 + \beta_1 h_1^4$$

d.h. T enthält keine quadratischen Terme in h_0 bzw. h_1 mehr

- damit ist T eine Integralapproximation vierter Ordnung (und nicht zweiter Ordnung, wie die Trapez-Regel)
- zur Bestimmung von α_0, α_1 setzen wir ein

$$\begin{aligned} T &= \alpha_0 T_{h_0} + \alpha_1 T_{h_1} \\ &= (\alpha_0 + \alpha_1) \int_a^b f(x)dx + \tau_1 (\alpha_0 h_0^2 + \alpha_1 h_1^2) + \alpha_0 R_4(h_0)h_0^4 + \alpha_1 R_4(h_1)h_1^4 \end{aligned}$$

- damit T weiterhin das Integral annähert und die quadratischen Terme verschwinden, muss

$$\alpha_0 + \alpha_1 = 1, \quad \alpha_0 h_0^2 + \alpha_1 h_1^2 = 0,$$

d.h. α_0, α_1 müssen das folgende lineare Gleichungssystem lösen

$$\begin{pmatrix} 1 & 1 \\ h_0^2 & h_1^2 \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

- mit

$$\alpha_0 = \frac{h_1^2}{h_1^2 - h_0^2}, \quad \alpha_1 = -\frac{h_0^2}{h_1^2 - h_0^2}$$

erhalten wir für T schließlich

$$T = \frac{h_1^2 T_{h_0} - h_0^2 T_{h_1}}{h_1^2 - h_0^2} = \int_a^b f(x)dx + R h^4, \quad h = \max(h_0, h_1)$$

- das Verfahren kann analog auf q verschiedene Trapezbreiten erweitert werden

- es sei $m_0, \dots, m_{q-1}, h_i = \frac{b-a}{m_i}$, und $I = \int_a^b f(x)dx$

- aus der Euler-Maclaurin Entwicklung folgt

$$\begin{aligned} T_{h_0} &= I + \tau_1 h_0^2 + \tau_2 (h_0^2)^2 + \dots + \tau_{q-1} (h_0^2)^{q-1} + R_{2q}(h_0)(h_0^2)^q \\ T_{h_1} &= I + \tau_1 h_1^2 + \tau_2 (h_1^2)^2 + \dots + \tau_{q-1} (h_1^2)^{q-1} + R_{2q}(h_1)(h_1^2)^q \\ &\vdots \\ T_{h_{q-1}} &= I + \tau_1 h_{q-1}^2 + \tau_2 (h_{q-1}^2)^2 + \dots + \tau_{q-1} (h_{q-1}^2)^{q-1} + R_{2q}(h_{q-1})(h_{q-1}^2)^q \end{aligned}$$

- wir setzen

$$\begin{aligned}
 T &= \sum_{i=0}^{q-1} \alpha_i T_{h_i} \\
 &= \left(\sum_{i=0}^{q-1} \alpha_i \right) I + \tau_1 \left(\sum_{i=0}^{q-1} \alpha_i h_i^2 \right) + \dots + \tau_{q-1} \left(\sum_{i=0}^{q-1} \alpha_i (h_i^2)^{q-1} \right) + R(h) h^{2q}
 \end{aligned}$$

mit $h = \max_{i=0, \dots, q-1} h_i$

- analog zu oben folgt

$$\begin{aligned}
 \sum \alpha_i &= 1 \\
 \sum \alpha_i h_i^2 &= 0 \\
 \sum \alpha_i (h_i^2)^2 &= 0 \\
 &\vdots \\
 \sum \alpha_i (h_i^2)^{q-1} &= 0
 \end{aligned}
 \Leftrightarrow
 \underbrace{\begin{pmatrix} 1 & \dots & 1 \\ h_0^2 & \dots & h_{q-1}^2 \\ (h_0^2)^2 & \dots & (h_{q-1}^2)^2 \\ \vdots & & \vdots \\ (h_0^2)^{q-1} & \dots & (h_{q-1}^2)^{q-1} \end{pmatrix}}_{=:A}
 \underbrace{\begin{pmatrix} \alpha_0 \\ \vdots \\ \alpha_{q-1} \end{pmatrix}}_{=: \alpha}
 = \underbrace{\begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}}_{=: e}$$

- bestimme α_i aus $A\alpha = e$:
 - A^T ist die Vandermonde-Matrix zu $x_i = h_i^2$
 - A ist genau dann regulär falls A^T regulär ist, also genau dann wenn die h_i paarweise verschieden sind
 - $A\alpha = e$ ist also immer eindeutig lösbar, falls für die T_{h_i} paarweise verschiedene Trapezbreiten $h_i > 0$ benutzt werden
- die Berechnung von T ist durch das Auflösen von $A\alpha = e$ sehr mühsam
- zur Implementierung wäre eine elegantere Form wünschenswert
- um diese herzuleiten, benutzen wir die folgende geometrische Interpretation von T
- die Trapezbreiten h_i sind gegeben, $h_0 > h_1 > \dots > h_{q-1} > 0$ ($h_i = 0$ geht nicht)
- zu jedem $x_i = h_i^2$ berechnen wir mit der summierten Trapezregel eine Integralnäherung $y_i = T_{h_i}$
- wir interpolieren durch die q Punkte (x_i, y_i) , $i = 0, \dots, q-1$ ein Polynom p_{q-1} vom Grad $q-1$
- die Koeffizienten p_i sind gegeben durch

$$\begin{pmatrix} 1 & x_0 & \dots & x_0^{q-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_{q-1} & \dots & x_{q-1}^{q-1} \end{pmatrix}
 \begin{pmatrix} p_0 \\ \vdots \\ p_{q-1} \end{pmatrix}
 =
 \begin{pmatrix} y_0 \\ \vdots \\ y_{q-1} \end{pmatrix}$$

also

$$\begin{pmatrix} 1 & h_0^2 & \dots & (h_0^2)^{q-1} \\ \vdots & \vdots & & \vdots \\ 1 & h_{q-1}^2 & \dots & (h_{q-1}^2)^{q-1} \end{pmatrix}
 \underbrace{\begin{pmatrix} p_0 \\ \vdots \\ p_{q-1} \end{pmatrix}}_{=:p}
 =
 \begin{pmatrix} T_{h_0} \\ \vdots \\ T_{h_{q-1}} \end{pmatrix}$$

- wir können dieses System schreiben als

$$A^T p = t$$

- andererseits gilt

$$T = \alpha^T t, \quad A\alpha = e, \quad e = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

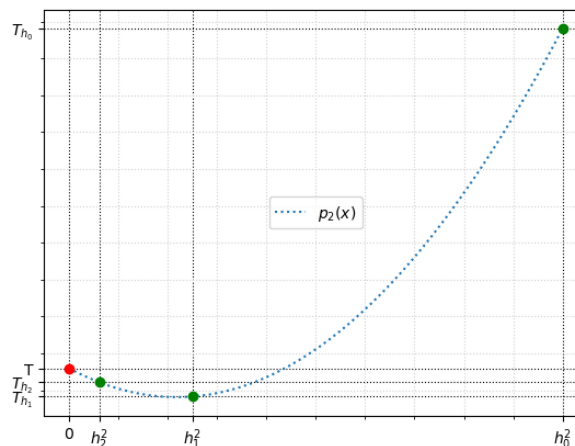
- damit erhalten wir

$$T = \alpha^T t \stackrel{A^T p = t}{=} \alpha^T A^T p = (A\alpha)^T p \stackrel{A\alpha = e}{=} e^T p = p_0$$

so dass

$$T = p_0 = p_{q-1}(0)$$

wobei $p_{q-1}(x)$ das Interpolationspolynom durch die Punkte (h_i^2, T_{h_i}) ist



- wieso liefert dieses Vorgehen vernünftige Ergebnisse?
- für den Fehler der summierten Trapez-Regel gilt

$$|T_h - \int_a^b f(x)dx| = c h^2, \quad h = \frac{b-a}{m}$$

- für $h \rightarrow 0$ (d.h. $m \rightarrow \infty$) konvergiert T_h gegen den exakten Integralwert
- idealerweise würde man also die Trapez-Regel T_h für $h = 0$ bzw. $m = \infty$ auswerten, was aber nicht möglich ist
- die Extrapolation bestimmt T_h für verschiedene h , versucht den Trend für $h \rightarrow 0$ abzuschätzen und damit eine gute Näherung für $h = 0$ (d.h. für das exakte Integral) zu erreichen
- analog zur Newton-Interpolation soll das Extrapolationsschema leicht erweiterbar aufgebaut werden
 - starte mit einer Schrittweite und berechne $T = T_{h_0}$
 - reicht die Genauigkeit nicht aus, so fügt man eine weitere Schrittweite hinzu und bestimmt ein neues T durch $p_{q-1}(0)$
 - das wiederholt man so lange, bis die gewünschte Genauigkeit erreicht ist
- pro Schrittweite erhalten wir einen neuen Interpolationspunkt, d.h. wir erweitern das Polynom wie bei der Newton-Interpolation
- nach jeder Erweiterung muss das neue Polynom an $\hat{x} = 0$ ausgewertet werden um das neue T zu erhalten
- das Auswerten des Interpolationspolynoms an einer gegebenen Stelle x kann sehr einfach durch das *Aitken-Neville-Schema* erfolgen, das mit den dividierten Differenzen verwandt ist:
 - gegeben sind die Punkte (x_i, y_i) , $i = 0, \dots, q-1$
 - setze $P_{i0} = y_i$ und

$$P_{i0} = y_i \quad i = 0, \dots, q-1$$

$$P_{ij} = P_{i,j-1} \frac{x - x_{i-j}}{x_i - x_{i-j}} + P_{i-1,j-1} \frac{x_i - x}{x_i - x_{i-j}} \quad i = 0, \dots, n, \quad j = 0, \dots, i.$$

d.h.

$$\begin{array}{ccccccc}
 y_0 & = & P_{00} & & & & \\
 & & \searrow & & & & \\
 y_1 & = & P_{10} & \rightarrow & P_{11} & & \\
 & & \searrow & & \searrow & & \\
 y_2 & = & P_{20} & \rightarrow & P_{21} & \rightarrow & P_{22} \\
 \vdots & & \vdots & & \vdots & & \ddots
 \end{array}$$

dann ist $P_{q-1,q-1} = p_q(x)$, wobei p_q das Interpolationspolynom durch $(x_0, y_0), \dots, (x_{q-1}, y_{q-1})$ ist

- in unserem Fall gilt $x = 0, x_i = h_i^2, y_i = T_{h_i}$, also $P_{i0} = T_{h_i}$,

$$P_{ij} = P_{ij-1} + \frac{P_{ij-1} - P_{i-1,j-1}}{\left(\frac{h_{i-j}}{h_i}\right)^2 - 1} \quad i \geq j \geq 1$$

und $T = P_{q-1,q-1}$ ist eine Approximation der Ordnung $2q$

- darüber hinaus sind alle Werte P_{ij} in Spalte j Approximationen der Ordnung $2(j+1)$, d.h.

Ordnung	2	4	6	...
$T_{h_0} =$	P_{00}			
$T_{h_1} =$	P_{10}	P_{11}		
$T_{h_2} =$	P_{20}	P_{21}	P_{22}	
$\vdots$	$\vdots$	$\vdots$		$\ddots$

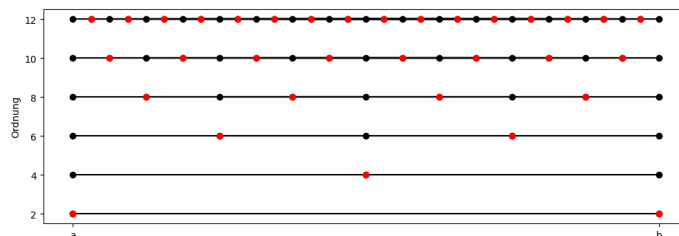
! Satz 321

Sei T die zu q Schrittweiten $h_0 > h_1 > \dots > h_{q-1} > 0$ extrapolierte äquidistante summierte Trapez-Regel. Dann ist $T = P_{q-1,q-1}$ mit

$$\begin{aligned}
 P_{i0} &= T_{h_i} & i &= 0, \dots, q-1 \\
 P_{ij} &= P_{ij-1} + \frac{P_{ij-1} - P_{i-1,j-1}}{\left(\frac{h_{i-j}}{h_i}\right)^2 - 1} & 1 \leq j \leq i \leq q-1.
 \end{aligned}$$

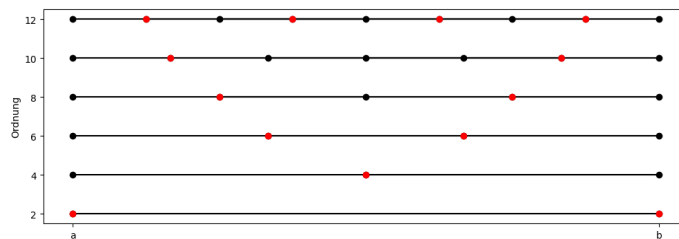
T approximiert $\int_a^b f(x) dx$ für $f \in C^{2q}[a, b]$ mit Ordnung $2q$.

- im Beispiel wurden die Trapezbreiten fortlaufend halbiert, weshalb man viele Funktionswerte wiederverwenden kann



- diese Folge der h_i heißt *Romberg-Folge*
- die Anzahl der Funktionsauswertungen steigt trotzdem sehr stark an
- die h_i müssen nicht notwendig fortlaufend halbiert werden, sondern nur paarweise verschieden sein
- eine alternative Wahl der Trapezbreiten ist durch die *Bulirsch-Folge* gegeben

$$h_0 = b - a, \quad h_1 = \frac{h_0}{2}, \quad h_2 = \frac{h_0}{3}, \quad h_i = \frac{h_{i-2}}{2} \quad i > 2$$



- für die Anzahl der Funktionsauswertungen bei optimaler Implementierung erhält man

Ordnung	2	4	6	8	10	12	14	16
Romberg	2	3	5	9	17	33	65	129
Bulirsch	2	3	5	7	9	13	17	25

- bei der Bulirsch-Folge sind für eine gegebene Fehlerordnung sehr viel weniger Auswertungen nötig, aber:
 - die einzelnen h_i sind größer als bei der Romberg-Folge, so dass die einzelnen $T_{h_i} = P_{i0}$ weniger genau sind
 - das Extrapolationsergebnis T ist eine Linearkombination der T_{h_i} und kann somit für alle q als Integrationsformel Stützstellen x_i und Gewichten β_i geschrieben werden

$$T = \sum_i \beta_i f(x_i)$$

- auf dem Intervall $[0, 1]$ erhält man für x_i, β_i bei Romberg

Ordnung \ i	0	1	2	3	4	5	6	7	8
2	0	1	—	—	—	—	—	—	—
4	0	$\frac{1}{2}$	1	—	—	—	—	—	—
6	0	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{3}{4}$	1	—	—	—	—
8	0	$\frac{1}{8}$	$\frac{1}{4}$	$\frac{3}{8}$	$\frac{1}{2}$	$\frac{5}{8}$	$\frac{3}{4}$	$\frac{7}{8}$	1

Ordnung \ i	0	1	2	3	4	5	6	7	8
2	$\frac{1}{2}$	$\frac{1}{2}$	—	—	—	—	—	—	—
4	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{6}$	—	—	—	—	—	—
6	$\frac{7}{90}$	$\frac{16}{45}$	$\frac{2}{15}$	$\frac{16}{45}$	$\frac{7}{90}$	—	—	—	—
8	$\frac{31}{810}$	$\frac{512}{2835}$	$\frac{176}{2835}$	$\frac{512}{2835}$	$\frac{218}{2835}$	$\frac{512}{2835}$	$\frac{176}{2835}$	$\frac{512}{2835}$	$\frac{31}{810}$

bzw. bei Bulirsch

Ordnung \ i	0	1	2	3	4	5	6
2	0	1	—	—	—	—	—
4	0	$\frac{1}{2}$	1	—	—	—	—
6	0	$\frac{1}{3}$	$\frac{1}{2}$	$\frac{2}{3}$	1	—	—
8	0	$\frac{1}{4}$	$\frac{1}{3}$	$\frac{1}{2}$	$\frac{2}{3}$	$\frac{3}{4}$	1

Ordnung \ i	0	1	2	3	4	5	6
2	$\frac{1}{2}$	$\frac{1}{2}$	—	—	—	—	—
4	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{6}$	—	—	—	—
6	$\frac{11}{120}$	$\frac{27}{40}$	$-\frac{8}{15}$	$\frac{27}{40}$	$\frac{11}{120}$	—	—
8	$\frac{151}{2520}$	$\frac{256}{315}$	$-\frac{243}{280}$	$\frac{104}{105}$	$-\frac{243}{280}$	$\frac{256}{315}$	$\frac{151}{2520}$

- man kann zeigen, dass für die Romberg-Folge für alle q immer $\beta_i > 0$ gilt
- bei der Bulirsch-Folge kommen ab Ordnung 6 negative Gewichte vor, was zu Stabilitätsproblemen führen kann
- Fazit:
 - bei hoher Ordnung ist die Romberg-Folge wegen der höheren Stabilität vorzuziehen
 - bei geringer Ordnung verursacht die Bulirsch-Folge den geringeren Aufwand

57.1 Problemstellung

- die Genauigkeit einer Integralapproximation kann erhöht werden durch
 - kleinere Teilintervalle h z.B. bei den summierten Newton-Cotes-Formeln
 - höhere Fehlerordnung (mehr Stützstellen oder Extrapolation)
- in beiden Fällen wächst der Aufwand, d.h. die Anzahl der Funktionsauswertungen
- wie wir im Beispiel oben gesehen haben, liefern die Romberg- bzw. Bulirsch-Extrapolation für bestimmte Integranden sehr effiziente, hochgenaue Approximationen
- unter bestimmten Umständen treten aber erhebliche Schwierigkeiten auf
- bisher haben wir summierte Formeln immer mit Teilintervallen konstanter Breite h betrachtet
- wie das letzte Beispiel zeigt, ist das nicht immer sinnvoll
- für spezielle Integranden wären lokal unterschiedliche Breiten angebracht
- in den folgenden Abschnitten werden wir sehen, wie diese adaptive Anpassung algorithmisch umgesetzt werden kann

57.2 Fehlerindikator

- das Integral

$$\int_a^b f(x) dx$$

wird mit einer auf Interpolationspolynomen basierenden Methode $I(f, a, b)$ (Newton-Cotes, Gauß) approximiert

- um die Qualität der Approximation zu überprüfen, soll ein Fehlerindikator hergeleitet werden
- für $I(f, a, b)$ gelten Fehlerabschätzungen der Art

$$E = \int_a^b f(x) dx - I(f, a, b) = ch^p f^{(q)}(\xi), \quad h = b - a, \quad \xi \in [a, b]$$

mit c unabhängig von f, h

- um einen Fehlerindikator abzuleiten, spaltet man das Integral in zwei Anteile auf

$$\int_a^b f(x) dx = \int_a^{\frac{a+b}{2}} f(x) dx + \int_{\frac{a+b}{2}}^b f(x) dx,$$

wendet auf beide Teile I an, addiert die Ergebnisse und erhält eine neue (in der Regel genauere) Approximation

$$\tilde{I}(f, a, b) = I\left(f, a, \frac{a+b}{2}\right) + I\left(f, \frac{a+b}{2}, b\right).$$

- für den Fehler von $\tilde{I}(f, a, b)$ erhält man

$$\begin{aligned} \tilde{E} &= \int_a^b f(x) dx - \tilde{I}(f, a, b) \\ &= \int_a^{\frac{a+b}{2}} f(x) dx - I\left(f, a, \frac{a+b}{2}\right) + \int_{\frac{a+b}{2}}^b f(x) dx - I\left(f, \frac{a+b}{2}, b\right) \\ &= c\left(\frac{h}{2}\right)^p f^{(q)}(\xi_l) + c\left(\frac{h}{2}\right)^p f^{(q)}(\xi_r) \\ &= ch^p \frac{1}{2^{p-1}} \frac{f^{(q)}(\xi_l) + f^{(q)}(\xi_r)}{2} \end{aligned}$$

mit $h = b - a$, $\xi_l \in [a, \frac{a+b}{2}]$, $\xi_r \in [\frac{a+b}{2}, b]$

- nimmt man nun an, dass

$$f^{(q)}(\xi) \approx f^{(q)}(\xi_l) \approx f^{(q)}(\xi_r)$$

dann gilt

$$\tilde{E} \approx \frac{1}{2^{p-1}} E$$

bzw.

$$E \approx 2^{p-1} \tilde{E}$$

also

$$\tilde{I}(f, a, b) - I(f, a, b) = E - \tilde{E} \approx (2^{p-1} - 1) \tilde{E}$$

und somit

$$\tilde{E} \approx \frac{1}{2^{p-1} - 1} (\tilde{I}(f, a, b) - I(f, a, b))$$

bzw.

$$E \approx \frac{2^{p-1}}{2^{p-1} - 1} (\tilde{I}(f, a, b) - I(f, a, b))$$

- als Fehlerindikator benutzt man dann

$$\tilde{F} = \frac{1}{2^{p-1} - 1} |\tilde{I}(f, a, b) - I(f, a, b)| \approx |\tilde{E}|$$

bzw.

$$F = \frac{2^{p-1}}{2^{p-1} - 1} |\tilde{I}(f, a, b) - I(f, a, b)| \approx |E|.$$

- die einzige Information die wir für den Fehlerindikator benötigen ist also die Fehlerordnung p von $I(f, a, b)$ bezüglich $h = b - a$
- um diese zu bestimmen reicht es aus, den Fehler

$$E(h) = \int_0^h f(x) dx - I(f, 0, h)$$

nach h in eine Taylorreihe um $h = 0$ zu entwickeln

57.3 Adaptive Newton-Cotes Verfahren

- für Newton-Cotes Formeln kann man die Fehlerordnung leicht berechnen
- wir betrachten hier zunächst die Trapezregel (Polynomgrad $n = 1$) und erhalten für den führenden Term in der Fehlerentwicklung

$$-\frac{h^3 \frac{d^2}{dh^2} f(h) \Big|_{h=0}}{12}$$

also $p = 3$ und somit

$$\tilde{F} = \frac{1}{3} |\tilde{I} - I|$$

Zusammenfassung

- Ziel des Kapitels ist, Integrale numerisch approximieren, wenn keine geschlossene Stammfunktion verfügbar ist oder nur Funktionswerte gegeben sind:

$$I = \int_a^b f(x) dx \approx Q(f)$$

- Newton–Cotes-Formeln: Approximation über Polynom-Interpolation von f auf äquidistanten Stützstellen (z. B. Trapez-, Simpsonregel) für $[0, 1]$ und allgemeine Intervalle $[a, b]$.
- Summierte Formeln: Unterteilung von $[a, b]$ in viele Teilintervalle
- Fehlerbetrachtung:
 - Abschätzung des Quadraturfehlers über Ableitungen von f
 - Exaktheitsgrad als Maß dafür, welche Polynome eine Regel exakt integriert
- Gauß-Integration: Wahl von Stützstellen und Gewichten so, dass für gegebene Anzahl von Funktionswerten maximale Genauigkeit erreicht wird (höherer Exaktheitsgrad als Newton–Cotes)
- Extrapolation: Kombination von Näherungen mit verschiedenen Schrittweiten zur Fehlerreduktion
- Adaptive Integration: automatische Schrittweitensteuerung durch lokalen Fehlerindikator, Verfeinerung nur dort, wo f „schwierig“ ist.
- Fejér- und Clenshaw–Curtis-Verfahren: Quadratur auf speziellen (nicht äquidistanten) Punkten, oft sehr effizient und stabil, besonders für glatte Funktionen

Teil XI

Nicht restringierte Optimierung

- numerische Methoden zur Minimierung einer Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ohne Nebenbedingungen, also

$$\min_{x \in \mathbb{R}^n} f(x)$$

- Grundidee: ausgehend von einem Startwert wird x schrittweise verbessert, bis $f'(x) \approx 0$ erreicht ist
- zwei Hauptklassen:
 - Abstiegsverfahren (nutzen nur Richtungsinformation von f')
 - Newton-artige Verfahren (nutzen zusätzlich Krümmungsinformation von f'')
- zentrale Bausteine: Wahl einer Suchrichtung und einer Schrittweite (Liniensuche) bzw. eines „sicheren“ Schrittraums (Trust-Region)
- Praxisaspekt: Kompromiss zwischen Rechenaufwand pro Schritt und Geschwindigkeit/Robustheit
- Spezialfall nichtlineare Ausgleichsprobleme

- wir suchen Minima einer gegebenen *Zielfunktion*

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad x \mapsto f(x),$$

wobei man x als die Parameter des Optimierungsproblems bezeichnet

- alternativ maximieren wir f indem $-f$ minimiert wird
- wir haben keine weiteren Einschränkungen an die Parameter x (wie z.B. $x_i \geq 0$)
- deshalb bezeichnet man diese Aufgabenstellung als Optimierung ohne Nebenbedingungen bzw. *nicht restriktierte Optimierung*
- wir unterscheiden verschiedene Typen von Minima

Definition 328

Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $\tilde{x} \in \mathbb{R}^n$.

- $\tilde{x}$ ist ein *globales Minimum* von f , falls

$$f(\tilde{x}) \leq f(x) \quad \forall x \in \mathbb{R}^n$$

- $\tilde{x}$ ist ein *lokales Minimum* von f , falls ein $\varepsilon > 0$ existiert mit

$$f(\tilde{x}) \leq f(x) \quad \forall x \in \mathbb{R}^n, \quad \|x - \tilde{x}\| < \varepsilon$$

- ist $f : \mathbb{R} \rightarrow \mathbb{R}$ hinreichend oft differenzierbar, so kann man über die Ableitungen von f hinreichende und notwendige Bedingungen für lokale Minima angeben:
 - ist $\tilde{x}$ ein lokales Minimum (Maximum), dann ist $f'(\tilde{x}) = 0$ und $f''(\tilde{x}) \geq 0$ (≤ 0) (notwendige Bedingung)
 - ist $f'(\tilde{x}) = 0$ und $f''(\tilde{x}) > 0$ (< 0), dann ist $\tilde{x}$ ein lokales Minimum (Maximum) (hinreichende Bedingung)
 - ist $f'(\tilde{x}) = 0$ und $f''(\tilde{x}) = 0$, dann sind zusätzliche Untersuchungen nötig, da auch ein Sattelpunkt vorliegen kann
- für Funktionen $f : \mathbb{R}^n \rightarrow \mathbb{R}$ erhalten wir analoge Ergebnisse

- auch sie können mit Hilfe der Taylor-Entwicklung von f bewiesen werden

! Lemma 330

Ist $f : \mathbb{R}^n \rightarrow \mathbb{R}$ in $\tilde{x}$ dreimal stetig differenzierbar, so gilt

$$f(x) = f(\tilde{x}) + f'(\tilde{x})^T(x - \tilde{x}) + \frac{1}{2}(x - \tilde{x})^T f''(\tilde{x})(x - \tilde{x}) + O(\|x - \tilde{x}\|^3)$$

mit *Gradient*

$$f'(x) = \begin{pmatrix} \partial_{x_1} f(x) \\ \vdots \\ \partial_{x_n} f(x) \end{pmatrix}$$

und *Hesse-Matrix*

$$f''(x) = \begin{pmatrix} \partial_{x_1} \partial_{x_1} f(x) & \dots & \partial_{x_1} \partial_{x_n} f(x) \\ \vdots & & \vdots \\ \partial_{x_n} \partial_{x_1} f(x) & \dots & \partial_{x_n} \partial_{x_n} f(x) \end{pmatrix}$$

- wir erhalten folgende Aussagen:

! Satz 331

$f : \mathbb{R}^n \rightarrow \mathbb{R}$ sei hinreichend oft differenzierbar, $\tilde{x} \in \mathbb{R}^n$.

- ist $\tilde{x}$ ein lokales Minimum (Maximum), dann gilt $f'(\tilde{x}) = 0$ und $f''(\tilde{x})$ ist positiv (negativ) semidefinit (notwendige Bedingung)
- ist $f'(\tilde{x}) = 0$ und $f''(\tilde{x}) \neq 0$, dann gilt:
 - ist $f''(\tilde{x})$ positiv (negativ) definit, dann ist $\tilde{x}$ ein lokales Minimum (Maximum) (hinreichende Bedingung)
 - ist $f''(\tilde{x})$ positiv (negativ) semidefinit, dann ist $\tilde{x}$ ein lokales Minimum (Maximum) oder ein Sattelpunkt
 - ist $f''(\tilde{x})$ indefinit, dann ist $\tilde{x}$ ein Sattelpunkt
- ist $f'(\tilde{x}) = 0$ und $f''(\tilde{x}) = 0$, dann müssen Ableitungen höherer Ordnung untersucht werden
- ist $f'(\tilde{x}) = 0$ und $f''(x)$ positiv (negativ) semidefinit für alle x in einer Umgebung von $\tilde{x}$, dann ist $\tilde{x}$ ein lokales Minimum (Maximum)

- wir gehen ab jetzt davon aus, dass die untersuchten Funktionen f hinreichend oft differenzierbar sind
- da die Bestimmung von globalen Minima für allgemeines f extrem kompliziert ist, werden wir hier zunächst nur Verfahren untersuchen, um lokale Minima von f zu finden
- prinzipiell läuft alles auf eine Nullstellensuche für f' hinaus, wobei durch zusätzliche Maßnahmen Maxima bzw. Sattelpunkte ausgeschlossen werden sollen
- da f' im allgemeinen nichtlinear ist, sind die Verfahren wieder iterativ

Abstiegsverfahren

61.1 Konstruktion

- wir wollen iterative Verfahren bauen, die ausgehend von $x_0 \in \mathbb{R}^n$ eine Folge $x_k \in \mathbb{R}^n$ erzeugen mit

$$f(x_{k+1}) < f(x_k) \quad \forall k$$

- optimal wäre, wenn x_k gegen ein lokales Minimum konvergiert, d.h.

$$x_k \xrightarrow{k \rightarrow \infty} \tilde{x}$$

und damit auch

$$f'(x_k) \xrightarrow{k \rightarrow \infty} f'(\tilde{x}) = 0$$

- die Verfahren die wir hier betrachten, sollen folgende Grundstruktur haben:
 - wähle zu x_k eine Suchrichtung p_k und bestimme x_{k+1} durch

$$x_{k+1} = x_k + \alpha p_k, \quad \alpha > 0$$

- berechne α so, dass

$$f(x_k) - f(x_{k+1})$$

möglichst groß ist

- stoppe die Iteration mit einem geeigneten Abbruchkriterium
- damit das Verfahren funktionieren kann, muss zur Suchrichtung p_k ein $\alpha > 0$ existieren, so dass

$$f(x_{k+1}) = f(x_k + \alpha p_k) < f(x_k)$$

ist

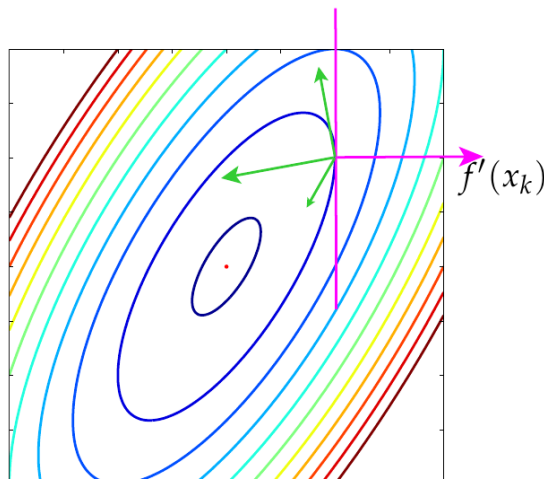
- für welche p_k ist das erfüllt?
- dazu betrachten wir die Taylorentwicklung von f um x_k

$$\begin{aligned} f(x_{k+1}) - f(x_k) &= f'(x_k)^T(x_{k+1} - x_k) + O(\|x_{k+1} - x_k\|^2) \\ &= \alpha f'(x_k)^T p_k + O(\alpha^2) \end{aligned}$$

- für $\alpha > 0$ genügend klein dominiert der erste Term auf der rechten Seite
- damit kann die rechte Seite wie gewünscht kleiner 0 gemacht werden, falls

$$f'(x_k)^T p_k < 0,$$

d.h. der Winkel θ_k zwischen p_k und dem negativen Gradienten $-f'(x_k)$ muss in $[0, \frac{\pi}{2})$ liegen



Definition 336

- $p \in \mathbb{R}^n$ ist *Abstiegsrichtung* von f in $x \in \mathbb{R}^n$, falls

$$f'(x)^T p < 0$$

- ein Abstiegsverfahren heißt *strikt*, falls für den Winkel θ_k zwischen p_k und dem negativen Gradienten $-f'(x_k)$

$$\theta_k \in [0, \frac{\pi}{2} - \delta) \quad \forall k$$

gilt, mit $\delta > 0$ unabhängig von k

- $-f'(x)$ scheint damit zunächst die ideale Abstiegsrichtung zu sein
- aus $-f'(x)$ lassen sich einfach weitere Abstiegsrichtungen konstruieren, die später noch eine wichtige Rolle spielen werden

Satz 338

Ist $f'(x) \neq 0$, $B \in \mathbb{R}^{n \times n}$ spd, dann sind

$$p = -B f'(x), \quad \text{bzw.} \quad p = -B^{-1} f'(x)$$

Abstiegsrichtungen in x

- damit haben wir genügend Möglichkeiten, Abstiegsrichtungen auszuwählen
- um einen implementierbaren Algorithmus zu erhalten, fehlt noch ein Verfahren zur Bestimmung der α sowie ein Abbruchkriterium

- als Abbruchkriterium ideal, aber nicht praktikabel, ist die Kontrolle des Fehlers

$$\|x_k - \tilde{x}\|$$

- deswegen benutzt man folgende Kriterien bzw. Kombinationen davon

$$\|x_{k+1} - x_k\| < \varepsilon_1, \quad f(x_k) - f(x_{k+1}) < \varepsilon_2, \quad \|f(x_k)'\| < \varepsilon_3$$

- die Qualität dieser Kriterien hängt sehr stark von der Kondition des Optimierungsproblem ab, d.h. bei schlecht konditionierten Problemen können die betrachteten Größen sehr klein sein, obwohl man noch sehr weit vom wahren Minimum $\tilde{x}$ entfernt ist

61.2 Liniensuche

- nachdem wir in x_k eine Abstiegsrichtung p_k gewählt haben, müssen wir entscheiden, wie weit wir absteigen, d.h. wie wir $\alpha_k > 0$ bestimmen für

$$x_{k+1} = x_k + \alpha_k p_k$$

- da wir dabei in $\mathbb{R}^n$ entlang der Linie

$$x_k + \alpha p_k, \quad \alpha \in (0, \infty)$$

suchen, nennt man dies *Liniensuche*

- dazu definieren wir

$$\varphi(\alpha) = f(x_k + \alpha p_k)$$

d.h.

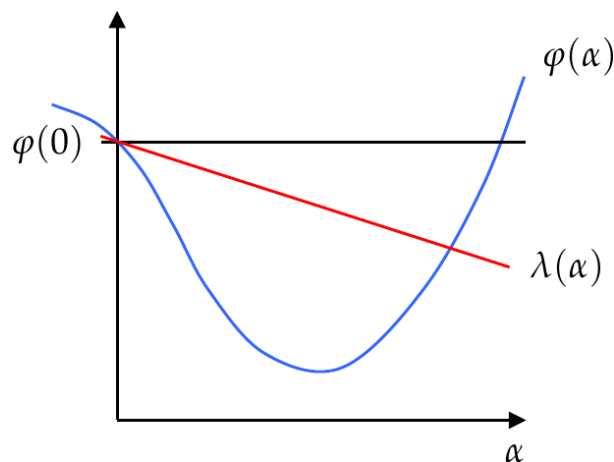
$$\varphi : \mathbb{R} \rightarrow \mathbb{R}$$

und

$$\varphi(0) = f(x_k), \quad \varphi(\alpha_k) = f(x_{k+1}), \quad \varphi'(0) = f'(x_k)^T p_k < 0$$

da p_k Abstiegsrichtung ist

- φ ist also eine einfache Funktion in $\mathbb{R}$, die in 0 streng monoton fallend ist
- daher gibt es auf jeden Fall ein $\alpha > 0$ mit $\varphi(\alpha) < \varphi(0) = f(x_k)$
- das Optimum an Abstieg erreichen wir, wenn wir φ auf $(0, \infty)$ minimieren, d.h. als α_k die globale Minimalstelle von φ auf $(0, \infty)$ benutzen (*exakte Liniensuche*)
- exakte Liniensuche wird selten benutzt, da sie zu aufwändig ist
- stattdessen begnügt man sich mit vereinfachten Varianten (*Soft Linesearch*)



- wir minimieren $\varphi(\alpha)$ nicht, sondern verlangen nur, dass wir einen hinreichenden Abstieg erzielen
- dazu betrachten wir die Gerade

$$\lambda(\alpha) = \varphi(0) + \rho \varphi'(0) \alpha, \quad \rho \in (0, \frac{1}{2})$$

und verlangen für α_k

$$\varphi(\alpha_k) \leq \lambda(\alpha_k)$$

- dies wird oft als *Armijo-Bedingung* bezeichnet
- wegen $\rho < 1$ ist die Existenz eines solchen α_k gewährleistet
- in der Praxis wird α_k z.B. durch *Backtracking* bestimmt:

- starte mit einem gegebenen Wert α (oft 1)
- ist $\varphi(\alpha) \leq \lambda(\alpha)$, dann setze $\alpha_k = \alpha$
- ist $\varphi(\alpha) \geq \lambda(\alpha)$, dann ersetze α durch

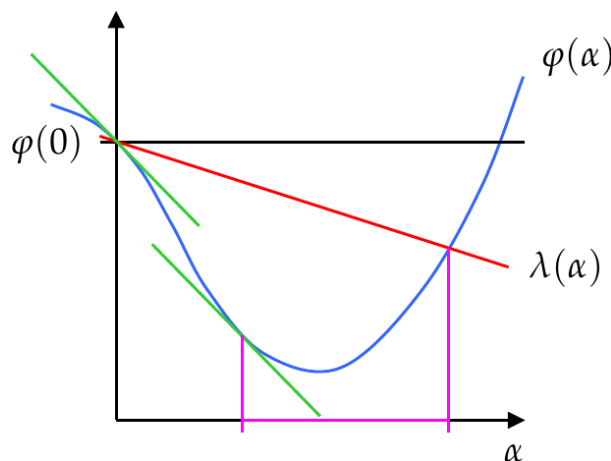
$$\tau \alpha, \quad \tau \in (0, 1)$$

und wiederhole die letzten beiden Schritte

- um unter Umständen unnötig kurze Abstiege zu verhindern (und damit den Aufwand zu reduzieren) wird zusätzlich noch die Einschränkung

$$\varphi'(\alpha_k) \geq \beta \varphi'(0), \quad \rho < \beta < 1$$

an α_k gestellt (Wolfe-Bedingung)



- α_k wird also erst aus dem Bereich gewählt, in dem φ nennenswert „flacher“ ist als in $\alpha = 0$
- in der Praxis bestimmt man zunächst ein Intervall $[a, b]$, das den interessanten Bereich einschließt und bestimmt dann in einem zweiten Schritt möglichst effizient α_k
- strikte Abstiegsverfahren mit einer der beiden Varianten der Liniensuche liefern stationäre Punkte von f

! Satz 339

Sei $f \in C^2(\mathbb{R}^n)$, $x_0 \in \mathbb{R}^n$ und $\{x \mid f(x) \leq f(x_0)\}$ kompakt. Ein striktes Abstiegsverfahren mit Liniensuche nach beiden Varianten von oben liefert eine Folge x_k mit

$$f(x_{k+1}) < f(x_k) \quad \forall k.$$

Entweder ist $f'(x_k) = 0$ für ein $k \in \mathbb{N}$ oder x_k besitzt mindestens einen Häufungspunkt $\tilde{x}$, wobei $f'(\tilde{x}) = 0$ für alle Häufungspunkte.

61.3 Methode des steilsten Abstiegs (Steepest-Descent)

- als Beispiel für ein Abstiegsverfahren betrachten wir das *Steepest-Descent-Verfahren*
- dabei benutzen wir als Abstiegsrichtung

$$p_k = -f'(x_k)$$

kombiniert mit exakter Liniensuche bzw. Liniensuche nach Armijo

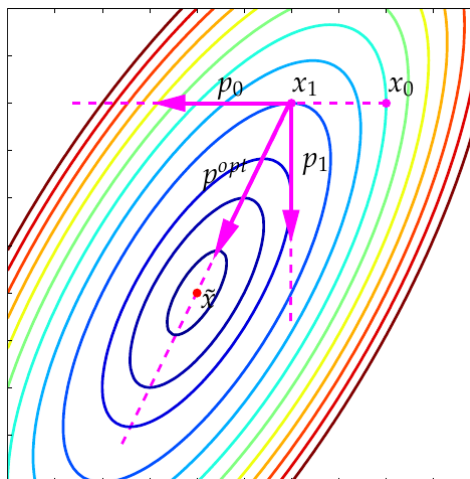
- insgesamt erhalten wir für das Steepest-Descent-Verfahren:
 - einfach zu implementieren
 - benötigt nur Auswertungen von f'
 - relativer guter Abstieg in den ersten Iterationen
 - relativ schwach, wenn man nahe am lokalen Minimum ist
 - leichte Unterschiede bei verschiedenen Verfahren zur Liniensuche
 - keine klaren Vorteile bei exakter Liniensuche
 - wird das Verfahren mit exakter Liniensuche auf quadratische Zielfunktionen

$$f(x) = \frac{1}{2}x^T A x - x^T b + c$$

mit A spd angewandt, so kann man lineare Konvergenz beweisen

61.4 Konjugierte Gradienten (CG)

- wie wir gesehen haben, ist das Steepest-Descent-Verfahren oft sehr langsam
- wie kann man das verbessern?
- dazu betrachten wir zwei aufeinander folgende Iterierte



- wir starten bei x_0 senkrecht zur Höhenlinie in Richtung $p_0 = -f'(x_0)$
- bei exakter Liniensuche erhalten wir x_1 genau dort, wo unsere Linie tangential zu einer Höhenlinie verläuft
- in x_1 starten wir erneut senkrecht zur Höhenlinie in Richtung $p_1 = -f'(x_1)$
- damit ist $p_1 \perp p_0$
- p_1 zeigt in der Regel *nicht* von x_1 auf das Minimum $\tilde{x}$

- $p^{opt} = \tilde{x} - x_1$ wäre die optimale Suchrichtung
- Idee:
 - bestimme neue Abstiegsrichtung als Linearkombination von p_0 und p_1

$$\hat{p}_1 = p_1 + \gamma p_0$$

- bestimme γ so, dass $\hat{p}_1$ näher an $p^{opt} = \tilde{x} - x_1$ ist
- wie soll man γ wählen?
- dazu betrachten wir zunächst quadratische Funktionen f , leiten ein Kriterium ab und wenden es später auch für andere f an
- es sei also $A \in \mathbb{R}^{n \times n}$ spd, $b \in \mathbb{R}^n$, $c \in \mathbb{R}$ und $f: \mathbb{R}^n \rightarrow \mathbb{R}$,

$$f(x) = \frac{1}{2} x^T A x - x^T b + c$$

- damit ist $f'(x) = Ax - b$ und f hat ein globales Minimum bei $\tilde{x} = A^{-1}b$
- für die Abstiegsrichtung beim Steepest-Descent-Verfahren erhalten wir

$$p_k = -f'(x_k) = b - Ax_k$$

- aus $p_1 \perp p_0$ folgt

$$0 = p_0^T p_1 = p_0^T (b - Ax_1) = p_0^T (A\tilde{x} - Ax_1) = p_0^T A(\tilde{x} - x_1) = p_0^T A p^{opt}$$

- wir bestimmen jetzt γ so, dass $\hat{p}_1 = p_1 + \gamma p_0$ diese Eigenschaft ebenfalls erfüllt

$$0 = p_0^T A \hat{p}_1 = p_0^T A(p_1 + \gamma p_0),$$

also

$$\gamma = -\frac{p_0^T A p_1}{p_0^T A p_0}$$

- diesen Zusammenhang wollen wir nicht nur für den ersten Schritt herstellen, sondern auch für alle folgenden Schritte, d.h.

$$\hat{p}_i^T A \hat{p}_j = 0 \quad \forall i \neq j$$

Definition 344

Sei $A \in \mathbb{R}^{n \times n}$ spd. Die Vektoren $p_i \in \mathbb{R}^n$ heißen *A-konjugiert* falls

$$p_i^T A p_j = 0 \quad \forall i \neq j.$$

- wir bauen also ein neues Verfahren mit modifizierten Steepest-Descent-Suchrichtungen auf
- wenn das Verfahren auf quadratische Funktionen vom obigen Typ angewandt wird, dann sind die Suchrichtungen A-konjugiert
- wir erhalten daraus das (nichtlineare) *CG-Verfahren*:
 - x_0 gegeben, $p_0 = p_0^{SD} = -f'(x_0)$
 - wiederhole für $k = 0, 1, 2, \dots$

$$f'(x_k)^T p_k \geq 0 \Rightarrow p_k = p_k^{SD}$$

$$\alpha_k = \text{linesearch}(x_k, p_k)$$

$$x_{k+1} = x_k + \alpha_k p_k$$

$$p_{k+1}^{SD} = -f'(x_{k+1})$$

$$\gamma_{k+1} \text{ berechnen}$$

$$p_{k+1} = p_{k+1}^{SD} + \gamma_{k+1} p_k$$

- zur Berechnung der γ_k können verschiedene Formeln angegeben werden, die für quadratisches f A -konjugierte Richtungen liefern:

$$\gamma_{k+1} = \frac{f'(x_{k+1})^T f'(x_{k+1})}{f'(x_k)^T f'(x_k)} \quad \text{Fletcher-Reeves}$$

$$\gamma_{k+1} = \frac{f'(x_{k+1})^T (f'(x_{k+1}) - f'(x_k))}{f'(x_k)^T f'(x_k)} \quad \text{Polak-Ribière}$$

! Satz 346

Sei f wie in quadratisch wie oben. Bei exakter Liniensuche und $f'(x_k) \neq 0 \forall k$ gilt für die Fletcher-Reeves- und Polak-Ribière-Version des nichtlinearen CG-Verfahrens:

- die p_k sind A -konjugiert
 - alle p_k sind Abstiegsrichtungen
 - der Algorithmus stoppt nach spätestens n Schritten
- insgesamt erhalten wir für das CG-Verfahren:
 - einfach zu implementieren
 - benötigt nur Auswertungen von f'
 - relativer guter Abstieg, auch nahe am lokalen Minimum
 - sensibel gegenüber verschiedenen Verfahren zur Liniensuche

62.1 Konstruktion

- lokale Minima von f sind stationäre Punkte, d.h. $f'(x) = 0$
- erste Herleitung:
 - wir bestimmen zunächst stationäre Punkte und überprüfen dann, ob es sich um Minima handelt
 - damit erhalten wir ein Nullstellenproblem für

$$g : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad g(x) = f'(x)$$

- zur Lösung wenden wir das Newton-Verfahren auf g an:
 - wir nähern g in einer Umgebung von x_k durch eine lineare Funktion (Taylor-Entwicklung) an

$$g(x) \approx l_k(x) = g(x_k) + g'(x_k)(x - x_k)$$

- als neue Näherung x_{k+1} benutzen wir eine Nullstelle von l_k , d.h.

$$0 = l_k(x_{k+1}) = g(x_k) + g'(x_k)(x_{k+1} - x_k)$$

bzw.

$$g'(x_k)(x_{k+1} - x_k) = -g(x_k)$$

- ist die Jacobi-Matrix $g'(x_k) \in \mathbb{R}^{n \times n}$ invertierbar, so folgt

$$x_{k+1} = x_k - g'(x_k)^{-1}g(x_k)$$

bzw.

$$x_{k+1} = x_k - f''(x_k)^{-1}f'(x_k)$$

- zweite Herleitung:
 - wir nähern f in einer Umgebung von x_k durch eine quadratische Funktion (Taylor-Entwicklung) an

$$f(x) \approx q_k(x) = f(x_k) + f'(x_k)^T(x - x_k) + \frac{1}{2}(x - x_k)^T f''(x_k)(x - x_k)$$

- als neue Näherung x_{k+1} benutzen wir einen Minimierer von q_k
- ist $f''(x_k)$ spd, so gibt es genau einen Minimierer, nämlich die eindeutige Lösung von $q'_k(x) = 0$, d.h.

$$0 = q'_k(x_{k+1}) = f'(x_k) + f''(x_k)(x_{k+1} - x_k),$$

also wieder

$$x_{k+1} = x_k - f''(x_k)^{-1} f'(x_k)$$

- die Konvergenzresultate für das Newton-Verfahren für skalare Funktionen lassen sich analog erweitern

! Satz 348

Sei $\tilde{x}$ ein lokales Minimum von f mit $f''(\tilde{x})$ spd und x_0 hinreichend nahe an $\tilde{x}$. Dann ist das Newton-Verfahren durchführbar und es konvergiert quadratisch gegen $\tilde{x}$.

- Vorteile:
 - quadratische Konvergenz
 - einfache Implementierung
- Nachteile:
 - lokale Konvergenz (Steepest-Descent: global)
 - wenn $f''(\tilde{x})$ singular ist kann die Iteration (zunächst) nicht fortgesetzt werden
 - benötige in jedem Schritt $f''(x_k)$, d.h. Auswertung von n^2 partiellen Ableitungen zweiter Ordnung
- das Newton-Verfahren wurde aus lokalen Modellen $l_k(x)$ bzw. $q_k(x)$ hergeleitet, die beide aus einer Taylor-Entwicklung gewonnen wurden
- diese Modelle sind nur lokal um den Entwicklungspunkt x_k gute Approximationen (d.h. für $\|x - x_k\|$ klein), was zum Teil die lokale Konvergenz erklärt
- wie kann man das Newton-Verfahren so modifizieren, dass
 - singuläres $f''(x_k)$ kein Problem mehr ist
 - sichergestellt ist, dass die lokalen Modelle sinnvolle Approximationen sind, also $\|x - x_k\|$ klein genug bleibt und damit eventuell globale Konvergenz erreicht wird, ohne dass die quadratische Konvergenz verloren geht
 - $f''(x_k)$ nicht ständig berechnet werden muss
- der erste Punkt führt zu *gedämpften Newton-Verfahren*:
 - wir versuchen, die gute Konvergenzrate des Newton-Verfahrens mit der globalen Konvergenz des Steepest-Descent-Verfahrens zu kombinieren
 - es ist

$$x_{k+1} = x_k - f''(x_k)^{-1} f'(x_k)$$

- ist $f''(x_k)$ spd, so ist nach oben

$$p_k = -f''(x_k)^{-1} f'(x_k)$$

eine Abstiegsrichtung und das Newton-Verfahren ein Abstiegsverfahren

$$x_{k+1} = x_k + \alpha_k p_k, \quad \alpha_k = 1$$

- statt $\alpha_k = 1$ könnte man α_k durch Liniensuche bestimmen und damit versuchen, globale Konvergenz zu erreichen (was unter einigen Einschränkungen auch funktioniert)
- allerdings muss dazu $f''(x_k)$ nach wie vor spd sein
- ist $f''(x_k)$ nicht spd, aber invertierbar, so kann die Situation wie folgt gerettet werden:
 - für $f'(x_k) \neq 0$ ist $p_k \neq 0$
 - ist p_k keine Abstiegsrichtung, dann ist $-p_k$ eine Abstiegsrichtung
- für singuläres $f''(x_k)$ kommt man damit aber auch nicht weiter
- andererseits wissen wir aber folgendes:
 - $f''(x_k)$ ist die Hesse-Matrix von f und damit symmetrisch
 - alle Eigenwerte von $f''(x_k)$ sind reell
 - ist $f''(x_k)$ nicht spd, dann ist

$$\lambda_{\min}(f''(x_k)) \leq 0$$

und für

$$\mu_k > -\lambda_{\min}(f''(x_k)) \geq 0$$

ist $f''(x_k) + \mu_k I$ spd

Definition 349

Eine Iteration des *gedämpften Newton-Verfahrens* ist definiert durch:

- wähle $\mu_k \in \mathbb{R}$ groß genug, s.d. $f''(x_k) + \mu_k I$ spd ist
- $p_k = -(f''(x_k) + \mu_k I)^{-1} f'(x_k)$
- $x_{k+1} = x_k + \alpha_k p_k$

wobei $\alpha_k = 1$ benutzt wird oder $\alpha_k > 0$ durch Liniensuche bestimmt wird.

- wie wählt man μ_k ?
 - λ_i seien die Eigenwerte von $f''(x_k)$
 - wähle μ_k so, dass

$$\lambda_{\min} + \mu_k > 0$$

und

$$\frac{\lambda_{\max} + \mu_k}{\lambda_{\min} + \mu_k}$$

nicht zu groß ist, damit die Kondition des linearen Gleichungssystems

$$(f''(x_k) + \mu_k I)p_k = -f'(x_k)$$

nicht zu schlecht ist

- in der Praxis setzt man verschiedene Methoden ein, um μ_k zu bestimmen
- Cholesky:
 - starte mit $\mu > 0$ und führe eine Cholesky-Zerlegung von $f''(x_k) + \mu I$ durch
 - teste dabei die Größe der Diagonalelemente vor dem Ziehen der Wurzel
 - ist ein Diagonalelement negativ, dann ist die Matrix nicht spd
 - setze $\mu := 2\mu$ und starte von vorne
- Abschätzung der Eigenwerte von $f''(x_k)$:
 - berechne (möglichst billig, z.B. mit dem Satz von Gerschgorin) eine untere Schranke c für die Eigenwerte von $f''(x_k)$
 - wähle $\mu_k > c$
- auf die Wahl der μ_k wird später nochmal genauer eingegangen

62.2 Trust-Region-Verfahren

- das ungedämpfte Newton-Verfahren konnte über das lokal quadratische Modell

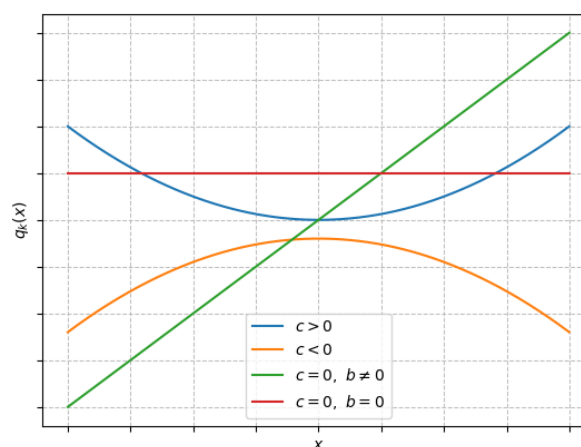
$$q_k(x) = f(x_k) + f'(x_k)^T(x - x_k) + \frac{1}{2}(x - x_k)^T f''(x_k)(x - x_k)$$

hergeleitet werden

- ist $f''(x_k)$ spd, dann ist x_{k+1} das eindeutige (globale) Minimum von q_k über ganz $\mathbb{R}^n$
- ist $f''(x_k)$ nicht spd, so kann q_k unendlich viele bzw. gar keine Minimierer besitzen
- im Fall $n = 1$ gilt

$$q_k(x) = a + bx + \frac{1}{2}cx^2, \quad a, b, c \in \mathbb{R}$$

mit $c = f''(x_k)$



- für $c \leq 0$ (d.h. $f''(x_k)$ nicht spd) erhalten wir kein eindeutiges Minimum:
 - $c = 0, b = 0$ liefert unendlich viele Minima
 - $c < 0$ bzw. $c = 0, b > 0$ liefert gar kein Minimum, da

$$\lim_{x \rightarrow -\infty} q_k(x) = -\infty$$

- andererseits macht es wenig Sinn, globale Minima von q_k auf ganz $\mathbb{R}^n$ zu suchen:
 - $q_k(x)$ ist eine lokale Approximation von $f(x)$
 - die Approximation taugt nur in einer (eventuell sehr kleinen) Umgebung von x_k was
 - Minima von $q_k(x)$ die weit von x_k weg liegen haben in der Regel mit dem tatsächlichen Verhalten von f wenig zu tun
- deshalb minimieren wir jetzt $q_k(x)$ nicht global über ganz $\mathbb{R}^n$, sondern nur lokal in

$$\Omega_k = \{x \mid \|x - x_k\| \leq r_k\}, \quad r_k > 0$$

- Ω_k heißt *Trust-Region (Vertrauensbereich)*, r_k *Vertrauensradius*
- da q_k stetig auf Ω_k ist und Ω_k kompakt ist, gibt es mindestens eine Lösung des Problems

$$\min_{x \in \Omega_k} q_k(x)$$

- wie berechnet man solche Probleme möglichst einfach?

! Satz 351

Ist $f''(x_k) + \mu I$ spd und x_μ die eindeutige Lösung von

$$(f''(x_k) + \mu I)(x_\mu - x_k) = -f'(x_k)$$

dann ist x_μ auch Lösung von

$$\min_{\|x - x_k\|_2 \leq r_\mu} q_k(x), \quad r_\mu = \|x_\mu - x_k\|_2.$$

Außerdem gilt $r_\mu \xrightarrow{\mu \rightarrow \infty} 0$.

- damit sieht unser Trust-Region-Verfahren wie folgt aus:
 - betreibe gedämpftes Newton-Verfahren mit Parameter μ_k
 - wähle μ_k so groß, dass $f''(x_k) + \mu_k I$ spd ist und x_{k+1} im Vertrauensbereich liegt, d.h. in einem Bereich wo q_k ein brauchbares Modell für f ist
- wie stellt man fest, ob q_k ein brauchbares Modell für f ist?
- wir betrachten dazu den *Gain-Faktor*

$$\rho_k = \frac{f(x_k) - f(x_{k+1})}{q_k(x_k) - q_k(x_{k+1})},$$

der den tatsächlichen Abstieg $f(x_k) - f(x_{k+1})$ mit dem durch das quadratische Modell q_k vorhergesagten Abstieg $q_k(x_k) - q_k(x_{k+1})$ in Relation setzt

- $\rho_k \leq 0$:
 - wegen $f''(x_k) + \mu_k I$ spd gilt für $f'(x_k) \neq 0$ immer $q_k(x_k) - q_k(x_{k+1}) > 0$
 - damit ist also $f(x_k) - f(x_{k+1}) \leq 0$, d.h. wir haben keinen Abstieg
 - an der Stelle x_{k+1} ist q_k also ein ganz schlechtes Modell für f
 - wir werfen x_{k+1} weg und verkleinern den Vertrauensbereich indem μ_k vergrößert wird und berechnen damit ein neues x_{k+1}

- $0 < \rho_k \ll 1$:
 - der tatsächliche Abstieg ist sehr gering
 - an der Stelle x_{k+1} ist q_k kein gutes Modell für f
 - wir behalten x_{k+1} , da ja ein Abstieg erzielt wurde
 - für den nächsten Schritt verkleinern wir den Vertrauensbereich, indem wir $\mu_{k+1} > \mu_k$ verwenden
- $\rho_k \approx 1$:
 - an der Stelle x_{k+1} ist q_k ein gutes Modell für f , da vorhergesagter und tatsächlicher Abstieg fast identisch sind
 - im nächsten Schritt vergrößern wir den Vertrauensbereich, indem wir $\mu_{k+1} < \mu_k$ verwenden
- $\rho_k \gg 1$:
 - an der Stelle x_{k+1} ist q_k ein schlechtes Modell für f , aber der tatsächliche Abstieg ist viel besser als der vorhergesagte
 - deshalb verfahren wir wie im Fall $\rho_k \approx 1$
- in der Praxis benutzt man z.B.

$$\mu_{\text{neu}} = \begin{cases} 2\mu & \rho < \frac{1}{4} \\ \frac{\mu}{3} & \rho > \frac{3}{4} \\ \mu & \text{sonst} \end{cases}$$

oder

$$\mu_{\text{neu}} = \begin{cases} 2\mu & \rho \leq 0 \\ \mu \max\left(\frac{1}{3}, 1 - (2\rho - 1)^3\right) & \rho > 0 \end{cases}$$

- durch die Faktoren 2, $\frac{1}{3}$ wird eine Stagnation durch alternierendes Vergrößern bzw. Verkleinern vermieden
- die zweite Variante ändert μ „kontinuierlich“ und hat in der Praxis oft Vorteile
- insgesamt erhalten wir damit ein gedämpftes Newton-Verfahren vom *Levenberg-Marquardt-Typ*:
 - starte mit $x := x_0$, $\mu := \mu_0$, x_0, μ_0 gegeben
 - wiederhole bis Genauigkeit erreicht ist:
 - wiederhole bis $f''(x) + \mu I$ spd ist:
 - $\mu := 2\mu$
 - löse $(f''(x) + \mu I)p = -f'(x)$
 - $x_{\text{neu}} := x + p$ (oder $x_{\text{neu}} := x + \alpha p$ bei Liniensuche)
 - berechne

$$\rho := \frac{f(x) - f(x_{\text{neu}})}{q(x) - q(x_{\text{neu}})}$$

mit $q(y) = f(x) + f'(x)^T(y - x) + \frac{1}{2}(y - x)^T f''(x)(y - x)$

- ist $\rho > \delta > 0$:
 - $x := x_{\text{neu}}$
 - $\mu := \mu \max(\frac{1}{3}, 1 - (2\rho - 1)^3)$
- ist $\rho \leq \delta$:
 - x_{neu} wegwerfen (aktuellen Schritt wiederholen)
 - $\mu := 2\mu$
- Vorteile:
 - globale Konvergenz (unter einigen Einschränkungen an f)
 - superlineare Konvergenz
 - einfach zu implementieren
- Nachteil:
 - Berechnung von $f''(x)$ in jedem Schritt

62.3 Quasi-Newton-Verfahren

- im letzten Abschnitt haben wir $f''(x)$ aus dem Original-Newton-Verfahren durch $f''(x) + \mu I$ ersetzt
- machen noch andere Modifikationen von f'' Sinn?
- wir betrachten das gedämpfte Newton-Verfahren vom Levenberg-Marquardt-Typ und ersetzen $f''(x)$ durch die 0-Matrix:

- das bedeutet, dass das quadratische Modell

$$q(y) = f(x) + f'(x)^T(y - x) + \frac{1}{2}(y - x)^T f''(x)(y - x)$$

abgeändert wird zu

$$q(y) = f(x) + f'(x)^T(y - x),$$

also nur noch ein lineares Modell für f ist

- ist $\mu > 0$ dann ist μI spd und

$$p = -\frac{1}{\mu} f'(x)$$

- benutzt man $x_{\text{neu}} = x + p$ so erhalten wir

$$x_{\text{neu}} = x - \frac{1}{\mu} f'(x)$$

also gerade das Steepest-Descent-Verfahren mit Parameter $\alpha = \frac{1}{\mu}$

- die Veränderung von μ anhand des Gain-Faktors ρ wirkt dann wie eine „softe“ Liniensuche
- offensichtlich braucht man $f''(x)$ nicht sehr genau zu kennen, um ein konvergentes Verfahren zu erhalten
- wichtig ist, dass die Näherung von $f''(x)$ spd ist
- ist die Näherung gut, dann wird man damit die Konvergenzgeschwindigkeit erhöhen
- Idee:
 - betrachte Standard-Newton (mit Liniensuche)

$$f''(x_k)p_k = -f'(x_k), \quad x_{k+1} = x_k + \alpha_k p_k$$

- ersetze $f''(x_k)$ durch eine Näherung $B_k \in \mathbb{R}^{n \times n}$, die am besten spd ist
- berechne damit die neue Näherung x_{k+1}

$$B_k p_k = -f'(x_k), \quad x_{k+1} = x_k + \alpha_k p_k$$

- führe anschließend ein sinnvolles Update

$$B_k \rightarrow B_{k+1}$$

durch

- solche Verfahren nennt man *Quasi-Newton-Verfahren*
- wie berechnet man ein Update für B_k ?
- dazu betrachten wir das Newton-Verfahren nochmal als Verfahren zur Nullstellensuche für $g(x) = f'(x)$
- für $\alpha_k = 1$ erhalten wir im eindimensionalen Fall

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)}$$

- vermeiden von $f''(x_k)$ bedeutet, dass wir das Verfahren so abändern müssen, dass $g'(x_k)$ nicht mehr berechnet werden muss
- das trifft z.B. auf das Sekanten-Verfahren zu

$$x_{k+1} = x_k - \frac{g(x_k)}{\frac{g(x_k) - g(x_{k-1})}{x_k - x_{k-1}}},$$

das superlinear konvergent ist

- hier wird die Ableitung durch einen Differenzenquotienten angenähert, d.h.

$$g'(x_k) \approx \frac{g(x_k) - g(x_{k-1})}{x_k - x_{k-1}}$$

bzw. (nach einem Index-Shift um 1)

$$g'(x_{k+1})(x_{k+1} - x_k) \approx g(x_{k+1}) - g(x_k)$$

und damit

$$f''(x_{k+1})(x_{k+1} - x_k) \approx f'(x_{k+1}) - f'(x_k)$$

- das Update B_{k+1} soll diese Bedingung exakt erfüllen

Definition 354

Es seien B_k Approximationen von $f''(x_k)$, $k \in \mathbb{N}$. B_k erfüllt die *Quasi-Newton-Bedingung*, falls

$$B_{k+1} h_k = y_k, \quad h_k = x_{k+1} - x_k, \quad y_k = f'(x_{k+1}) - f'(x_k)$$

- spalten wir das Update auf in

$$B_{k+1} = B_k + W, \quad W \in \mathbb{R}^{n \times n}$$

so liefert die Quasi-Newton-Bedingung

$$Wh_k = y_k - B_k h_k,$$

- h_k und die rechte Seite ist gegeben, d.h. wir erhalten n lineare Gleichungen für die n^2 Koeffizienten von W
- wir brauchen also zusätzliche Einschränkungen, um W eindeutig fest zu legen
- erste Idee:

- benutze für W eine einfache Matrix mit Rang 1, d.h.

$$W = ab^T, \quad a, b \in \mathbb{R}^n,$$

d.h. wir haben $2n$ freie Parameter

- neben der Quasi-Newton-Bedingung verlangen wir, dass

$$Wv = 0 \quad \forall v \perp h_k,$$

d.h. B_{k+1} unterscheidet sich von B_k nur „in Richtung h_k “

- damit erhalten wir *Broyden-Rang-1-Updates*

$$B_{k+1} = B_k + \frac{(y_k - B_k h_k) h_k^T}{h_k^T h_k}$$

- man startet meistens mit $B_0 = I$
- in der Regel sind die B_{k+1} nicht spd
- Problem kann beseitigt werden, indem für W geeignete Matrizen mit Rang 2 benutzt werden
- die erfolgreichste Update-Variante ist die *BFGS-Update-Formel* (Broyden, Fletcher, Goldfarb, Shanno)

$$B_{k+1} = B_k + \frac{y_k y_k^T}{h_k^T y_k} - \frac{u_k u_k^T}{h_k^T u_k}$$

mit

$$h_k = x_{k+1} - x_k, \quad y_k = f'(x_{k+1}) - f'(x_k), \quad u_k = B_k h_k$$

- für $h_k^T y_k > 0$ ist B_{k+1} spd, falls B_k spd ist ($h_k^T u_k = h_k^T B_k h_k > 0$, da B_k spd)
- damit erhalten wir insgesamt als BFGS-Quasi-Newton-Verfahren:
- starte mit $x_0, B_0 = I$
- wiederhole:

- löse $B_k p_k = -f'(x_k)$
- bestimme $x_{k+1} = x_k + \alpha_k p_k$ mit Liniensuche
- berechne Update $B_{k+1} = B_k + \frac{y_k y_k^T}{h_k^T y_k} - \frac{u_k u_k^T}{h_k^T u_k}$

- in jedem Schritt muss das lineare Gleichungssystem $B_k p_k = -f'(x_k)$ gelöst werden, was einen Aufwand $O(n^3)$ verursacht
- man kann die Update-Formeln auch direkt für $D_k = B_k^{-1}$ herleiten so dass p_k dann direkt als

$$p_k = -D_k f'(x_k),$$

berechnet werden kann, wofür nur noch $O(n^2)$ Operationen nötig sind

- für BFGS für die Inverse erhält man

$$D_{k+1} = D_k + \kappa_1 h_k h_k^T - \kappa_2 (h_k v_k^T + v_k h_k^T)$$

mit

$$\begin{aligned}h_k &= x_{k+1} - x_k \\y_k &= f'(x_{k+1}) - f'(x_k) \\v_k &= D_k y_k \\ \kappa_2 &= \frac{1}{h_k^T y_k} \\ \kappa_1 &= \kappa_2(1 + \kappa_2 y_k^T v_k)\end{aligned}$$

- insgesamt erhalten wir damit Verfahren mit folgenden Eigenschaften:
 - globale Konvergenz (unter einigen Einschränkungen an f)
 - superlineare Konvergenz
 - einfach zu implementieren
 - Berechnung von $f''(x)$ nicht erforderlich

Nichtlineare Ausgleichsprobleme

- oben hatten wir bereits lineare Ausgleichsprobleme untersucht:
 - zu gegebenen Punkten $(x_i, y_i) \in \mathbb{R}^2, i = 1, \dots, n$, soll ein Polynom p vom Grad $m-1$ mit Koeffizienten $p_1, \dots, p_m$ bestimmt werden, so dass die Summe der Fehlerquadrate

$$f(p) = \frac{1}{2} \sum_{i=1}^n (p(x_i) - y_i)^2$$

minimal wird

- $f(p)$ kann man schreiben als

$$f(p) = \frac{1}{2} F(p)^T F(p), \quad F(p) = Ap - y,$$

mit

$$A = \begin{pmatrix} 1 & x_1 & \dots & x_1^{m-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \dots & x_n^{m-1} \end{pmatrix} \in \mathbb{R}^{n \times m}, \quad p = \begin{pmatrix} p_1 \\ \vdots \\ p_m \end{pmatrix} \in \mathbb{R}^m, \quad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n$$

- F ist also linear affin in p
- wie wir oben gesehen haben ist das zugehörige Ausgleichsproblem über die Normalgleichungen

$$A^T A p = A^T y$$

einfach lösbar, falls A vollen Rang hat

- hat A keinen vollen Rang, so ist der Minimierer von f nicht eindeutig
- sollen die Punkte (x_i, y_i) statt mit einem Polynom p z.B. mit einer Funktion $p(x) = p_1 e^{p_2 x}$ durch Minimieren der Summe der Fehlerquadrate approximiert werden, so erhalten wir analog mit $p = (p_1, p_2)^T$

$$f(p) = \frac{1}{2} \|F(p)\|_2^2, \quad F(p) = \begin{pmatrix} p_1 e^{p_2 x_1} - y_1 \\ \vdots \\ p_1 e^{p_2 x_n} - y_n \end{pmatrix},$$

wobei F jetzt eine nichtlineare Funktion in p ist

Definition 356

Es sei $F : \mathbb{R}^m \rightarrow \mathbb{R}^n$ nichtlinear und hinreichend oft differenzierbar. Das Problem

$$\min_{x \in \mathbb{R}^m} f(x), \quad f(x) = \frac{1}{2} \|F(x)\|_2^2 = \frac{1}{2} F(x)^T F(x)$$

heißt *nichtlineares Ausgleichsproblem*

- das nichtlineare Ausgleichsproblem ist damit ein nicht restringiertes Minimierungsproblem, bei dem die Zielfunktion eine spezielle Struktur hat
- damit können wir prinzipiell die Verfahren aus den vorherigen Kapiteln anwenden
- wegen der speziellen Struktur von f ergeben sich teilweise Vereinfachungen
- die Verfahren aus den letzten Abschnitten greifen auf den Gradienten $f'(x)$ bzw. die Hesse-Matrix $f''(x)$ zu
- wir erhalten

$$f'(x) = F'(x)^T F(x)$$

wobei $F'(x) \in \mathbb{R}^{n \times m}$ die Jacobi-Matrix von F ist, bzw.

$$f''(x) = F'(x)^T F'(x) + \sum_{i=1}^n F_i(x) F_i''(x),$$

wobei $F_i''(x) \in \mathbb{R}^{m \times m}$ die Hesse-Matrix von $F_i(x)$ ist

- zur Herleitung eines ersten iterativen Verfahrens nähern wir $F(x)$ lokal durch eine linear affine Approximation

$$L_k(x) = F(x_k) + F'(x_k)(x - x_k)$$

an

- statt $f(x) = \frac{1}{2} \|F(x)\|_2^2$ minimieren wir nun

$$\tilde{q}_k(x) = \frac{1}{2} \|L_k(x)\|_2^2$$

und benutzen einen Minimierer als neue Näherung x_{k+1}

- diese Aufgabenstellung können wir auf zwei Arten interpretieren
- einerseits ist

$$L_k(x) = F'(x_k)x - (F'(x_k)x_k - F(x_k)) = A_k x - b_k$$

und wir erhalten

$$\tilde{q}_k(x) = \frac{1}{2} \|A_k x - b_k\|_2^2,$$

d.h. x_{k+1} ist Lösung eines linearen Ausgleichsproblems

- andererseits gilt

$$\begin{aligned} \tilde{q}_k(x) &= \frac{1}{2} L_k^T(x) L_k(x) \\ &= \frac{1}{2} (F(x_k)^T F(x_k) + F(x_k)^T F'(x_k)(x - x_k) \\ &\quad + (x - x_k)^T F'(x_k)^T F(x_k) \\ &\quad + (x - x_k)^T F'(x_k)^T F'(x_k)(x - x_k)) \\ &= \frac{1}{2} (F(x_k)^T F(x_k) + 2(F'(x_k)^T F(x_k))^T (x - x_k) \\ &\quad + (x - x_k)^T F'(x_k)^T F'(x_k)(x - x_k)) \end{aligned}$$

so dass

$$\tilde{q}_k(x) = f(x_k) + f'(x_k)^T(x - x_k) + \frac{1}{2}(x - x_k)^T F'(x_k)^T F'(x_k)(x - x_k)$$

ein lokales quadratisches Modell für f ist mit einer Näherung

$$B_k = F'(x_k)^T F'(x_k) = A_k^T A_k$$

der Hesse-Matrix

- hat $A_k = F'(x_k)$ vollen Rang, dann ist B_k spd und x_{k+1} ist eindeutig bestimmt durch

$$\begin{aligned} x_{k+1} &= (A_k^T A_k)^{-1} A_k^T b_k \\ &= B_k^{-1} A_k^T b_k \\ &= (F'(x_k)^T F'(x_k))^{-1} F'(x_k)^T (F'(x_k)x_k - F(x_k)) \\ &= x_k - (F'(x_k)^T F'(x_k))^{-1} F'(x_k)^T F(x_k) \\ &= x_k - F'(x_k)^+ F(x_k) \end{aligned}$$

Definition 357

Das *Gauß-Newton-Verfahren* für das nichtlineare Ausgleichsproblem ist definiert durch

$$x_{k+1} = x_k - F'(x_k)^+ F(x_k)$$

- wenn $F'(x_k)$ keinen vollen Rang hat, dann ist $B_k = F'(x_k)^T F'(x_k)$ symmetrisch positiv semidefinit, also singular, d.h. $\tilde{q}_k$ hat keinen eindeutigen Minimierer
- welchen Minimierer benutzen wir als x_{k+1} ?
- primitive Lösung:
 - benutze Pseudo-Inverse $F'(x_k)^+$
 - wie wir aus dem Kapitel über Regularisierung wissen, kann die Multiplikation mit $F'(x_k)^+$ sehr schlecht konditioniert sein
- zweite Lösung:
 - x_{k+1} soll $\tilde{q}_k$ minimieren, wird also im Prinzip mit einem (Quasi-)Newtonschritt ermittelt, wobei statt $f''(x_k)$ die Näherung B_k benutzt wird
 - hat $F'(x_k)$ keinen vollen Rang hat, dann ist $B_k = F'(x_k)^T F'(x_k)$ symmetrisch positiv semidefinit, also

$$B_k + \mu I$$

spd für alle $\mu > 0$

- das entspricht der Idee des gedämpften Newton-Verfahrens
- wir müssen nicht das selbe μ für alle k benutzen, sondern können μ von Schritt zu Schritt anpassen, indem wir z.B. die Levenberg-Marquardt-Strategie wie bei den gedämpften Newton-Verfahren benutzen
- insgesamt erhalten wir das *Levenberg-Marquardt-Verfahren* für nichtlineare Ausgleichsprobleme:
 - starte mit $x := x_0, \mu := \mu_0, x_0, \mu_0 > 0$ gegeben
 - wiederhole bis Genauigkeit erreicht ist:
 - löse $(F'(x)^T F'(x) + \mu I)p = -f'(x) = -F'(x)^T F(x)$
 - $x_{\text{neu}} := x + p$
 - berechne

$$\rho := \frac{f(x) - f(x_{\text{neu}})}{\tilde{q}(x) - \tilde{q}(x_{\text{neu}})}$$

mit $\tilde{q}(y) = \frac{1}{2} \|F(x) + F'(x)(y - x)\|_2^2$

- ist $\rho > 0$:
 - $x := x_{\text{neu}}$
 - $\mu := \mu \max\left(\frac{1}{3}, 1 - (2\rho - 1)^3\right)$
 - $\nu = 2$
- ist $\rho \leq 0$:
- x_{neu} wegwerfen (aktuellen Schritt wiederholen)
- $\mu := \nu\mu$
- $\nu := 2\nu$

Zusammenfassung

- Ziel ist die Minimierung einer glatten Funktion ohne Nebenbedingungen, also

$$\min_{x \in \mathbb{R}^n} f(x)$$

- Abstiegsverfahren: Iterationen der Form $x_{k+1} = x_k + \alpha_k p_k$ mit Abstiegsrichtung p_k und geeigneter Schrittweite α_k
- Liniensuche: Wahl von α_k zur sicheren Verbesserung von $f(x)$
- Steepest-Descent: einfaches Verfahren mit $p_k = -\nabla f(x_k)$, robust, aber oft langsam
- Konjugierte Gradienten (CG): nutzt „bessere“ Suchrichtungen (insb. für quadratische Modelle), oft deutlich schneller als Steepest-Descent
- Newton-Verfahren: verwendet lokale Krümmungsinformation (f'') und konvergiert nahe der Lösung sehr schnell, benötigt aber Stabilisierung (f'' singulär)
- Trust-Region: statt freier Schrittweite wird ein lokales Modell nur in einer „Vertrauensregion“ minimiert, erhöht Robustheit
- Quasi-Newton: approximiert die Hesse-Matrix (z. B. BFGS) und verbindet gute Konvergenz mit geringerem Aufwand pro Iteration.
- Nichtlineare Ausgleichsprobleme: Spezialfall

$$\min_x \|f(x)\|_2^2$$

mit angepassten Methoden (Gauß-Newton/Levenberg–Marquardt)

Literaturverzeichnis

- [DH19] Peter Deufhard and Andreas Hohmann. *Numerische Mathematik 1*. De Gruyter, Berlin, Boston, 2019. ISBN 9783110614329. URL: <https://doi.org/10.1515/9783110614329>, doi:doi:10.1515/9783110614329.
- [FJBNT04] Poul Erik Frandsen, Kristian Jonasson, Hans Bruun Nielsen, and Ole Tingleff. *Unconstrained Optimization, Lecture Notes*. Technical University of Denmark, 2004. URL: <http://www2.imm.dtu.dk/pubdb/edoc/imm3217.pdf>.
- [FH07] R.W. Freund and R.W. Hoppe. *Stoer/Bulirsch: Numerische Mathematik 1*. Springer-Lehrbuch. Springer Berlin Heidelberg, 2007. ISBN 9783540453901. URL: <https://link.springer.com/book/10.1007/978-3-540-45390-1>.
- [Gro13] Peter Groth. *FEM-Anwendungen*. Springer Berlin, Heidelberg, 2013. URL: <https://link.springer.com/book/10.1007/978-3-642-56224-2>.
- [HB09] Martin Hanke-Bourgeois. *Grundlagen der Numerischen Mathematik und des Wissenschaftlichen Rechnens*. Vieweg + Teubner, 2009. ISBN 978-3-8348-0708-3. URL: <https://link.springer.com/book/10.1007/978-3-8348-9309-3>.